Data-Driven Analysis to Predict Bullying Victimization Risk Among Adolescent Students Using Machine Learning

By

YAP JIA XIN

TP066475

APU3F2411CS(DA)

A project submitted in partial fulfillment of the requirements of Asia Pacific University of Technology and Innovation for the degree of

B.Sc. (Hons) in Computer Science (Data Analytics)

Supervised by Ms. NUR AMIRA BINTI ABDUL MAJID

2nd Marker: Ms. AZIAH ABDOLLAH

2025

DECLARATION OF THESIS CONFIDENTIALITY

Author's full name: **YAP JIA XIN**

IC No./Passport No.: **030211-14-0562**

Thesis/Project title: **Data-Driven Analysis to Predict Bullying Victimization Risk Among Adolescent Students Using Machine Learning**

I declare that this thesis is classified as:

☐ CONFIDENTIAL

☐ RESTRICTED

☑ OPEN ACCESS

I acknowledged that Asia Pacific University of Technology & Innovation (APU) reserves the right as follows:

1. The thesis is the property of Asia Pacific University of Technology & Innovation (APU).
2. The Library of Asia Pacific University of Technology & Innovation (APU) has the right to make copies for the purpose of research only.
3. The Library has the right to make copies of the thesis for academic exchange.

Author's Signature: Yap Jia Xin

Date: 22 July 2025

Supervisor's Name: NUR AMIRA BINTI ABDUL MAJID

Date: 22 July 2025

Signature: nmr

Please fill in **all** the following details for library cataloguing purposes.

First Name: Jia Xin

Middle Name (only if applicable) : -

Last Name: Yap

Title of the Final Year Project / Dissertation / Thesis :

Data-Driven Analysis to Predict Bullying Victimization Risk Among Adolescent Students Using Machine Learning

Abstract :

Bullying is a pervasive issue that significantly affects adolescents' mental health, academic performance, and overall well-being. With the rise of digital platforms, cyberbullying has emerged as an additional challenge. This project aims to develop a predictive system to identify students at risk of being bullied, using machine learning algorithms and the Global School-Based Student Health Survey (GSHS) dataset. The study employs various models, including Logistic Regression, Random Forest, Support Vector Machine (SVM), and XGBoost, to assess their accuracy in predicting bullying victimization. The goal is to provide actionable insights to educators, parents, and policymakers to facilitate early interventions, aligning with the United Nations' Sustainable Development Goal (SDG) 3, which emphasizes ensuring good health and well-being for all. The results reveal that XGBoost outperforms other models in terms of accuracy and F1-score, offering a robust tool for bullying prediction. Despite its promising outcomes, the system's reliance on a single dataset and complexity of the models are identified as limitations. Future improvements will focus on expanding the dataset and enhancing model interpretability to further contribute to the reduction of bullying and improvement of school environments.

Keywords: Bullying, Machine Learning, Prediction, Adolescents, Mental Health

A few keywords associated with the work : Bullying, Prediction, Machine Learning

General Subject: Data Management (DTM), Program of Data Analysis (PFDA), Text Analytics and Sentiment Analysis (TXSA)
Date of Submission : 23 July 2025

ACKNOWLEDGEMENT

I would like to express my deepest gratitude to my supervisor, Miss Nur Amira Abdul Majid, for their unwavering support, invaluable guidance, and constant encouragement throughout this research project. Her extensive expertise, thoughtful insights, and constructive feedback have been instrumental in shaping the direction and quality of this study. From the early stages of developing my Project Proposal Form to refining the final Investigation Report, Miss Amira's meticulous attention to detail and dedication to my academic growth have significantly contributed to the successful completion of this work. I am truly fortunate to have had her as my supervisor.

Additionally, I would like to extend my heartfelt thanks to my senior, Nicky Tan, whom I had the pleasure of working with during my internship at Smith & Nephew. Nicky's support and guidance during our time working together have played a vital role in my learning experience. Our insightful discussions and collaborative problem-solving approach have expanded my understanding of the research field and have been a continuous source of motivation. Nicky's encouragement has not only enhanced my technical skills but also boosted my confidence in tackling challenging aspects of the study. I am deeply grateful for their generosity in sharing their knowledge and their support in every step of this journey.

I would also like to thank my family and friends for their unending support and belief in me throughout this research. Their love, understanding, and encouragement have been a constant source of strength.

ABSTRACT

Bullying is a pervasive issue that significantly affects adolescents' mental health, academic performance, and overall well-being. With the rise of digital platforms, cyberbullying has emerged as an additional challenge. This project aims to develop a predictive system to identify students at risk of being bullied, using machine learning algorithms and the Global School-Based Student Health Survey (GSHS) dataset. The study employs various models, including Logistic Regression, Random Forest, Support Vector Machine (SVM), and XGBoost, to assess their accuracy in predicting bullying victimization. The goal is to provide actionable insights to educators, parents, and policymakers to facilitate early interventions, aligning with the United Nations' Sustainable Development Goal (SDG) 3, which emphasizes ensuring good health and well-being for all. The results reveal that XGBoost outperforms other models in terms of accuracy and F1-score, offering a robust tool for bullying prediction. Despite its promising outcomes, the system's reliance on a single dataset and complexity of the models are identified as limitations. Future improvements will focus on expanding the dataset and enhancing model interpretability to further contribute to the reduction of bullying and improvement of school environments.

Keywords: Bullying, Machine Learning, Prediction, Adolescents, Mental Health

SDG Goal 3: Ensure healthy lives and promote well-being for all at all ages

Table of Contents

Table of Figures

Table of Tables

CHAPTER 1: INTRODUCTION

1.1 Introduction

Bullying is one of the most pervasive social issues that negatively affect individuals and communities. It is generally defined as repeated aggressive behavior, either verbal, physical, or psychological, intended to harm or scare someone who is thought to be weak. Bullying can occur in various settings, including schools, workplaces, homes, and online platforms which are known as cyberbullying. Bullying can be physically such as hitting, kicking, pushing or breaking belongings. Besides, it also can be verbal such as threatening or inappropriate comments and cyberbullying through online which involve harassment or spreading harmful content about someone (Bibi et al., 2019). These will result in adverse outcomes such as depression, anxiety and suicidal ideation.

Bullying among students has emerged as a significant global issue. During school years, bullying is an expression of violence in the peer context, with significant implications for their mental health, academic performance and overall well-being. Several international organizations have identified violence as a significant serious issue. Bullying harms students' rights, especially their rights to education. Students with disabilities, students who migrated from other places, students have different skin color from others, or students who belong to a minority group are among the vulnerable groups who are particularly at risk (Menesini & Salmivalli, 2017). Therefore, addressing bullying is critical to achieving this goal, as it not only affects the individuals involved but also compromises the safety and inclusivity of learning environments.

In recent years, machine learning techniques have been increasingly applied to predict and mitigate bullying in schools. For example, a study utilizing a Gradient Boosting Decision Tree (GBDT) model demonstrated the potential of machine learning in forecasting bullying victimization among students, thereby enhancing the effectiveness of intervention strategies (Qiu et al., 2024). At the same time, research has identified various factors associated with bullying behavior, such as age, sex, school performance, psychological factors, and ethnicity (Galán-Arroyo et al., 2023).

This project aims to develop a predictive model using machine learning algorithms to identify students at risk of bullying. By analyzing the Global School-Based Student Health Survey (GSHS) dataset, the study seeks to uncover significant predictors of bullying, enabling educators and policymakers to implement targeted interventions. This initiative aligns with the

United Nations' Sustainable Development Goal 3, which emphasizes ensuring healthy lives and promoting well-being for all at all ages.

1.2 Problem Background

1. Effectiveness of Anti-Bullying Policies in Schools

Despite the implementation of anti-bullying policies in schools worldwide, many students report that these measures often fail to translate into meaningful action. Surveys indicate that a significant number of students feel unsupported when reporting bullying incidents, with school authorities frequently dismissing their concerns or lacking the tools to intervene effectively. This disconnect between policy and practice underscores the need for a more comprehensive approach, including regular training for educators, clear accountability systems, and involvement of parents and community stakeholders to ensure policies are enforced consistently (Juvonen & Graham, 2014). Addressing this issue aligns with Sustainable Development Goal (SDG) 3, which emphasizes ensuring good health and well-being. Strengthening anti-bullying policies contributes to students' mental and emotional health, reducing the long-term psychological effects of victimization.

2. The Impact of Bullying

Bullying among adolescents has profound and multifaceted impacts on mental health, emotional well-being, and social development, often leading to long-term consequences. Victims of bullying experience heightened levels of anxiety, depression, and suicidal ideation, which can persist into adulthood if left unaddressed (Menesini & Salmivalli, 2017). Studies also reveal that bullying significantly affects academic performance, with victims demonstrating lower engagement, higher absenteeism, and diminished academic outcomes (Galán-Arroyo et al., 2023). Moreover, bystanders who witness bullying situations may experience secondary trauma, fear and feelings of helplessness, thereby exacerbating the ripple effects of bullying within school communities (Rigby, 2003). These findings highlight the necessity of SDG 3, which advocates for mental health support and overall well-being. Implementing mental health programs in schools can help mitigate the adverse effects of bullying and promote a healthier learning environment.

3. Emerging Threat of Cyberbullying in Schools

Other than traditional bullying such as physical bullying, cyberbullying has emerged as a significant concern among school-aged children and adolescents with the increasing penetration of digital technology. Unlike traditional bullying, cyberbullying extends beyond the physical confines of the school and can occur at any time, leaving victims with little respite. This form of bullying often involves the use of social media platforms, messaging apps, or gaming communities to harass, intimidate, or spread false information about others. The pervasive nature of cyberbullying can lead to severe emotional and psychological distress for victims, including anxiety, depression, and in extreme cases, suicidal ideation. Schools and policymakers need to develop robust cyberbullying prevention and intervention strategies that include digital literacy programs, monitoring tools, and stronger regulations for online behavior (Källmén & Hallgren, 2021).

4. Parental Awareness and Involvement in Combating Bullying

Parental awareness and involvement play a critical role in mitigating bullying behaviors. Many parents remain unaware of their child's involvement in bullying, either as a victim or a perpetrator, due to a lack of open communication or misunderstanding of the signs. Parents equipped with the knowledge to recognize behavioral changes and actively participate in their child's school life can serve as a vital support system. Besides, schools must work closely with parents through workshops and awareness campaigns to ensure a collaborative approach to addressing bullying (Källmén & Hallgren, 2021). Encouraging parental engagement aligns with SDG 3, as fostering a supportive home environment contributes to children's emotional and psychological health. When parents actively participate in anti-bullying initiatives, they help create a protective and nurturing atmosphere, reducing the risks of long-term mental health consequences.

1.3 Project Aim

The aim of this project is to develop an advanced machine learning model to predict bullying incidents among students, using the Global School-Based Student Health Survey (GSHS) dataset. In order to reduce the rate of bullying, the model will provide actionable insights to support early intervention strategies, contributing to SDG 3: Good Health and Well-being.

1.4 Objectives

The objectives of this project are:

1. To perform exploration data analysis (EDA) on the Global School-Based Student Health Survey (GSHS) dataset to identify significant predictors of bullying among students.
2. To build a predictive model using machine learning techniques with clean and pre-processed datasets to classify and predict bullying incidents.
3. To validate the predictive model by evaluating its performance using appropriate metrics such as accuracy and f1 score.
4. To provide evidence-based recommendations for students, parents and educators based on the identified predictors to enable early interventions.

1.5 Scope

1.5.1 Task To be executed

The deliverables for this project include a machine learning-based detection solution for bullying in school. This is to identify the key contributing factors that lead to bullying among students.

1. **Target Users**

   The target users for this project are students, parents and educators. For students, this project will help them to identify individuals at risk of being bullied, allowing them to make timely interventions to protect mental health, enhance peer relationships, and improve their well-being. Besides, parents can use the insights to better understand their child and protect them from bullying. For educators such as teachers and school administrators, this project can guide them to focus on at-risk students and enable them to create a safer school environment by implementing anti-bullying programs.

2. **Data Acquisition**

   This project will be using an open dataset which related to bullying among students from Kaggle. This dataset is the Global School-Based Student Health Survey (GSHS). It is a school-based survey which uses a self-administered questionnaire to obtain data on young people's health behavior and protective factors related to the leading causes of morbidity and mortality. The survey was conducted in Argentina in 2018. A total of 56,981 students participated. From the GHSH, the survey questions related to bullying were selected.

3. **Data Understanding and Processing**

   Exploratory Data Analysis (EDA) will be conducted to uncover trends, correlations, and insights within the data. Data cleaning processes will ensure quality by addressing missing values, duplicate entries, and outliers. Feature engineering will focus on extracting meaningful variables, such as bullying frequency or severity scores. The dataset will be separated into training and testing datasets for model training and evaluation after preprocessing.

4. **Model Building**

   A fully developed machine learning model designed to predict the likelihood of bullying incidents among students. The model will identify individuals at risk, either as victims or perpetrators, based on demographic, behavioral, and contextual data.

5. **Model Training and Evaluation**

   The dataset is split into training, validation, and test sets, with cross-validation to prevent overfitting. Metrics such as accuracy, precision, recall, F1-score, and AUC-ROC are used to evaluate performance, especially for imbalanced data. Iterative refinements improve the model through feature adjustments and alternative algorithms, ensuring robust and accurate predictions.

6. **Interpreting Model Result**

   Detailed interpretability tools and techniques such as SHAP and LIME will integrate into the model to provide clear and actionable explanations for predictions, enabling educators and stakeholders to understand the key factors contributing to bullying risks.

1.5.2 Constraints

1. The Global School-Based Student Health Survey (GSHS) 2018 dataset may not accurately reflect real-time or region-specific bullying trends. Besides, this is a school-based survey that uses a self-report questionnaire to obtain data. Therefore, there could raise the possibility of bias.
2. The dataset does not include all possible risk factors of bullying, such as family relationships, social media activity, or the effectiveness of school policies. These factors could improve the predictive model.
3. This project only predicts bullying victimization risk among adolescent students but not both bullies and the bullied.

1.5.3 What will be done as part of the project?

1. Conduct Exploratory Data Analysis (EDA) to understand the dataset by identifying the characteristics of the bullied and missing values in the dataset.
2. Perform Data Cleaning and Pre-processing to handle missing values, and outliers.
3. Apply Data Transformation techniques such as features engineering to prepare a clear dataset for modeling.
4. Implement machine learning algorithms to build a predictive model by using pre-processed dataset to predict students at risk of being bullied.
5. Conduct model training and evaluation by using appropriate metrics such as accuracy, f1 score, precision, and ROC-AUC curve.
6. Create a simple system for the users to test the predictive models on the bullied.

1.5.4 What will not be done as part of the project?

1. This project does not conduct surveys for collecting data but uses open dataset from Kaggle.
2. This project will not build a fully functional system, it is just a simple system for testing on the best predictive model to predict at-risk students.
3. The system will not identify and predict the bullies, but the bullied.

1.6 Potential Benefits

The goal of this project is to create a prediction model that uses machine learning to find adolescents who are likely to be bullied. By analyzing the Global School-Based Student Health Survey (GSHS) dataset, this project can provide valuable insights to educators, parents, and policymakers to prevent bullying. The benefits of this project can be categorized into tangible and intangible benefits.

1.6.1 Tangible Benefits

Tangible benefits are the outcomes that can be measured, such as reduced costs or enhanced decision making. For this project, the tangible benefits include:

1. Reduction of bullying incidents: Schools can take early detections by identifying at-risk students and predicting possible bullying incidents. This will eventually result in a quantifiable decrease in bullying incidents.
2. Improved Academic Performance: Reducing bullying through early detection can help students perform better on academic, which can be shown by their improved grades. This is because bullying has negatively impacted students' concentration on study and their attendance.
3. Optimized Resource Allocation: By utilizing the insights gathered from this approach, education departments and schools may effectively distribute counseling resources, ensuring that students who need help acquire it immediately. As a result, the operating expenses for mental health therapies may decrease.

1.6.2 Intangible Benefits

Tangible benefits are the outcomes that cannot be measured by the currency but can see benefits other than the currency, such as well-being and mental health. For this project, the intangible benefits include:

1. Enhanced Mental Health & Well-being: Early intervention can help students avoid the long-term emotional consequences of bullying, such as depression, anxiety, and suicidal ideation, providing them a safer and healthier school environment.
2. Stronger Student-Teacher Relationships: Teacher can more understand their students' emotional well-being and boost stronger relationship between teachers and students.

3. Increased Awareness & Preventive Culture: This project raise the awareness of risk of bullying among students, teachers, and parents, leading to more proactive and preventive actions rather than reactive responses.

1.7 Overview of the FYP Documentation

The Final Year Project (FYP) documentation presents a comprehensive study and development of a **Bullying Victimization Risk Prediction System**. The project aimed to create a predictive tool that helps identify students at risk of being bullied, with the goal of enabling early intervention and prevention strategies. The documentation covers various aspects of the project, including its objectives, methodology, system design, implementation, evaluation, and the results obtained from using machine learning algorithms to predict bullying risks.

Project Background and Objectives

The project was motivated by the growing concern about bullying in schools and its detrimental effects on students' mental health and well-being. The primary objective was to build a system that can predict the likelihood of a student being bullied, using a variety of factors and machine learning techniques. The system was developed to provide educators, parents, and policymakers with actionable insights that could help reduce bullying incidents and improve the overall school environment.

Methodology

The project used the **CRISP-DM methodology**, a popular framework for data mining and machine learning projects. The six phases of CRISP-DM—Business Understanding, Data Understanding, Data Preparation, Modeling, Evaluation, and Deployment—guided the development process. The project utilized data from the **Global School-Based Student Health Survey (GSHS)**, which provided detailed information about students' experiences with bullying, their emotional well-being, and other relevant factors.

Machine Learning Models

The documentation explores the use of four machine learning models to predict bullying victimization: **Logistic Regression, Random Forest, Support Vector Machine (SVM), and XGBoost**. The models were trained on the GSHS dataset and evaluated based on accuracy, precision, recall, and F1-score. Among these models, XGBoost demonstrated the best performance, achieving the highest accuracy and F1-score.

System Design and Implementation

The system was developed using Python and the **Streamlit** framework, providing a simple and intuitive user interface for educators and parents to input student data and receive predictions. The system was designed to offer real-time risk assessments, allowing users to take immediate action if necessary. The documentation includes details of the system's architecture, code implementation, and user interface design.

Results and Evaluation

The evaluation of the models revealed that the **Random Forest** and **SVM** models showed significant improvements after hyperparameter tuning. XGBoost, however, remained the top-performing model throughout the project. The system's performance was evaluated using various metrics, including the ROC-AUC curve, learning curves, and feature importance analysis. The system's ability to predict bullying risk with high accuracy demonstrates its potential as a tool for preventing bullying in schools.

Conclusion

The project successfully developed a predictive system that can help identify students at risk of being bullied. The documentation highlights the system's strengths, including its accuracy and ease of use, while also acknowledging its limitations, such as the reliance on a single dataset and the complexity of some machine learning models. Recommendations for future improvements include expanding the dataset, improving model interpretability, and integrating real-time interventions.

Overall, this FYP documentation showcases a valuable contribution to bullying prevention in schools, demonstrating the potential of machine learning to address complex social issues. The system's development, evaluation, and recommendations for future improvements lay the groundwork for further research and enhancement in the field of student well-being and safety.

1.8 Project Plan

Task Name	Duration	Start Date	End Date	Status
Project Proposal Form	20 Days	Tue 17/12/2025	Mon 6/1/2025	Completed
Chapter 1: Introduction	**8 Days**	**Mon 17/2/2025**	**Mon 24/2/2023**	**Completed**
1.1 Introduction	1 Day	Mon 17/2/2025	Mon 17/2/2025	Completed
1.2 Problem Background	1 Day	Mon 17/2/2025	Mon 17/2/2025	Completed
1.3 Project Aim	1 Day	Mon 17/2/2025	Mon 17/2/2025	Completed
1.4 Objectives	1 Day	Mon 17/2/2025	Mon 17/2/2025	Completed
1.5 Scope	2 Days	Wed 19/2/2025	Fri 20/2/2025	Completed
1.6 Potential Benefits	1 Days	Sun 23/2/2025	Sun 23/2/2025	Completed
1.7 Overview of IR	1 Day	Mon 24/2/2023	Mon 24/2/2023	Completed
1.8 Project Plan	1 Day	Mon 17/2/2025	Mon 17/2/2025	Completed
Chapter 2: Literature Review	**10 Days**	**Tue 25/2/2025**	**Wed 10/3/2025**	**Completed**
2.1 Introduction	1 Day	Tue 25/2/2025	Tue 25/2/2025	Completed
2.2 Domain Research	5 Days	Sun 2/3/2025	Wed 6/3/2025	Completed
2.3 Similar Systems/Works	1 Day	Wed 26/2/2025	Wed 26/2/2025	Completed
2.4 Technical Research	2 Days	Fri 28/2/2025	Sat 1/3/2025	Completed
2.5 Summary	1 Day	Mon 3/3/2025	Mon 3/3/2025	Completed
Chapter 3: Methodology	**15 Days**	**Tue 25/2/2025**	**Sun 9/3/2025**	**Completed**
3.1 Introduction	1 Days	Tue 4/3/2025	Tue 4/3/2025	Completed
3.2 Methodology	2 Days	Tue 25/2/2025	Wed 26/2/2025	Completed
3.3 Data Collection	1 Day	Wed 26/2/2025	Wed 26/2/2025	Completed
3.3 Initial Data Understanding	10 Days	Thus 27/2/2025	Sat 8/3/2025	Completed
3.4.1 Data Understanding	10 Days	Thus 27/2/2025	Sat 8/3/2025	Completed
3.4.2 Data Pre-processing	10 Days	Thus 27/2/2025	Sat 8/3/2025	Completed
3.5 Summary	1 Days	Sun 9/3/2025	Sun 9/3/2025	Completed
Chapter 4: Conclusion	**1 Days**	**Wed 12/3/2025**	**Wed 12/3/2025**	**Completed**
References	<1 Day	Wed 12/3/2025	Wed 12/3/2025	Completed
Appendices	<1 Day	Wed 12/3/2025	Wed 12/3/2025	Completed

Table 1: Project plan 1

Task Name	Duration	Start Date	End Date	Status
Chap 1: Introduction	**3 days**	**6/6/2025**	**23/7/2025**	Completed
1.1 Introduction	<1 Day	6/6/2025	6/6/2025	Completed
1.2 Problem Background	<1 Day	6/6/2025	6/6/2025	Completed
1.3 Project Aim	<1 Day	6/6/2025	6/6/2025	Completed
1.4 Objectives	<1 Day	6/6/2025	6/6/2025	Completed
1.5 Scope	<1 Day	6/6/2025	6/6/2025	Completed
1.6 Potential Benefits	<1 Day	6/6/2025	6/6/2025	Completed
1.7 Overview of FYP Docs	1 Day	1/7/2025	1/7/2025	Completed
1.8 Project Plan	1 Day	6/6/2025	23/7/2025	Completed
Chap 2: Literature Review	**3 Days**	**6/6/2025**	**2/7/2025**	Completed
2.1 Introduction	<1 Day	6/6/2025	6/6/2025	Completed
2.2 Domain Research	1 Day	1/7/2025	2/7/2025	Completed
2.3 Similar Works	<1 Day	6/6/2025	6/6/2025	Completed
2.4 Technical Research	<1 Day	6/6/2025	6/6/2025	Completed
2.5 Summary	<1 Day	6/6/2025	6/6/2025	Completed
Chap 3: Methodology	**1 Days**	**6/6/2025**	**7/6/2025**	Completed
3.1 Introduction	1 Day	7/6/2025	7/6/2025	Completed
3.2 Methodology	<1 Day	6/6/2025	7/6/2025	Completed
3.5 Summary	1 Day	7/6/2025	7/6/2025	Completed
Chap 4: Design & Implementation	**14 Days**	30/6/2025	**13/7/2025**	Completed
4.1 Introduction	1 Day	30/6/2025	30/6/2025	Completed
4.2 Data Collection	1 Day	30/6/2025	30/6/2025	Completed
4.3 Data Pre-processing	3 Days	1/7/2025	4/7/2025	Completed
4.4 Data Understanding	2 Days	4/7/2025	6/7/2025	Completed
4.5 Model Building	7 Days	7/7/2025	13/7/2025	Completed
4.6 Summary	1 Day	13/7/2025	13/7/2025	Completed
Chap 5: Result & Discussion	**9 Days**	**14/7/2025**	**23/7/2025**	Completed
5.1 Introduction	1 Days	14/7/2025	14/7/2025	Completed
5.2 Model Evaluation & Discussion	5 Days	15/7/2025	20/7/2025	Completed

5.3 Model Deployment	3 Days	20/7/2025	23/7/2025	Completed
5.4 Summary	1 Day	23/7/2025	23/7/2025	Completed
Chap 6: Conclusion	**3 Days**	**21/7/2025**	**23/7/2025**	Completed
6.1 Critical Evaluation	2 Days	21/7/2025	23/7/2025	Completed
6.2 Limitation	2 Days	22/7/2025	23/7/2025	Completed
6.3 Recommendation	1 Day	23/7/2025	23/7/2025	Completed
References	1 Day	6/6/2025	23/7/2025	Completed
Appendices	1 Day	23/7/2025	23/7/2025	Completed

Table 2: Project Plan 2

CHAPTER 2: LITERATURE REVIEW

2.1 Introduction

Bullying victimization impacts millions of adolescents globally and is a serious public health and educational issue. It is an important topic for researchers, educators, and policymakers because of the significant impact it has on teenagers' mental health, academic performance, and overall well-being. Understanding of the root causes of bullying as well as its long-term effects is important for developing effective intervention and prevention strategies.

In literature review for this project, it will cover the different factors that influence victimization, the theoretical frameworks that explain bullying behavior, and the effects of bullying on students. The Social-Ecological Framework is analyzed to provide an in-depth overview of the ways the social, organizational, interpersonal, and human factors influence bullying dynamics. Furthermore, the review discusses the main risk factors which are personal traits, family history, peer interactions, and educational environments which are linked to bullying victimization. By identifying those components, teachers and students may create more focused strategies to stop bullying and assist children who are in danger.

Additionally, this review highlights the social, academic, and psychological repercussions of bullying victimization. Bullying victims frequently suffer from long-term consequences that last until adulthood, including social withdrawal, anxiety, despair, and poor academic performance. Understanding these consequences is important for developing effective networks of support and policies aimed at decreasing bullying occurrences.

This study supports Sustainable Development Goal (SDG) 3: Good Health and Well-Being by incorporating current research and theoretical viewpoints, highlighting the necessity of fostering safer and more encouraging learning environments in schools. Additionally, machine learning models may be integrated into bullying prevention programs to anticipate and detect at-risk adolescents early, enabling prompt intervention, thanks to technological advancements and data-driven techniques. This evaluation lays the groundwork for future research using data analytics and predictive modeling to reduce bullying victims and improve student wellbeing.

2.2 Domain Research

2.2.1 Theorical Framework Influencing Bullying

Theoretical frameworks impacting bullying victims include many kinds of theories that describe the dynamics and root causes of bullying behaviors. These frameworks integrate individual, social, and environmental factors, providing a comprehensive understanding of how bullying occurs and its impact on victims.

2.2.1.1 Social-Ecological Framework

The social-ecological framework is a common framework for studying bullying behavior focusing on the interaction of individual, interpersonal, organizational, community, and societal elements.

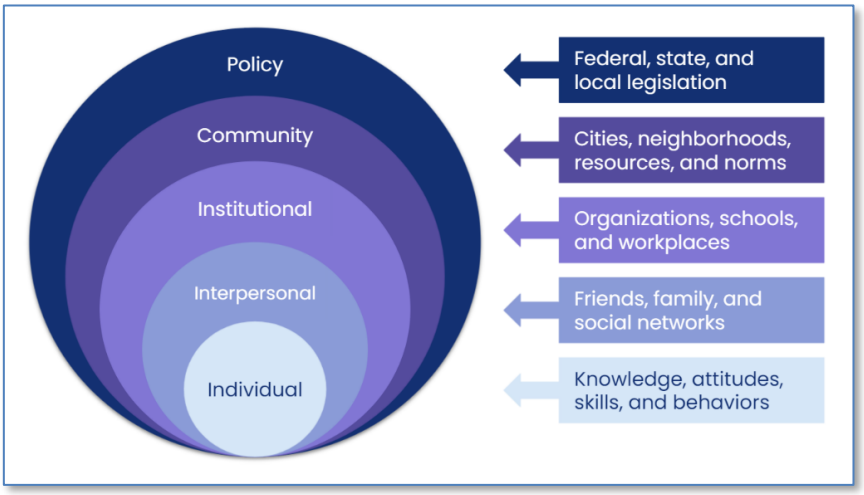


Figure 1: Social-Ecological Framework

In the **individual level**, the individual characteristics such as low self-control, aggression, and poor emotional regulation increase vulnerability to bullying. Girls are more likely to exhibit empathy and respond to interventions, while boys may resist behavioral changes despite training (Shams et al., 2018). Educational programs aim to increase individual awareness about bullying such as harmful actions can minimize victimization, although attitudes may stay intact after intervention (Shams et al., 2018). Moving forward to the **interpersonal level**, it highlights the importance of peer dynamics, in which spectator responses, such as supporting or intervening in bullying have a significant effect on results. Bully-victim subgroups have a strong connection with delinquent peer groups, while supportive interactions with parents and instructors act as protective buffers. For example, higher parental education levels and strong relationships between teachers and students are

associated with reduced bullying rates, but a perceived lack of adult support raises the risk (Napolitano & Espelage, 2011).

At the **organizational level**, school environment is important. Bullying occurrences are reported fewer times in classrooms with explicit anti-bullying rules, regular rule enforcement, and strong relationship between teachers and students. Programs such as the Olweus Bullying Prevention Program show the effectiveness of multi-level efforts that include staff training, student participation, and parental involvement (Napolitano & Espelage, 2011). Bullying behaviors are shaped by broader **community and cultural variables**. The cultural standards, such as acceptance for aggressiveness or gender stereotypes, legitimize particular acts, but digital circumstances extend bullying into online areas by providing anonymity and permanence. For example, cyberbullying requires changes to the social-ecological framework as it expands traditional bullying into the digital ecosystem (Patel & Quan-Haase, 2022).

2.2.2 Factors Influencing Bullying Victimization

Bullying victimization is influenced by a combination of factors such as individual characteristics, family background, peer interactions, and the school environment. Understanding these factors is important for identifying at-risk students and implementing effective intervention strategies. In previous research, they were mainly research on single factors only. Therefore, their research lacks comprehensive and structured proof (Qiu,T et al., 2024).

2.2.2.1 Individual factor

Bullying was related to many individual characteristics such as gender, age, race, appearance, and mental health are the factors that affect bullying victimization. These factors affect a student's risk of being a victim of bullying. Some of the students are at more risk than others because of their social and personal characteristics. The figure below shown the percentage of students between 12-18 years old who reported being bullied due to different personal characteristics in the year 2021-2022 (*COE - Student Bullying*, 2023).

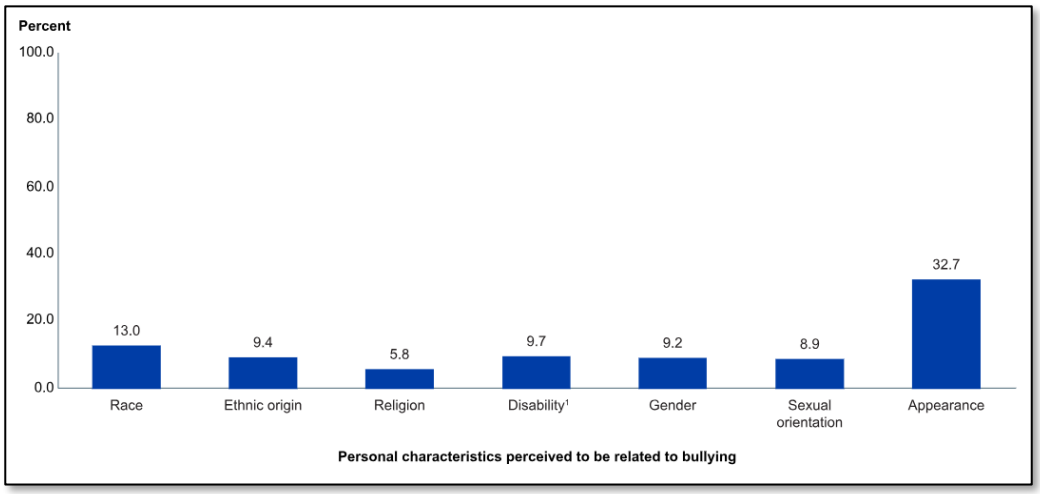


Figure 2: Percentage of different personal characteristics with bullying

Gender is the main and easiest characteristic to determine because it only has 2 options, boy or girl. Normally, boys are stronger than girls, which means boys usually act as the bullies and girls are the victims in bullying. However, these kinds of stereotypes influence boys negatively. Therefore, they show behaviors related to violence (Galán-Arroyo et al., 2023). Most of the research shows that boys are more likely to be involved in physical bullying, while girls are more commonly targeted in indirect forms of bullying, such as spreading rumors and social isolation (Ahmed et al., 2022) because they frequently behave

more to spread rumor about their classmates or to be excluded or rejected by social circles (Galán-Arroyo et al., 2023).

When it comes to **race**, some research found out that most of the black students are more likely categorized as bullies and less likely are categorized as the bullied victims compared with the Asian and White students (Marsh, 2018). However, some people argue that Black students are most likely to be classified as bully-victims and white men are more likely than men or women of any other race to get involved in bullying. Asian students are the least likely to participate in bullying (Marsh, 2018). The Black students usually experience more verbal attack such as being name calling and discriminate on their skin color. This same as the **ethnic**. Students who are from minor ethnic groups are easily get bullied compared with students who are from major ethnic group (Ahmed et al., 2022). Because of their small groups of different races, different cultures and different racial, the school environment is leading to discrimination and social exclusion.

Besides, bullying victimization is significantly influenced by **age**. Bullying is most common in secondary school, as seen by the fact that it generally increases over the primary school years, peaks during the early secondary school years, and then slightly reduces during the high school years (Marsh, 2018). Bullying among students with disabilities occurs at a prevalence of 24.5% in primary school, 34.1% in secondary school, and 26.6% in high school (Marsh, 2018). The figure below shows the frequency of bullying in school among 12-18 years old students.

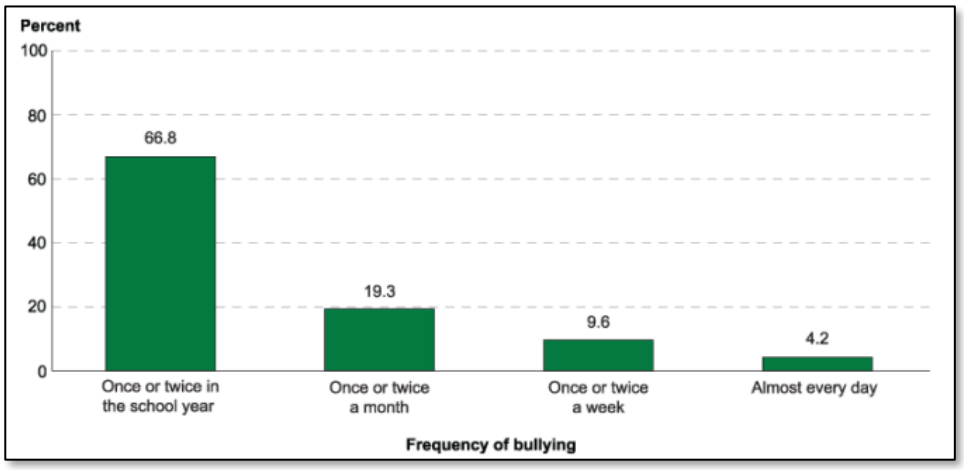


Figure 3: Frequency of bullying in school between 12-18 years old

In previous studies, researchers have found out that the **psychological** and behavioral characteristics of students are also an important factor that affect bullying in school. When

compared between students, the bullied victims often show higher degrees of neuroticism, irritability, and impulsiveness than the students who are not bullied. Additionally, they frequently show their negative emotion and always lack of confidence and self-care. This may make them choose to escape whenever they face challenges (Qiu et al., 2024). Mental health problems such as social anxiety, depression, and PTSD will increase the probability that they may be bullied as they always struggle with peer connections and due to their internal behaviors. Additional characteristics such as loneliness and low self-esteem also lead to bullying victimization. However, students with externalizing characteristics such as disobedience, violence or criminal behaviors are more likely to be bullies (Ahmed et al., 2022). This demonstrates how personality characteristics and a student's potential involvement in bullying occurrences are related.

Another important aspect that impacts bullying victimization is students' **physical appearance**. According to the research, a U-shape relationship has been observed. Students who are underweight that look skinny and overweight that look fat are more likely to be bullied while students who look big and strong in their body size are more likely to be the bullies (Ahmed et al., 2022). Moreover, **disabled students** are the most targeted bullied group as they face a lot of challenges on social and communication skills. This category of students is likely to face rejection from their peers. In addition, at-risk student group are also including **LGBTQ** student group. This group is identified as Lesbian, Gay, Bisexual, Transgender and Queer and they have the higher rate to get bullied including physically, verbally and relationally such as homophobic, name calling and physical attack. LGBTQ students are especially vulnerable in school environments because they not only face bullying from their peers, but they also suffer a lack of teacher intervention on their behaves as well as direct insults from school teachers, staff, and administrators (Marsh, 2018).

Last but not least, bullying victimization also can be influenced by **academic performance** and social skills. According to some research, students who have low grades in their academics are more likely to be bullied because their classmates may consider them as weak. On the other hand, in some cases, students who have high achievement in their academics also might experience bullying. This is caused by the jealousy among the classmates (Ahmed et al., 2022). In addition, students who are poor on social skills are hard to make friends, which increases the possibility of loneliness and bullying.

2.2.2.2 Family factor

Family plays an important role in shaping their child's behavior and well-being. which has a direct impact on their likelihood of participating in bullying. Parenting methods, parent-child relationships, family conflict, and home environment are all factor that could increase a child's likelihood of becoming a bully, victim, or both. A positive family environment may protect children from bullying, but a broken family might increase the likelihood of participating in incidents of bullying.

One of the most important family-related factors is **parenting style**. According to research, children who grow up with harsh or violent parents are more likely to be bullied at school (Qiu et al., 2024). This is because they observe the violence and conflict situation at home and their mental state are being hurt (Fraga et al., 2021). The likelihood of victimization is significantly increased when parents and children are emotionally distant from one another, poor communication, and lack of care. On the other hand, children who grow up in a healthy home environment, with supporting and loving parents, are less likely to become victims of bullying (Qiu et al., 2024).

Bullying behavior is also influenced by **family discord** and family disputes. Children grew in families whose parents always have conflict, abuse drugs, or engage in physical violence have a higher chance of being bullied and becoming bullies themselves (Ahmed et al., 2022). As the saying goes, parents are the role model for their children. Their children will learn everything because they do not know right or wrong but just follow their parents behavior. The **family structure** is also affecting bullying victimization. Children in stable two-parent households show better levels of life satisfaction and lower rates of bullying than those in single-parent or stepfamilies. The presence of both parents frequently offers a more supportive atmosphere, which helps protect them from the negative effects of bullying. (Ramasamy et al., 2024)

2.2.2.3 Peer factor

In school, peers play an important part in a student's social development and daily school life. Hence, peer interactions and group dynamics play key variables in bullying and victimization. Peer support, group norms, and delinquent conduct can all have an impact on whether a kid becomes a bully, victim, or both.

Strong peer connections are a fundamental protective aspect for bullying. According to research, students who have strong relationships are less likely to be bullied since their peers may give emotional and social support while students who lack peer support are more likely to be bullied (Qiu et al., 2024).

Besides, a study says that the quality of friendship has a significant impact on the probability of being bullied. Students with strong, supportive connections report fewer incidents of victimization while students with weak friendship quality are at greater risk to being bullied (Afriani Afriani & Denisa Denisa, 2021). Students who engage in vandalism, gang activities, or carry weapons at school are more likely to be bullied or to be offenders or victims. In contrast, students with pro-social conduct and low social anxiety are less likely to experience bullying and frequently perform better academically (Ahmed et al., 2022).

Moreover, alcohol and drug use are also significant risk factors. Research has proven a relationship between bullying and substance addiction, with research proving that students who engage in bullying are more likely to use alcohol and drugs throughout their school years and later in life (Ahmed et al., 2022).

2.2.2.4 School Factor

The school environment is also an important factor that influences students' experiences and their probability of being involved in bullying, whether as bullies or the bullied. Bullying victimization is strongly affected by several factors, including the classroom environment, school setting, relationship between teachers and students, and a sense of connection.

The most significant factor is the relationship between teachers and students. According to research, students who have respectful and encouraging interactions with their instructors are less likely to experience bullying (Qiu et al., 2024). Although their perspectives on bullying might differ, teachers play an important role in recognizing and dealing with bullying. However, when it comes to verbal or relational bullying, some teachers may view bullying as a natural part of social development, leading to a lack of intervention (Ahmed et al., 2022).

Bullying behavior is also influenced by the overall environment of the school. This is because if the school environment is positive and structured, the bullying cases are also less common in the school. However, it is more likely to happen in school if the school have poor discipline and unclear rules and regulation (Ahmed et al., 2022). In addition, students with learning difficulties or poor academic performance are at a higher risk of being bullied, as they may struggle with adapting to the school environment (Qiu et al., 2024).

Last but not lease, the feeling of belonging to school is a further important factor. Students are less likely to experience bullying if they feel satisfaction in their academic performance, actively participate in school events, and have a sense of belonging. However, bullying is more likely to occur among students who have poor relationships with peers and teachers (Markkanen et al., 2019). Therefore, this shows that fostering a strong, supportive school community can help prevent bullying.

In conclusion, bullying victimization is influenced by individual, family, peer, and school factors, all of which contribute to a student's likelihood of being targeted. Mental health, personality traits, socioeconomic background, and academic performance shape an individual's risk, while family dynamics, parenting styles, and household environment also play a crucial role. Additionally, peer group influences, social norms, delinquency, and substance use can either increase or decrease the chances of bullying. The school

environment, including teacher support, school climate, and student engagement, further affects bullying prevalence.

Identifying these risk signs is important for creating focused intervention plans that protect adolescents who are at danger. With this information, schools, educators, and legislators may develop proactive, inclusive, and supportive anti-bullying initiatives that tackle the underlying causes of victimization. Additionally, by incorporating machine learning models into bullying prevention initiatives, at-risk adolescents may be identified early on, facilitating prompt assistance and intervention.

This aligns with Sustainable Development Goal (SDG) 3: Good Health and Well-being, which emphasizes ensuring healthy lives and promoting well-being for all. By reducing bullying incidents, improving mental health support, and fostering safe school environments, this research contributes to the broader goal of enhancing adolescent well-being and creating healthier learning spaces for future generations.

2.2.3 Impact of Bullying Victimization

The impacts of bullying victimization on a student's social well-being, academic achievement, and mental health are severe and continuous. Bullying victims usually suffer from social isolation, psychological frustration, and poor self-esteem, which can negatively affect their personal development and future well-being. Bullying affects an individual's mental health and life outcomes, and its effects continue over the school years.

2.2.3.1 *Consequences of Mental Health*

Mental health conditions including depression, anxiety, and suicidal thoughts are strongly connected with bullying. Bullying victims frequently suffer from long-term stress, emotional pain, and a sense of helplessness, which raises their risk of mental health issues (Menesini & Salmivalli, 2017). According to research, bullying among teenagers increases the risk of developing symptoms of post-traumatic stress disorder (PTSD), which could last into adulthood (Ahmed et al., 2022). In severe situations, bullying can lead to suicide ideation and self-harm, which makes it a serious public health issue (Juvonen & Graham, 2013).

2.2.3.2 *Academic Performance and school Engagement*

Besides mental health, bullying also has negative effect on academic achievement. The stress, anxiety, and low self-esteem always makes the bullied victims difficult to focus in class, which result in poor educational achievement (Galán-Arroyo et al., 2023). A lot of victims develop a negative attitude toward school, associating it with threats, humiliation, and rejection, which reduces their motivation to participate in classroom activities (Rigby, 2003).

School absences are a serious issue as well. Research shows that since they are afraid to fall into their bullies, students that have been bullied are more likely to miss class or possibly quit school altogether (Patchin, 2019). Students' future educational and professional prospects are impacted by chronic absenteeism, which also leads to learning gaps and decreased academic achievement. Since bullying results in a negative and dangerous learning environment that affects the educational experience for both victims and spectators schools with high bullying rates also report worse overall student performance (Gupta et al., 2020).

2.2.3.3 Long-term Effects on Adulthood

Bullying victimization affects victims well into adulthood, with long-term consequences that go beyond childhood and adolescence. According to studies, those who suffered from bullying at school are more likely to struggle in their careers, have low self-esteem, and have a higher risk of mental health issues (Gupta et al., 2020). Bullying victimization has also been related to reduced job satisfaction, unstable relationships at work, and lower earning potential since people who experienced bullying as individual frequently struggle with assertiveness, confidence, and leadership abilities (Qiu et al., 2024). Additionally, bullying might have an effect on people of all ages. Studies indicate that adults who were bullied as adolescents could struggle to develop positive parenting practices, which could cause their own children to struggle emotionally (Ahmed et al., 2022). To avoid long-term psychological and social harm, early interventions are highly needed.

In conclusion, the impact of bullying victimization extends far beyond the school environment, affecting mental health, academic success, social relationships, and long-term well-being of students. Bullying prevention is aligned with SDG 3: Good Health and Well - Being. It places a high priority on supporting mental health and ensuring good lifestyles for people of all ages. In order to decrease bullying and its negative effects, early intervention programs, counseling services, and supportive learning environments must be developed in collaboration with schools, and mental health specialists. By getting active, society may protect students who are at risk and create a safe and friendly future for all.

2.2.4 Machine Learning Algorithms for Bullying Prediction

Machine Learning (ML) provides an effective approach to analyze and predict bullying victims by combining multiple data sets and algorithms. Recent research has used machine learning approaches to identify key variables and create predictive models for bullying.

2.2.4.1 Logistic Regression

Logistic Regression (LR) is a widely recognized and foundational statistical method that is often used for binary classification tasks, including the prediction of bullying behaviors. It is particularly favored for its simplicity, interpretability, and computational efficiency, making it an ideal choice for initial modeling in bullying prediction studies. The ability of LR to produce clear and interpretable results has made it a valuable tool in various machine learning tasks, including those related to cyberbullying detection.

In the context of bullying prediction, LR has been effectively employed in several studies. One such study compared various machine learning techniques for detecting cyberbullying on Twitter and found that LR performed admirably in balancing training time with prediction accuracy. The study emphasized LR's effectiveness when combined with natural language processing (NLP) techniques, showcasing its ability to handle large datasets and accurately identify instances of cyberbullying (Muneer & Fati, 2020). Additionally, another study focused on cyberbullying detection in multilingual datasets, particularly in Hindi-English contexts, where LR was instrumental in classifying abusive and bullying messages. The integration of NLP with machine learning enhanced LR's ability to identify offensive content, making it a suitable model for identifying bullying behavior in diverse linguistic settings (Mehendale et al., 2022).

Despite its many strengths, Logistic Regression does have certain limitations. One notable disadvantage is its assumption of a linear relationship between features and the log-odds of the outcome, which may not always capture the complex and nonlinear patterns often seen in bullying behaviors. This linear assumption could limit the model's ability to detect intricate bullying patterns that do not follow a straightforward, linear correlation. Furthermore, LR is sensitive to outliers, which can significantly degrade its performance. The presence of extreme data points in the dataset may cause the model to produce inaccurate predictions, thus affecting its overall reliability. However, LR’s advantages in terms of interpretability, efficiency, and scalability make it a solid choice, particularly in cases where real-time prediction and clarity in feature importance are essential.

2.2.4.2 Random Forest

Random Forest (RF) is a powerful ensemble learning technique that constructs multiple decision trees during training and outputs the class that represents the mode of the classes for classification tasks. This method is particularly advantageous in handling complex datasets with high dimensionality and non-linearity, making it a valuable tool for predicting bullying behaviors. The robustness of RF comes from its ensemble approach, which aggregates the results of several decision trees, improving both accuracy and reliability. This capability is crucial for tasks such as cyberbullying detection, where data is often noisy and diverse.

In bullying prediction, Random Forest has been successfully applied in various studies. For instance, one study focused on cyberbullying detection across social media platforms and used RF to classify instances of bullying. The model's ability to combine the predictions of multiple decision trees significantly enhanced its accuracy and robustness in identifying bullying behaviors. The study demonstrated that RF was effective in classifying both overt and subtle forms of bullying in social media posts (Akter et al., 2025). Moreover, another study that compared different machine learning algorithms for cyberbullying detection found that RF achieved the highest recall rate, signifying its effectiveness in identifying true positive instances of bullying. This strong recall was attributed to the model's ensemble nature, which mitigated the risk of overfitting and improved its generalization across various datasets (Amgad Muneer & Suliman Mohamed Fati, 2020).

Despite its strengths, Random Forest does have some limitations. One of the main drawbacks is its interpretability. While RF provides feature importance scores, allowing users to understand which features are most influential in making predictions, the overall model remains more difficult to interpret compared to simpler models like Logistic Regression. This lack of transparency can be a challenge when trying to explain the decision-making process behind the model's predictions. Additionally, training multiple decision trees can be computationally expensive, especially when working with large datasets. This can lead to increased training times and resource consumption, particularly in real-time applications where speed is critical. Nevertheless, the advantages of Random Forest, such as its ability to handle non-linear relationships and its robustness, make it a valuable tool for bullying prediction in complex data environments.

2.2.4.3 SVM

Support Vector Machine (SVM) is a supervised learning model that excels in both classification and regression tasks, particularly when dealing with high-dimensional spaces. Its ability to model complex, non-linear decision boundaries makes it an ideal candidate for tasks such as bullying prediction, where the relationships between features can be intricate and non-linear. SVM constructs an optimal hyperplane that maximizes the margin between different classes, which helps in achieving high accuracy even in challenging scenarios. This characteristic is especially useful in bullying prediction, where the data may be highly variable and difficult to separate using linear boundaries alone.

In the context of bullying prediction, SVM has been applied successfully in various studies. For instance, a case study focused on detecting cyberbullying on Twitter utilized SVM with a linear kernel function, achieving an impressive accuracy rate of 92.7%. This high level of accuracy demonstrates SVM's proficiency in classifying sentiment-related issues, such as bullying, within social media content. The model's ability to effectively identify instances of cyberbullying, even in the noisy and diverse environment of Twitter, highlights its potential in real-world applications (None Al-Khowarizmi et al., 2023). Additionally, another study further emphasized SVM's effectiveness when combined with feature extraction techniques. By selecting relevant features that capture the nuances of bullying behavior, the model's performance was significantly enhanced, enabling it to more accurately detect subtle forms of online bullying.

While SVM offers numerous advantages, it does come with certain limitations. One of the primary drawbacks is its high memory consumption, particularly when dealing with large datasets. The computational resources required to train an SVM model can be substantial, especially when the number of features exceeds the number of samples, a common occurrence in text classification tasks. Additionally, the performance of SVM is highly dependent on the choice of kernel function and its associated parameters. Selecting the right kernel—whether linear, polynomial, or radial basis function (RBF)—can have a significant impact on the model's effectiveness, and this process often requires careful tuning to achieve optimal results. Despite these challenges, SVM's ability to handle high-dimensional and non-linear data makes it a powerful tool for bullying prediction, particularly in the context of cyberbullying detection on social media platforms.

2.2.4.4 XGBoost

XGBoost (Extreme Gradient Boosting) has emerged as a highly efficient and scalable machine learning algorithm, particularly valued for its robustness, ability to handle large datasets, and effectiveness in capturing complex patterns. These features make XGBoost particularly suitable for bullying prediction tasks, where data often consists of high-dimensional, noisy, and diverse information. Its popularity in predictive modeling tasks, including the detection and classification of bullying behaviors, stems from its ability to deliver high performance in a wide range of applications.

In the domain of bullying prediction, XGBoost has demonstrated significant potential. For instance, a study by Yao et al. (2022) employed XGBoost in combination with Fuzzy C-Means clustering to detect various types of cyberbullying. The model achieved an impressive accuracy of 91.75%, outperforming more traditional machine learning algorithms. This result underscores XGBoost's capability to handle complex, high-dimensional data typically encountered in bullying detection tasks (Ali Süzen & Duman, 2023). Another study analyzing cyberbullying detection methods across social media platforms also highlighted XGBoost's superior performance. The research found that XGBoost outperformed other models, such as Logistic Regression and Support Vector Machine, in terms of prediction accuracy, recall, and F1-score, demonstrating its ability to effectively identify bullying behaviors in diverse datasets (Mubeen et al., 2025). Moreover, a separate study focusing on sentiment analysis of Instagram comments utilized XGBoost combined with TF-IDF vectorization, achieving an accuracy of 75.20%, precision of 71%, recall of 87%, and an F1-score of 78%. This result further reflects XGBoost's proficiency in detecting subtle forms of cyberbullying in social media interactions (Muhamad Riza Kurniawanda & Fenina Adline Twince Tobing, 2022).

Despite its impressive advantages, XGBoost does have some limitations. One significant drawback is its complexity, which can make the model difficult to interpret, especially when transparency is required in applications. Additionally, XGBoost requires careful tuning of hyperparameters to achieve optimal performance, a process that can be time-consuming. Another limitation is its demand for substantial computational resources, particularly when training on large datasets, although its support for parallel processing helps alleviate some of this burden. Despite these limitations, XGBoost remains a powerful tool in bullying prediction due to its high performance, regularization capabilities,

parallelization, and the ability to provide insights into feature importance, all of which contribute to its continued success in this field.

2.3 Similar Works

Authors	Description	Dataset Used	Analysis Techniques	Evaluation Techniques	Outcomes	Limitations
Qiu et al. (2024)	Develops a machine learning model to predict students at risk of bullying victimization.	Data from 10,000 primary and secondary school students in China.	Decision trees, XGBoost, and neural networks.	Precision, Recall, and F1-score.	Machine learning models can predict bullying victimization with high accuracy, allowing for early intervention.	Data imbalance may affect model performance, and ethical concerns regarding data privacy.
Ahmed et al. (2022)	Explores the link between bullying victimization and psychiatric comorbidities.	Cross-sectional survey of 1,500 adolescents in Egypt.	Descriptive analysis, logistic regression.	Confusion Matrix and AUC-ROC analysis.	Victims of bullying are more likely to suffer from anxiety, depression, and PTSD.	Cultural differences may affect generalizability, and self-reported measures could introduce bias.
Gupta et al. (2020)	Assesses school factors influencing bullying victimization rates.	Data from 300 schools in India, including surveys and teacher interviews.	Factor analysis, multiple regression analysis.	Mean Absolute Error (MAE) and R-squared metrics.	Positive school environments and strong teacher support reduce bullying victimization rates.	Limited to Indian schools, may not consider other environmental influences.
Morgan et al. (2023)	Identifies predictive factors for bullying victimization among U.S. elementary students.	Nationally representative dataset from U.S. schools.	Logistic regression, structural equation modeling (SEM).	Area Under the Curve (AUC), Model Fit Index.	Risk factors include gender, disability status, and school climate. Early interventions can help reduce bullying rates.	Focuses only on elementary students, may not generalize to older age groups.
Eid et al. (2023)	Uses graph machine learning to detect bullying patterns from social networks.	Friends dataset, a social network dataset.	Neo4j's Unsupervised Graph Learning, node embedding, clustering algorithms.	Precision, Recall, and F1-score.	Graph-based ML models can effectively detect bullying behavior in social networks.	Limited to social network interactions, may not detect offline bullying.
Galán-Arroyo et al. (2023)	Explores the relationship between bullying, self-concept, and mental health in adolescents.	Cross-sectional survey of school adolescents in Spain.	Descriptive analysis, factor analysis, correlation study.	Structural Equation Modeling (SEM), Confirmatory Factor Analysis (CFA).	Low self-concept and negative mental health outcomes increase bullying victimization risk.	Self-reported data may introduce bias, and findings may not apply to all cultures.

Table 3: Similar system

2.4 Technical Research

This section outlines the hardware and software involved in developing the project, including the operating system, libraries, programming language, and development environment. To ensure smooth data processing, effective machine learning model execution, and seamless predictive system deployment, every component was carefully chosen.

2.4.1 Hardware and Software used for this project

Selecting the right hardware and software plays a critical role in the computational efficiency of machine learning models. This is due to the Machine Learning require high processing power, memory and storage space to handle large datasets and do complex calculations while building models. (GeeksforGeeks, 2024)

Devices Specification	
Processor	11th Gen Intel(R) Core(TM) i5-11320H @ 3.20GHz 2.50 GHz
RAM	16GB
Operating System	64-bit
GPU	NVIDIA GeForce MX450
Window Specification	
Edition	Windows 11 Home Single Language
Version	23H2

Table 4: Hardware Specification

During this project, a laptop named Dellxin-2021 is used with an Intel Core i5-11320H processor, 16GB RAM, and 64-bit operating system which running on Window 11 Home Language. These specs offer enough processing capacity to manage large datasets, develop machine learning models, and run simulations efficiently. Besides, a GPU-enabled device, such as NVIDIA GeForce MX450 series graphics card, may significantly improve computational performance for complex deep learning tasks or processing high-dimensional data, even though this hardware is appropriate for basic model training.

2.4.2 Programming Language chosen – Python

Python is an open-source alternative for traditional programming techniques because it is a strong programming language with an easy-to-understand syntax and instructions that resemble English. It is widely used in machine learning due to its ease of use, flexibility, and extensive library support. Additionally, Python can run on multiple operating systems, including Windows, macOS, and Linux, providing a high level of power and customization for machine learning applications (GeeksforGeeks, 2021). Python supports various functionalities, including statistical analysis, visualization, and model deployment, making it an best choice for machine learning project.

Machine learning and artificial intelligence are fast growing technologies that enable automated fraud detection, virtual assistants, spam filters, search engines, and recommendation systems. AI research is becoming more important for resolving complicated tasks that would be challenging to handle manually as the need for intelligent automation increases. Python is recognized as the greatest language for artificial intelligence because of its simplicity, consistency, and efficiency. Furthermore, its active developer community fosters collaboration, making it easier to refine and enhance code (Team, 2025).

Python's strong library support contributes significantly to its supremacy in machine learning. Libraries such as NumPy, Pandas, Scikit-learn, TensorFlow, and Keras provide essential tools for analyzing, manipulating, and creating machine learning models. These libraries handle the majority of the complexity needed in machine learning, allowing developers to focus on increasing model accuracy rather than dealing with low-level calculations (GeeksforGeeks, 2021).

Python is also very user-friendly, making it a great option for individuals new to machine learning. Its simple syntax and rich tools, such as Scikit-learn, make model development easier. To get started, it is essential to understand data types, statistics, data sourcing, and data cleaning. Choosing the right libraries and implementing algorithms effectively are crucial steps in building successful models. Python frameworks perform many complex tasks, allowing developers to focus on improving model performance (Anthony, 2023).

Python remains the most popular machine learning language due to its simplicity, powerful frameworks, and significant community support. It provides a user-friendly and efficient environment for both new and experienced developers to create accurate and effective models.

2.4.3 IDE chosen – Visual Studio Code with Jupyter Notebook

For machine learning work, Visual Studio Code (VS Code) with Jupyter Notebook integration is a good option since it combines the flexibility of Jupyter Notebooks with the powerful features of a professional code editor. Users may create, open, and save notebooks right within Visual Studio Code thanks to its built-in support for Jupyter Notebooks. Using Markdown to provide smooth transitions between code execution and documentation, this connection streamlines the workflow (Microsoft, 2021). For environment setup, users can easily set up a Python environment using Anaconda or other Python distributions. For instance, creating an Anaconda environment with essential libraries like Pandas, Scikit-learn, and TensorFlow can be done with a simple command:

2.4.4 Libraries / Tools chosen

For this project, various libraries and tools were selected to facilitate data preprocessing, exploration data analysis, and predictive modeling. Pandas and NumPy were used for data manipulation, preprocessing, and handling missing values, while Matplotlib and Seaborn provided visualization capabilities for exploratory data analysis. Scikit-learn was employed to implement machine learning algorithms such as Logistic Regression, Decision Trees, and XGBoost, and TensorFlow and Keras were utilized for deep learning model development. To enhance model interpretability, SHAP and LIME were integrated to understand feature importance. Additionally, NLTK and TextBlob applied for natural language processing tasks related to analyzing bullying-related textual data. Tabulate was used to present structured tabular data, and StandardScaler and LabelEncoder were implemented for data preprocessing to standardize numerical features and encode categorical variables. These libraries provide robust functionalities to streamline the machine learning workflow and ensure optimal model performance.

2.5 Summary

This chapter provided a comprehensive review of relevant literature and technical aspects of the project, highlighting both theoretical foundations and practical implementations. It explored the key factors influencing bullying victimization, categorized into individual, family, peer, and school-related elements. These aspects were examined through various existing studies on bullying prediction and intervention strategies, which helped identify research gaps that this project aims to address.

The technical framework of the project was also discussed in detail, particularly focusing on the selection of Python as the programming language due to its vast ecosystem of libraries and efficiency in handling machine learning tasks. Visual Studio Code with Jupyter Notebook was chosen as the integrated development environment, offering flexibility in writing, debugging, and running data science workflows. A wide range of libraries and tools were employed for data preprocessing, modeling, and visualization, including Pandas, NumPy, Scikit-learn, TensorFlow, Keras, SHAP, LIME, Matplotlib, and Seaborn. These tools facilitated efficient handling of the dataset, from cleaning and feature extraction to model training and evaluation.

By synthesizing theoretical insights with technical implementations, this chapter laid the groundwork for developing a robust predictive model for bullying victimization. The literature review guided the identification of key variables, while the technological choices ensured an efficient and scalable analytical approach. This holistic integration of research and computational methodologies is essential for achieving the project's objectives effectively.

CHAPTER 3: METHODOLOGY

3.1 Introduction

The CRISP-DM (Cross-Industry Standard Process for Data Mining) methodology is a well-established framework used to guide data mining projects. It is particularly effective for machine learning tasks as it provides a structured, iterative approach to problem-solving and model development. This methodology includes six phases: Business Understanding, Data Understanding, Data Preparation, Modeling, Evaluation, and Deployment, ensuring that all aspects of a data-driven project are thoroughly addressed. In the context of predicting bullying risk among adolescent students, CRISP-DM proves to be an ideal approach due to its flexibility and comprehensive structure.

By focusing on mental health and aligning with SDG Goal 3 (Good Health and Well-being), this project aims to provide actionable insights for educators and policymakers to detect bullying behaviors early and intervene effectively. The methodology's ability to facilitate continuous improvement and adaptability makes it well-suited for analyzing the Global School-Based Student Health Survey (GSHS) dataset, which serves as the foundation for predicting bullying risks in schools.

3.2 Methodology

CRISP-DM, also known as Cross-Industry Standard Process for Data Mining in full, is the most common methodology for organizing machine learning projects. In CRISP-DM methodology, there are 6 phases included which are the Business Understanding phase, Data Understanding phase, Data Preparation phase, Modeling phase, Evaluation phase and lastly is the Deployment phase. Therefore, it covers all aspects of a data-driven project due to its flexibility, comprehensiveness and widespread adoption in data science field.

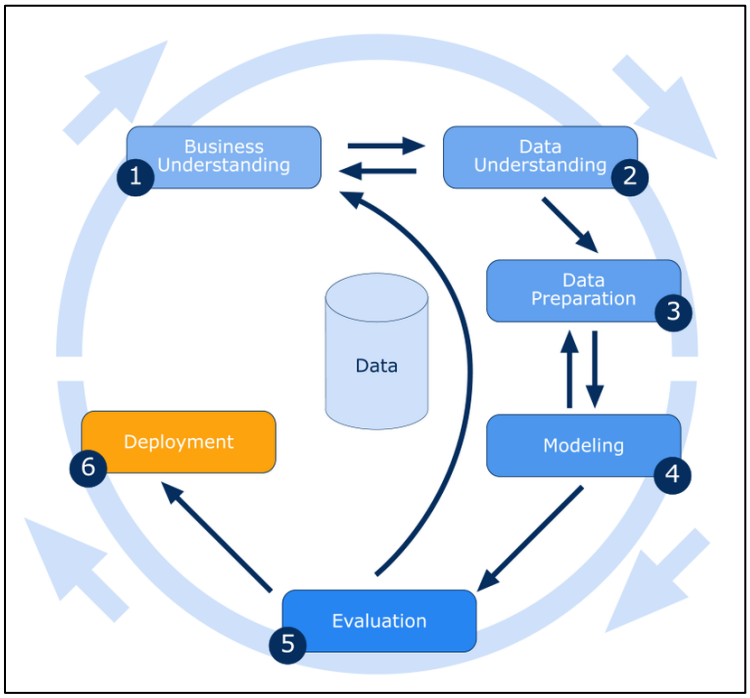


Figure 4: Framework of CRISP-DM

CRISP-DM methodology is chosen to use in this project because it provides a structured, iterative approach to managing data mining and machine learning projects. It ensures that data-driven insights can successfully apply to real-time issues. For predicting risk of bullying among adolescent students, CRISP-DM offers a systematic approach to analyzing data, identifying key risk indicators, and developing a predictive model that can support educators and policymakers, especially for the complexity of bullying-related factors and the need for actionable insights. By enhancing early detection and intervention techniques that support students' mental health, this project is aligned with SDG Goal 3, Good Health and Well-being.

Compared to KDD and SEMMA, CRISP-DM is more suitable for this project. This is because KDD is more focused on knowledge extraction rather than practical application. Same to SEMMA, it is useful for model exploration but lack of dedicated deployment phase,

making it less effective for integrating findings into actionable policies. (Hotz, 2018) However, the flexibility of CRISP-DM allows for continuous improvements in model accuracy and relevance, making it ideal for analyzing the Global School-Based Student Health Survey (GSHS) dataset to predict students at risk of bullying.

3.2.1 Phase 1 – Business Understanding

Business Understanding is the first phase of CRISP-DM methodology. It is important to understand the requirements and objectives in business understanding. There are 4 key tasks which is identifying the business goal and scenarios, defining data mining purpose, and developing comprehensive project plans. It involves tasks such as establishing the project's goals, scope, and context, as well as the activities' schedule. Therefore, they provide a strong framework for how the goals of the business can be transformed into activities for data mining analysis. (AdamBrian, 2020)

In this project, it outlines the goals of the study by identifying bullying victimization as a major problem impacting the mental health of adolescents. This stage involves researching bullying, understanding its psychological effects, and aligning prediction objectives with SDG 3, which places a high priority on mental health.

3.2.2 Phase 2 – Data Understanding

After Business Understanding, the second phase comes to Data Understanding. This phase focuses on understanding the collected data. There are 4 primary tasks to focus on analyzing and evaluating the dataset. Firstly, data collecting involves documenting any issues that are experienced. Then, describing the data entails assessing surface features and evaluating formats. Next is data exploration, which documents findings and theories. Lastly, data quality evaluation checks for conflicts, missing characteristics, blank fields, and spelling mistakes to guarantee the integrity of the data. (AdamBrian, 2020)

In this project, Global School-Based Student Health Survey (GSHS) dataset is a school-based survey which is used to focus on exploring in this phase. It uses exploratory data analysis (EDA) to check missing values, anomalies, and correlations, as well as identifying relevant variables such as peer relationships and emotional health indicators.

3.2.3 Phase 3 – Data Preparation

After understanding the data, next is data preparation. In this phase, it is one of the most time-consuming phases among CRISP-DM. Data Preparation are focusing on cleaning dataset from a raw dataset and prepare to build model in the next phase. Data cleaning includes removing duplications, handling missing values, replacing values, and inputting invalid or erroneous values.

In this project, different pre-processing techniques may be used to improve the quality of the data during the Data Preparation stage, such as numerical characteristics are normalized, categorical variables are encoded, and missing values are handled in multiple ways. Besides, SMOTE can be used in this dataset if the dataset has imbalance class. Then, the dataset could split into 7:3 or 8:2 for training and testing the dataset to optimize the mode performance. After done all pre-processing, the data is ready for modeling in the next phase.

3.2.4 Phase 4 – Modeling

When comes to the fourth phase, data modeling, this is where the major task is to construct models and study their effectiveness in solving the project requirements. In this phase, some tasks need to be done such as selecting models, testing models, creating models and accessing the models.

In this project, this phase involves training various machine learning algorithms to predict bullying risk in school. Different models have different functionalities to test interpretability, explore complex pattern recognition and build model comparison. To maximize accuracy, the models go through cross-validation and hyperparameter adjustment. This stage is important because it shows how well the predictive algorithm detects students who are at risk. Since better prediction accuracy makes proactive anti-bullying interventions possible and reduces the long-term psychological impacts of victimization, SDG 3 is directly supported here.

3.2.5 Phase 5 – Evaluation

In CRISP-DM methodology, this phase is to make sure that the models chosen are matching the goals. It assesses the models using performance metrics such as Accuracy, Precision, Recall, F1-Score, and AUC-ROC Curve. Since this project are sensitively in bullying-related predictions, high recall is prioritized to minimize the false negatives. This evaluation process helps refining the model before deployment, ensuring its reliability in real-world applications.

3.2.6 Phase 6 – Deployment

The last phase of CRISP-DM is deployment phase. In this phase for this project, the best-performing model will be integrated into an API or dashboard for educators and policymakers. With the use of this program, which offers real-time risk assessments, schools can effectively support students who are at risk. Key findings and recommendations are compiled in a report, which ensures that the insights result in data-driven policy changes and provide a safer school environment. This stage strengthens the objective of SDG 3, which is to ensure healthy lives and promote the well-being for everyone, by decreasing the gap between data science and social impact.

3.3 Summary

In summary, the CRISP-DM methodology offers a systematic and structured approach to tackling complex problems such as bullying prediction. By following its phases, this project ensures that the data is well-understood, properly prepared, and modeled effectively to identify at-risk students. The evaluation phase allows for continuous refinement of the predictive models, while the final deployment phase enables real-world applications of the insights gained. The integration of machine learning with the CRISP-DM framework enhances the potential for early detection of bullying, which can lead to timely interventions that positively impact student mental health. Ultimately, this project aligns with SDG 3 by promoting well-being and fostering a safer educational environment for all students.

CHAPTER 4: DESIGN AND IMPLEMENTATION

4.1 Introduction

This project focuses on the design and implementation process used to predict bullying risk among adolescent students. The chapter begins with data collection, where a dataset titled "Bullying_2018.csv" from Kaggle.com is used. This open-source dataset contains information about bullying experiences and the characteristics of students. Following data collection, an initial understanding of the dataset is conducted, including an exploration of its structure, data types, and key attributes. The chapter then dives into the data pre-processing phase, where missing values, inconsistent data, and outliers are addressed to prepare the data for analysis. After pre-processing, the data is transformed into a clean and structured format, ready for modeling.

The modeling phase begins with the implementation of several machine learning algorithms, including Logistic Regression, Random Forest, Support Vector Machine (SVM), and XGBoost. Each model is trained and evaluated to determine its effectiveness in predicting bullying risk. The process includes hyperparameter tuning to optimize each model's performance. The results of these models will be discussed and compared to assess which one provides the most accurate and reliable predictions.

4.2 Data Collection

This data is collected from Kaggle.com and it is an open-source dataset. This project only uses one dataset which is in csv file and shown in table below.

Dataset Name	Source (From Kaggle.com)
Bullying_2018.csv	Bullying in schools

Table 5: Dataset from Kaggle

4.3 Initial Data Understanding

This project uses an open dataset from Kaggle which is Bullying_2018. This dataset contains the records of whether the students were bullied and the characteristics of the student. Before proceeding to data pre-processing, it is important to understand the dataset first.

4.3.1. Data Dictionary

No	Column Name	Description	Data Type	Value
1.	record	Number of records	Numerical - Integer	1 - 57095
2.	Bullied on school property in past 12 months	Does student got bullied at school	Categorical - Boolean	Yes, No, (Blanks)
3.	Bullied not on school property in past 12 months	Does student got bullied outside school	Categorical - Boolean	Yes, No, (Blanks)
4.	Cyber bullied in past 12 months	Does student got cyberbullied	Categorical - Boolean	Yes, No, (Blanks)
5.	Custom_Age	Age of student	Categorical - String	- 11 years old or younger - 12 years old - 13 years old - 14 years old - 15 years old - 16 years old - 17 years old - 18 years old or older - (Blanks)
6.	Sex	Sex of student	Categorical - String	Female, Male, (Blanks)
7.	Physically attacked	Times of student who got physical attacked	Categorical - String	- 0 times - 1 time - 2-3 times - 4-5 times - 6-7 times - 8-9 times - 10-11 times - 12 or more times - (Blanks)
8.	Physical fighting	Times of student who got physical fights	Categorical - String	- 0 times - 1 time - 2-3 times - 4-5 times - 6-7 times - 8-9 times - 10-11 times - 12 or more times - (Blanks)

9.	Felt lonely	How often felt lonely	Categorical - String	- Never - Rarely - Sometimes - Most of the time - Always - (Blanks)
10	Close friends	Number of close friends	Categorical - String	0, 1, 2, 3 or more, (Blanks)
11.	Miss school no permission	Missed school without permission	Categorical - String	- 0 days - 1-2 days - 3-5 days - 6-9 days - 10 or more days - (Blanks)
12.	Other students kind and helpful	How often others help	Categorical - String	- Never - Rarely - Sometimes - Most of the time - Always - (Blanks)
13.	Parents understand problems	How often parents understand	Categorical - String	- Never - Rarely - Sometimes - Most of the time - Always - (Blanks)
14.	Most of the time or always felt lonely	Frequently lonely	Categorical - Boolean	Yes, No, (Blanks)
15.	Missed classes or school without permission	Missed classes / school without permission	Categorical - Boolean	Yes, No, (Blanks)
16.	Were underweight	Underweight	Categorical - Boolean	Yes, No, (Blanks)
17.	Were overweight	Overweight	Categorical - Boolean	Yes, No, (Blanks)
18.	Were obese	Obese	Categorical - Boolean	Yes, No, (Blanks)

4.3.2. Import Libraries

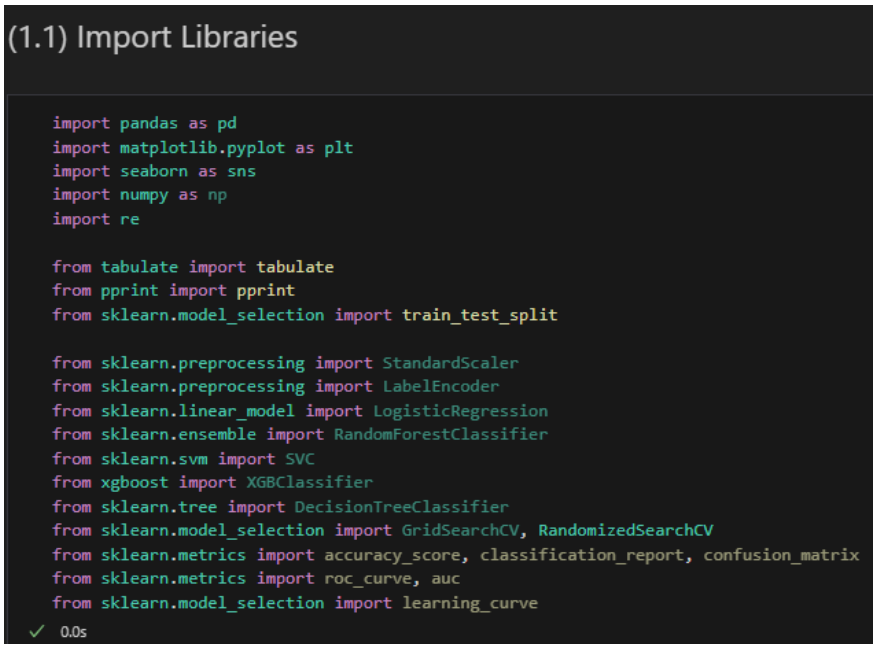


Figure 5: Libraries Import

Figure above shows all the libraries import that uses in this project from importing dataset to model building and evaluation.

4.3.3. Load Dataset and Check Basic Information

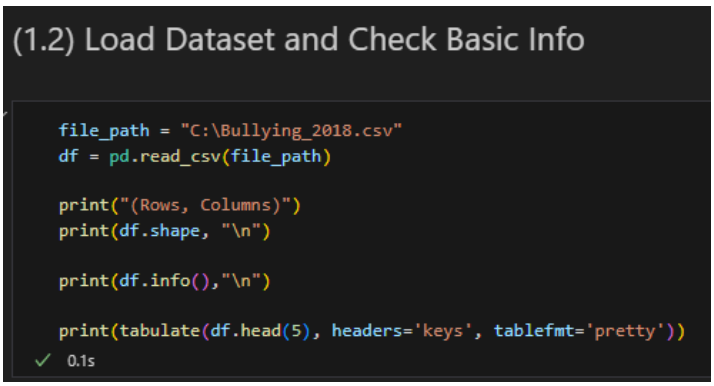


Figure 6: Source code of Import dataset and check basic information

The figure above shows the source code of using Pandas Python Library to import the bullying dataset from device. After importing data, check for the basic information of the dataset such as the number of rows and columns for the dataset, and the datatype of each of the attributes in the dataset.

```
(Rows, Columns)
(56981, 18)
```

Figure 7: Output of basic information (1)

This figure shows the output of checking the rows and columns of the bullying dataset. There are 56981 rows of data and 18 columns of attributes. The 56981 rows of data are the responses of students who participated in this Global School-Based Student Health Survey (GSHS) and the 18 columns of attributes are the questions that are included in this survey.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 56981 entries, 0 to 56980
Data columns (total 18 columns):
 #   Column                                        Non-Null Count  Dtype
---  ------                                        --------------  -----
 0   record                                        56981 non-null  int64
 1   Bullied_on_school_property_in_past_12_months  56981 non-null  object
 2   Bullied_not_on_school_property_in_past_12_months  56981 non-null  object
 3   Cyber_bullied_in_past_12_months               56981 non-null  object
 4   Custom_Age                                    56981 non-null  object
 5   Sex                                           56981 non-null  object
 6   Physically_attacked                           56981 non-null  object
 7   Physical_fighting                             56981 non-null  object
 8   Felt_lonely                                   56981 non-null  object
 9   Close_friends                                 56981 non-null  object
 10  Miss_school_no_permission                     56981 non-null  object
 11  Other_students_kind_and_helpful               56981 non-null  object
 12  Parents_understand_problems                   56981 non-null  object
 13  Most_of_the_time_or_always_felt_lonely        56981 non-null  object
 14  Missed_classes_or_school_without_permission   56981 non-null  object
 15  Were_underweight                              56981 non-null  object
 16  Were_overweight                               56981 non-null  object
 17  Were_obese                                    56981 non-null  object
dtypes: int64(1), object(17)
memory usage: 7.8+ MB
None
```

Figure 8: Output of basic information (2)

The figure above shows the output of checking the attributes' name and their datatype of all the 18 columns. As shown in the output, the attributes are about the characteristics of the bullying victimization. There are most of the columns are categorical data which represented as "object" datatype in Pandas library.

4.3.4. Initial Features Analysis

(1.3) Initial Features Analysis

```
numeric_cols = df.select_dtypes(include=["int"]).columns
categorical_cols = df.select_dtypes(include=["object"]).columns

for col in numeric_cols:
    fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(10, 4))

    # Box Plot
    sns.boxplot(x=df[col], ax=axes[0])
    axes[0].set_title(f"Box Plot of {col}")

    # Histogram
    sns.histplot(df[col], bins=30, kde=True, ax=axes[1], color="purple")
    axes[1].set_title(f"Histogram of {col}")

    plt.tight_layout()
    plt.show()

for col in categorical_cols:
    unique_vals = df[col].dropna().unique()
    try:
        sorted_order = sorted(unique_vals, key=lambda x: float(x))
    except:
        # Fallback: sort alphabetically
        sorted_order = sorted(unique_vals)

    plt.figure(figsize=(14, 5))
    ax = sns.countplot(
        x=df[col],
        hue=df[col],
        palette="muted",
        order=sorted_order,
        hue_order=sorted_order,
    )

    for p in ax.patches:
        ax.annotate(
            f'{int(p.get_height())}',
            (p.get_x() + p.get_width() / 2., p.get_height()),
            ha='center', va='bottom', fontsize=10, color='black'
        )
    plt.title(f"Distribution of {col}")
    plt.xticks(rotation=0)
    plt.show()
```

✓ 3.7s

Figure 9: Source code of Features Analysis for each of the attributes

The above figure shows the source code for exploring each of the attributes by using Boxplot and Bar Chart to visualize. Boxplot is used to visualize the numerical column while the Bar Chart is used to visualize categorical column which is in “object” datatype in Pandas library. In this dataset, there are only 1 numerical column, and the others are categorical columns.

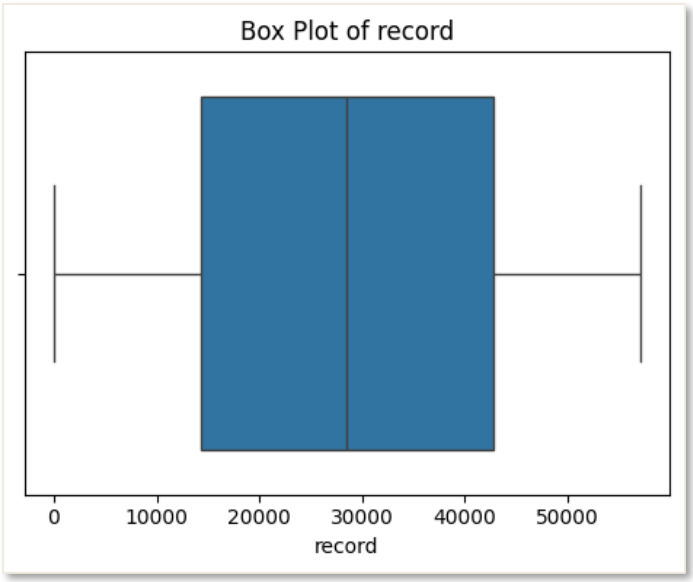


Figure 10: Record Visualization

This figure is the output of features analysis for record. The above data information shows that there is only 1 column in integer datatype which means that only this column is in numerical data. Therefore, Boxplot has used to visualize this column.

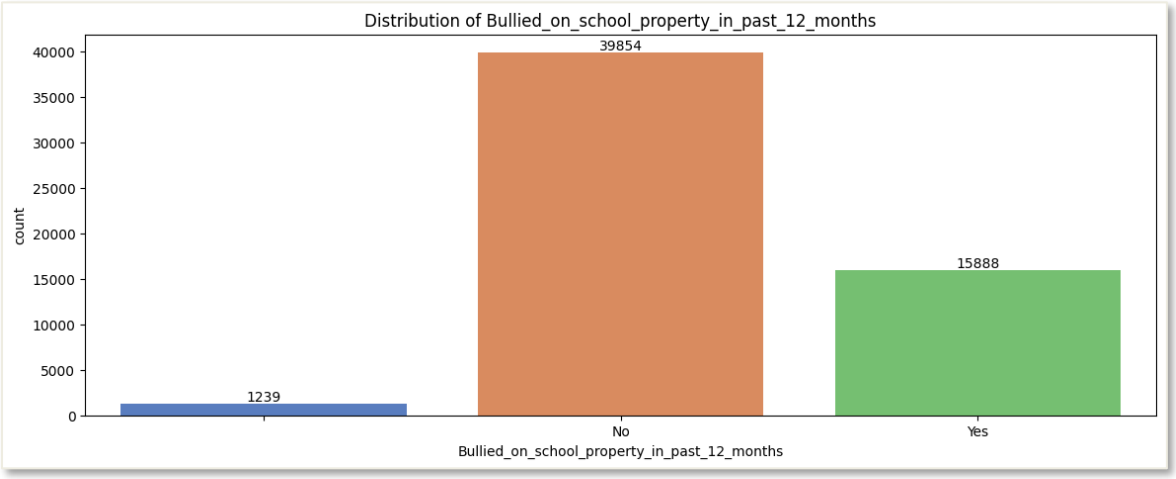


Figure 11: Bar Chart of Bullied_on_school_property_in_past_12month

The figure above shows the number of students who got bullied in school in the past 12 months. The bar chart shows that there are 115888 students in this dataset who have been bullied in school in the past 12 months, while there are 39854 students who have not been bullied. In addition, there are 1239 students leaving blank for this question. Therefore, it is considered as null value in the dataset.

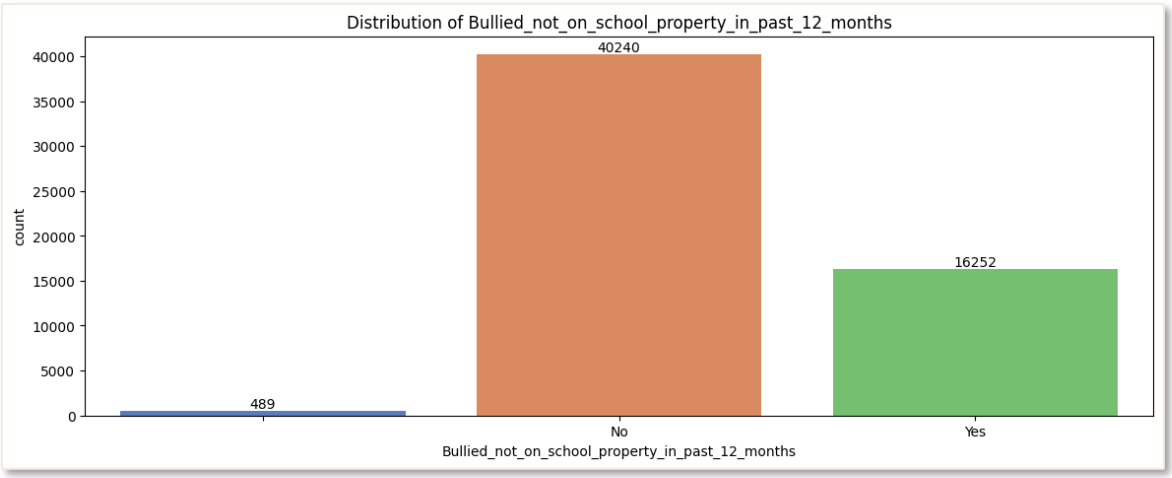


Figure 12: Bar Chart of Bullied_not_on_school_property_in_past_12month

The figure above shows the number of students who got bullied but not in school in the past 12 months. The bar chart shows that there are 16252 students who have been bullied outside the school, while 40240 students have not been bullied. In addition, there are 489 null values in this column, which are not sure if these numbers of students have been bullied outside the school.

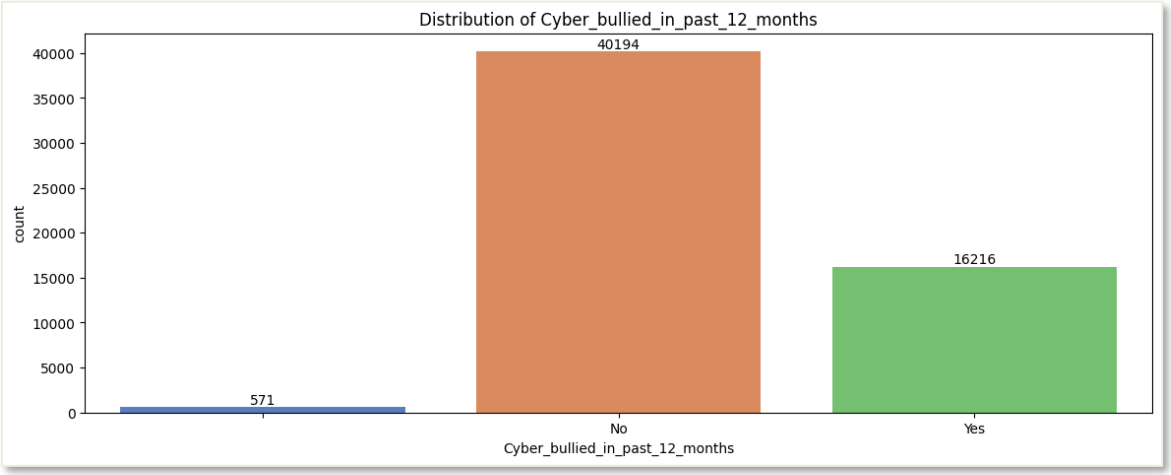


Figure 13: Bar Chart of Cyber_bullied_in_past_12month

The figure above shows the number of students who got cyberbullied in the past 12 months. The bar chart shows that there are 16216 students who have been faced with cyberbullying, 40194 students who do not face cyberbullying, and 571 of students have left this question blank.

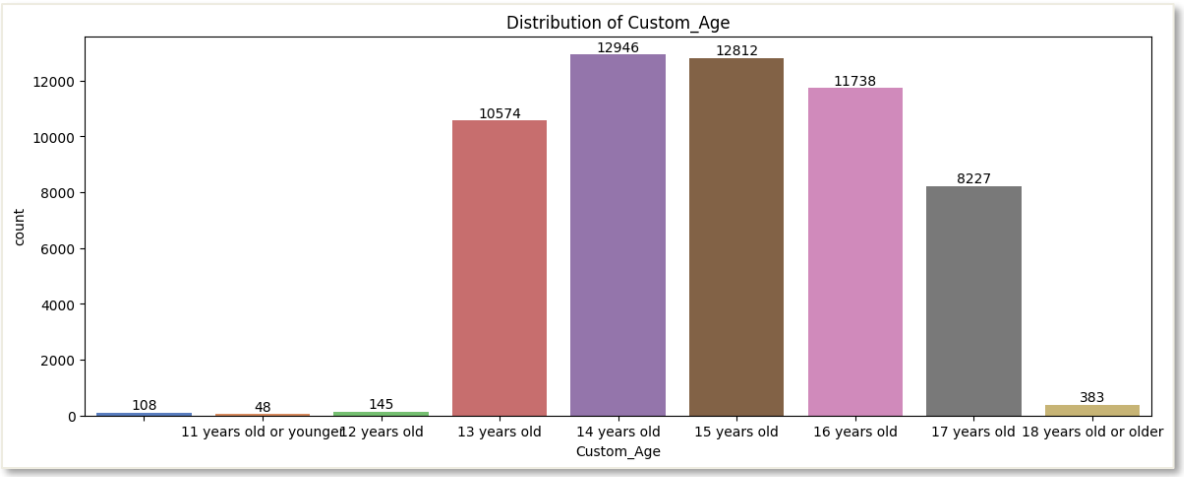


Figure 14: Bar Chart for Custom_Age

Based on the above figure, it shows the students who enrolled in this Global School-Based Student Health Survey (GSHS) are in different ages. There are 48 students who are below 11 years old, 145 students are 12 years old, 10574 students are 13 years old, 12946 students are 14 years old, 12812 of 15 years old students, 11738 of 16 years old students, 8227 of 17 years old students, and 383 of 18 years old or older students contributed in the survey. As shown by the above bar chart, most of the participance are in the age range of 13 – 17 years old. However, there are 108 students who prefer not to say their age during this survey.

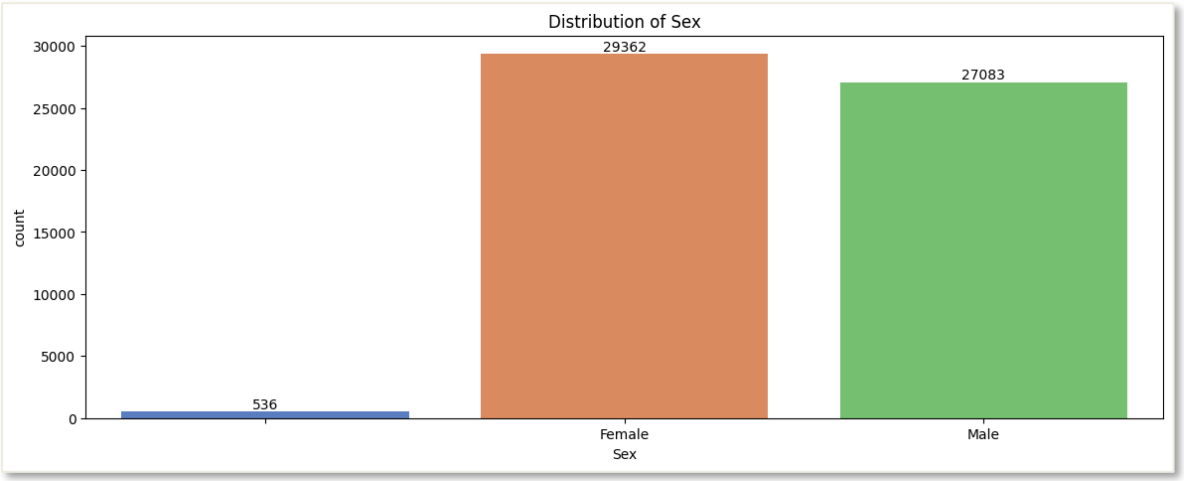


Figure 15: Bar Chart of Sex

The figure above shows the sex of the students in this dataset. Female students have more participance than male students, and there are 536 students prefer to say their sex, lead to null value in dataset.

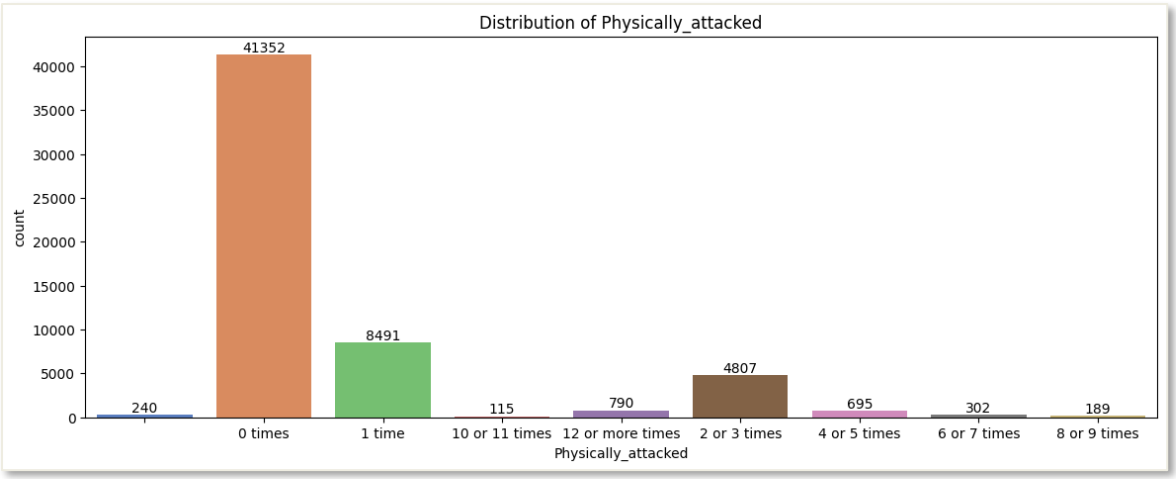


Figure 16: Bar Chart of Physically_attacked

The above figure shows how many times that students in this dataset have been faced physically attacked. Luckily, most of the students dose not face physical attack but there are 790 students experienced 12 or more times. Some of the students (8491 students) have faced physically attacked 1 time and 4807 students have faced 2 or 3 times.

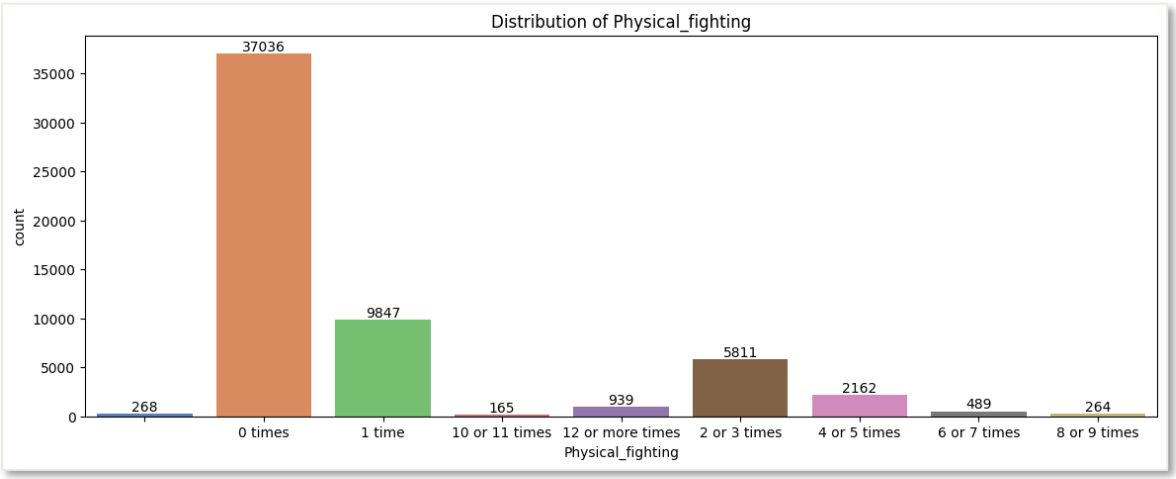


Figure 17: Bar Chart of Physical_fighting

The above figure shows how many times that students in this dataset have been involved in physical fighting. Most of the students are not involved in fighting. However, there are 9847 students had involved 1 time, 55811 students had involved 2-3 times, and 2162 students had involved 4-5 times.

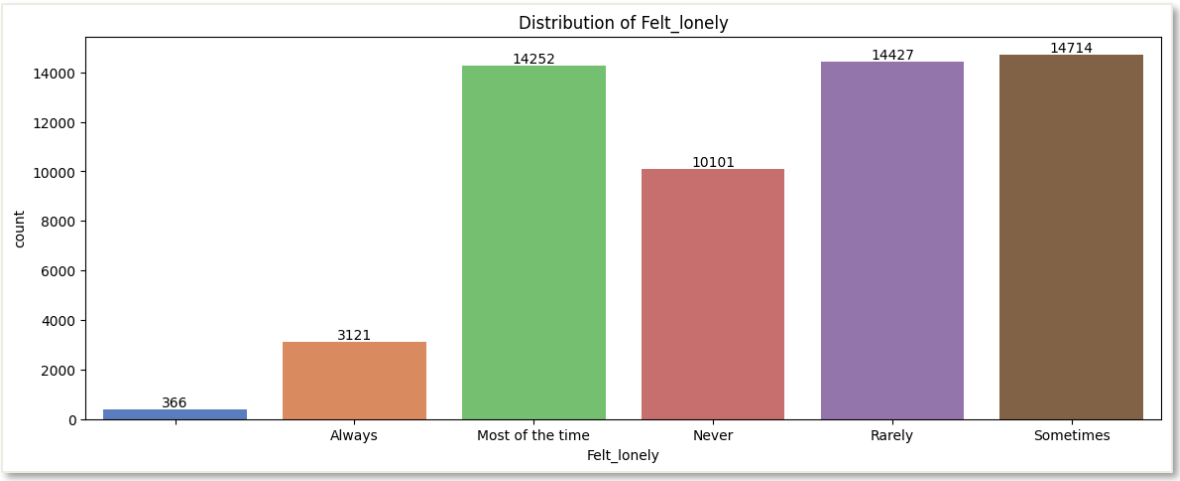


Figure 18: Bar Chart of Felt_lonely

The above figure shows how frequently that the students felt lonely. From the bar chart, it shows that there are 10101 students who have never felt lonely, 14427 students rarely felt lonely, and 14714 students only sometimes felt lonely. However, there are 14252 students felt lonely most of the time and even 3121 of students felt lonely most of the time.

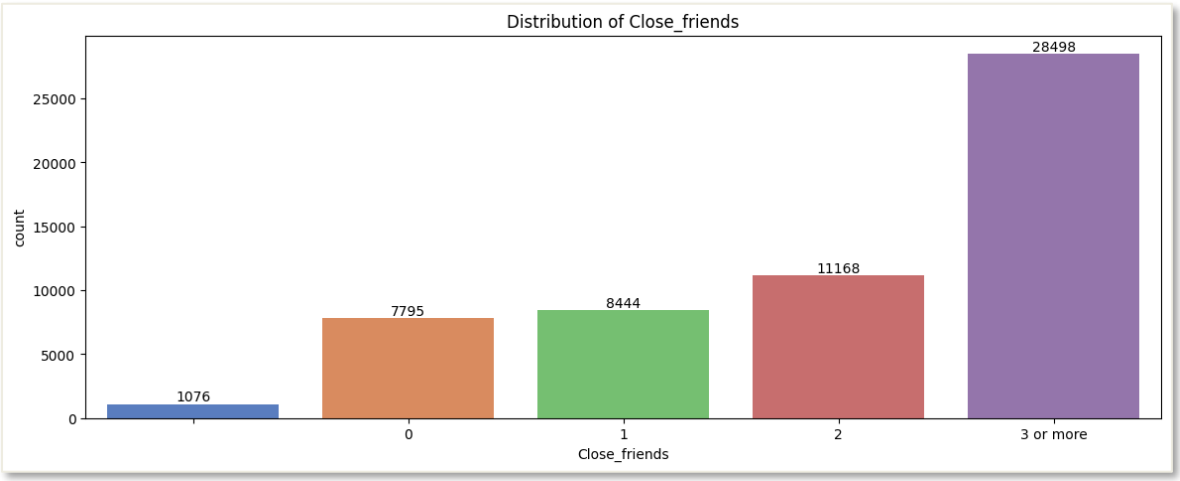


Figure 19: Bar Chart of Close_friends

The figure above shows how many close friends the students have. Most of the students have 3 or more close friends. However, there are 7795 students have response that they have no close friend at all.

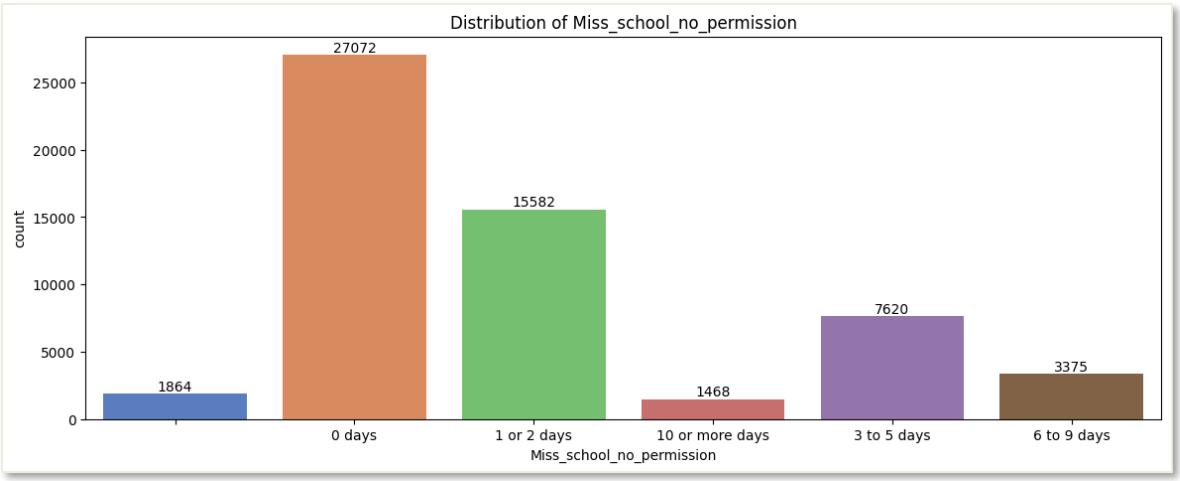


Figure 20: Bar Chart of Miss_school_no_permission

The figure above shows how many days that student missed school with no permission. As we can see, most of the students do not miss school but there are 15582 students missed school 1-2 days, 7620 students missed school 3-5 days, 3375 students missed school 6-9 days, 1468 students missed school around 10 or more days, and 1864 students does not answer this question.

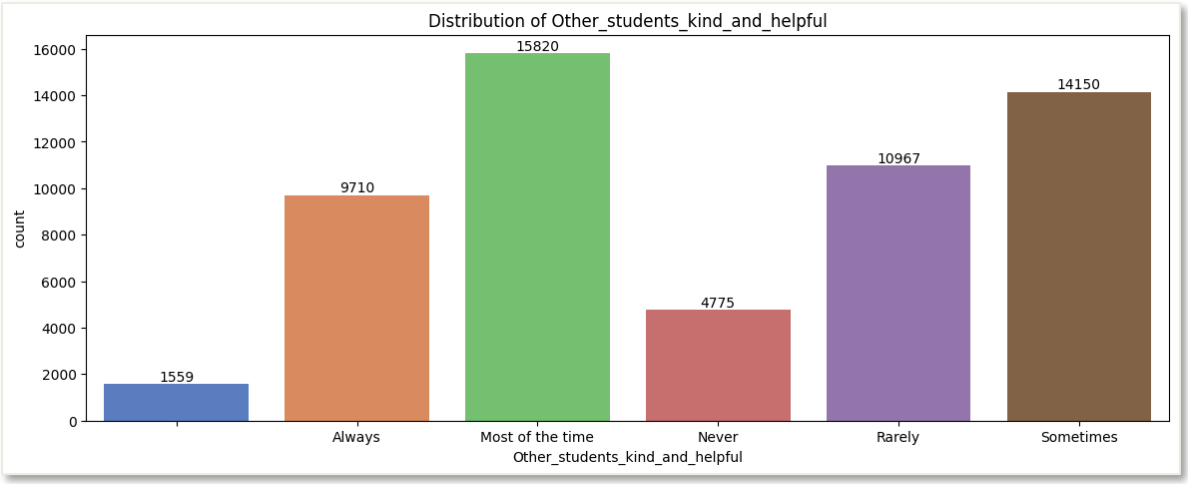


Figure 21: Bar Chart of Other_students_kind_and_helpful

This figure shows how frequently students get help from other students. The bar chart above shows that 15820 students got help from other students most of the time. Sadly, there are 4775 students who have never got help from others before.

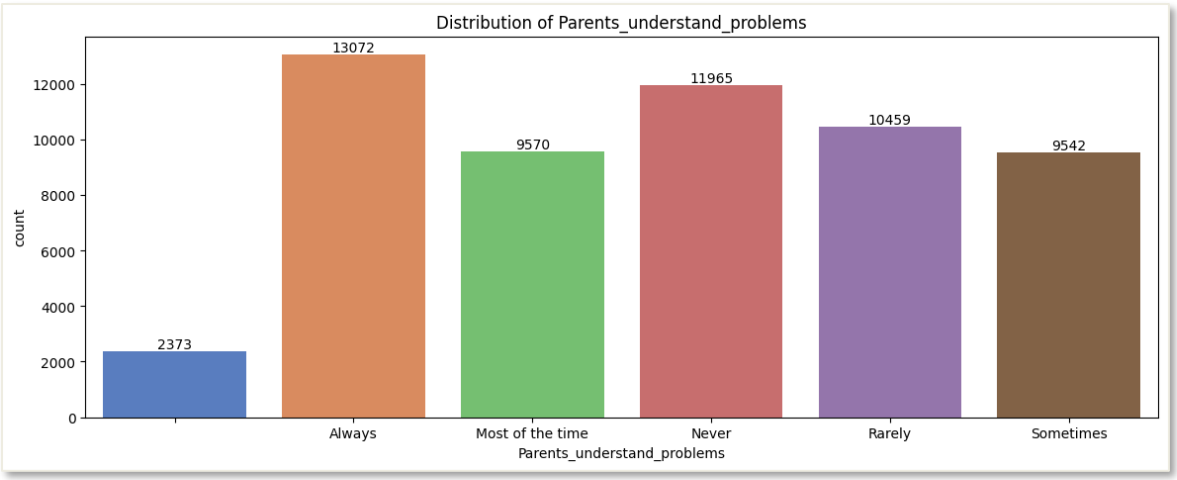


Figure 22: Bar Chart of Parents_understand_problems

The above figure shows the bar chart of how frequently the students' parents understand their problem that they faced. Although most of them (13072 students) are understood by their parents, there are still a lot of students (11965 students) who are never understood by their parents. This may be due to the students' parents having traditional mindset and less communication with their kids.

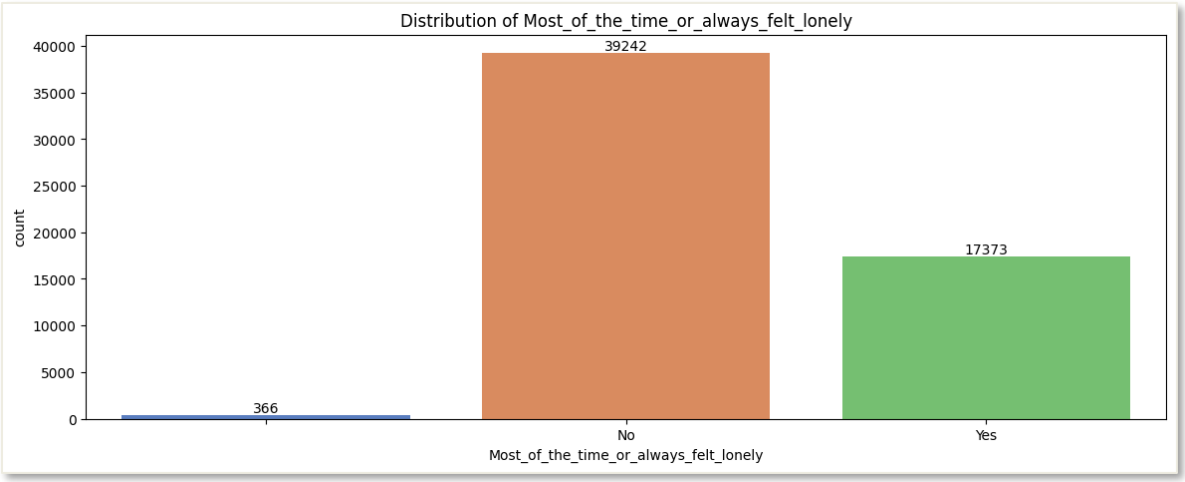


Figure 23: Bar Chart of Most_of_the_time_or_always_felt_lonely

The above figure shows whether the students always felt lonely. The bar chart shows that there are 17373 students who have marked Yes, they always or most of the time felt lonely. However, there are 39242 students marked No which does not always felt lonely but maybe only sometimes felt lonely.

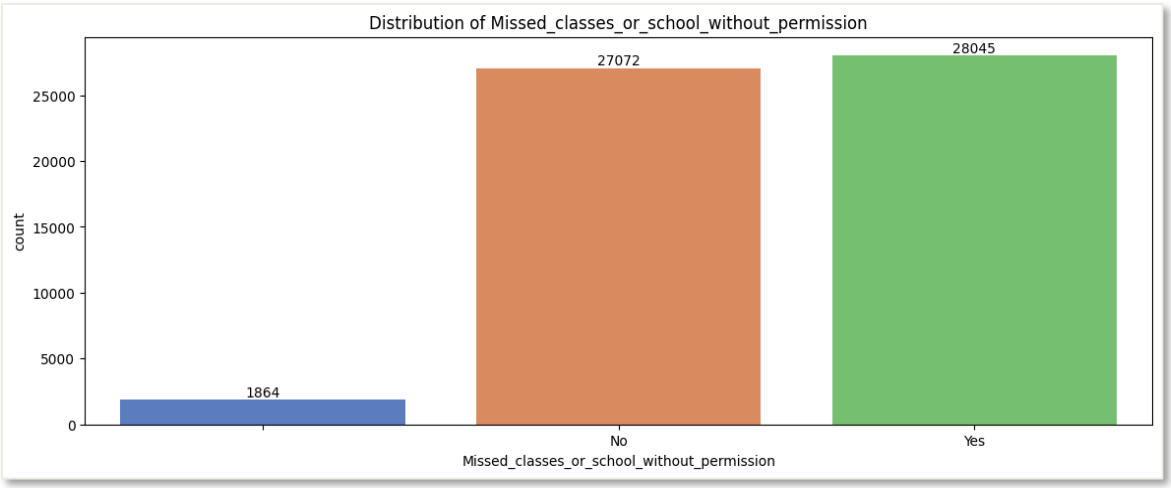


Figure 24: Bar Chart of Missed_classes_or_school_without_permission

The above bar chart shows whether the students have missed classes or schools without permission. There are 28045 of them marked as Yes, which means that they have missed class or school without permission. Besides, there are 27072 students marked as No, which means they does not miss class or school without permission.

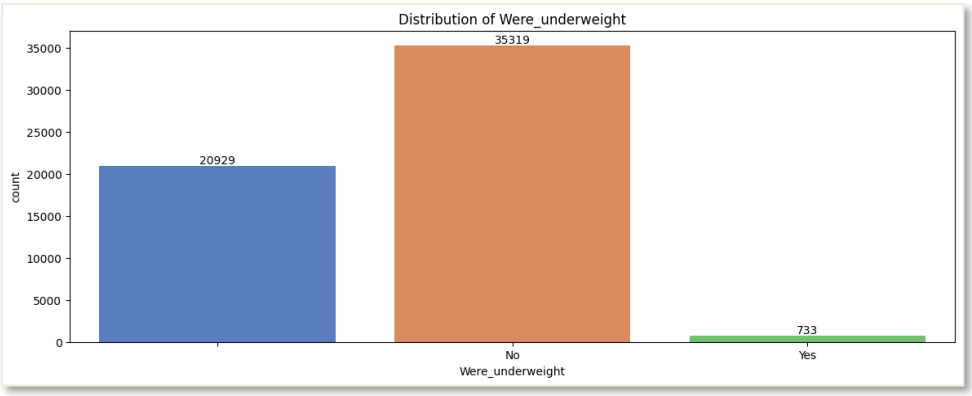


Figure 25: Bar Chart of Were_underweight

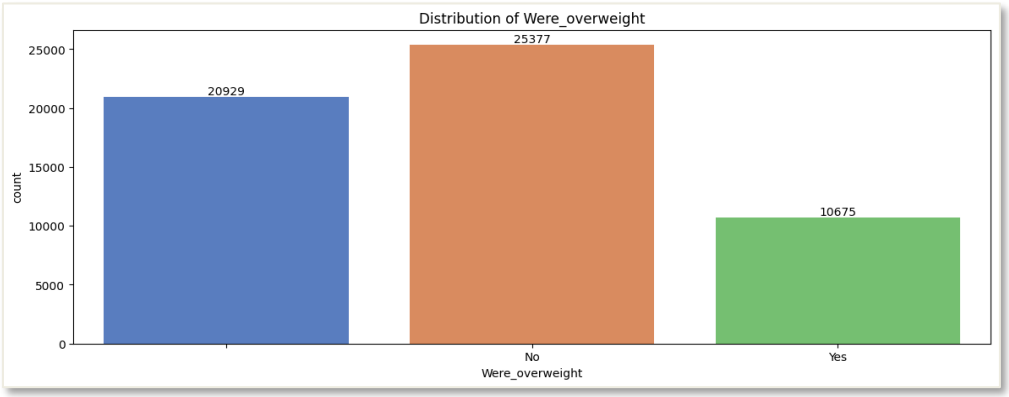


Figure 26: Bar Chart of Were_overweight

Figure 27: Bar Chart of Were_obese

The 3 figures above show the bar chart of the physical appearance of the students, exploring whether the students were underweighted, overweight or obese. By observing these characteristics can basically determine whether a student easily gets bullied. Figure 25 shows that 733 students were underweight, Figure 26 shows that 10675 students were overweight and Figure 27 shows that 2655 students were obese. However, these 3 columns in this dataset have the most blank data, which have 20929 null values. Therefore, these 3 columns may affect the accuracy of predicting the bullying risk in school as they have too many null values. This issue will be handled in the data pre-processing part.

4.3.5. Check Missing Value

(1.4) Check Missing Value

```
df_csv = pd.read_csv(file_path, dtype=str, na_values=["", " "])

# Calculate missing count and percentage
missing_count = df_csv.isnull().sum()
missing_percentage = (missing_count / len(df_csv)) * 100

missing_info = missing_count[missing_count > 0].astype(str) + " (" + missing_percentage[missing_percentage > 0].round(2).astype(str) + "%)"
print("Number of Missing Value:")
print("----------------------------------------------------------------")
print(missing_info)
```

✓ 0.1s

Figure 28: Code for checking missing value

```
Number of Missing Value:
----------------------------------------------------------------
Bullied_on_school_property_in_past_12_months          1239 (2.17%)
Bullied_not_on_school_property_in_past_12_months       489 (0.86%)
Cyber_bullied_in_past_12_months                        571 (1.0%)
Custom_Age                                             108 (0.19%)
Sex                                                    536 (0.94%)
Physically_attacked                                    240 (0.42%)
Physical_fighting                                      268 (0.47%)
Felt_lonely                                            366 (0.64%)
Close_friends                                         1076 (1.89%)
Miss_school_no_permission                             1864 (3.27%)
Other_students_kind_and_helpful                       1559 (2.74%)
Parents_understand_problems                           2373 (4.16%)
Most_of_the_time_or_always_felt_lonely                 366 (0.64%)
Missed_classes_or_school_without_permission           1864 (3.27%)
Were_underweight                                     20929 (36.73%)
Were_overweight                                      20929 (36.73%)
Were_obese                                           20929 (36.73%)
dtype: object
```

Figure 29: Output of missing value checking

The figures above show the code for checking missing value and its output. The output has shown the number and percentage of missing values for each column in the dataset. The column “Were_underweight”, “Were_overweight”, and “Were_obese” have the most of missing value. All these missing values will be handled in data pre-processing as this will affect the accuracy of bully prediction.

4.3.6. Check Inconsistence Value

```
(1.5) Check Inconsistence Value

columns_to_check = [
    "Physically_attacked",
    "Physical_fighting",
    "Close_friends",
    "Miss_school_no_permission"
]

unique_values = {col: df[col].unique().tolist() for col in columns_to_check}
pprint(unique_values)
```

Figure 30: Code for checking inconsistent value

```
{'Close_friends': ['2', '3 or more', '0', '1', ' '],
 'Miss_school_no_permission': ['10 or more days',
                               '6 to 9 days',
                               '0 days',
                               '3 to 5 days',
                               ' ',
                               '1 or 2 days'],
 'Physical_fighting': ['0 times',
                       '2 or 3 times',
                       '1 time',
                       '4 or 5 times',
                       '6 or 7 times',
                       '8 or 9 times',
                       '10 or 11 times',
                       ' ',
                       '12 or more times'],
 'Physically_attacked': ['2 or 3 times',
                         '0 times',
                         '1 time',
                         '12 or more times',
                         '4 or 5 times',
                         '10 or 11 times',
                         '8 or 9 times',
                         '6 or 7 times',
                         ' ']}
```

Figure 31: Output for checking inconsistency value

The above figure shows the code and output for checking the column in dataset which contains special data such as “12 or more times” in Physically_attacked column and “4 or 5 times” in Physical_fighting column. These kinds of data make the dataset messy due to inconsistency value.

4.3.7. Check Outliers

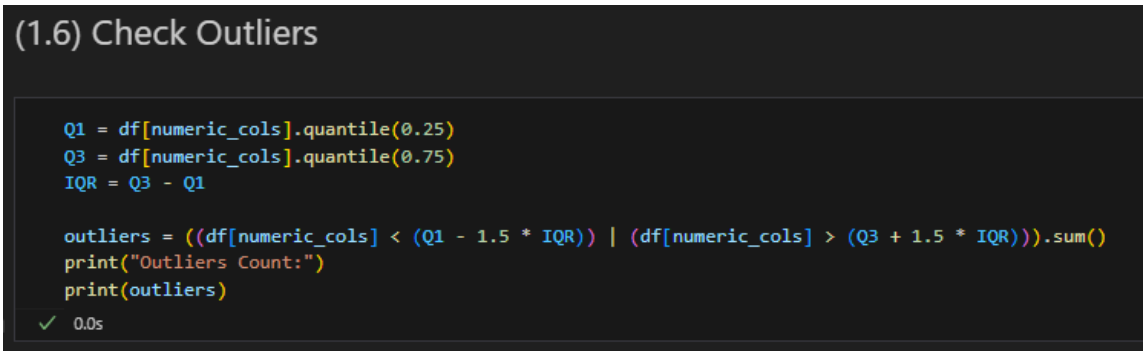


Figure 32: Code for checking outliers

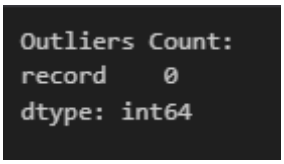


Figure 33: Output for checking outliers

The figures above show the source code and the output of checking the outliers in the raw dataset. Due to only numerical data can undergo this process and there is only one column which is the “record” column are numerical data, therefore the output only shows the outliers count of “record” column. Some of the columns cannot be used for outlier detection because they have only 2 possible values such as Yes/No. As shown in the output, there are no outliers in “record” column.

4.4 Data Pre-processing

4.4.1. Rename Variables

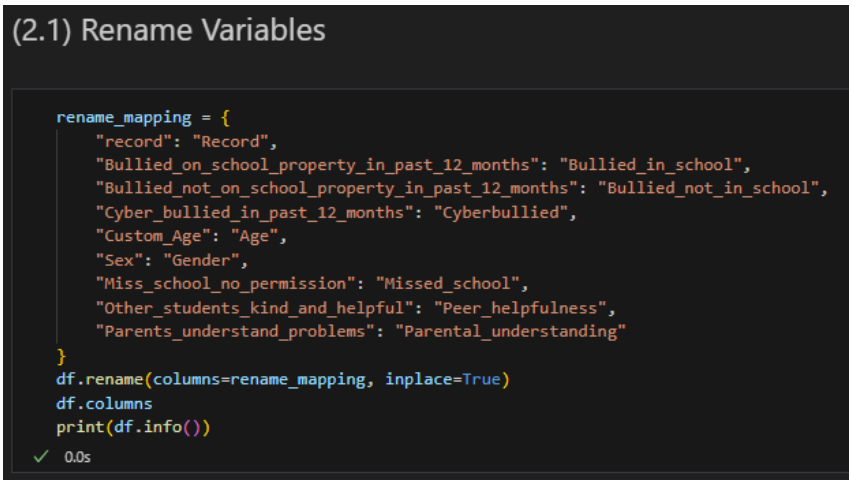


Figure 34: Code of renaming the column name

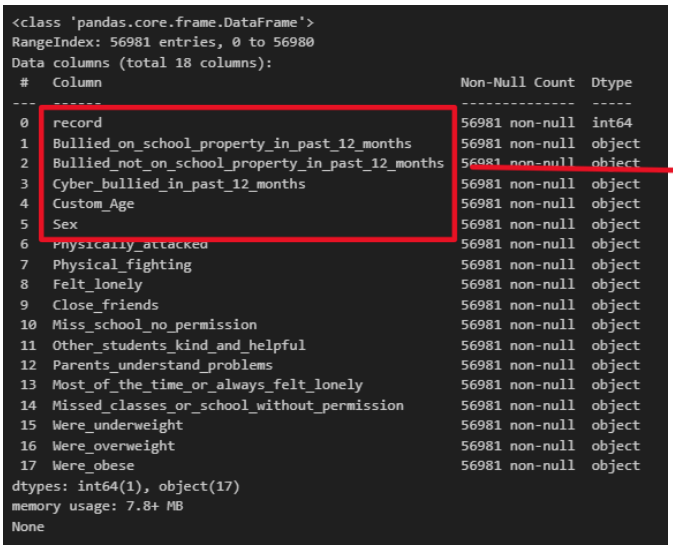


Figure 35: Output before renaming

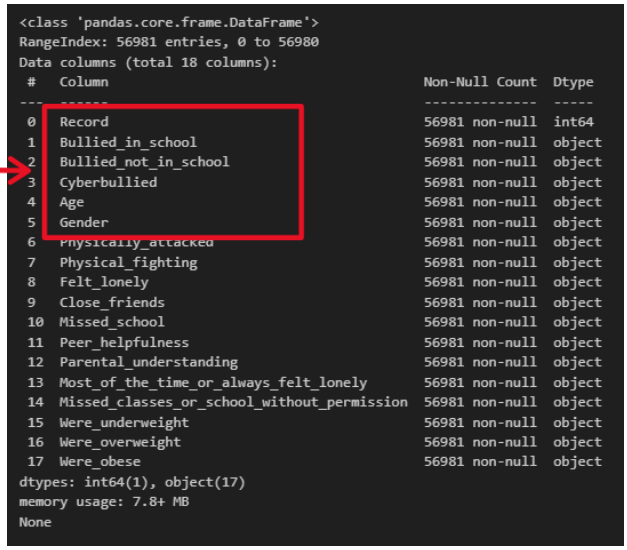


Figure 36: Output after renaming

The figures above are the code of renaming the columns and the output before undergoing pre-processing and after renaming the columns. Since some of the column's names are too long, this process is to shorten and simplify the column name. For example, Figure 35 is the output before renaming, the "Bullied_on_school_property_in_past_12_months" have renamed to "Bullied_in_school" which shown in Figure 36.

4.4.2. Handle Missing Value

(2.2) Hnadling Missing Value

```
# replace empty cell to null value
df.replace(r'^\s*$', pd.NA, regex=True, inplace=True)
df.fillna({"column_name": "default_value"}, inplace=True)


# 1. Yes/No data: Assume "Yes"
yes_no_columns = [
    "Bullied_in_school", "Bullied_not_in_school",
    "Cyberbullied", "Most_of_the_time_or_always_felt_lonely",
    "Missed_classes_or_school_without_permission",
    "Were_overweight", "Were_obese"
]
df[yes_no_columns] = df[yes_no_columns].fillna("Yes")
df["Were_underweight"] = df["Were_underweight"].fillna("No")


# 2. Physically_attack & Physical_fighting: Assume "0 times"
attack_columns = ["Physically_attacked", "Physical_fighting"]
df[attack_columns] = df[attack_columns].fillna("0 times")


# 3. Close_friends: Assume "0"
df["Close_friends"] = df["Close_friends"].fillna("0")


# 4. Miss_school: Assume "0 days"
df["Missed_school"] = df["Missed_school"].fillna("0 days")


# 5. Age: Extract numerical values , calculate median, reformat as string
numeric_age = df["Age"].str.extract('(\d+)').astype(float)
median_age = numeric_age.median()  # Calculate median
df["Age"] = df["Age"].fillna(f"{int(median_age)} years old")


# 6. Gender: Use mode
df["Gender"] = df["Gender"].fillna(df["Gender"].mode()[0])


# 7. Never-Always data: Assume "Never"
never_always_columns = [
    "Peer_helpfulness",
    "Parental_understanding"
]
df[never_always_columns] = df[never_always_columns].fillna("Never")
df["Felt_lonely"] = df["Felt_lonely"].fillna("Always")


print("Number of Missing Value After Handle:")
print("-------------------------------------------------")
print(df.isnull().sum(),"\n")
print(tabulate(df.head(10), headers='keys', tablefmt='pretty'))
```

✓ 0.5s

Figure 37: Code for handling missing values

Since the missing value in this dataset is an empty cell, it does not have any value inside the specific cells and Pandas cannot detect it as null value. Therefore, before handling the missing values, first is to replace the empty cell to NA. This is to let the program detect the NA as the missing value in the dataset.

After replacing the empty cell into NA value, different methods of handling missing value have applied.

- **Yes/No Data:** Assumed that the null value is "Yes" and replace with it. But for the column "Were_underweight", the null value is replaced as "No". This is because in logic, if a student is overweight or obese, they cannot be underweighted. Therefore, "Were_underweight" is opposite sides with "Were_overweight" and "Were_obese".

- **Ordinal Data**:
  - Assume "0 times" for "Physically_attacked" and "Physical_fighting"
  - Assume "0" close friend for "Close_friends"
  - Assume "0 days" for "Missed_school"
  - Assume "Never" for "Peer_helpfulness" and "Parental_understanding"
  - Assume "Always" for" Felt_lonely"

- For "Age", calculate the median and replace to the median value
- For "Gender", replace the null value to the mode data, which is Female in this dataset.

```
Number of Missing Value:
-----------------------------------------------------------------
Bullied_on_school_property_in_past_12_months          1239 (2.17%)
Bullied_not_on_school_property_in_past_12_months       489 (0.86%)
Cyber_bullied_in_past_12_months                        571 (1.0%)
Custom_Age                                             108 (0.19%)
Sex                                                    536 (0.94%)
Physically_attacked                                    240 (0.42%)
Physical_fighting                                      268 (0.47%)
Felt_lonely                                            366 (0.64%)
Close_friends                                         1076 (1.89%)
Miss_school_no_permission                             1864 (3.27%)
Other_students_kind_and_helpful                       1559 (2.74%)
Parents_understand_problems                           2373 (4.16%)
Most_of_the_time_or_always_felt_lonely                 366 (0.64%)
Missed_classes_or_school_without_permission           1864 (3.27%)
Were_underweight                                     20929 (36.73%)
Were_overweight                                      20929 (36.73%)
Were_obese                                           20929 (36.73%)
dtype: object
```

Figure 38: Before handling missing value

```
Number of Missing Value After Handle:
-----------------------------------------------------
Record                                              0
Bullied_in_school                                   0
Bullied_not_in_school                               0
Cyberbullied                                        0
Age                                                 0
Gender                                              0
Physically_attacked                                 0
Physical_fighting                                   0
Felt_lonely                                         0
Close_friends                                       0
Missed_school                                       0
Peer_helpfulness                                    0
Parental_understanding                              0
Most_of_the_time_or_always_felt_lonely              0
Missed_classes_or_school_without_permission         0
Were_underweight                                    0
Were_overweight                                     0
Were_obese                                          0
dtype: int64
```

Figure 39: After handling missing values

Figure 39 shows the output after handling the missing value by using different methods. Now, there is no any missing value in the dataset and could proceed to the next.

4.4.3. Feature Engineering

(2.3) Feature Engineering

- Combine 3 classification of bullied

```
# Combine & Create "Bullied"
df["Bullied"] = df[["Bullied_in_school",
                    "Bullied_not_in_school",
                    "Cyberbullied"]].max(axis=1)

columns_to_remove = ["Bullied_in_school",
                     "Bullied_not_in_school",
                     "Cyberbullied"]
df.drop(columns=columns_to_remove, inplace=True)

# Reorder columns
cols = list(df.columns)
cols.remove("Bullied")

if "Record" in cols:
    first_two_cols = ["Record"]
    remaining_cols = [col for col in cols if col != "Record"]
    new_order = first_two_cols + ["Bullied"] + remaining_cols
    df = df[new_order]

print("(Rows, Columns)")
print(df.shape, "\n")

print(df.info(),"\n")
print(tabulate(df.head(10), headers='keys', tablefmt='pretty'))
```

✓ 0.0s

Figure 40: Code for features engineering

Figure above shows the code for features engineering. In this process, 3 categories of bully (Bullied_in_school, Bullied_not_in_school, and Cyberbullied) are combined into one column which named as “Bullied”. After combining these columns, their original columns will be dropped.

```
(Rows, Columns)
(56981, 16)
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 56981 entries, 0 to 56980
Data columns (total 16 columns):
 #   Column                                  Non-Null Count  Dtype
---  ------                                  --------------  -----
 0   Record                                  56981 non-null  int64
 1   Bullied                                 56981 non-null  object
 2   Age                                     56981 non-null  object
 3   Gender                                  56981 non-null  object
 4   Physically_attacked                     56981 non-null  object
 5   Physical_fighting                       56981 non-null  object
 6   Felt_lonely                             56981 non-null  object
 7   Close_friends                           56981 non-null  object
 8   Missed_school                           56981 non-null  object
 9   Peer_helpfulness                        56981 non-null  object
 10  Parental_understanding                  56981 non-null  object
 11  Most_of_the_time_or_always_felt_lonely  56981 non-null  object
 12  Missed_classes_or_school_without_permission  56981 non-null  object
 13  Were_underweight                        56981 non-null  object
 14  Were_overweight                         56981 non-null  object
 15  Were_obese                              56981 non-null  object
dtypes: int64(1), object(15)
memory usage: 7.0+ MB
None
```

Figure 41: Output of features engineering

After combining the bullied column into one column named “Bullied” and drop the 3 original column of bullied, the column in dataset is reduced from 18 columns to 16 columns.

4.4.4. Remove Redundant Data

(2.4) Remove Redundant Data

```
columns_to_remove = ["Record",
                     "Most_of_the_time_or_always_felt_lonely",
                     "Missed_classes_or_school_without_permission"
                     ]
df.drop(columns=columns_to_remove, inplace=True)

print("(Rows, Columns)")
print(df.shape, "\n")

print(df.info(),"\n")
print(tabulate(df.head(10), headers='keys', tablefmt='pretty'))
```

✓ 0.0s

Figure 42: Code for removing redundant data

Figures above show the code for removing redundant data. In this dataset, there are two columns which named "Felt_lonely" and "Most_of_the_time_or_always_felt_lonely" are similar but different data, one is scale data (Never – Always) and another is Yes/No data. This is same for the "Miss_school_no_permission" (Never – Always) column and "Missed_classes_or_ school_without_permission" (Yes/No) column. Therefore, I choose to remove "Most_of_the_time_or_always_felt_lonely" and "Missed_classes_or_ school_without_permission". Besides, the "record" column is also removed due to it is no use for the future process.

```
(Rows, Columns)
(56981, 13)
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 56981 entries, 0 to 56980
Data columns (total 13 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   Bullied                 56981 non-null  object
 1   Age                     56981 non-null  object
 2   Gender                  56981 non-null  object
 3   Physically_attacked     56981 non-null  object
 4   Physical_fighting       56981 non-null  object
 5   Felt_lonely             56981 non-null  object
 6   Close_friends           56981 non-null  object
 7   Missed_school           56981 non-null  object
 8   Peer_helpfulness        56981 non-null  object
 9   Parental_understanding  56981 non-null  object
 10  Were_underweight        56981 non-null  object
 11  Were_overweight         56981 non-null  object
 12  Were_obese              56981 non-null  object
dtypes: object(13)
memory usage: 5.7+ MB
None
```

Figure 43: Output of removing redundant data

After removing the 3 redundant columns , the column in dataset reduced from 16 columns to 13 columns.

4.4.5. Data Transformation

- ***Extract Number Only***

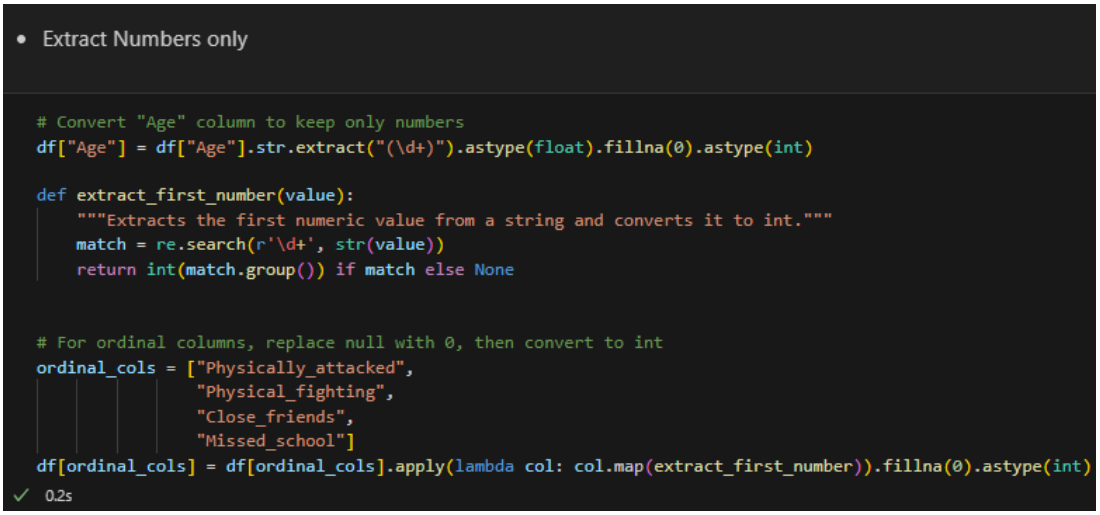


Figure 44: Code for extract number only

The figure above shows the code of extracting the number only from the data in column "Age", "Physically_attacked", "Physical_fighting", "Close_friends", and "Missed_school". This is because these data suppose is numerical data, but in this dataset their data is for example "13 years old", "0 times", "3 to 5 days", and "3 or more". Therefore, this process is to extract the number only to change their datatype from categorical data to numerical data.

- ***Yes/No → 1/0***

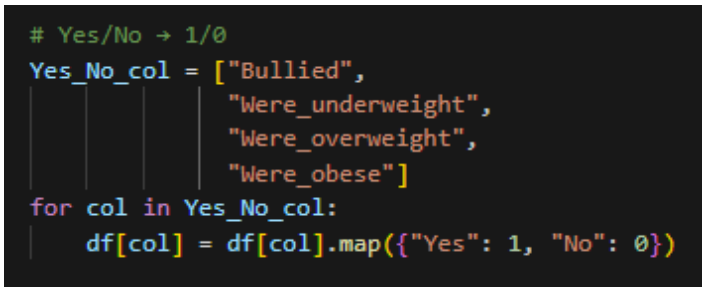


Figure 45: Code for converting Yes/No to 1/0

The figure above shows the code of transforming Yes/No data into Yes=1 and No=0. By doing this is to change the datatype of these columns from categorical data to numerical data.

- ***Scale Mapping (Never – Always → 1-5)***

```
# Scale Mapping (Never - Always -> 1-5)
scale_cols = ["Felt_lonely",
              "Peer_helpfulness",
              "Parental_understanding"]

scale_map = {"Never": 1,
             "Rarely": 2,
             "Sometimes": 3,
             "Most of the time": 4,
             "Always": 5}
df[scale_cols] = df[scale_cols].replace(scale_map)
```

Figure 46: Code for converting Never-Always to 1-5

The figure above shows the code of transforming scale mapping data to numerical data. 1 representing Never, 2 representing Rarely, 3 representing Sometimes, 4 representing Most of the time, and 5 representing Always. By doing this process is to change the datatype of these columns from categorical data to numerical data.

- ***Gender Mapping (Female=0, Male=1)***

```
# Gender Mapping (Female=0, Male=1)
df["Gender"] = df["Gender"].map({"Female": 0,
                                 "Male": 1})
```

Figure 47: Code for converting Female/Male to 0/1

The figure above shows the code of transforming Female = 0 and Male = 1. This process is to change the datatype of Gender from categorical data to numerical data.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 56981 entries, 0 to 56980
Data columns (total 13 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   Bullied                 56981 non-null  int64
 1   Age                     56981 non-null  int64
 2   Gender                  56981 non-null  int64
 3   Physically_attacked     56981 non-null  int64
 4   Physical_fighting       56981 non-null  int64
 5   Felt_lonely             56981 non-null  int64
 6   Close_friends           56981 non-null  int64
 7   Missed_school           56981 non-null  int64
 8   Peer_helpfulness        56981 non-null  int64
 9   Parental_understanding  56981 non-null  int64
 10  Were_underweight        56981 non-null  int64
 11  Were_overweight         56981 non-null  int64
 12  Were_obese              56981 non-null  int64
dtypes: int64(13)
memory usage: 5.7 MB
None
```

	Bullied	Age	Gender	Physically_attacked	Physical_fighting	Felt_lonely	Close_friends	Missed_school	Peer_helpfulness	Parental_understanding	Were_underweight	Were_overweight	Were_obese
0	1	13	0	2	0	5	2	10	1	5	0	1	1
1	1	13	0	0	0	4	3	6	3	5	0	1	1
2	1	14	1	1	0	4	3	6	3	5	0	0	0
3	0	16	1	0	2	1	3	0	3	1	0	0	0
4	0	13	0	0	0	2	3	0	4	4	0	1	1
5	1	13	1	0	1	4	0	3	4	5	0	0	0
6	0	14	0	1	0	3	3	0	4	5	0	1	1
7	0	12	0	0	0	2	3	0	4	1	0	1	1
8	1	13	1	1	2	4	3	6	4	4	0	1	1
9	1	14	0	1	0	5	0	6	3	1	0	1	1

Figure 48: Output after data transformation

Figures above show the output after data transforming from categorical data to numerical data. The number of columns in this dataset is also reduced to 13 columns.

After data pre-processing, the dataset is now clean and structured. It is ready to continue with the next step, data modeling.

4.5 Data Understanding After Pre-processing

4.5.1. Data Dictionary (After Pre-processing)

No	Column Name	Description	Assumption
1.	Bullied	Does student got bullied	1 = Yes 0 = No
2.	Age	Age of student	-
3.	Gender	Gender of student	1 = Male 0 = Female
4.	Physically_attacked	Times of student who got physical attacked	-
5.	Physical_fighting	Times of student who got physical fights	-
6.	Felt_lonely	How often felt lonely	Never = 1 Rarely = 2 Sometimes = 3 Most of the time =4 Always = 5
7.	Close_friends	Number of close friends	-
8.	Missed_school	Missed school without permission	-
9.	Peer_helpfulness	How often others help	Never = 1 Rarely = 2 Sometimes = 3 Most of the time =4 Always = 5
10.	Parental_understanding	How often parents understand	Never = 1 Rarely = 2 Sometimes = 3 Most of the time =4 Always = 5
11.	Were_underweight	Underweight	1 = Yes 0 = No
12.	Were_overweight	Overweight	1 = Yes 0 = No
13.	Were_obese	Obese	1 = Yes 0 = No

4.5.2. Features Analysis (After Pre-processing)

2b Features Analysis (after pre-processing 1)

```
categorical_cols = df.select_dtypes(include=["object"]).columns

for col in categorical_cols:
    unique_vals = df[col].dropna().unique()
    try:
        sorted_order = sorted(unique_vals, key=lambda x: float(x))
    except:
        # Fallback: sort alphabetically
        sorted_order = sorted(unique_vals)

    plt.figure(figsize=(14, 5))
    ax = sns.countplot(
        x=df[col],
        hue=df[col],
        palette="muted",
        order=sorted_order,
        hue_order=sorted_order,
    )

    for p in ax.patches:
        ax.annotate(
            f'{int(p.get_height())}',
            (p.get_x() + p.get_width() / 2., p.get_height()),
            ha='center', va='bottom', fontsize=10, color='black'
        )

    plt.title(f"Distribution of {col}")
    plt.xticks(rotation=0)
    plt.show()
```

✓ 2.2s

Figure 49: Code for features analysis (After Pre-processing)

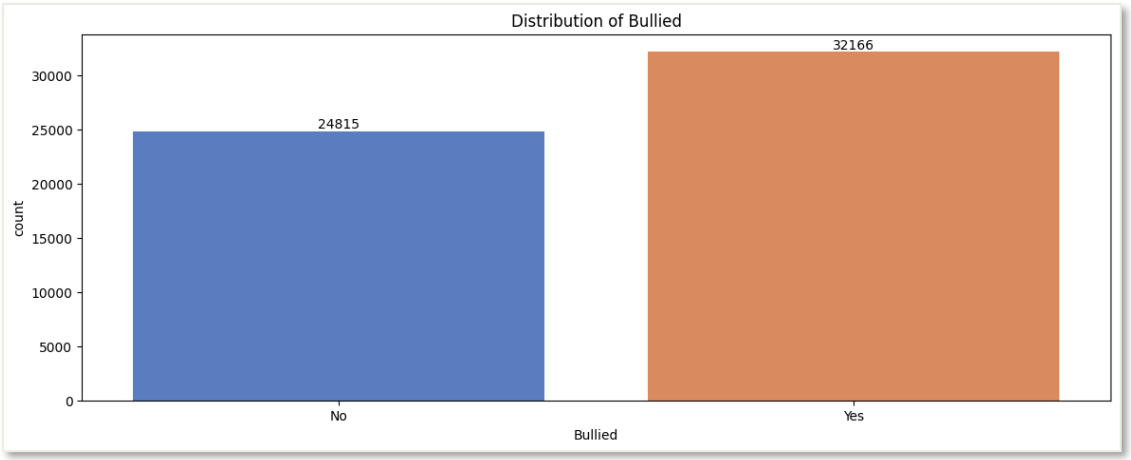


Figure 50: Bar chart of Bullied

The figure above shows the bar chart of bullied is the combination of the 3 categories bullied in the raw data which is the "Bullied_on_school_property_in_past_12_months", "Bullied_not_on_school_property_in_past_12_months", and "Cyber_bullied_in_past_12_month". The combination method is based on if one of the categories of the bullied is marked "Yes", then consider as "Yes" while combine. If 3 of the categories of the bullied in raw dataset is marked as "No", then here only marked as "No". As we can see from the above bar chart, there are 24815 students does not get bullied while 32166 students have been bullied.

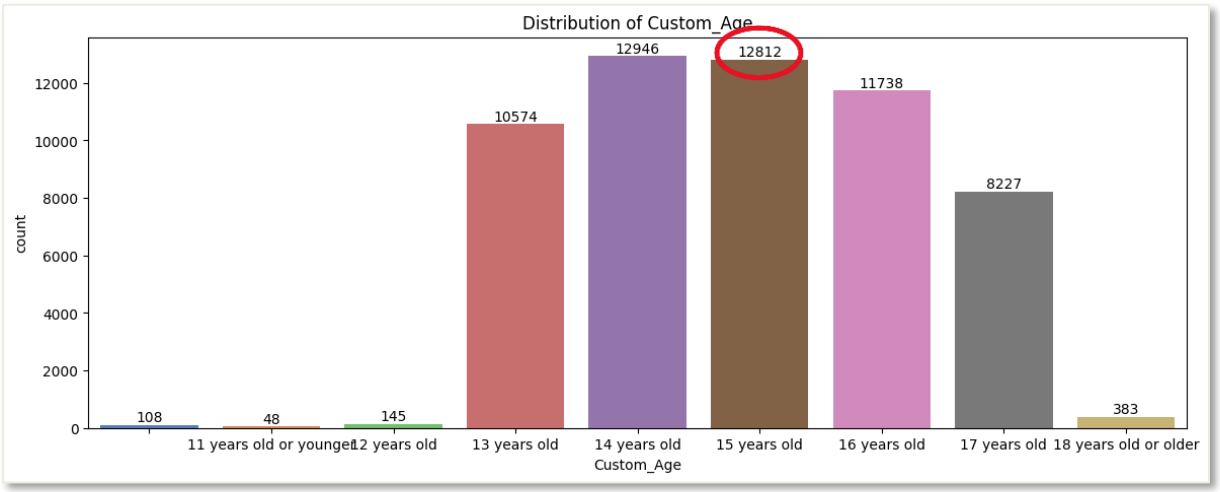


Figure 51: Bar Chart for student's Age (Before pre-processing)

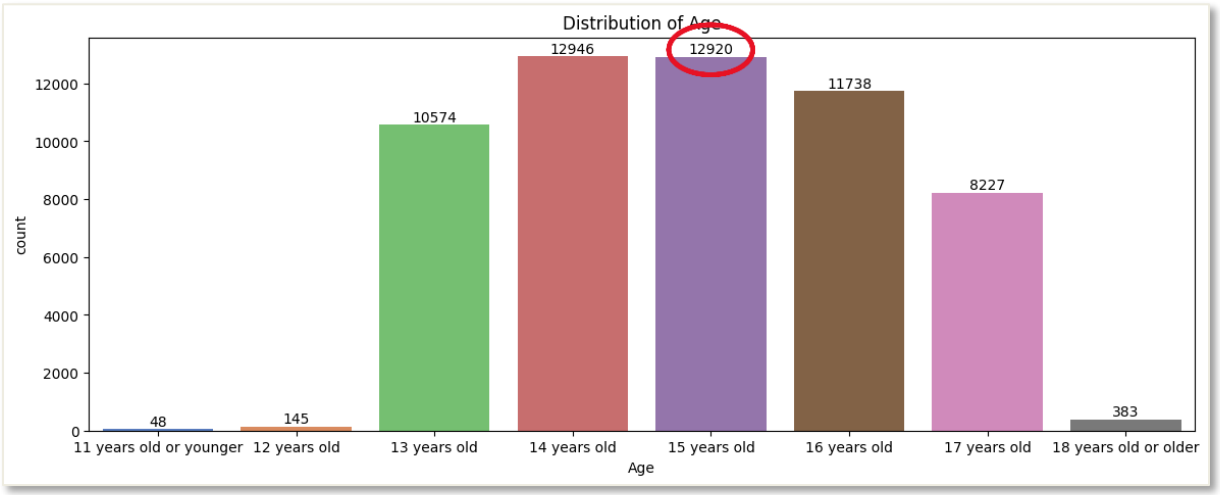


Figure 52: Bar Chart for student's Age (After Pre-processing)

The figures above shows the output of student age before and after pre-processing. In the stage of data pre-processing, missing value were handled by calculating median of the column of age. As shown by the bar chart above, there is no missing value in the output after pre-processing and the age "15 years old" have increased from 12812 to 12920, which means that 15 years old is the median and the missing value are assuming in 15 years old. Besides, the name of "Custom_Age" have been remaned to "Age".

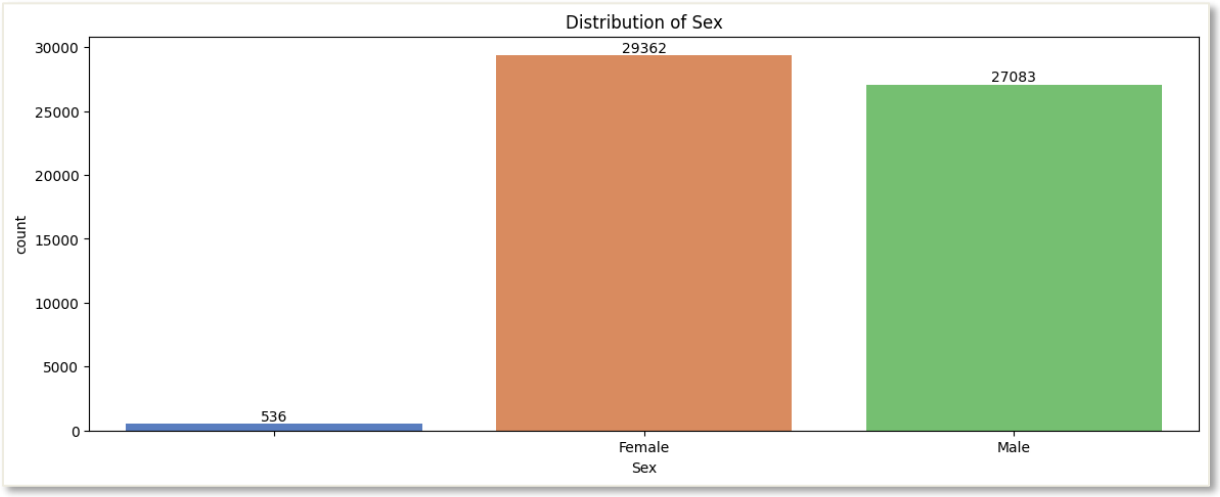


Figure 53: Bar Chart of Sex of student (Before pre-processing)

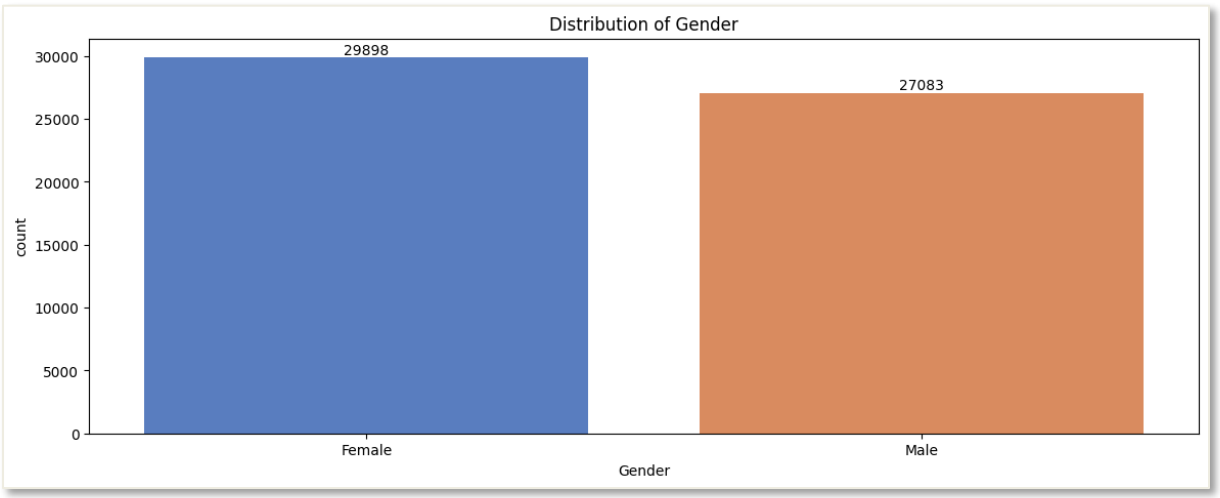


Figure 54: Bar Chart for Gender (After Pre-processing)

The figures above shows the output before and after pre-processing of Gender. The missing value of Gender are handled by calculating the mode in this column. As shown in the bar chart above, the count of Female have increased from 29362 to 29898. Meaning that female is the mode and the missing value have assumed as Female while handling pre-processing. Besides, the name of this column have been renamed to Gender.

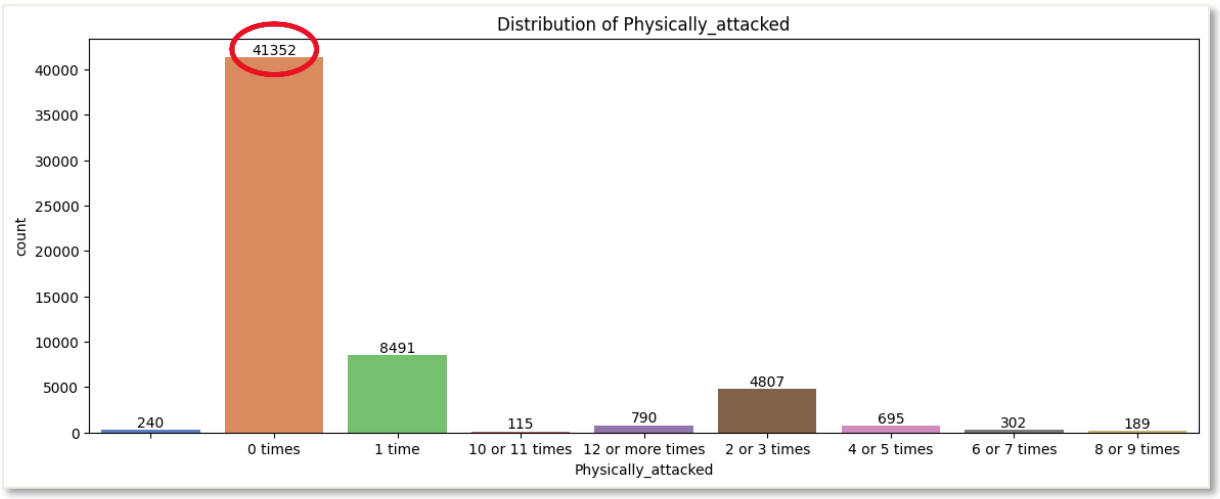


Figure 55: Bar Chart for Phycsically_attacked (Before pre-processing)

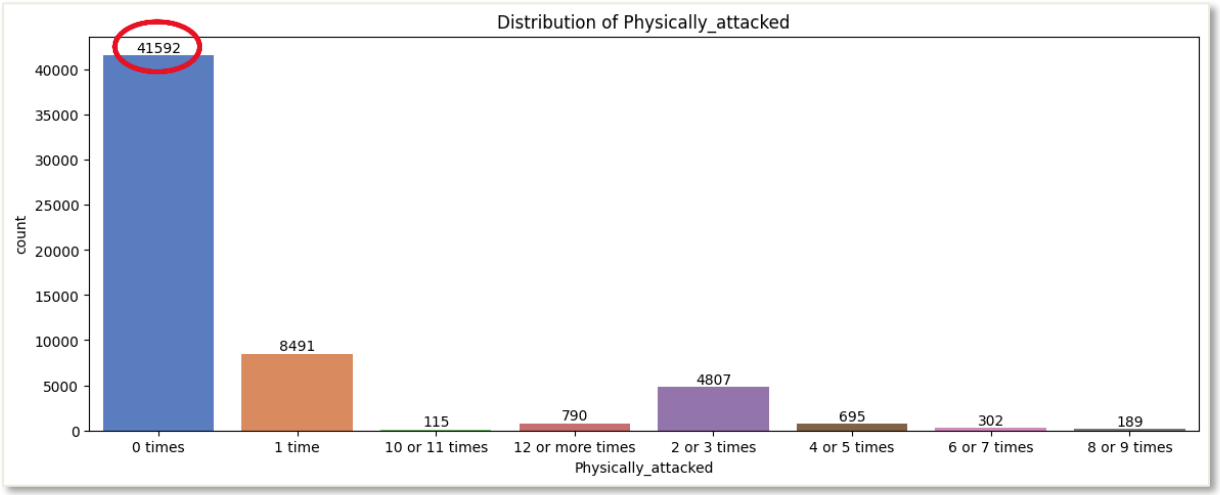


Figure 56: Bar Chart for Phycsically_attacked (After Pre-processing)

The figures above shows the output before and after pre-processing of Physically_attacked. The bar chart after pre-porocessing have no more missing value since it have been handled during pre-processing. The missing value of this column have assumed as 0 times. Therefore, student who faced 0 time of physical attacked have increased from 41352 to 41592.

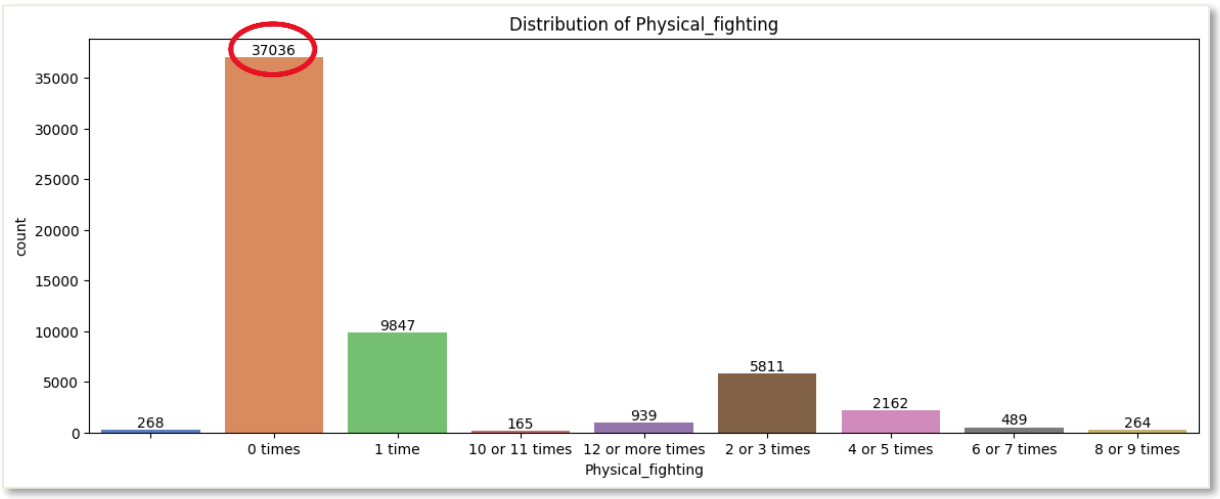


Figure 57: Bar Chart for Phycsical_fighting (Before pre-processing)

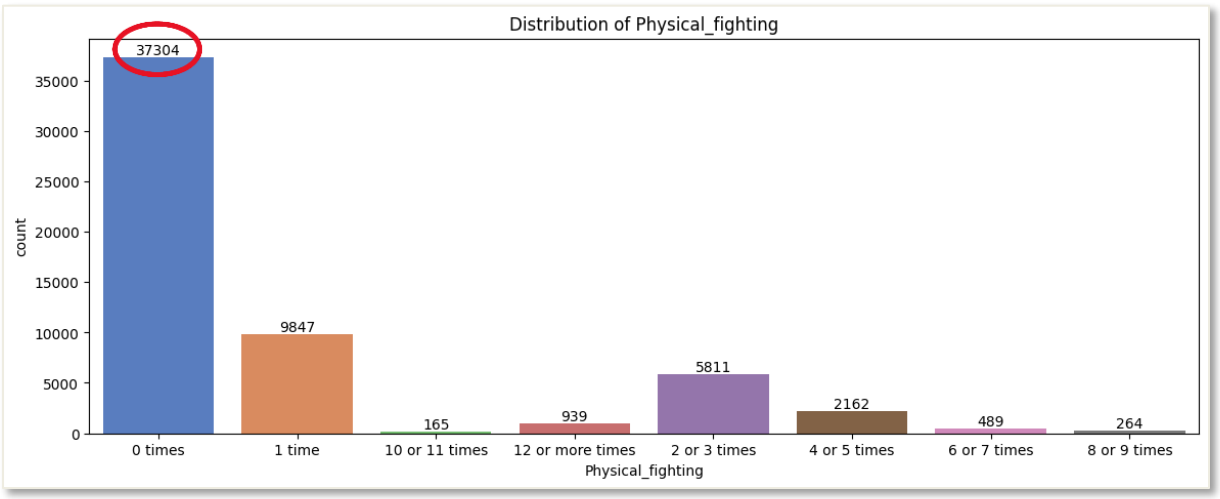


Figure 58: Bar Chart for Physical_fighthing (After Pre-processing)

The figures above shows the output before and after pre-processing of Physical_fighthing. The missing value have handled same as the column "Physically_attacked", which is assumed as 0 times and the student who faced 0 time of physical fighting have been increased from 37036 to 37304.

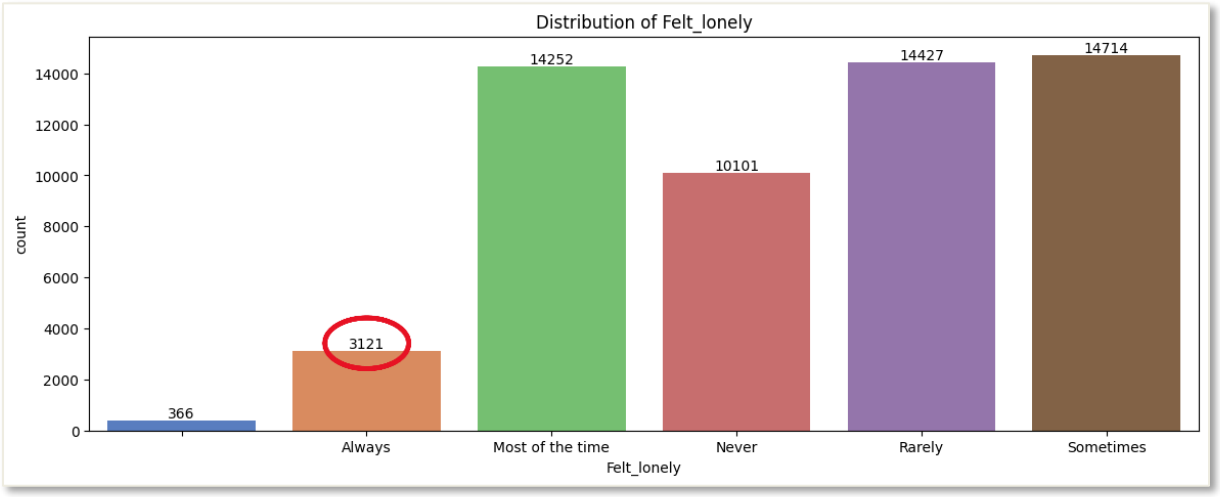


Figure 59: Bar Chart for Felt_lonely (Before pre-processing)

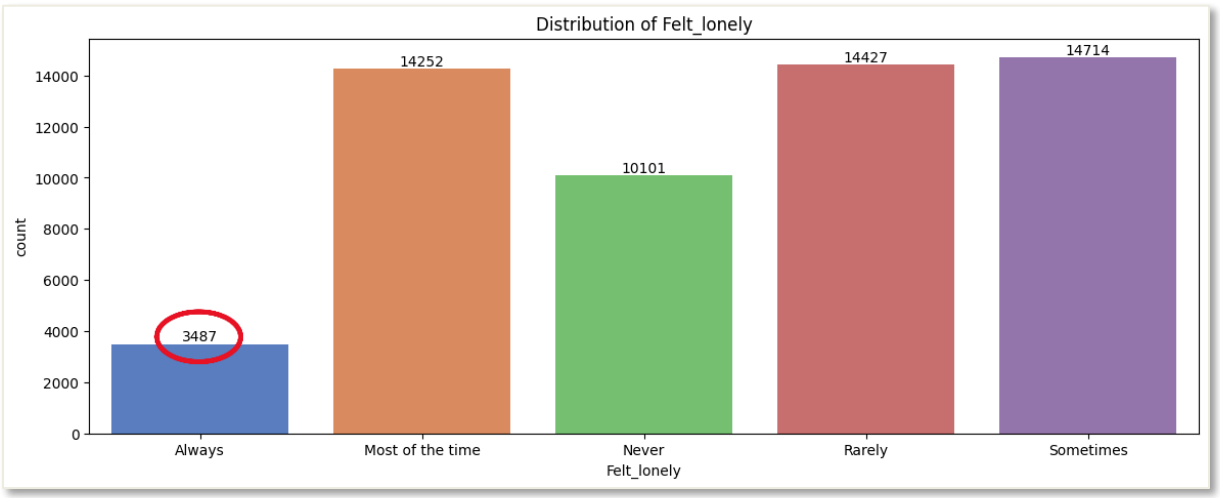


Figure 60: Bar Chart for Felt_lonely (After pre-processing)

The figures above shows the output before and after pre-processing of Felt_lonely. The missing value for this column have assumed as Always after pre-processing. As shown in the bar chart after pre-processing, the is no missing value and the count of Alway have increase from 3132 to 3487.

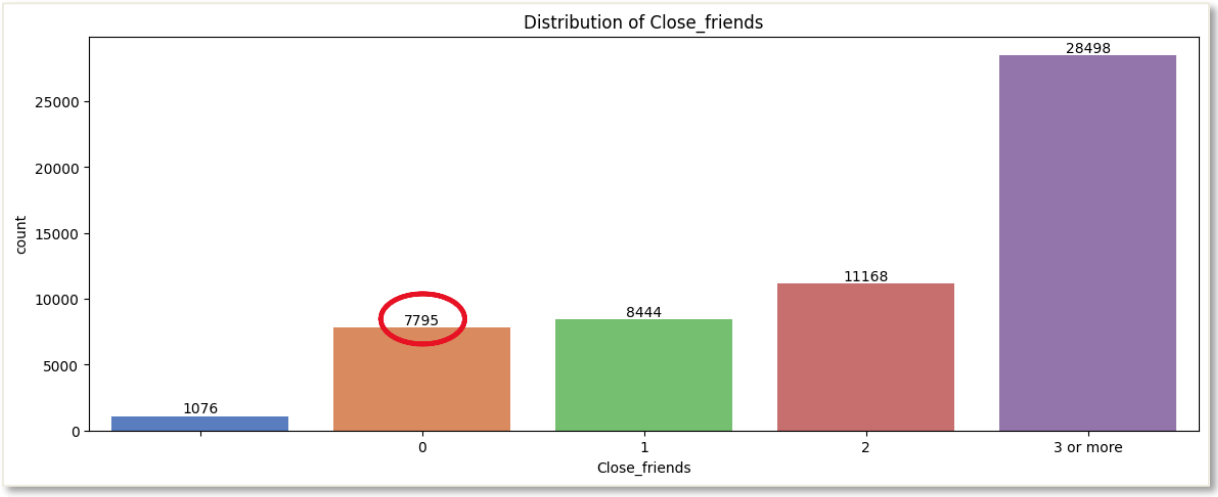


Figure 61: Bar Chart for Close_friends (Before pre-processing)

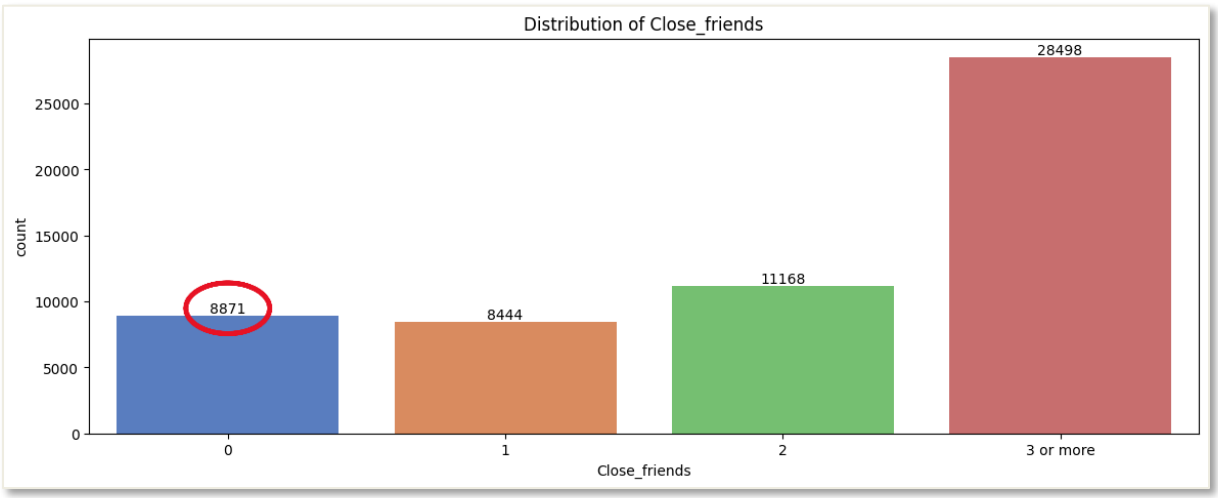


Figure 62: Bar Chart for Close_friends (After Pre-processing)

The figures above shows the output before and after pre-processing of Close_friends. The missing value in the bar chart before pre-processing have been handled by assuming it as 0. Therefore, the 0 count have been increase from 7795 to 8871.

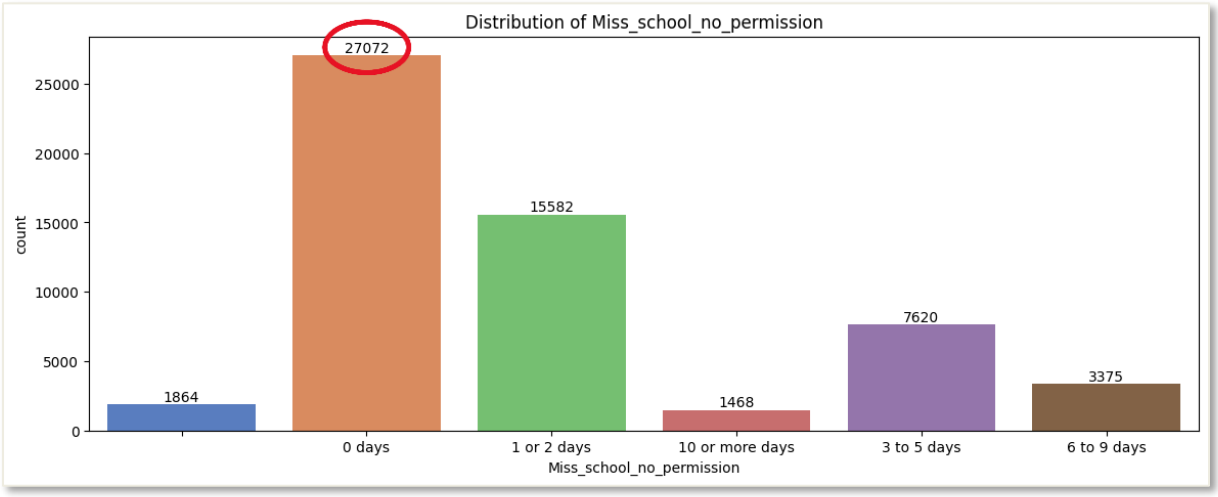


Figure 63: Bar Chart for Miss_school_no_permission (Before pre-processing)

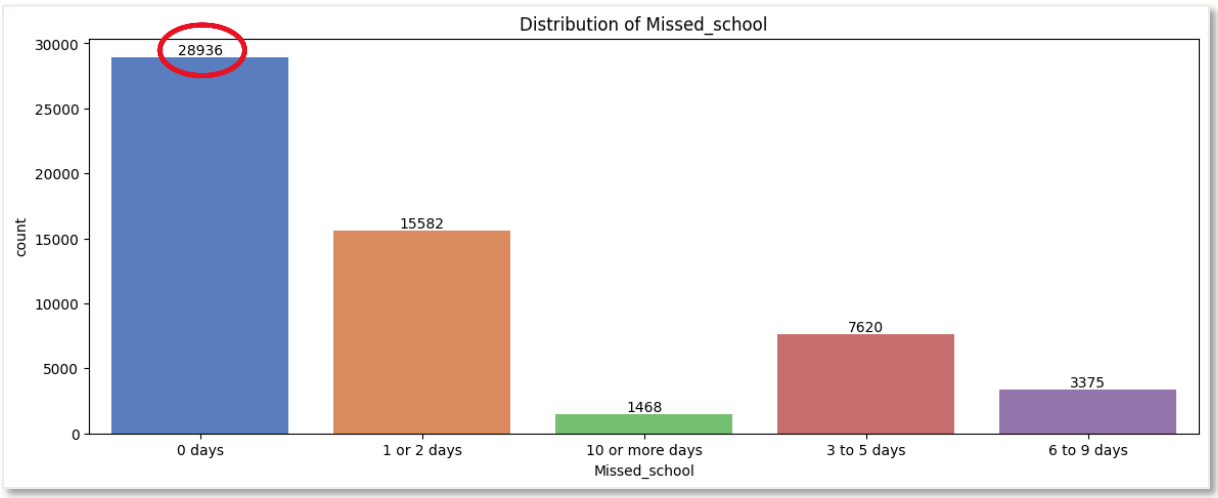


Figure 64: Bar Chart for Missed_school (After Pre-processing)

The figures above shows the output before and after pre-processing of Missed_school. After preprocessing, the missing value have been handled by assuming it as 0 days. Therefore, the value of 0 days have been increased from 27072 to 28936. Besides, the name of this column have been renamed to “Missed_school” beasause the name before pre-processing is too long.

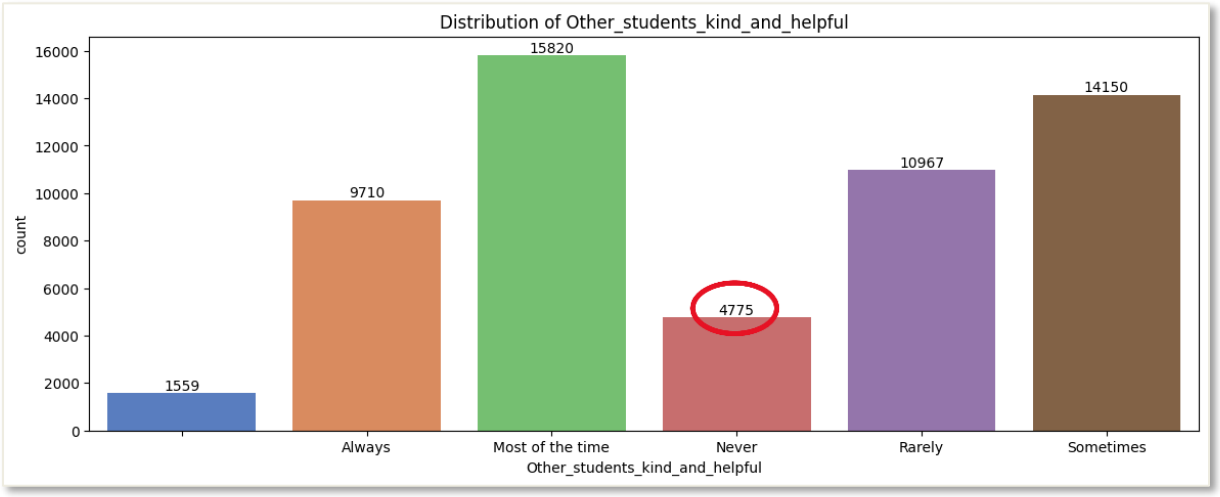


Figure 65: Bar Chart for Other_students_kind_and _helpful (Before pre-processing)

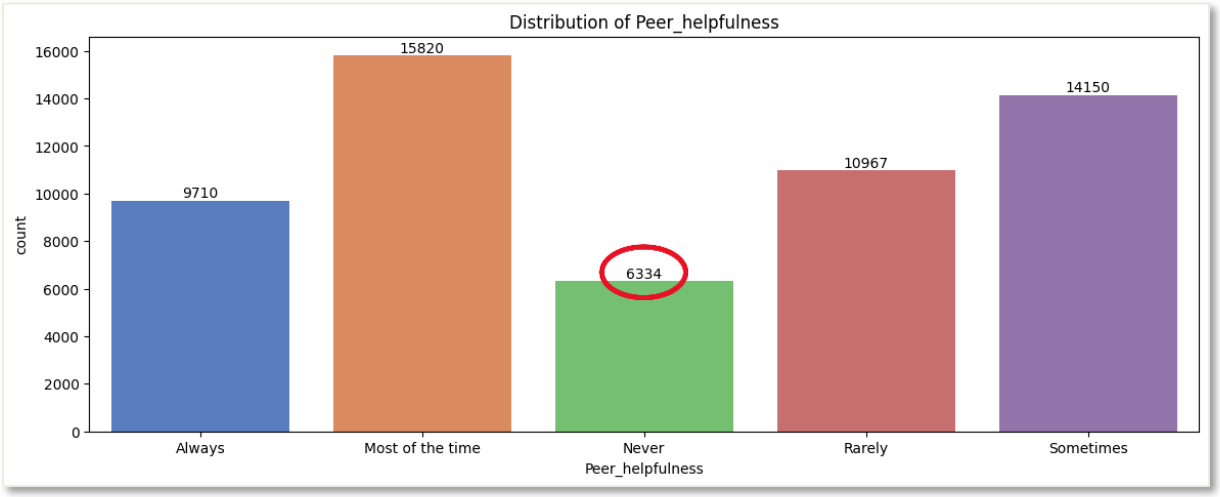


Figure 66: Bar Chart for Peer_helpfulness (After Pre-processing)

The figures above shows the output before and after pre-processing of Peer_helpfulness. After preprocessing, the missing value have been handled by assuming it as Never. Therefore, the count of student never received help from other is increase from 4775 to 6334. Besides, the name of this column have been renamed to Peer_helpfulness because the name in raw data is too long.

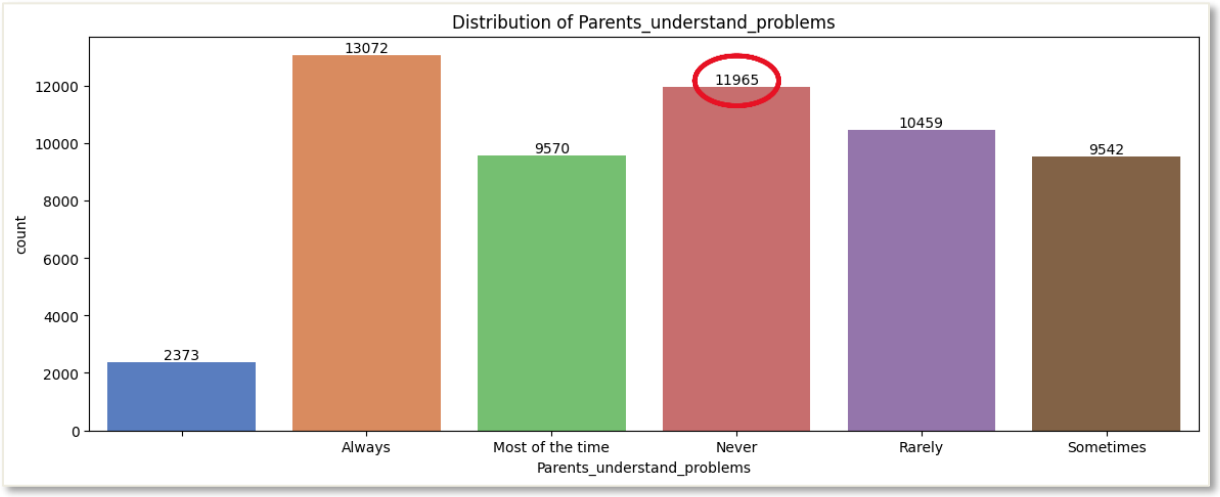


Figure 67: Bar Chart for Parents_understand_problem (Before pre-processing)

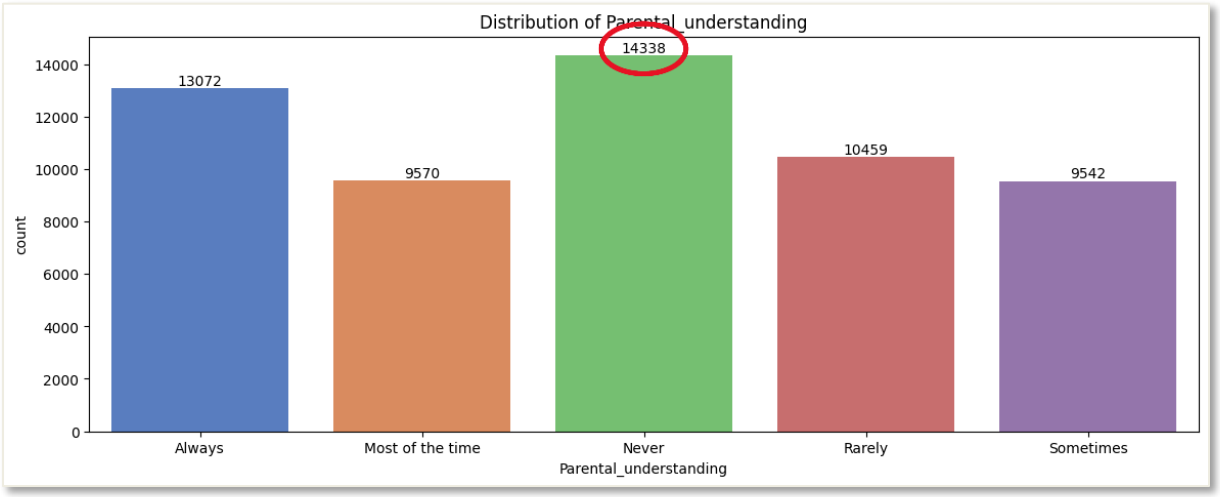


Figure 68: Bar Chart for Parental_understanding (After pre-processing)

The figures above shows the output before and after pre-processing of Perental_understanding. There are 2373 missing value in the raw data. After pre-processing, the missing value have been handled by assuming it as Never. Therefore, the count of Never have increased from 11965 to 14338. Moreover, the column name are also remaned to Parental_understanding.

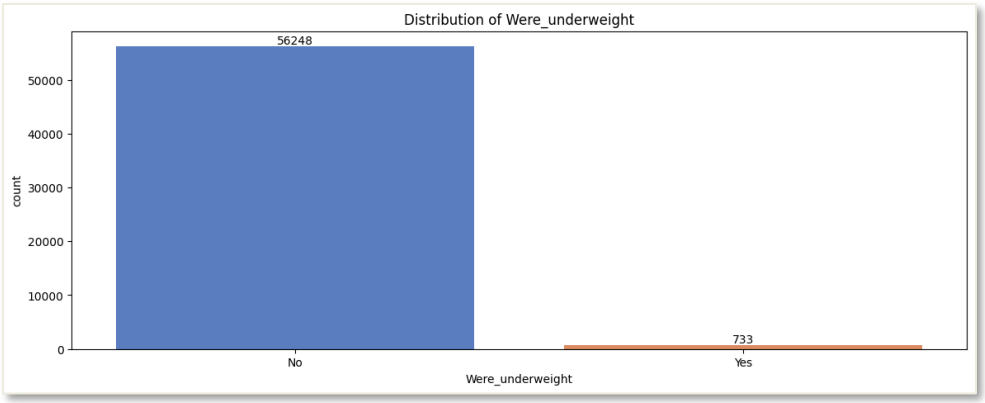


Figure 69: Bar Chart for Were_underweight

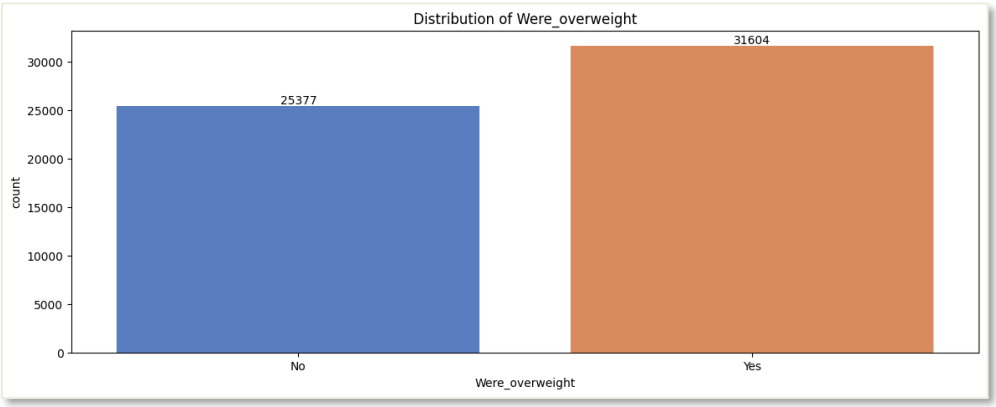


Figure 70: Bar Chart for Were_overweight

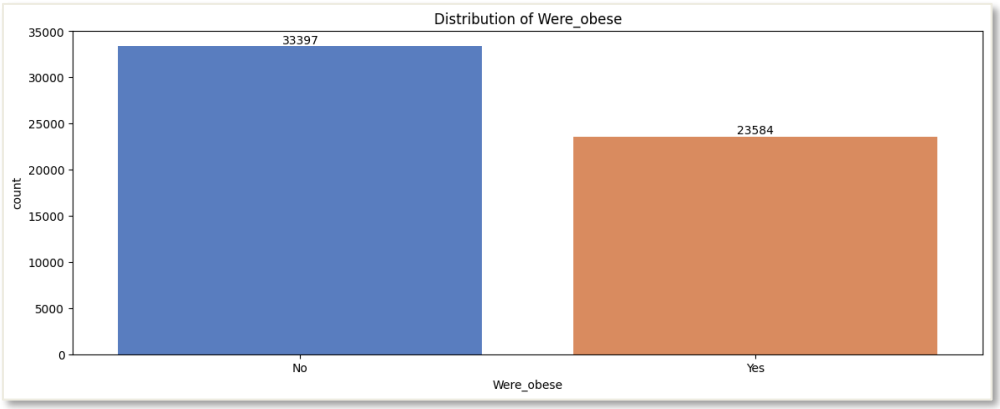


Figure 71: Bar Chart for Were_obese

Figures 69, 70, and 71 is the output after pre-processing of Were_underweight, Were_overweight, and Were_obese. After pre-processing, the missing value in Were_underweight has assumed as No, while the missing value for Were_overweight and Were_obese has assumed as Yes. The reason that cannot assume all missing value of these 3 column as Yes is because if a student is overweight or obese, they cannot be underweighted. Therefore, as we can see from these bar charts, students who are overweight are more likely to be bullied.

4.5.3. Check Outliers (After Pre-processing)

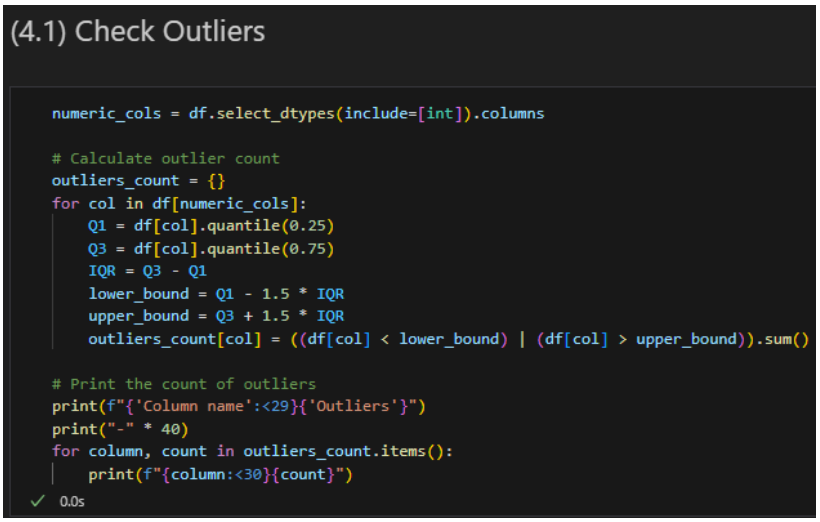


Figure 72: Code for checking the outliers (After Pre-processing)

```
Column name                    Outliers
----------------------------------------
Bullied                        0
Age                            0
Gender                         0
Physically_attacked            2091
Physical_fighting              4019
Felt_lonely                    0
Close_friends                  0
Missed_school                  12463
Peer_helpfulness               0
Parental_understanding         0
Were_underweight               733
Were_overweight                0
Were_obese                     0
```

Figure 73: Output of checking the outliers

The figures above show the code and output for checking the outliers. In the output, it shows that the dataset has 4 column having outliers, which is “Physically_attacked” with 2091 outliers, “Physical_fighting” with 4019 outliers, “Missed_school” with 12463 outliers, and “Were_underweight” with 733 outliers. These outliers will be handle afterwards.

4.5.4. Create new column for outliers (Flagging)

(4.2) Create new column for outliers

```
numeric_cols = df.select_dtypes(include='number').columns

# Precompute IQR bounds
bounds = {}
for col in numeric_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    bounds[col] = (Q1 - 1.5 * IQR, Q3 + 1.5 * IQR)

# Check outliers in a row
def detect_outliers(row):
    for col in numeric_cols:
        lower, upper = bounds[col]
        if row[col] < lower or row[col] > upper:
            return 1
    return 0

# Create new column "Has_Outlier"
df["Has_Outlier"] = df.apply(detect_outliers, axis=1)

print("(Rows, Columns)")
print(df.shape, "\n")
print(tabulate(df.head(10), headers='keys', tablefmt='pretty'))
```

Figure 74: Code for Create an outlier column

The figure above is the code for creating a column for outliers and named "Has_outlier". If there are outliers in the specific row, then "1", else "0". This method is called flagging. Instead of removing or modifying the outliers, flagging is a process of identifying and marking data points that deviate significantly from the norm.

```
(Rows, Columns)
(56981, 14)
```

Figure 75: Output after creating outlier column (1)

	Bullied	Age	Gender	Physically_attacked	Physical_fighting	Felt_lonely	Close_friends	Missed_school	Peer_helpfulness	Parental_understanding	Were_underweight	Were_overweight	Were_obes	Has_Outlier
0	1	13	0	2	0	5	2	10	1	5	0	1	1	1
1	1	13	0	0	0	4	3	6	3	5	0	1	1	1
2	1	14	1	1	0	4	3	6	3	5	0	0	0	1
3	0	16	1	0	2	1	3	0	3	1	0	0	0	0
4	0	13	0	0	0	2	3	0	4	4	0	1	1	0
5	1	13	1	0	1	4	0	3	4	5	0	0	0	1
6	0	14	0	1	0	3	3	0	4	5	0	1	1	0
7	0	12	0	0	0	2	3	0	4	1	0	1	1	0
8	1	13	1	1	2	4	3	6	4	4	0	1	1	1
9	1	14	0	1	0	5	0	6	3	1	0	1	1	1

Figure 76: Output after creating outlier column (2)

Figures above show the output after flagging the outliers by creating a new column for it. The dataset is now having 56981 rows and 14 columns. This is the final cleaned dataset that is ready to proceed to model building and evaluations.

4.5.5. Correlation Heatmap

(4.3) Correlation Heatmap

```
corr_matrix = df.corr()

# Plot heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(corr_matrix, annot=True, cmap="coolwarm", fmt=".2f", linewidths=0.5)
plt.title("Correlation Heatmap")
plt.show()
```

Figure 77:Code for correlation heatmap

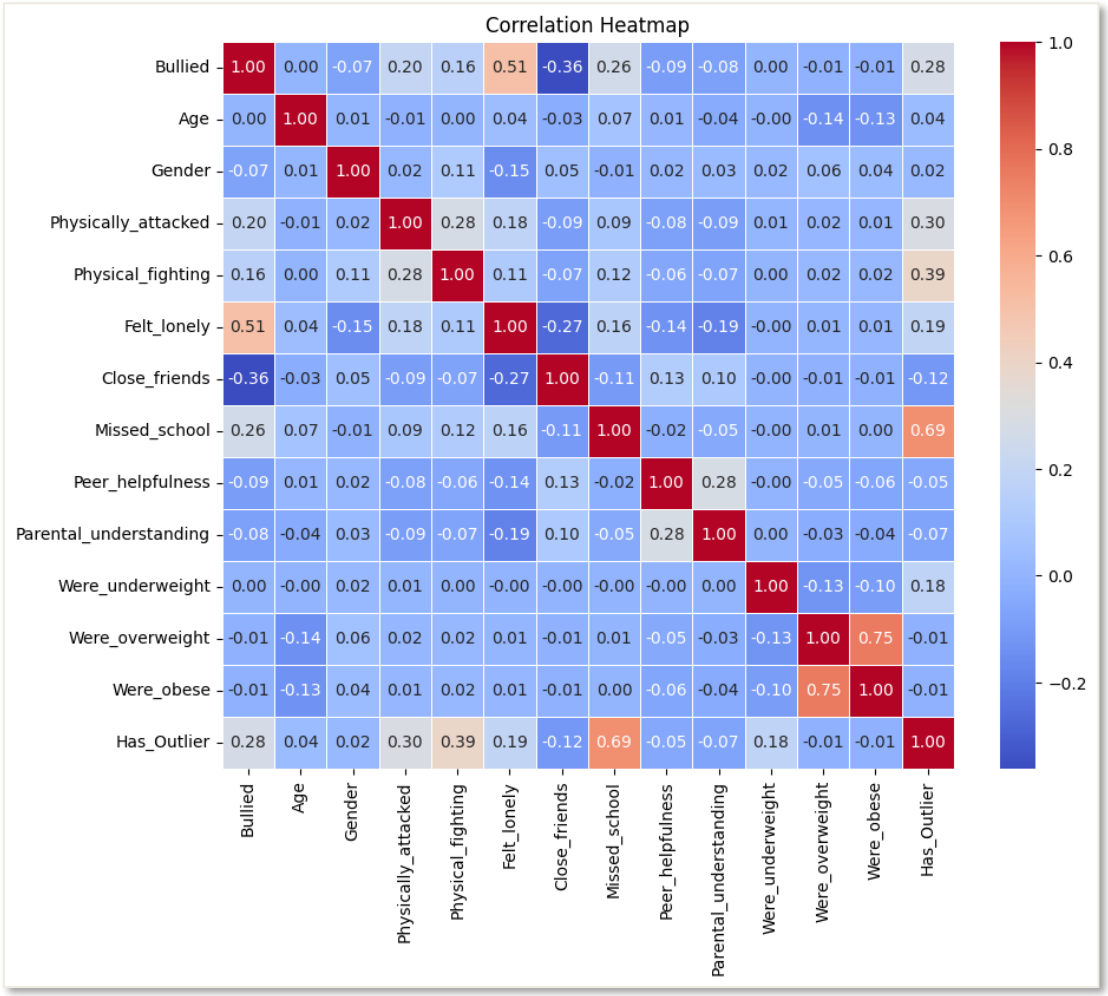


Figure 78:Correlation heatmap

The figures above are the code and output of correlation heatmap. As the topic of this project is about bullying, so the target variable is “Bullied”. The correlation heatmap highlights the key relationship between “Bullied” and other variables. As we can see that the strongest correlation with “Bullied” is **“Felt_lonely” (0.51)**, meaning that students who get bullied are more likely to feel lonely. Besides, physical bully such as **“Physically_attacked” (0.20)** and **“Physical_fighting” (0.16)** are positively connected with “Bullied”.

In addition, having fewer **close friends (-0.36)** and **missing school more often (0.26)** also correlated strongly. Weak negative correlations with **Peer_helpfulness (-0.09)** and **Parental_understanding (-0.08)** suggest some protective benefits. However, weight-related factor like “Were_underweight” (0.00), “Were_overweight” (-0.01) and “ Were_obese” (-0.01) have only small linking with “Bullied”. In conclusion, this correlation heatmap shows that loneliness and physical violence are the most important factors that cause bullying.

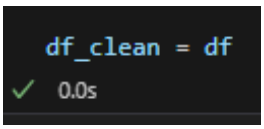


Figure 79: Create new data frame

This figure is the code of creating a new data frame which called “df_clean” for the cleaned dataset. “df_clean = df” means it is direct copy the previous data frame and continue proceed to the next step using df_clean.

4.6 Model Building

4.6.1 Check Target Variable count

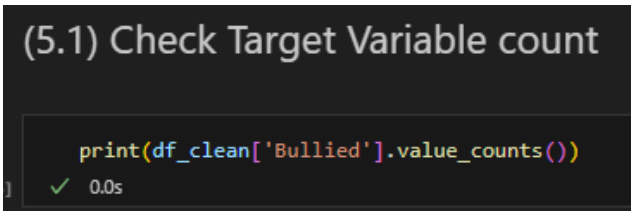


Figure 80: Code for checking target variable count

```
Bullied
1    32166
0    24815
Name: count, dtype: int64
```

Figure 81: Output of target variable count

The figures above show the code and output of checking the count of target variable. The target variable in this dataset is “Bullied”. Figure 81 shows that ‘1’ which is the bullied classes have 32166 cases while ‘0’ which is non-bullied classes have 24815 cases.

4.6.2 Data Splitting

(5.2) Split Data (Train & Test)

```
X = df_clean.drop('Bullied', axis=1)
y = df_clean['Bullied']

# Split into 80% training, 20% testing
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

#Training Set
print("[Train] Set Shape:")
train_count = y_train.value_counts()
print(train_count)
print("Total:", train_count.sum(), "\n")

#Testing Set
print("[Test] Set Shape:")
test_count = y_test.value_counts()
print(test_count)
print("Total:", test_count.sum(), "\n")
```

✓ 0.0s

Figure 82: Code for data splitting

```
[Train] Set Shape:
Bullied
1    25696
0    19888
Name: count, dtype: int64
Total: 45584

[Test] Set Shape:
Bullied
1    6470
0    4927
Name: count, dtype: int64
Total: 11397
```

Figure 83: Output for data splitting

The above figures show the code and the output of data splitting. The dataset is split into 80% for training with total 45584 samples and 20% for testing with total of 11397 samples. The '1' represent to Bullied which '0' represent to No Bullied. The 80-20 train-test split is the common ratio for data splitting. The balanced split preserves the original dataset's distribution, providing reliable data for model training and evaluation.

4.6.3 Logistic Regression (LR)

4.6.3.1 Advantage of Logistic Regression

- **Ease of implementation and training**

  Logistic regression requires very low computational cost, making it easier to implement, analyse, and train compared to more complex machine learning models (Kanade, 2022). Therefore, it able to handle large dataset effectively.

- **Suitable for binary classification**

  Logistic Regression can reflect binary outcomes effectively such as yes/no, 0/1 by calculating the probability of an occurrence, making it useful in diverse fields such as healthcare, finance, and marketing for decisions which requiring binary classification (Keylabs, 2024).

4.6.3.2 Base Model Building

```python
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.preprocessing import StandardScaler

# Optionally scale features for Logistic Regression
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Logistic Regression Model
log_reg_model = LogisticRegression(max_iter=200)
log_reg_model.fit(X_train_scaled, y_train)

# Predict and evaluate
y_pred_log_reg = log_reg_model.predict(X_test_scaled)
accuracy = accuracy_score(y_test, y_pred_log_reg)
confusion = confusion_matrix(y_test, y_pred_log_reg)
report = classification_report(y_test, y_pred_log_reg)

# Print results
print(f"Confusion Matrix:\n {confusion}")
print(f"\n📊 Classification Report:\n{report}")
print(f"✅ Accuracy: {accuracy:.4f}")
```

Figure 84: Code of building Logistic Regression (Base Model)

The figure above shows the source code of building the base model of Logistic Regression and evaluate its performance on test set by using scikit-learn. Some necessary libraries are imported which include Logistic Regression for the mode building, accuracy_score, classification_report, and confusion_matrix for model evaluation, and StandardScaler for scaling the features. Then, the code starts with scaling the training and test data using StandardScaler to ensure the features are standardized. A Logistic Regression model is then trained on the scaled training data, and predictions are made on the test data. The performance

of the model is evaluated using accuracy, confusion matrix, and classification report, which are printed at the end. These evaluations show how well the model predicts outcomes and the quality of its classifications. The output will be discussed and compared with other machine learning models in Chapter 5.

4.6.3.3 Hyperparameter Tuning

```
from sklearn.model_selection import GridSearchCV

# Logistic Regression Model
logreg = LogisticRegression()

param_grid_logreg = {
    'C': [0.01, 0.1, 1, 10, 100],
    'solver': ['liblinear', 'saga'],
    'penalty': ['l1', 'l2'],
    'max_iter': [50, 100, 200],
}

search_logreg = GridSearchCV(logreg, param_grid_logreg, cv=5, n_jobs=-1)

# Fit the model
search_logreg.fit(X_train_scaled, y_train)

# Predict using the best model
best_logreg = search_logreg.best_estimator_
y_pred_logreg = best_logreg.predict(X_test_scaled)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred_logreg)
confusion = confusion_matrix(y_test, y_pred_logreg)
report = classification_report(y_test, y_pred_logreg)

# Display results
print(f"Confusion Matrix (After Tuning):\n {confusion}")
print(f"\n📊Classification Report:\n{report}")

# Best parameters and best score
print("✅Best Hyperparameters for LOGISTIC REGRESSION: ")
for param, value in search_logreg.best_params_.items():
    print(f"  {param.capitalize()}: {value}")

print(f"\nBest Cross-Validation Score: {search_logreg.best_score_:.4f}")
print(f"Accuracy (After Tuning): {accuracy:.4f}")
```

Figure 85: Code of Hyperparameter Tuning (Logistic Regression)

The figure above is the code of using GridSearchCV (Grid Search Cross-Validation) from scikit-learn to perform hyperparameter tuning for Logistic Regression model and evaluate its performance. A parameter grid (param_grid_logreg) is created to tune the key hyperparameters, such as **C** (regularization strength), **solver (liblinear or saga)** , **penalty (L1 or L2)**, and **max_iter** which is the maximum number of iterations for optimization process.

By using GridSearchCV for hyperparameter tuning, it involves systematically searching over these predefined parameters with 5-fold cross validation and fitting the model on the scaled training data. Once the best hyperparameters are found, the model is used to make predictions on the scaled test data. The best hyperparameter and best cross-validation score will be listed out in the output together with the model performance after hyperparameter tuning such as accuracy, confusion matrix, and classification report with precision, recall, and f1-socre. This process helps improve the performance of Logistic Regression by selecting the optimal set of hyperparameters for the dataset. The output of the model evaluation after hyperparameter tuning will be discussed in Chapter 5.

4.6.3.4 Best Hyperparameter

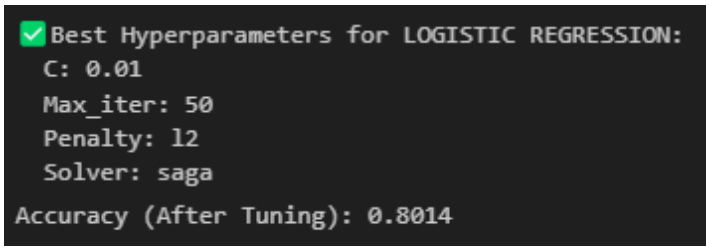


Figure 86: Output of Best Hyperparameter (Logistic Regression)

This figure displays the results of hyperparameter tuning foe Logistic Regression model by using GridSearchCV. The best hyperparameter found after tuning include a **regularization strength (C) of 0.01**, a **maximum iteration count (max_iter) of 50**, a **penalty of L2**, and the **saga solver**, which is particularly efficient for large datasets. After applying these tuned parameters, the model achieved an **accuracy** of **0.8014 (80.14%)** on the test data. This output highlights the model's improved performance after the hyperparameter tuning process.

4.6.4 Random Forest (RF)

4.6.4.1 Advantage of Random Forest

- **Handling imbalance data and missing value**

  Random Forest can handle dataset which having class imbalance and can estimate missing values effectively by using surrogate splits during tree building. This capability ensures that the model remains robust even when some data points are incomplete, which is common in real-world clinical datasets.

- **High accuracy and Robustness**

  Random Forest provide high predictive accuracy by combining many decision trees which reducing variance and improving stability compared to single decision trees. This combined strategy reduces overfitting risk and increases adaptation to previously unknown data (Coursera, 2024).

4.6.4.2 Base Model Building

```
from sklearn.ensemble import RandomForestClassifier

# Random Forest Model
rf_model = RandomForestClassifier(random_state=42)
rf_model.fit(X_train, y_train)

# Predict and evaluate
y_pred_rf = rf_model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred_rf)
confusion = confusion_matrix(y_test, y_pred_rf)
report = classification_report(y_test, y_pred_rf)

# Print results
print(f"Confusion Matrix:\n {confusion}")
print(f"\n📊 Classification Report:\n{report}")
print(f"✅ Accuracy: {accuracy:.4f}")
```

Figure 87: Code of building Random Forest (Base Model)

The figure above shows the source code of using scikit-learn to build and train a Random Forest classifier and evaluate its performance. The Random Forest Classifier is imported from sklearn.ensemble library. This model is trained on the training data, which is X_train and y_train, and evaluated on testing data (X_test). It calculates accuracy, generates a confusion matrix, and produces a classification report with metrics like precision and recall for model evaluations. The output of these evaluations will be discussed and compared with other machine learning models in Chapter 5.

4.6.4.3 *Hyperparameter Tuning*

```python
rf = RandomForestClassifier()

param_grid_rf = {
    'n_estimators': [50, 100, 200, 300],
    'max_depth': [10, 20, 30, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'bootstrap': [True, False],
}

# Perform GridSearchCV
search_rf = GridSearchCV(rf, param_grid_rf, cv=5, n_jobs=-1)

# Fit the model
search_rf.fit(X_train, y_train)

# Predict using the best model
best_rf = search_rf.best_estimator_
y_pred_rf = best_rf.predict(X_test)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred_rf)
confusion = confusion_matrix(y_test, y_pred_rf)
report = classification_report(y_test, y_pred_rf)

# Display results
print(f"confusion_matrix (After Tuning):\n {confusion}")
print(f"\n📊Classification Report:\n{report}")

print("✅Best Hyperparameters for RANDOM FOREST: ")
for param, value in search_rf.best_params_.items():
    print(f"  {param.capitalize()}: {value}")

print(f"\nBest Cross-Validation Score: {search_rf.best_score_:.4f}")
print(f"Accuracy (After Tuning): {accuracy:.4f}")
```

Figure 88: Code of Hyperparameter Tuning (Random Forest)

The figure above shows the code of hyperparameter tuning for Random Forest. A GridSearchCV is used to optimize the hyperparameters of Random Forest classifier and evaluate its performance. The model is set up with a specified hyperparameter grid that includes options for **n_estimators** (number of trees), **max_depth** (maximum depth of trees), **min_samples_split** (minimum number of samples required to split an internal node), **min_samples_leaf** (minimum number of samples required to be at a leaf node), and **bootstrap** (whether to use bootstrap sampling for trees).

GridSearchCV performs an exhaustive search over this grid using 5-fold cross-validation to find the best combination of hyperparameters. The model is then trained on the scaled training data, and predictions are made on the test data. The best hyperparameter will and the best cross-validation score will be listed out in the output with the model performance. The model performance after hyperparameter tuning is evaluated using accuracy, confusion matrix and classification report. These evaluations will be discussed in Chapter 5. This process provides a comprehensive view of the model's performance and improvements.

4.6.4.4 Best Hyperparameter

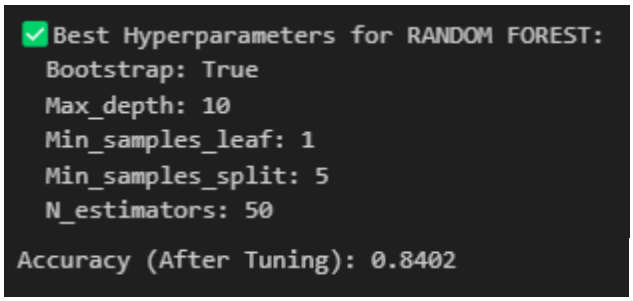


Figure 89: Output of Best Hyperparameter (Random Forest)

This figure shows the result of hyperparameter tuning for Random Forest model with the best hyperparameter after optimization. The best hyperparameter for Random Forest for **Booststrap** is set to Ture, meaning bootstrap sampling is used when building the trees. Then, the **Max_depth** is 10 which is limiting the maximum depth of each tree. The **Min_samples_leaf** is set to 1, meaning that each leaf node must have at least one sample and the **Min_samples_split** is 5, meaning a node will only be split if it has at least five samples. Lastly, the **N_estimators** are 50 specifying that the model will use 50 trees in the forest. With these parameters, the model achieved an **accuracy of 0.8402**, or **84.02%,** on the test data, reflecting an improvement in performance after the hyperparameter tuning process.

4.6.5 Support Vector Machine (SVM)

4.6.5.1 Advantage of SVM

- **Effective in High-Dimensional Space**

  SVM perform well in high-dimensional space. For example, when the number of features exceeds the number of samples, it avoids the curse of dimensionality by mapping data into high-dimensional space using kernel functions (Guido et al., 2024).

- **Applicability to classification and regression**

  SVM can handle with both linear and non-linear classification task by using the kernel functions. It can use for both type of problems , extending its value across other areas. This versatility allows SVMs to model complex relationships in data, such as in the classification of non-linearly separable datasets (Wang et al., 2017).

4.6.5.2 Base Model Building

```
from sklearn.svm import SVC

# SVM Model (use scaled data)
svm_model = SVC(kernel="linear", probability=True)
svm_model.fit(X_train_scaled, y_train)

# Predict and evaluate
y_pred_svm = svm_model.predict(X_test_scaled)
accuracy = accuracy_score(y_test, y_pred_svm)
confusion = confusion_matrix(y_test, y_pred_svm)
report = classification_report(y_test, y_pred_svm)

# Print results
print(f"Confusion Matrix:\n {confusion}")
print(f"\n📊 Classification Report:\n{report}")
print(f"✅ Accuracy: {accuracy:.4f}")
✓ 4m 37.5s
```

Figure 90: Code of building SVM (Base Model)

The figure above shows the source code of using scikit-learn to train and evaluate the SVM model. The SVM model is initialized with a linear kernel and the probability=True parameter to enable probability estimates. The model is then trained by using the scaled training data (X_train_scaled) and corresponding target values (y_train). After training, the predictions are made on the scaled test data (X_test_scaled). The performance of SVM will be evaluated by calculating its accuracy, generating a confusion matrix and creating classification reports with metrics like precision, recall, and f1-score. The output of these evaluations will be discussed and compared with other models in Chapter 5.

4.6.5.3 *Hyperparameter Tuning*

```
# SVM Model
svm =  SVC(probability=True)

param_grid_svm = {
    'C': [0.1, 1, 10],
    'kernel': ['linear', 'rbf'],
    'gamma': ['scale', 'auto', 0.1, 1],
}

search_svm = RandomizedSearchCV(svm, param_grid_svm, n_iter=10,
                                cv=3, n_jobs=-1, random_state=42)

# Fit the model
search_svm.fit(X_train_scaled, y_train)

# Predict using the best model
best_svm = search_svm.best_estimator_
y_pred_svm = best_svm.predict(X_test_scaled)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred_svm)
confusion = confusion_matrix(y_test, y_pred_svm)
report = classification_report(y_test, y_pred_svm)

# Display results
print(f"Confusion Matrix (After Tuning):\n {confusion}")
print(f"\n📊Classification Report:\n{report}")

print("✅Best Hyperparameters for SVM : ")
for param, value in search_svm.best_params_.items():
    print(f"  {param.capitalize()}: {value}")

print(f"\nBest Cross-Validation Score: {search_svm.best_score_:.4f}")
print(f"Accuracy (After Tuning): {accuracy:.4f}")
```

Figure 91:Code of Hyperparameter Tuning (SVM)

The figure above shows the hyperparameter tuning for a Support Vector Machine (SVM) model using RandomizedSearchCV. The model is configured with probability=True to enable probability estimates. A hyperparameter grid is defined, which includes value of **C** with options [0.1, 1, and 10], **kernel** which can either be [linear or rbf], and **gamma** with possible value of [scale, auto, 0.1, and 1]. The RandomizedSearchCV is a function that searches randomly through these grids. It performs with 10 iterations (n_iter =10) and 3-fold cross-validation (cv=3) to identify the best combination of hyperparameters. After fitting the model with the best parameters, predictions are made on the scaled test data and the best hyperparameter will be listed out in the output with the model evaluation after hyperparameter tuning. The model evaluation with accuracy, confusion matrix and classification report will be discussed in Chapter 5.

4.6.5.4 Best Hyperparameter

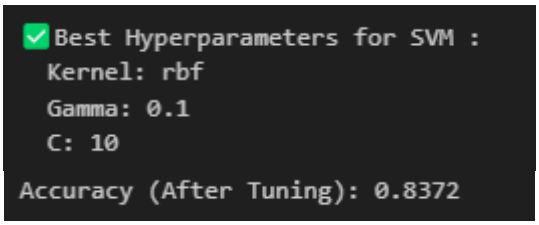


Figure 92: Output of Best Hyperparameter (SVM)

This figure shows the result of hyperparameter tuning for SVM with the best hyperparameter and accuracy after tuning. As the output show that the best combination of hyperparameters received through the tuning process includes the use of the **RBF kernel**, a **gamma** value of 0.1, and a **C** value of 10. These parameters were selected as they provided the most effective balance between bias and variance during the training process. With this best configuration, the model achieved an **accuracy of 0.8372**, which is also 83.72% of the predictions made on the test data were correct. This shows an improvement in the model's performance after tuning.

4.6.6 XGBoost (XGB)

4.6.6.1 Advantage of XGBoost

- **High predictive accuracy**

  XGBoost often outperforms other classic algorithms such as logistic regression, SVM, and random forest in terms of precision, recall, and total accuracy. It usually achieving accuracy rates of more than 90% in different fields including sales forecasting and risk warning (Chen, 2023).

- **Fast training speed and parallelization**

  XGBoost allows parallel processing, which greatly speed up the training time compared to the traditional gradient boosting methods. Therefore, it can handle large datasets and complex method with high efficiency (Tarwidi et al., 2023).

- **Flexibility and tunability**

  XGBoost provides multiple hyperparameters for controlling model complexity and performance, which allows fine-tuning transformed to specific datasets and applications (Wiens et al., 2025).

4.6.6.2 Base Model Building

```
from xgboost import XGBClassifier

# XGBoost Model
xgboost_model = XGBClassifier(eval_metric="mlogloss", random_state=42)
xgboost_model.fit(X_train, y_train)

# Predict and evaluate
y_pred_xgboost = xgboost_model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred_xgboost)
confusion = confusion_matrix(y_test, y_pred_xgboost)
report = classification_report(y_test, y_pred_xgboost)

# Print results
print(f"Confusion Matrix:\n {confusion}")
print(f"\n📊 Classification Report:\n{report}")
print(f"✅ Accuracy: {accuracy:.4f}")
```

Figure 93: Code of building XGBoost (Base Model)

The figure above shows the source code for building the base model of XGBoost. XGBoost is a powerful gradient boosting algorithm to train and evaluate a classification model. The model is initialized using the XGBClassifier from the **xgboost** library with the evaluation metric set to **"mlogloss"** (logarithmic loss), and a fixed the random_state=42 to ensure reproducibility. The classifier is trained on the scaled training data (X_train) and corresponding target values (y_train) using the fit() method. After training the model, it will

predict the outcomes for the scaled test data (X_test). The performance of XGBoost will be evaluated by accuracy, confusion matrix and the classification reports. The output of XGBoost evaluation will be discussed and compared with other models in Chapter 5.

4.6.6.3 Hyperparameter Tuning

```python
from sklearn.model_selection import RandomizedSearchCV

# XGBoost Model
xgb = XGBClassifier()

param_grid_xgb = {
    'n_estimators': [50, 100, 200, 300],
    'max_depth': [3, 6, 10, 15],
    'learning_rate': [0.01, 0.1, 0.2, 0.5],
    'subsample': [0.6, 0.8, 1.0],
    'colsample_bytree': [0.6, 0.8, 1.0],
    'gamma': [0, 0.1, 0.2, 0.5],
}

# Perform RandomizedSearchCV
search_xgb = RandomizedSearchCV(xgb, param_grid_xgb, n_iter=10,
                                cv=5, n_jobs=-1, random_state=42)

# Fit the model
search_xgb.fit(X_train, y_train)

# Predict using the best model
best_xgb = search_xgb.best_estimator_
y_pred_xgb = best_xgb.predict(X_test)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred_xgb)
confusion = confusion_matrix(y_test, y_pred_xgb)
report = classification_report(y_test, y_pred_xgb)

# Display results
print(f"Confusion Matrix (After Tuning):\n{confusion}")
print(f"\n📊 Classification Report:\n{report}")

print("✅ Best Hyperparameters for XGBoost: ")
for param, value in search_xgb.best_params_.items():
    print(f"  {param.capitalize()}: {value}")

print(f"\nBest Cross-Validation Score: {search_xgb.best_score_:.4f}")
print(f"Accuracy (After Tuning): {accuracy:.4f}")
```

Figure 94: Code of Hyperparameter Tuning (XGBoost)

The figure above shows the code of hyperparameter tuning for XGBoost by using RandomizedSearchCV. The XGBClassifier has been set up and a complete parameter grid is defined, including the values for the number of estimators **(n_estimators),** tree depth **(max_depth), learning rate**, **subsample ratio**, column sampling ratio **(colsample_bytree)**, and **gamma** (minimum loss reduction for further partitioning). The randomized search is set to run 10 iterations (n_iter=10) with 5-fold cross-validation (cv=5) to explore the hyperparameter space efficiently. After fitting the model on the training data, the best

combination of hyperparameters is selected, and predictions are made on the test set. The best hyperparameters for XGBoost will be listed out in the result together with the model evaluation such as accuracy, confusion matrix and classification report. The model evaluation will be discussed in Chapter 5 with other models.

4.6.6.4 Best Hyperparameter

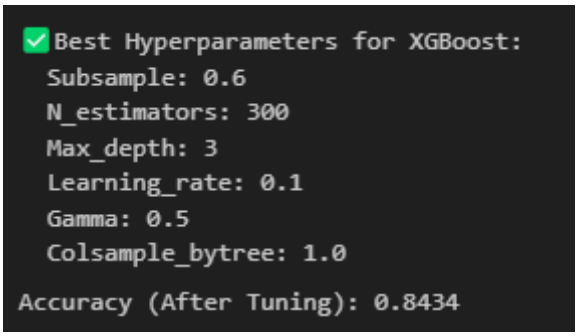


Figure 95: Output of Best Hyperparameter (XGBoost)

This figure shows the output of best hyperparameter for XGBoost model. The best hyperparameter that founded through the tuning process include a **subsample** value of 0.6, meaning that 60% of the training data is used for each tree to help prevent overfitting. The model uses **300 trees** (**n_estimators**), and each tree has a **maximum depth** of 3. The **learning rate** is set to 0.1, controlling the contribution of each individual tree, while the **gamma** value is 0.5, representing the minimum loss reduction required to make further splits. Lastly, the **colsample_bytree** is set to 1.0, meaning all features are used for constructing each tree. With these tuned parameters, the model achieved an accuracy of **84.34%** on the test data, reflecting an improvement in performance after hyperparameter optimization.

4.7 Summary

In this chapter, the design and implementation of the bullying prediction model are thoroughly discussed. The project relies on the Bullying_2018 dataset from Kaggle, which is carefully cleaned and pre-processed to ensure accurate analysis. The dataset undergoes several stages of handling missing values, inconsistent entries, and outliers, making it ready for the machine learning phase. Through the application of models like Logistic Regression, Random Forest, SVM, and XGBoost, the project aims to identify which model performs best in predicting bullying risk. These models are evaluated using various metrics such as accuracy, precision, recall, and F1-score. Hyperparameter tuning further enhances the performance of each model. By the end of this chapter, the data is fully prepared for modeling, and the implementation details have been outlined, setting the stage for a comprehensive evaluation of the model performance in the following chapter.

CHAPTER 5: RESULT AND DISCUSSION

5.1 Introduction

Chapter 5 focuses on the results and discussions of the various machine learning models used to predict bullying victimization risk among adolescent students. In this chapter, we evaluate the performance of four machine learning models: Logistic Regression (LR), Random Forest (RF), Support Vector Machine (SVM), and XGBoost. We analyze how well these models perform in predicting whether a student has been bullied or not, using different metrics such as accuracy, precision, recall, F1-score, and ROC-AUC. Each model is assessed both before and after tuning the hyperparameters to see how much improvement can be made. The chapter also explores the importance of various features, such as "Felt_lonely" and "Missed_school," to understand what factors are most influential in predicting bullying. Finally, the best-performing model, XGBoost, is saved and deployed into a real-world application where it can be used for bullying risk prediction. This chapter provides a thorough overview of the model performance, highlighting which models are most effective and how they can be used to support early intervention in schools.

5.2 Model Evaluations and Discussions

5.2.1 Logistic Regression (LR)

5.2.1.1 LR Base Model Result

```
Confusion Matrix:
 [[3749 1178]
 [1095 5375]]

📊 Classification Report:
              precision    recall  f1-score   support

           0       0.77      0.76      0.77      4927
           1       0.82      0.83      0.83      6470

    accuracy                           0.80     11397
   macro avg       0.80      0.80      0.80     11397
weighted avg       0.80      0.80      0.80     11397

✅ Accuracy: 0.8006
```

Figure 96: Output of Base Model (Logistic Regression)

The figure above shows the performance evaluation of the base Logistic Regression model for predicting bullying victimization risk among adolescent students. Firstly, the **confusion matrix** shows that Logistic Regression accurately predicted 3749 cases of students does not being bullied (Ture Negative) and 5375 cases of student have been bullied (True Positive). However, the model also has misclassified 1178 of student does not being bullied cases as student have been bullied (False Positives) and 1095 as student have been bullied cases as student does not being bullied (False Negatives).

Then, the **classification report** also shows that the model had a **precision** of 0.77 for student does not being bullied (Class 0) and 0.82 for students have been bullied (Class 1), which represents the proportion of correct positive predictions for each class. The **recall** is 0.76 for student does not being bullied (Class 0) and 0.83 for students who have been bullied (Class 1), showing that this model can accurately identify each class. The **f1-score** for student does not being bullied (Class 0) is 0.77 while for the students who have been bullied (Class 1) is 0.83. This has balanced precision and recall for both classes.

The overall **accuracy** of the model is 80.06%, with both the macro average and weighted average for precision, recall, and F1-score all being 0.80, highlighting a balanced performance across the two classes.

5.2.1.2 LR Learning Curve

A learning curve is a graph that shows how a machine learning model's performance improves as it is trained with more data or over more iterations. It typically plots the amount of training data or time on the x-axis and a measure of accuracy or error on the y-axis. Learning curves help diagnose whether a model is underfitting or overfitting and indicate if adding more data could improve results. They are useful tools for monitoring training progress and guiding model tuning to achieve better performance.

```
from sklearn.model_selection import learning_curve

# Base model (Logistic Regression) learning curve
train_sizes, train_scores, test_scores = learning_curve(
    log_reg_model,
    X_train_scaled,
    y_train,
    cv=5,
    n_jobs=-1,
    train_sizes=[0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1]
)

# Calculate mean and std for train and test scores
train_mean = train_scores.mean(axis=1)
test_mean = test_scores.mean(axis=1)
train_std = train_scores.std(axis=1)
test_std = test_scores.std(axis=1)

# Plot Learning Curve
plt.figure(figsize=(10, 6))
plt.plot(train_sizes, train_mean, label=f'Training Score', linestyle='-', marker='o')
plt.plot(train_sizes, test_mean, label=f'Cross-Validation Score', linestyle='--', marker='x')
plt.fill_between(train_sizes, train_mean - train_std, train_mean + train_std, alpha=0.1)
plt.fill_between(train_sizes, test_mean - test_std, test_mean + test_std, alpha=0.1)
plt.title('Learning Curve - Logistic Regression (Base Model)')
plt.xlabel('Training Set Size')
plt.ylabel('F1 Score')
plt.legend(loc='best')
plt.grid(True)
plt.show()
```

Figure 97: Code of Learning Curve (Logistic Regression)

Figure above shows the code of learning curve for Logistic Regression model. The learning_curve function is used to compute the training and cross-validation scores for various training set sizes, ranging from 10% to 100% of the dataset. This allows for a detailed evaluation of the model's performance as more data is introduced. The training scores and test scores are calculated for different training set sizes and stored in train_scores and test_scores, respectively.

The code then calculates the mean and standard deviation of the training and test scores, which is useful for understanding the variability in model performance. The learning curve is plotted

with the training scores and cross-validation scores, using filled areas to show the standard deviation, providing a visual representation of the model's performance and its generalization ability.

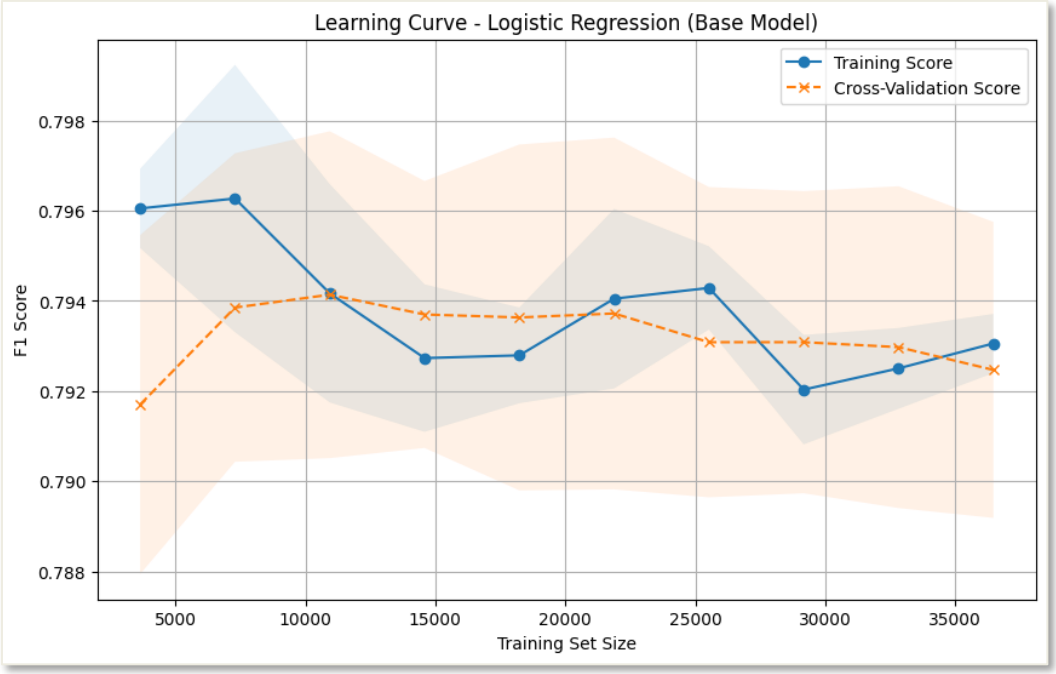


Figure 98: Visualization of Learning Curve(Logistic Regression)

The figure above shows the learning curve for the base Logistic Regression model. It describes how the F1 score changes as the training set size increases. The blue solid line represents the training score, while the orange dashed line represents the cross-validation score. The shaded area around each line indicates the standard deviation, providing a sense of variability in performance.

As the training set size increases, the training score starts high (f1-score = 0.796) but decreases slightly, which is typical in many models as the model is exposed to more data. However, the cross-validation score has shown more fluctuation, which means that the model is less consistent when generalizing to unseen data. However, both scores stabilize as the training set size continues to grow. The model seems to achieve a **relatively stable F1 score**, with the **training score** staying higher than the **cross-validation score**, suggesting that the model may be overfitting with smaller datasets. However, as more data is used, the gap between the two scores narrows, implying better generalization with larger training sets.

5.2.1.3 LR ROC-AUC

```
from sklearn.metrics import roc_curve, auc

# Predict probabilities for ROC Curve
y_prob_logreg = log_reg_model.predict_proba(X_test_scaled)[:, 1]

# Compute ROC curve
fpr, tpr, thresholds = roc_curve(y_test, y_prob_logreg)
roc_auc = auc(fpr, tpr)

# Plot ROC-AUC Curve
plt.figure(figsize=(10, 6))
plt.plot(fpr, tpr, lw=2, label='Logistic Regression (AUC = %0.4f)' % roc_auc)
plt.plot([0, 1], [0, 1], color='gray', linestyle='--')
plt.title('ROC Curve - Logistic Regression (Base Model)')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend(loc="lower right")
plt.grid(True)
plt.show()
```

Figure 99: Code of ROC-AUC (Logistic Regression)

The figure above shows the code of generating ROC curve and calculating the AUC (Area under the curve) for the Logistic Regression base model. The ROC curve is used to evaluate the performance of the model by plotting the True Positive Rate (TPR) against the False Positive Rate (FPR) at various classification thresholds.

The model first predicts the probabilities for each class using the predict_proba method on the test set (X_test_scaled). Then, the roc_curve function computes the FPR, TPR, and thresholds. The AUC is calculated using the auc function, which summarizes the area under the ROC curve as a single number, representing the model's ability to distinguish between the classes. The AUC score is a key measure of the model's classification performance, with values closer to 1 indicating better performance.

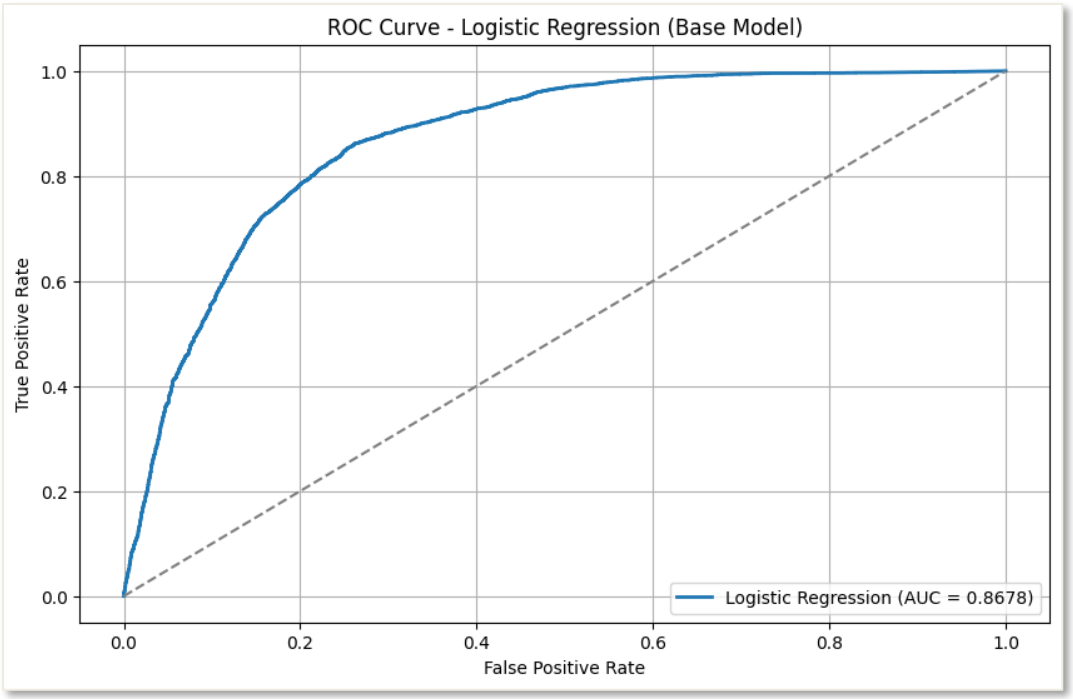


Figure 100: Visualization of ROC Curve (Logistic Regression)

This figure shows the ROC curve for Logistic Regression base model. The blue curve represents the model's performance, plotting the True Positive Rate (TPR) on the y-axis against the False Positive Rate (FPR) on the x-axis. The dashed gray line represents a random classifier, which serves as the baseline for comparison. The AUC (Area Under the Curve) score is **0.8678**, which indicates that the model has good performance. An AUC value closer to 1 suggests that the model is able to distinguish between the positive and negative classes effectively. In this case, the curve rises steeply, showing that the model performs well with relatively few false positives, and the AUC of 0.8678 indicates a strong ability to classify instances correctly.

5.2.1.4 LR Tuned Model Result

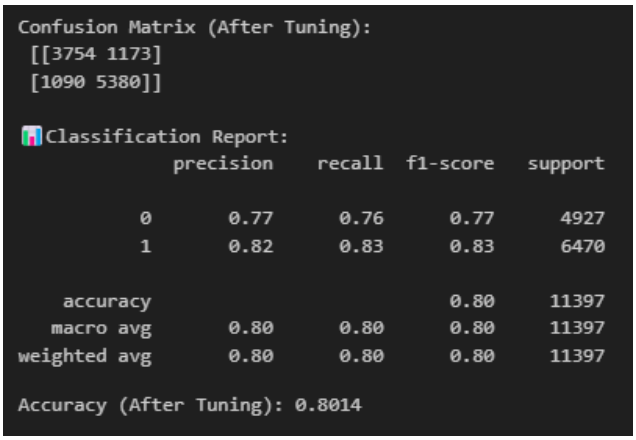


Figure 101: Output of Hyperparameter Tuning (Logistic Regression)

The output shows the performance of the Logistic Regression model after hyperparameter tuning. The **confusion matrix** shows that the model correctly predicted 3754 cases non-victimization (True Negatives) and 5380 cases victimization (True Positive), with some misclassifications.

The **classification report** shows that Class 0 (non-victimization) had a **precision** of 0.77 and recall of 0.76, while Class 1 (victimization) had better results with a precision of 0.82 and **recall** of 0.83. The overall **accuracy** is **0.8014**, and both the macro average and weighted average F1-scores are 0.80, indicating balanced performance across classes.

Model Type	Accuracy	Precision (Class 0)	Precision (Class 1)	Recall (Class 0)	Recall (Class 1)	F1-Score (Class 1)
LR (Base Model)	0.8006	0.77	0.82	0.76	0.83	0.83
LR (Tuned Model)	0.8014	0.77	0.82	0.76	0.83	0.83

Table 6: Comparison of Base and Tuned Model (Logistic Regression)

The table compares the performance of the Base Model and the Tuned Model for Logistic Regression (LR) across several evaluation metrics. The **accuracy** of the Base Model is 0.8006, while the Tuned Model shows a slight improvement with an accuracy of 0.8014. Both models have identical **precision** values for Class 0 (0.77) and Class 1 (0.82), suggesting that tuning did not affect precision for either class. Same to the **recall**, the Class 0 for recall remains 0.76, and Class 1 recall stays at 0.83 for both models, indicating no change in how well the models identify each class.

The **F1-score for class 1** is consistent at 0.83 for both models. Although the performance metrics are almost the same, the small increase in accuracy suggests that tuning helped the model make slightly better predictions overall, even if the improvement is minimal. This shows that while tuning did not dramatically change the results, it helped to slightly optimize the model's performance.

5.2.1.5 LR Learning Curve (After Tune)

```
# After Hyperparameter Tuning Model (Best model from GridSearchCV)
train_sizes, train_scores, test_scores = learning_curve(
    best_logreg,
    X_train_scaled,
    y_train,
    cv=5,
    n_jobs=-1,
    train_sizes=[0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1]
)

# Calculate mean and std for train and test scores
train_mean = train_scores.mean(axis=1)
test_mean = test_scores.mean(axis=1)
train_std = train_scores.std(axis=1)
test_std = test_scores.std(axis=1)

# Plot Learning Curve for tuned model
plt.figure(figsize=(10, 6))
plt.plot(train_sizes, train_mean, label=f'Training Score', linestyle='-', marker='o')
plt.plot(train_sizes, test_mean, label=f'Cross-Validation Score', linestyle='--', marker='x')
plt.fill_between(train_sizes, train_mean - train_std, train_mean + train_std, alpha=0.1)
plt.fill_between(train_sizes, test_mean - test_std, test_mean + test_std, alpha=0.1)
plt.title('Learning Curve - Logistic Regression (After Tune)')
plt.xlabel('Training Set Size')
plt.ylabel('F1 Score')
plt.legend(loc='best')
plt.grid(True)
plt.show()
```

Figure 102: Code of Learning Curve – After Tune (Logistic Regression)

This figure shows the code of learning curve after hyperparameter tuning for Logistic Regression model. This is the same process with the learning curve for the base model, the only changes is changing the variable from base model (log_reg_model) to the best tuned model which is represented by (best_logreg) in the code.

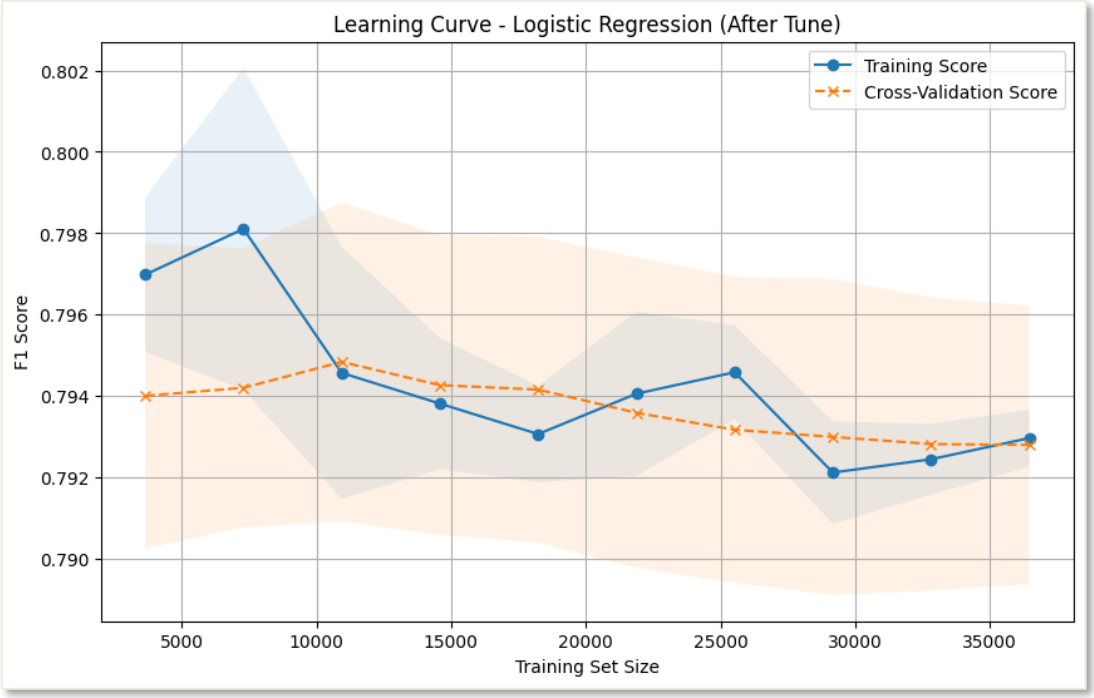


Figure 103: Visualization of Learning Curve – After Tune (Logistic Regression)

The learning curve for Logistic Regression (After Tune) shows how the model performs with different amounts of data. The training score (blue line) starts high but decreases a bit as the training data size increases. However, it stabilizes around 0.798, meaning the performance of model on the training data becomes steady after a certain point.

The cross-validation score (orange line) is always lower than the training score, starting with some fluctuations but also stabilizing at about 0.792. This shows that the model might be overfitting, meaning that it is doing better on the training data than on new, unseen data.

Both scores stabilize after a certain amount of training data, showing that adding more data doesn't improve performance much after that point. Overall, the model has a balanced F1-score between 0.79 and 0.80, which means it is performing well in terms of both precision and recall. In conclusion, the learning curve shows that the model has achieved its best performance and adding more data won't make a big difference.

5.2.1.6 LR ROC-AUC (After Tune)

```
# Predict probabilities for ROC Curve (tuned model)
y_prob_best_logreg = best_logreg.predict_proba(X_test_scaled)[:, 1]

# Compute ROC curve for the tuned model
fpr_best, tpr_best, thresholds_best = roc_curve(y_test, y_prob_best_logreg)
roc_auc_best = auc(fpr_best, tpr_best)

# Plot ROC-AUC Curve for tuned model
plt.figure(figsize=(10, 6))
plt.plot(fpr_best, tpr_best, lw=2, label='Logistic Regression (Tuned, AUC = %0.4f)' % roc_auc_best)
plt.plot([0, 1], [0, 1], color='gray', linestyle='--')
plt.title('ROC Curve - Logistic Regression (After tune)')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend(loc="lower right")
plt.grid(True)
plt.show()
```

Figure 104: Code of ROC-AUC -– After Tune (Logistic Regression)

This figure shows the code of ROC-AUC curves after hyperparameter tuning for Logistic Regression model. This is the same process with the ROC-AUC curve for the base model, the only change is the variable from base model (log_reg_model.predict_proba), which is to calculate the AUC (Area Under Curve) for the base model to the best tuned model which is represented by (best_logreg.predict_proba) in this code. This variable is to calculate the AUC for the tuned model.

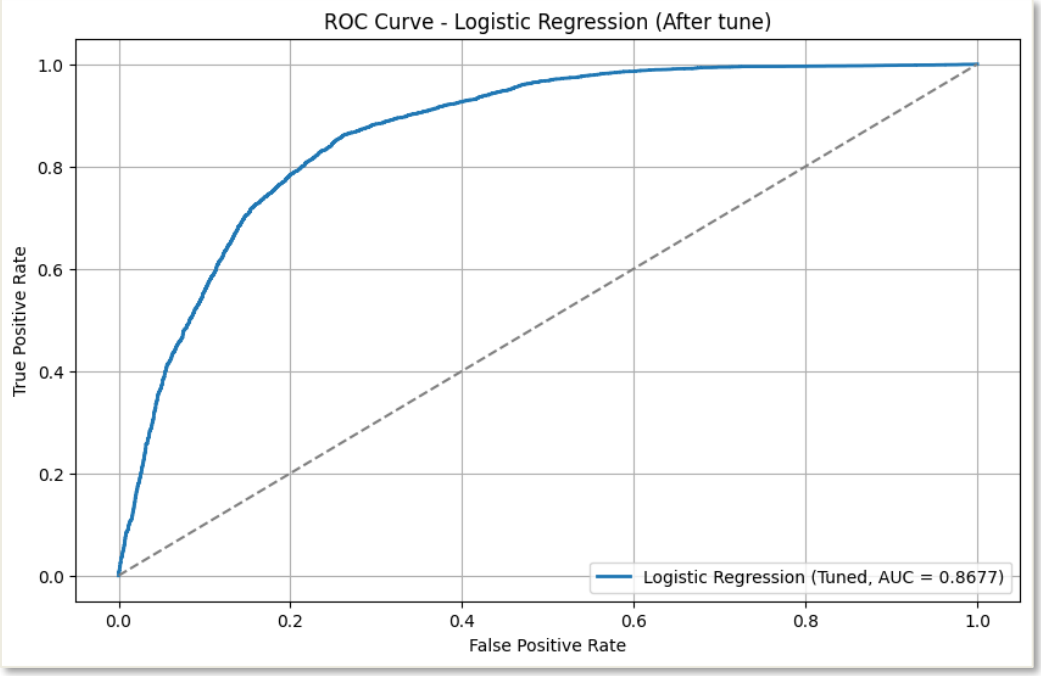


Figure 105: Visualization of ROC Curve – After Tune (Logistic Regression)

The figure above shows the ROC Curve and AUC score for Logistic Regression after hyperparameter tuning. It shows how well the model distinguishes between the two classes (0 and 1) as the decision threshold varies. The curve is steep and moves close to the top-left corner, which indicates that the model has a high true positive rate (sensitivity) and a low false positive rate. The dashed grey diagonal line represents the performance of a random classifier, so the further the ROC curve is from this line, the better the model performs. In this case, the **area under the curve (AUC) is 0.8677**, which is a good score, suggesting that the tuned model is effective in classifying the data. However, there is still room for improvement as the model could potentially achieve a higher AUC score.

5.2.2 Random Forest (RF)

5.2.2.1 RF Base Model Result

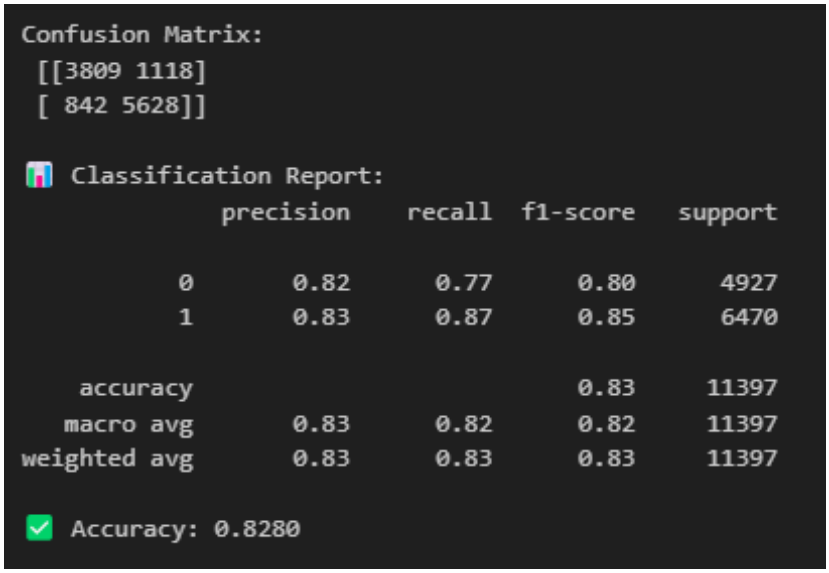


Figure 106: Output of Base Model (Random Forest)

The figure above shows the performance evaluation of the base Random Forest model for predicting bullying victimization risk among adolescent students. Firstly, the **confusion matrix** show that Random Forest have predicted correctly on 3809 cases of non-victimization (True Negatives) and 5628 cases of victimization (True Positives). However, it has misclassified 1118 non-victimized cases as victimized (False Positives) and 842 victimized cases as non-victimized (False Negatives).

For the **classification report**, it shows that the model has achieved a **precision** of 0.82 for non-victimization (Class 0) and 0.83 for victimization (Class 1). This represents the proportion of valid positive representing the proportion of valid positive predictions for each class. Besides, the **recall** is 0.77 for non-victimization (Class 0) and 0.87 for victimization (Class 1), which highlights the ability of Random Forest to accurately identify each class. Then, the **f1-score** for non-victimization (Class 0) is 0.80, while for the victims (Class 1) is 0.85, which balancing the precision and recall.

The overall accuracy for Random Forest is 82.80%, with both the **macro average** and **weighted average** for precision, recall, and F1-score being 0.83, reflecting a well-balanced model performance across both classes.

5.2.2.2 RF Learning Curve

```
# Random Forest Base Model Learning Curve
train_sizes, train_scores, test_scores = learning_curve(
    rf_model,
    X_train,
    y_train,
    cv=5,
    n_jobs=-1,
    train_sizes=[0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1]
)

# Calculate mean and std for train and test scores
train_mean = train_scores.mean(axis=1)
test_mean = test_scores.mean(axis=1)
train_std = train_scores.std(axis=1)
test_std = test_scores.std(axis=1)

# Plot Learning Curve
plt.figure(figsize=(10, 6))
plt.plot(train_sizes, train_mean, label=f'Training Score', linestyle='-', marker='o')
plt.plot(train_sizes, test_mean, label=f'Cross-Validation Score', linestyle='--', marker='x')
plt.fill_between(train_sizes, train_mean - train_std, train_mean + train_std, alpha=0.1)
plt.fill_between(train_sizes, test_mean - test_std, test_mean + test_std, alpha=0.1)
plt.title('Learning Curve - Random Forest (Base Model)')
plt.xlabel('Training Set Size')
plt.ylabel('F1 Score')
plt.legend(loc='best')
plt.grid(True)
plt.show()
```

Figure 107: Code of Learning Curve (Random Forest)

This figure shows the code of learning curve for a Random Forest base model to evaluate how the model performs as the training set size increases. The function learning_curve is used to calculate the training and test scores for various training set sizes ranging from 10% to 100% of the data. This allows the evaluation of the model's performance as more data is added.

The mean and standard deviation of both the training and test scores are computed for each training set size to provide a better understanding of the variability and performance. The learning curve is then plotted, where the training score is shown by blue circles, and the cross-validation score is shown with orange crosses. This learning curve helps assess whether the model is underfitting or overfitting, providing insights into its generalization capability as more data is used for training.

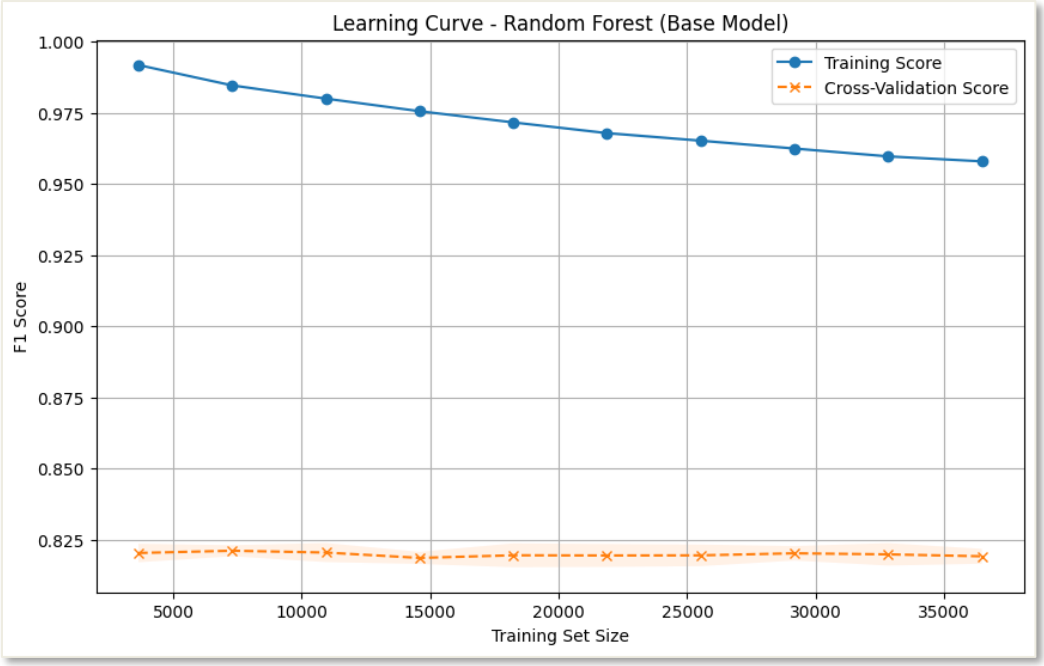


Figure 108: Visualization of Learning Curve (Random Forest)

The figure above shows the learning curve for the Random Forest base model, which illustrates the F1 score at various training set sizes. The training score, which is the blue line, starts high but decreases as the training size increases. This suggests that the model is overfitting the training data at smaller sizes, but as more data is used, the model's performance begins to stabilize and potentially generalize better.

For the cross-validation score, which is the orange dashed line, it remains quite stable and low across different training set sizes. This indicates that the model's generalization capability on unseen data (cross-validation) is limited and does not improve significantly with more data, which might suggest underfitting or insufficient model complexity.

The gap between the training and cross-validation scores suggests that the model is likely to overfit the training data at smaller sizes and does not generalize well. The model's performance appears to peak when additional data is added, with a cross-validation score of roughly 0.825, indicating the need for hyperparameter adjustment or a more sophisticated model for improved generalization.

5.2.2.3 RF ROC-AUC

```
# Predict probabilities for ROC Curve
y_prob_rf = rf_model.predict_proba(X_test)[:, 1]

# Compute ROC curve
fpr_rf, tpr_rf, thresholds_rf = roc_curve(y_test, y_prob_rf)
roc_auc_rf = auc(fpr_rf, tpr_rf)

# Plot ROC-AUC Curve for Base Model
plt.figure(figsize=(10, 6))
plt.plot(fpr_rf, tpr_rf, lw=2, label='Random Forest (AUC = %0.4f)' % roc_auc_rf)
plt.plot([0, 1], [0, 1], color='gray', linestyle='--')
plt.title('ROC Curve - Random Forest (Base Model)')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend(loc="lower right")
plt.grid(True)
plt.show()
```

Figure 109: Code of ROC-AUC (Random Forest)

The figure above shows the code that used to generate the ROC curve and calculate the AUC (Area Under the Curve) for the Random Forest base model. The model predicts the probabilities for the test data using the predict_proba method, which outputs the predicted probabilities for each class. The probabilities for the positive class are selected ([:, 1]) to compute the ROC curve. The ROC curve is computed using the roc_curve function, which returns the False Positive Rate (FPR), True Positive Rate (TPR), and the classification thresholds. The AUC is then calculated using the auc function, summarizing the area under the ROC curve as a single value.

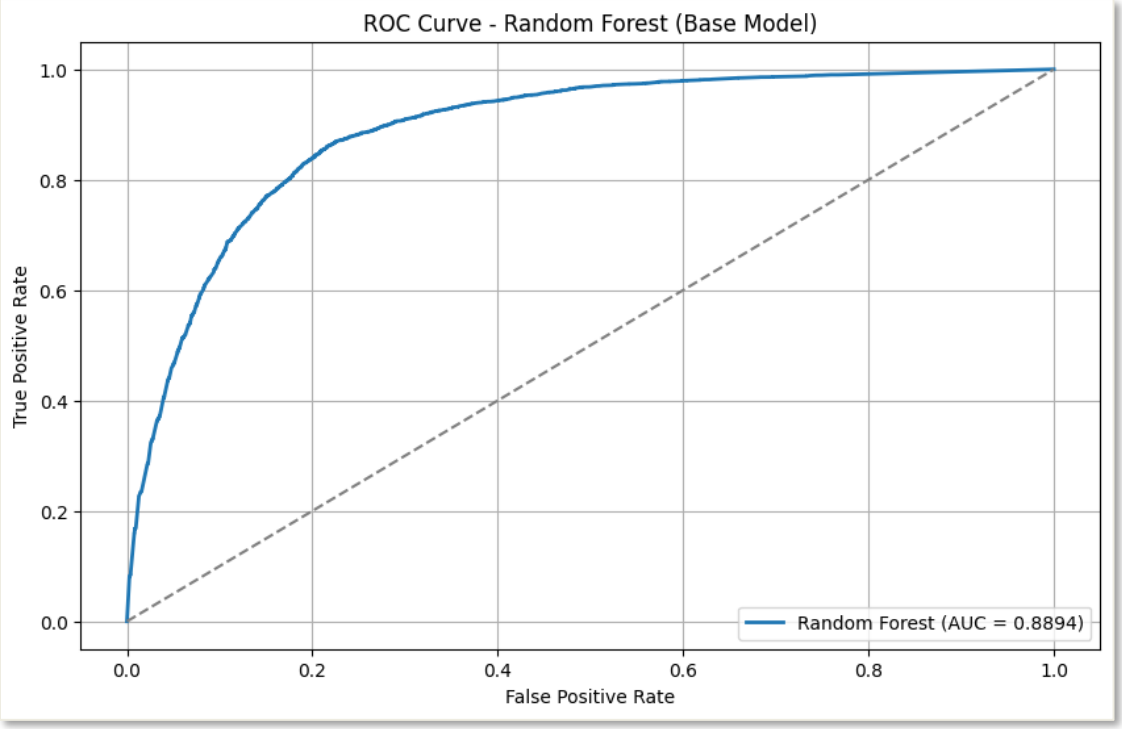


Figure 110: Visualization of ROC Curve (Random Forest)

The figure above shows the ROC curve for the Random Forest base model, with the AUC (Area Under the Curve) value of **0.8894**. The curve plots the True Positive Rate (TPR) on the y-axis against the False Positive Rate (FPR) on the x-axis, representing the model's ability to distinguish between the positive and negative classes.

The blue curve shows the performance of the model, with a significant upward slope, showing that the model performs well in distinguishing between classes. The gray dashed line represents a random classifier, serving as the baseline for comparison. The AUC value of 0.8894 suggests that the model has a strong ability to correctly classify instances, with values closer to 1 indicating better performance.

This ROC curve demonstrates that the Random Forest model achieves a high classification performance, with a good trade-off between sensitivity (True Positives) and specificity (True Negatives).

5.2.2.4 RF Tuned Model Result

```
confusion_matrix (After Tuning):
 [[3875 1052]
 [ 769 5701]]

Classification Report:
              precision    recall  f1-score   support

           0       0.83      0.79      0.81      4927
           1       0.84      0.88      0.86      6470

    accuracy                           0.84     11397
   macro avg       0.84      0.83      0.84     11397
weighted avg       0.84      0.84      0.84     11397

Accuracy (After Tuning): 0.8402
```

Figure 111: Output of Hyperparameter Tuning (Random Forest)

The figure above shows the output of hyperparameter tuning for the Random Forest model, showing the confusion matrix and classification report and accuracy after tuning. For **confusion matrix**, Random Forest has correctly predicted 3875 non-bullied (True Negatives) and 5701 bullied victims (True Positives) but misclassified 769 False Positives (FP) and 1052 False Negatives (FN).

For **classification report**, it highlights that the **precision** for Class 0 is 0.83, meaning that 83% of cases predicted as Class 0 were accurate, while for Class 1, the precision is 0.84. The **recall** for Class 0 is 0.79, meaning that the model correctly identified 79% of actual Class 0 cases, while for Class 1, recall is 0.88, showing better performance in identifying Class 1. The **F1-score** for Class 0 is 0.81 and for Class 1 is 0.84, showing a balance between precision and recall. Overall, the model achieved an **accuracy of 0.8402** after tuning, with both macro average and weighted average F1-scores of 0.84, indicating a well-balanced performance across both classes.

Model Type	Accuracy	Precision (Class 0)	Precision (Class 1)	Recall (Class 0)	Recall (Class 1)	F1-Score (Class 1)
RF (Base Model)	0.8280	0.82	0.83	0.77	0.87	0.85
RF (Tuned Model)	0.8402	0.83	0.84	0.79	0.88	0.86

Table 7: Comparison of Base and Tuned Model (Random Forest)

The table above compares the performance of the base model and the tuned model for Random Forest (RF) across several metrics. The base model achieved an **accuracy** of 0.8280, while the tuned model showed a slight improvement, reaching 0.8402.

In terms of **precision and recall**, class 0 represents the student who is not being bullied, and class 1 represents the students who have been bullied. The base model of precision had 0.82 for Class 0 and 0.83 for Class 1, while the tuned model improved to 0.83 for Class 0 and 0.84 for Class 1.

Besides, the **recall** l for Class 0 was 0.77 in the base model and increased to 0.79 in the tuned model, while Class 1 had a recall of 0.87 in the base model and 0.88 in the tuned model, showing a slight improvement in identifying Class 1 instances.

The **F1-score for class 1** improved from 0.85 to 0.86. These improvements across all metrics show that the tuned model performs better in identifying both classes correctly. This means the Random Forest model became more accurate and balanced in predictions after tuning, showing that the changes in parameters helped improve its overall performance.

5.2.2.5 RF Learning Curve (After Tune)

```
# After Hyperparameter Tuning (Best Random Forest Model from GridSearchCV)
train_sizes, train_scores, test_scores = learning_curve(
    best_rf,
    X_train,
    y_train,
    cv=5,
    n_jobs=-1,
    train_sizes=[0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1]
)

# Calculate mean and std for train and test scores
train_mean = train_scores.mean(axis=1)
test_mean = test_scores.mean(axis=1)
train_std = train_scores.std(axis=1)
test_std = test_scores.std(axis=1)

# Plot Learning Curve for Tuned Model
plt.figure(figsize=(10, 6))
plt.plot(train_sizes, train_mean, label=f'Training Score', linestyle='-', marker='o')
plt.plot(train_sizes, test_mean, label=f'Cross-Validation Score', linestyle='--', marker='x')
plt.fill_between(train_sizes, train_mean - train_std, train_mean + train_std, alpha=0.1)
plt.fill_between(train_sizes, test_mean - test_std, test_mean + test_std, alpha=0.1)
plt.title('Learning Curve - Random Forest (After Tune)')
plt.xlabel('Training Set Size')
plt.ylabel('F1 Score')
plt.legend(loc='best')
plt.grid(True)
plt.show()
```

Figure 112: Code of Learning Curve – After Tune (Random Forest)

This figure shows the code of learning curve after hyperparameter tuning for Random Forest model. This is the same process with the learning curve for the base model, the only changes are changing the variable from base model (rf_model) to the best tuned model which is represented by (best_rf) in this code.

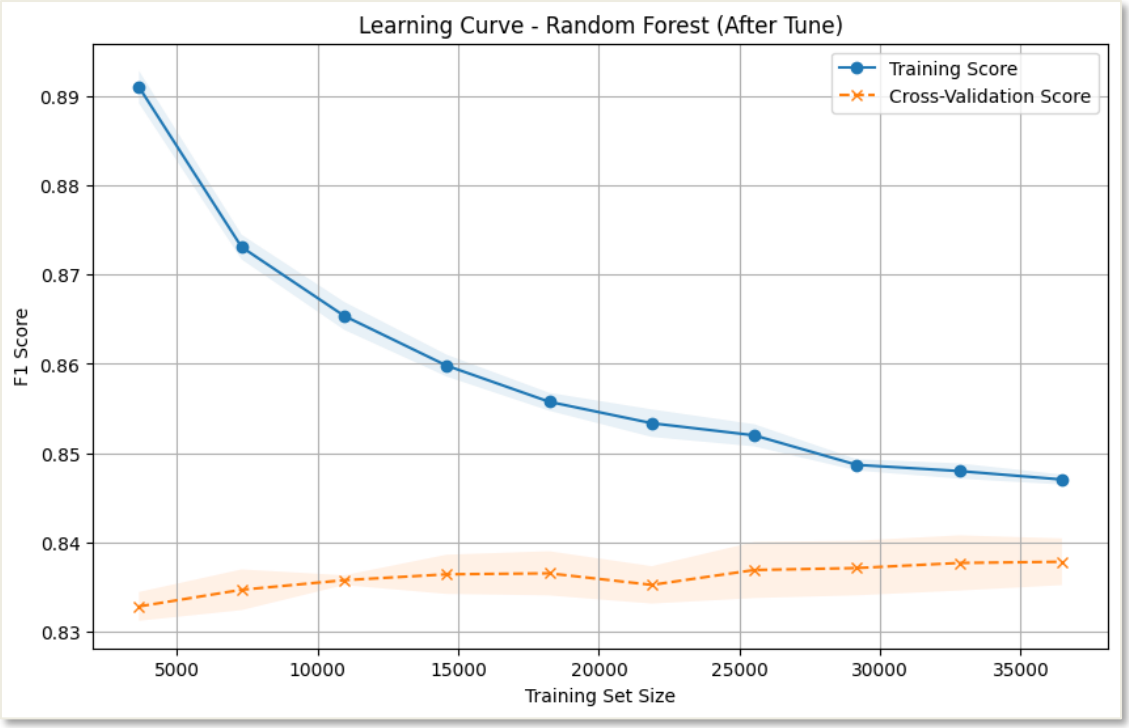


Figure 113: Visualization of Learning Curve – After Tune (Random Forest)

The figure above shows the learning curve of Random Forest after hyperparameter tuning. It shows how the model's performance changes with more data. As shown in the figure, the training score (blue line) starts high but decreases as more data is added, stabilizing at around 0.86. This shows that while the model initially improves with more data, it starts to level off after a certain point.

The cross-validation score (orange line) remains lower than the training score throughout this process, and it also stabilizes at around 0.84. This suggests that the model may be overfitting the training data because it performs better on the training data than on the validation data. The gap between the training and cross-validation scores suggests that adding more data will not significantly improve the model's performance on new data. In conclusion, the model's performance stabilizes with the given data, and adding more data beyond this point might not lead to a big improvement.

5.2.2.6 RF ROC-AUC (After Tune)

```
# Predict probabilities for ROC Curve (Tuned Model)
y_prob_best_rf = best_rf.predict_proba(X_test)[:, 1]

# Compute ROC curve for Tuned Model
fpr_best_rf, tpr_best_rf, thresholds_best_rf = roc_curve(y_test, y_prob_best_rf)
roc_auc_best_rf = auc(fpr_best_rf, tpr_best_rf)

# Plot ROC-AUC Curve for Tuned Model
plt.figure(figsize=(10, 6))
plt.plot(fpr_best_rf, tpr_best_rf, lw=2, label='Random Forest (Tuned, AUC = %0.4f)' % roc_auc_best_rf)
plt.plot([0, 1], [0, 1], color='gray', linestyle='--')
plt.title('ROC Curve - Random Forest (After tune)')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend(loc="lower right")
plt.grid(True)
plt.show()
```

Figure 114: Code of ROC-AUC --- After Tune (Random Forest)

This figure shows the code of ROC-AUC curves after hyperparameter tuning for Random Forest model. This is the same process with the ROC-AUC curve for the base model, the only change is the variable from base model (rf_model.predict_proba), which is to calculate the AUC for the base model to the best tuned model which is represented by (best_rf.predict_proba) in this code. This best_rf variable is to calculate the AUC for the tuned model.

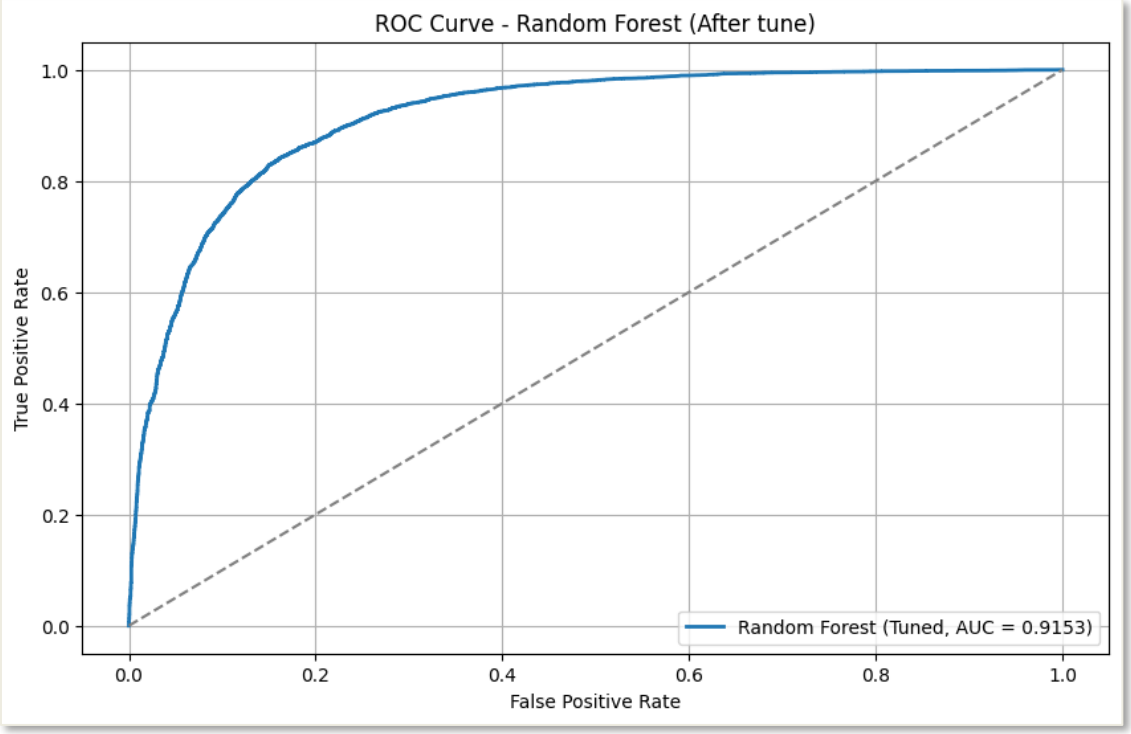


Figure 115: Visualization of ROC Curve --- After Tune (Random Forest)

The figure above shows the ROC Curve and AUC score for Random Forest after hyperparameter tuning. The ROC curve for the tuned Random Forest model shows that it performs well with the curve quickly rising towards the top-left corner. This shows that the model has a high true positive rate (correctly identifying class 1 cases) while maintaining a low false positive rate (incorrectly classifying class 0 cases as class 1). The AUC score of 0.9153 is very good, as it is close to 1, which means that the model is excellent at distinguishing between the two classes. The curve is significantly above the dashed diagonal line, which represents a random classifier. This reinforces the idea that the tuned Random Forest model has good performance and is more reliable than random guessing.

5.2.3 Support Vector Machine (SVM)

5.2.3.1 SVM Base Model Result

```
Confusion Matrix:
 [[3661 1266]
 [ 961 5509]]

📊 Classification Report:
              precision    recall  f1-score   support

           0       0.79      0.74      0.77      4927
           1       0.81      0.85      0.83      6470

    accuracy                           0.80     11397
   macro avg       0.80      0.80      0.80     11397
weighted avg       0.80      0.80      0.80     11397

✅ Accuracy: 0.8046
```

Figure 116: Output of Base Model (SVM)

The figure above shows the performance evaluation of the base SVM model for predicting bullying victimization risk among adolescent students. The **confusion matrix** has shown that the model correctly predicted 3661 cases of non-victimization (True Negatives) and 5509 cases of victimization (True Positives). However, the SVM have misclassified 1266 non-victimized cases as victimized (False Positives) and 961 victimized cases as non-victimized (False Negatives).

For the **classification report**, Class 0 represents the student who is not being bullied, and Class 1 represents the students who have been bullied. The **precision** for Class 0 is 0.79, while for Class 1 is 0.81, which means that the proportion of correct positive predictions for each class. The **recall** for Class 0 is 0.74, while for Class 1 is 0.85, reflecting that this model is able to identify each class correctly. Then, the **f1-score** is 0.77 for Class 0 and 0.83 for Class 1, which have balanced the precision and recall.

The overall **accuracy** of the model is 80.46%, with both the macro average and weighted average for precision, recall, and F1-score all being 0.80, showing a well-balanced performance across both classes.

5.2.3.2 SVM Learning Curve

```
# Learning Curve for SVM (Base Model)
train_sizes, train_scores, test_scores = learning_curve(
    svm_model,
    X_train_scaled,
    y_train,
    cv=3,
    n_jobs=-1,
    train_sizes=[0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1]
)

# Calculate mean and std for train and test scores
train_mean = train_scores.mean(axis=1)
test_mean = test_scores.mean(axis=1)
train_std = train_scores.std(axis=1)
test_std = test_scores.std(axis=1)

# Plot Learning Curve
plt.figure(figsize=(10, 6))
plt.plot(train_sizes, train_mean, label=f'Training Score', linestyle='-', marker='o')
plt.plot(train_sizes, test_mean, label=f'Cross-Validation Score', linestyle='--', marker='x')
plt.fill_between(train_sizes, train_mean - train_std, train_mean + train_std, alpha=0.1)
plt.fill_between(train_sizes, test_mean - test_std, test_mean + test_std, alpha=0.1)
plt.title('Learning Curve - SVM (Base Model)')
plt.xlabel('Training Set Size')
plt.ylabel('F1 Score')
plt.legend(loc='best')
plt.grid(True)
plt.show()
```

Figure 117: Code of Learning Curve (SVM)

The figure above shows the code of learning curve for a Support Vector Machine (SVM) base model. The learning_curve function calculates the training and test scores for various training set sizes, ranging from 10% to 100%. The code also calculates the mean and standard deviation of the training and test scores, which are then used to generate a plot that shows the F1 score for both the training and cross-validation (test) sets. The plot includes filled areas to indicate the standard deviation, providing a sense of variability in the performance of the model. The resulting learning curve helps assess how the SVM model performs as the size of the training data increases, and whether the model is overfitting or underfitting.

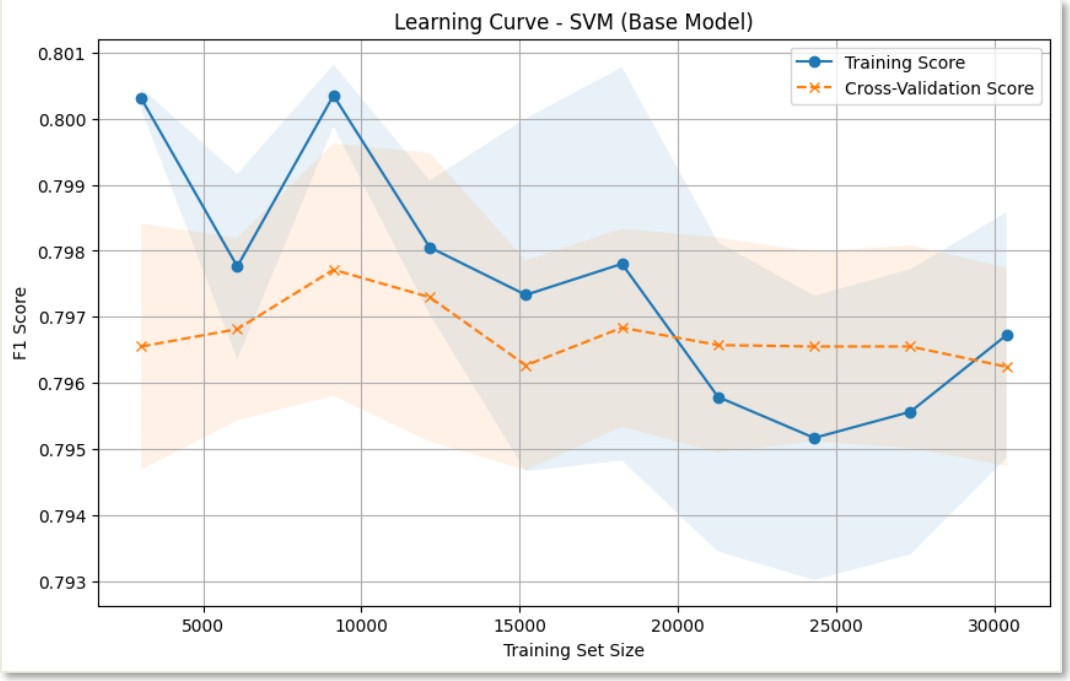


Figure 118:Visualization of Learning Curve (SVM)

The figure above shows the learning curve of SVM base model. The graph illustrates the F1 score as the training set size increases. The blue line represents the training score, while the orange dashed line represents the cross-validation score. As the training set size increases, the training score starts high but slowly decreases, meaning that the model may be overfitting at smaller data sizes. Besides, for the cross-validation score, it remains regularly consistent and lower than the training score, showing that the model fails to generalize to unseen data, even when more data is added for training. The gap between the training and cross-validation scores suggests that the model could benefit from further tuning to improve generalization and reduce overfitting. The shaded areas around each line represent the standard deviation, giving a sense of variability in performance across different training set sizes.

5.2.3.3 SVM ROC-AUC

```
# Predict probabilities for ROC Curve (Base Model)
y_prob_svm = svm_model.predict_proba(X_test_scaled)[:, 1]

# Compute ROC curve
fpr_svm, tpr_svm, thresholds_svm = roc_curve(y_test, y_prob_svm)
roc_auc_svm = auc(fpr_svm, tpr_svm)

# Plot ROC-AUC Curve for Base Model
plt.figure(figsize=(10, 6))
plt.plot(fpr_svm, tpr_svm, lw=2, label='SVM (AUC = %0.4f)' % roc_auc_svm)
plt.plot([0, 1], [0, 1], color='gray', linestyle='--')
plt.title('ROC Curve - SVM (Base Model)')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend(loc="lower right")
plt.grid(True)
plt.show()
```

Figure 119: Code of ROC-AUC (SVM)

The figure above shows the code used to generate the ROC curve and calculate the AUC (Area Under the Curve) for the SVM (Support Vector Machine) base model. The model first predicts the probabilities for the test data using the predict_proba method on the scaled test set (X_test_scaled). The probabilities for the positive class ([:, 1]) are selected to compute the ROC curve.

The ROC curve is computed with the roc_curve function, which returns the False Positive Rate (FPR), True Positive Rate (TPR), and the classification thresholds. The AUC is then calculated using the auc function, providing a single value summarizing the area under the ROC curve.

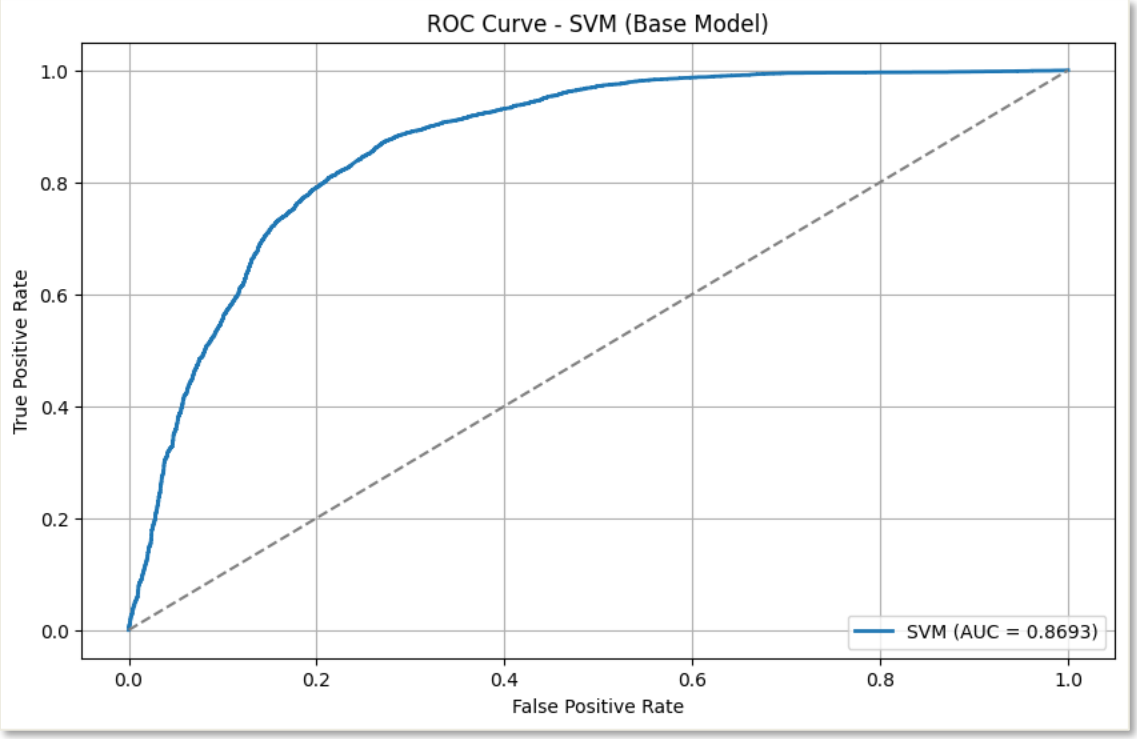


Figure 120: Visualization of ROC Curve (SVM)

The figure above shows the ROC curve for the SVM (Support Vector Machine) base model, with an AUC (Area Under the Curve) of **0.8693**. The blue curve represents the model's performance, plotting the True Positive Rate (TPR) on the y-axis against the False Positive Rate (FPR) on the x-axis. The gray dashed line represents a random classifier, serving as a baseline.

The curve demonstrates that the model performs well, with a significant upward slope, indicating it correctly identifies positive instances (high TPR) while minimizing false positives (low FPR). The AUC value of 0.8693 suggests that the model has strong discriminative power, effectively distinguishing between positive and negative classes. The closer the AUC score is to 1, the better the model is at classification.

5.2.3.4 SVM Tuned Model Result

```
Confusion Matrix (After Tuning):
 [[3792 1135]
 [ 720 5750]]

Classification Report:
              precision    recall  f1-score   support

           0       0.84      0.77      0.80      4927
           1       0.84      0.89      0.86      6470

    accuracy                           0.84     11397
   macro avg       0.84      0.83      0.83     11397
weighted avg       0.84      0.84      0.84     11397

Accuracy (After Tuning): 0.8372
```

Figure 121: Output of Hyperparameter Tuning (SVM)

The figure above shows the output of model evaluation after hyperparameter tuning for the SVM model. The model evaluation is shown by the confusion matrix, classification report and accuracy after tuning. For the **confusion matrix**, it shows that the SVM model correctly predicted 3792 non-bullied students (True Negatives) and 5750 bullied students (True Positives), while it misclassified 1135 False Positives (FP) and 720 False Negatives (FN).

The **classification report** shows a **precision** of 0.84 for both non-bullied students (Class 0) and bullied students (Class 1), meaning that the performance is balanced in predicting both classes. The recall for non-bullied students (Class 0) is 0.77, meaning the model correctly identified 77% of actual Class 0 cases, while for bullied students (Class 1), the **recall** is 0.89, indicating that 89% of actual Class 1 cases were correctly identified. The **F1-score** for Class 0 is 0.80, and for Class 1 is 0.86, showing a better balance between precision and recall for Class 1.

The overall **accuracy** after tuning is 0.8372, meaning the model correctly predicted 83.72% of the instances. The macro average and weighted average F1-scores are both 0.83, reflecting a well-balanced model performance across both classes.

Model Type	Accuracy	Precision (Class 0)	Precision (Class 1)	Recall (Class 0)	Recall (Class 1)	F1-Score (Class 1)
SVM (Base Model)	0.8046	0.79	0.81	0.74	0.85	0.83
SVM (Tuned Model)	0.8372	0.84	0.84	0.77	0.89	0.86

Table 8: Comparison of Base and Tuned Model (SVM)

The table compares the performance of the base model and the tuned model for SVM with various metrics. The base model achieved an **accuracy** of 0.8046, while the tuned model showed a slight improvement with an accuracy of 0.8372, indicating better overall performance after hyperparameter tuning.

The **precision** for Class 0 (non-bullied students) increased from 0.79 in the base model to 0.84 in the tuned model, while the precision for Class 1 (bullied students) improved from 0.81 to 0.84. For **recall**, the base model achieved 0.74 for Class 0, which improved to 0.77 in the tuned model, and Class 1 recall increased from 0.85 to 0.89.

Lastly, the **F1-score for class 1** also increased from 0.83 to 0.86. These enhancements show that the tuned SVM model performs better at correctly identifying both classes. Overall, the tuning process helped the SVM model become more accurate, precise, and balanced in making predictions.

5.2.3.5 SVM Learning Curve (After Tune)

```
# Learning Curve for Tuned SVM
train_sizes, train_scores, test_scores = learning_curve(
    best_svm,
    X_train_scaled,
    y_train,
    cv=3,
    n_jobs=-1,
    train_sizes=[0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1]
)

# Calculate mean and std for train and test scores
train_mean = train_scores.mean(axis=1)
test_mean = test_scores.mean(axis=1)
train_std = train_scores.std(axis=1)
test_std = test_scores.std(axis=1)

# Plot Learning Curve for Tuned Model
plt.figure(figsize=(10, 6))
plt.plot(train_sizes, train_mean, label=f'Training Score', linestyle='-', marker='o')
plt.plot(train_sizes, test_mean, label=f'Cross-Validation Score', linestyle='--', marker='x')
plt.fill_between(train_sizes, train_mean - train_std, train_mean + train_std, alpha=0.1)
plt.fill_between(train_sizes, test_mean - test_std, test_mean + test_std, alpha=0.1)
plt.title('Learning Curve - SVM (After Tune)')
plt.xlabel('Training Set Size')
plt.ylabel('F1 Score')
plt.legend(loc='best')
plt.grid(True)
plt.show()
```

Figure 122: Code of Learning Curve – After Tune (SVM)

This figure shows the code of learning curve after hyperparameter tuning for SVM model. This is the same process with the learning curve for the base model, the only change is the variable from base model (svm_model) to the best tuned model which is represented by (best_svm) in this code.

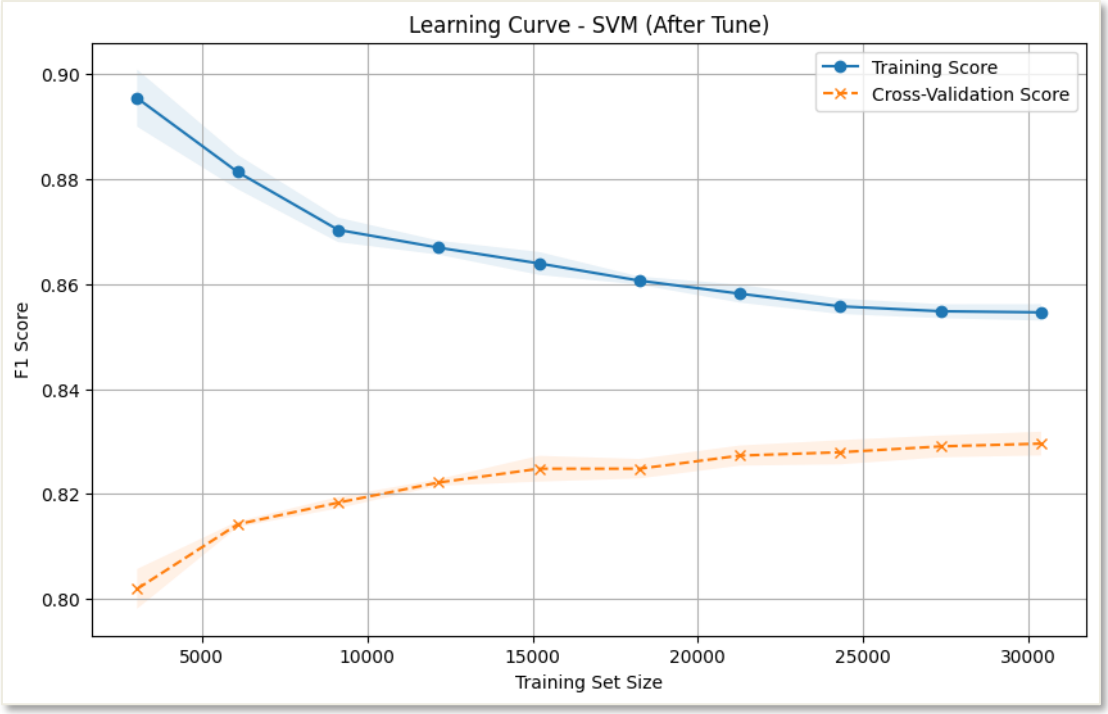


Figure 123: Visualization of Learning Curve – After Tune (SVM)

The figure above shows the learning curve of SVM after hyperparameter tuning. It shows how the model's performance changes as more data is used. The training score (blue line) starts high and then slowly decreases, stabilizing at around 0.86. This shows that the model initially performs well, but as the amount of data increases, its performance on the training set becomes more consistent and less variable.

The cross-validation score (orange line) starts lower than the training score and slowly increases, stabilizing around 0.82. This indicates that while the model is doing well on the training data, it still has space for improvement when tested on new, unseen data. The gap between the training and cross-validation scores shows that the model may be overfitting the training data.

As more data is added, the performance improves slightly, but the model still doesn't perform as well on the cross-validation data compared to the training data. In conclusion, the model's performance improves with more data, but there is still a gap between its performance on training and validation data.

5.2.3.6 SVM ROC-AUC (After Tune)

```
# Predict probabilities for ROC Curve (Tuned Model)
y_prob_best_svm = best_svm.predict_proba(X_test_scaled)[:, 1]

# Compute ROC curve for Tuned Model
fpr_best_svm, tpr_best_svm, thresholds_best_svm = roc_curve(y_test, y_prob_best_svm)
roc_auc_best_svm = auc(fpr_best_svm, tpr_best_svm)

# Plot ROC-AUC Curve for Tuned Model
plt.figure(figsize=(10, 6))
plt.plot(fpr_best_svm, tpr_best_svm, lw=2, label='SVM (Tuned, AUC = %0.4f)' % roc_auc_best_svm)
plt.plot([0, 1], [0, 1], color='gray', linestyle='--')
plt.title('ROC Curve - SVM (After tune)')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend(loc="lower right")
plt.grid(True)
plt.show()
```

Figure 124:Code of ROC-AUC --- After Tune (SVM)

This figure shows the code of ROC-AUC curves after hyperparameter tuning for SVM model. This is the same process with the ROC-AUC curve for the base model, the only change is the variable from base model (svm_model.predict_proba), which is to calculate the AUC for the base model to the best tuned model which is represented by (best_svm.predict_proba) in this code. This best_svm variable is to calculate the AUC for the tuned SVM model.

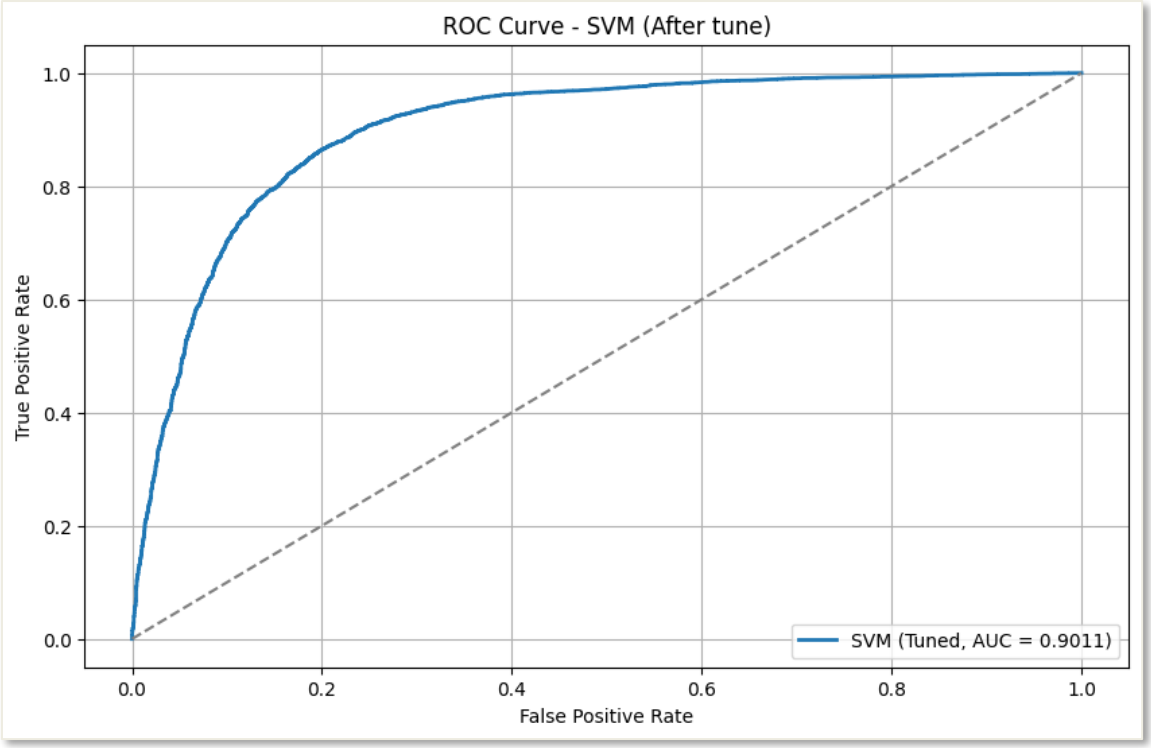


Figure 125: Visualization of ROC Curve – After Tune (SVM)

The figure above shows the ROC Curve and AUC score for SVM after hyperparameter tuning. The ROC curve for the tuned SVM model shows great performance, with the curve quickly rising to the top-left corner. This shows that the model has a high true positive rate (correctly identifying class 1 cases) and a low false positive rate (incorrectly labeling class 0 cases as class 1). The AUC score of 0.9011 is very high, indicating the model's strong ability to distinguish between the two classes. The curve is well above the dashed diagonal line, which represents random guessing, confirming that the tuned SVM model is highly effective at classification.

5.2.4 XGBoost (XGB)

5.2.4.1 XGB Base Model Result

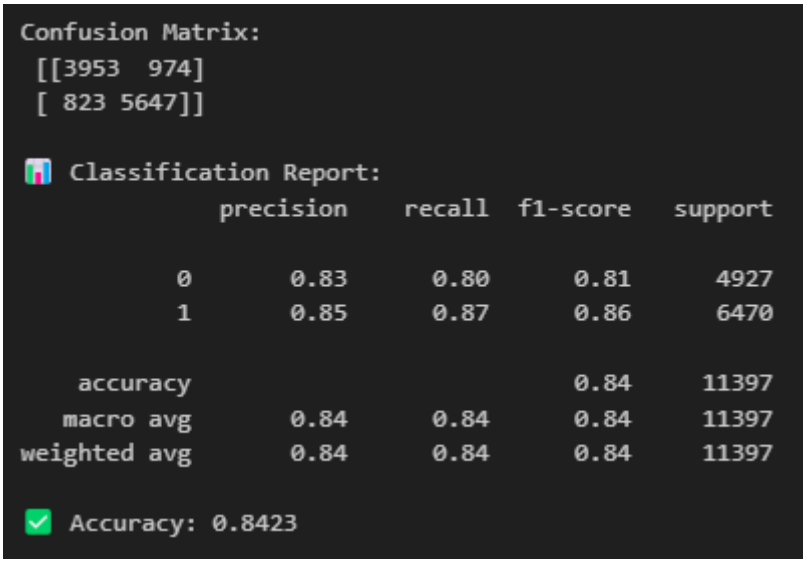


Figure 126: Output of Base Model (XGBoost)

The figure above shows the performance evaluation of the base XGBoost model for predicting bullying victimization risk among adolescent students. The **confusion matrix** shows that the model has accurately predicted 3953 non-bullied students (True Negatives) and 5647 bullied students (True Positives). The model has misclassified 974 non-bullied students as bullied students (False Positives) and 823 bullied students as non-bullied students (False Negatives).

In **classification report**, it shows a **precision** of 0.83 for non-bullied students (Class 0) and 0.85 for bullied students (Class 1), which represents the proportion of correct positive predictions in each class. Then, the **recall** has 0.80 for non-bullied students (Class 0) and 0.87 for bullied students (Class 1), indicating the model's ability to identify each class correctly. The **F1-score** is 0.81 for non-bullied students (Class 0) and 0.86 for bullied students (Class 1), providing a balanced measure of precision and recall.

The overall **accuracy** for XGBoost is 84.23%, with both the macro average and weighted average for precision, recall, and F1-score all being 0.84, highlighting a well-balanced performance across both classes.

5.2.4.2 XGB Learning Curve

```
# Learning Curve for XGBoost (Base Model)
train_sizes, train_scores, test_scores = learning_curve(
    xgboost_model,
    X_train,
    y_train,
    cv=3,
    n_jobs=-1,
    train_sizes=[0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1]
)

# Calculate mean and std for train and test scores
train_mean = train_scores.mean(axis=1)
test_mean = test_scores.mean(axis=1)
train_std = train_scores.std(axis=1)
test_std = test_scores.std(axis=1)

# Plot Learning Curve
plt.figure(figsize=(10, 6))
plt.plot(train_sizes, train_mean, label=f'Training Score', linestyle='-', marker='o')
plt.plot(train_sizes, test_mean, label=f'Cross-Validation Score', linestyle='--', marker='x')
plt.fill_between(train_sizes, train_mean - train_std, train_mean + train_std, alpha=0.1)
plt.fill_between(train_sizes, test_mean - test_std, test_mean + test_std, alpha=0.1)
plt.title('Learning Curve - XGBoost (Base Model)')
plt.xlabel('Training Set Size')
plt.ylabel('F1 Score')
plt.legend(loc='best')
plt.grid(True)
plt.show()
```

Figure 127: Code of Learning Curve (XGBoost)

The figure above shows the code of learning curve for the XGBoost base model to evaluate its performance at different training set sizes. The function learning_curve is used to compute the training and cross-validation scores for various training sizes ranging from 10% to 100% of the training data, with 3-fold cross-validation (cv=3). The training and test scores are then used to calculate the mean and standard deviation for both the training and validation scores at each training set size.

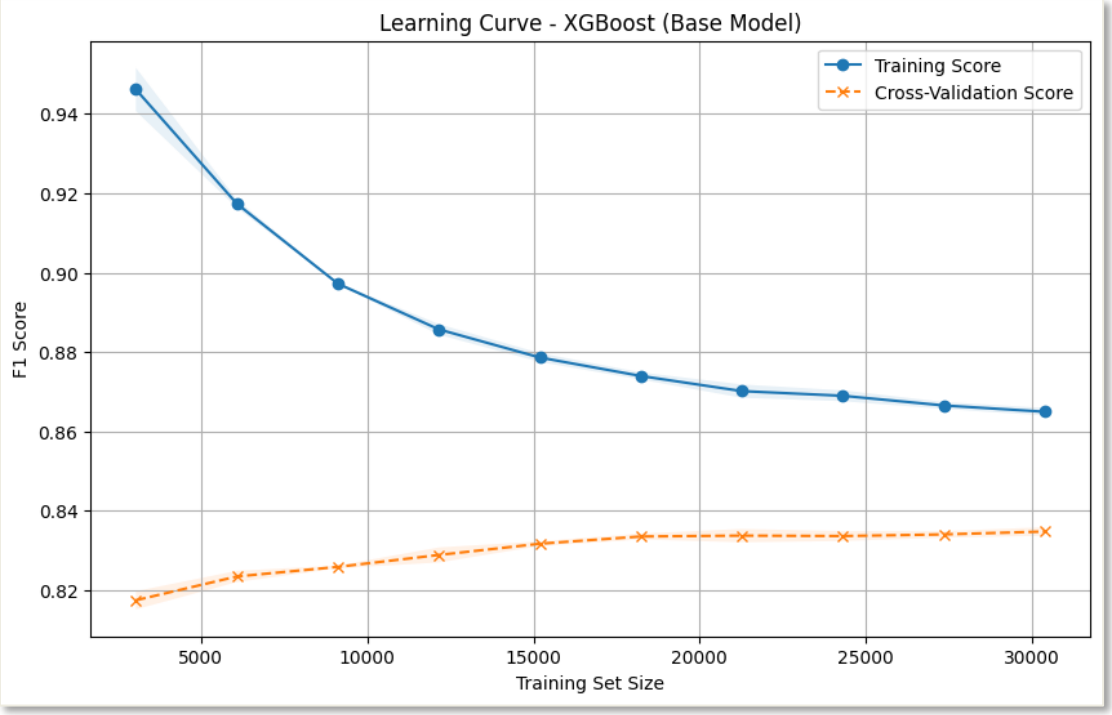


Figure 128: Visualization of Learning Curve (XGBoost)

The figure above shows the visualization of learning curve for XGBoost base model. The graph illustrates how the F1 score varies as the training set size increases. The training score is represented with solid line and circle markers, while the cross-validation score is represented by a dash line and 'x' markers. As shown in the curve, the training score starts high but decreases as more data is used for training. This decrease shows that the model initially performs well with small amounts of data, but as the training data increases, it struggles to maintain its performance, and this is possibly due to overfitting.

On the other hand, as the training set size increases, the cross-validation remains fairly stable and low. This pattern shows that the model may not be generalizing well with larger data sets, as the performance on unseen data (cross-validation) is consistently lower than on the training data. Both scores seem to stabilize as the training set size grows, with the cross-validation score hovering around 0.84, indicating that the model is performing consistently after a certain point.

5.2.4.3 XGB ROC-AUC

```
# Predict probabilities for ROC Curve (Base Model)
y_prob_xgb = xgboost_model.predict_proba(X_test)[:, 1]

# Compute ROC curve
fpr_xgb, tpr_xgb, thresholds_xgb = roc_curve(y_test, y_prob_xgb)
roc_auc_xgb = auc(fpr_xgb, tpr_xgb)

# Plot ROC-AUC Curve for Base Model
plt.figure(figsize=(10, 6))
plt.plot(fpr_xgb, tpr_xgb, lw=2, label='XGBoost (AUC = %0.4f)' % roc_auc_xgb)
plt.plot([0, 1], [0, 1], color='gray', linestyle='--')
plt.title('ROC Curve - XGBoost (Base Model)')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend(loc="lower right")
plt.grid(True)
plt.show()
```

Figure 129: Code of ROC-AUC (XGBoost)

The figure above shows the code that generate the ROC curve and calculate the **AUC (Area Under the Curve)** for the **XGBoost** base model. The model first predicts the probabilities for the test set using the predict_proba method, which outputs the predicted probabilities for both classes. The code then selects the probabilities for the positive class ([:, 1]) to compute the **ROC curve**.

The **ROC curve** is computed using the roc_curve function, which calculates the **False Positive Rate (FPR)**, **True Positive Rate (TPR)**, and the classification thresholds. The **AUC** is computed using the auc function, summarizing the model's performance in a single value.

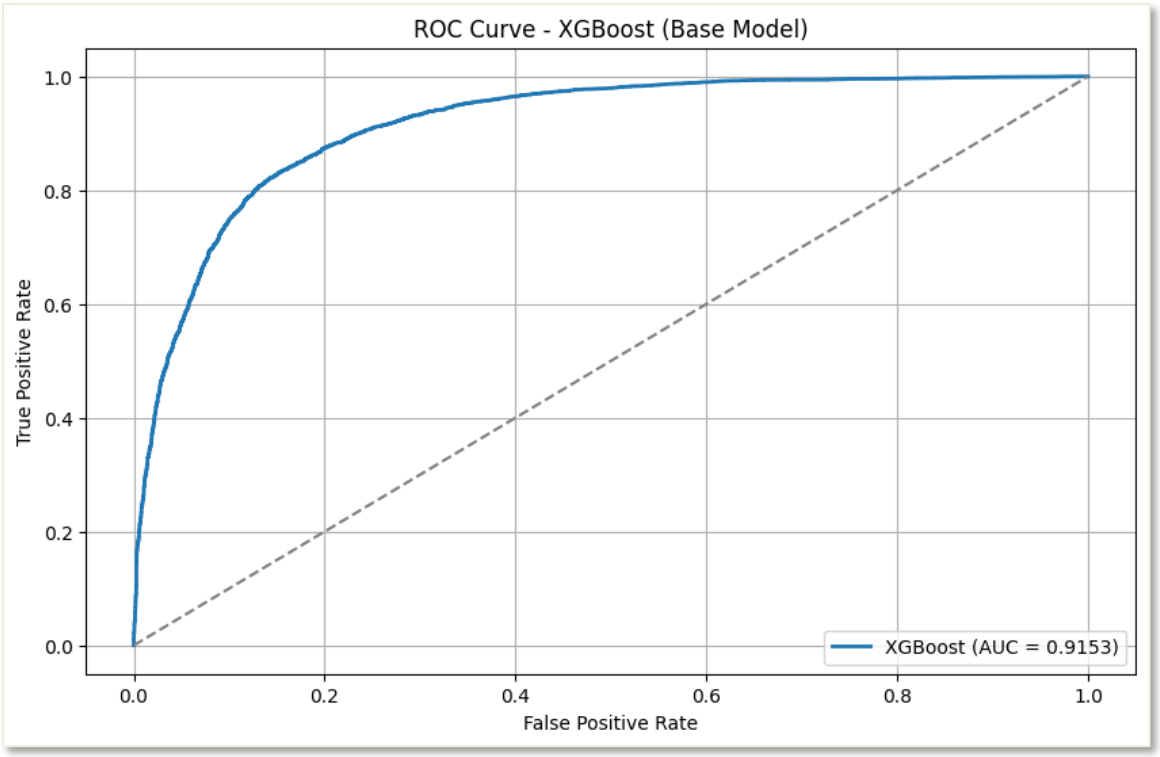


Figure 130:Visualization of ROC Curve (XGBoost)

The figure above presents the ROC curve for the XGBoost base model, showing a high AUC (Area Under the Curve) of **0.9153**. The blue curve represents the model's performance, plotting the True Positive Rate (TPR) on the y-axis against the False Positive Rate (FPR) on the x-axis. The gray dashed line represents a random classifier, which serves as a baseline for comparison.

The curve shows that the XGBoost model performs well, with a steep rise in the initial part of the curve, indicating a high True Positive Rate (TPR) and low False Positive Rate (FPR). The AUC value of 0.9153 indicates excellent classification performance, suggesting that the model effectively distinguishes between the positive and negative classes. This strong performance is reflected by the curve, which moves well above the random classifier baseline.

5.2.4.4 XGB Tuned Model Result

```
Confusion Matrix (After Tuning):
[[3927 1000]
 [ 785 5685]]

Classification Report:
              precision    recall  f1-score   support

           0       0.83      0.80      0.81      4927
           1       0.85      0.88      0.86      6470

    accuracy                           0.84     11397
   macro avg       0.84      0.84      0.84     11397
weighted avg       0.84      0.84      0.84     11397

Accuracy (After Tuning): 0.8434
```

Figure 131: Output of Hyperparameter Tuning (XGBoost)

The figure above shows the output of model evaluation after hyperparameter tuning for the XGBoost model. The model evaluation is shown by the confusion matrix, classification report and accuracy after tuning. For **confusion matrix**, XGBoost has accurately predicted 3927 students does not being bullied (True Negatives) and 5685 students have been bullied (True Positives), misclassifying 1000 False Positives (FP) and 785 False Negatives (FN).

The **classification report** shows that for students **does not being bullied** (Class 0), the **precision** is **0.83**, and the **recall** is **0.80**, resulting in an **F1-score of 0.81**. For students who **have been bullied** (Class 1), the **precision is 0.85**, and the **recall is 0.88**, giving an **F1-score of 0.86**, meaning that a better performance in identifying Class 1 instances.

The overall **accuracy** after tuning is **0.8434**, which means the model correctly predicted 84.34% of the instances. The macro average and weighted average F1-scores are both 0.84, reflecting a balanced performance across both classes.

Model Type	Accuracy	Precision (Class 0)	Precision (Class 1)	Recall (Class 0)	Recall (Class 1)	F1-Score (Class 1)
XGB (Base Model)	0.8423	0.83	0.85	0.80	0.87	0.86
XGB (Tuned Model)	0.8434	0.83	0.85	0.80	0.88	0.86

Table 9: Comparison of Base and Tuned Model (XGBoost)

The table above shows the comparison of the base model and the tune model for XGBoost by evaluating different metrics. The base model has an **accuracy** of 0.8423, while the tuned model showed a slight improvement with an accuracy of 0.8434, indicating a small positive impact from hyperparameter tuning.

In terms of **precision and recall,** class 0 represents the student who is not being bullied, and class 1 represents the students who have been bullied. The precision for both models has the same precision for Class 0 (0.83) and Class 1 (0.85), showing no change after tuning.

For **recall**, the base model achieved 0.80 for Class 0, and the tuned model maintained the same recall for Class 0, while Class 1 recall improved 0.01 from 0.87 in the base model to 0.88 in the tuned model, indicating better identification of Class 1 instances.

The **F1-score for class 1** remained unchanged at 0.86. These results indicate that the tuning process gave a small improvement in accuracy and recall for class 1, helping the model better detect positive cases. Overall, the tuned XGBoost model remains consistent and reliable, with slight performance gains.

5.2.4.5 RF Learning Curve (After Tune)

```
# Learning Curve for Tuned XGBoost
train_sizes, train_scores, test_scores = learning_curve(
    best_xgb, X_train, y_train, cv=3, n_jobs=-1,
    train_sizes=[0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1]
)

# Calculate mean and std for train and test scores
train_mean = train_scores.mean(axis=1)
test_mean = test_scores.mean(axis=1)
train_std = train_scores.std(axis=1)
test_std = test_scores.std(axis=1)

# Plot Learning Curve for Tuned Model
plt.figure(figsize=(10, 6))
plt.plot(train_sizes, train_mean, label=f'Training Score', linestyle='-', marker='o')
plt.plot(train_sizes, test_mean, label=f'Cross-Validation Score', linestyle='--', marker='x')
plt.fill_between(train_sizes, train_mean - train_std, train_mean + train_std, alpha=0.1)
plt.fill_between(train_sizes, test_mean - test_std, test_mean + test_std, alpha=0.1)
plt.title('Learning Curve - XGBoost (After Tune)')
plt.xlabel('Training Set Size')
plt.ylabel('F1 Score')
plt.legend(loc='best')
plt.grid(True)
plt.show()
```

Figure 132: Code of Learning Curve – After Tune (XGBoost)

This figure shows the code of learning curve after hyperparameter tuning for XGBoost model. This is the same process with the learning curve for the base model, the only change is the variable from base model (xgboost_model) to the best tuned model which is represented by (best_xgb) in this code.

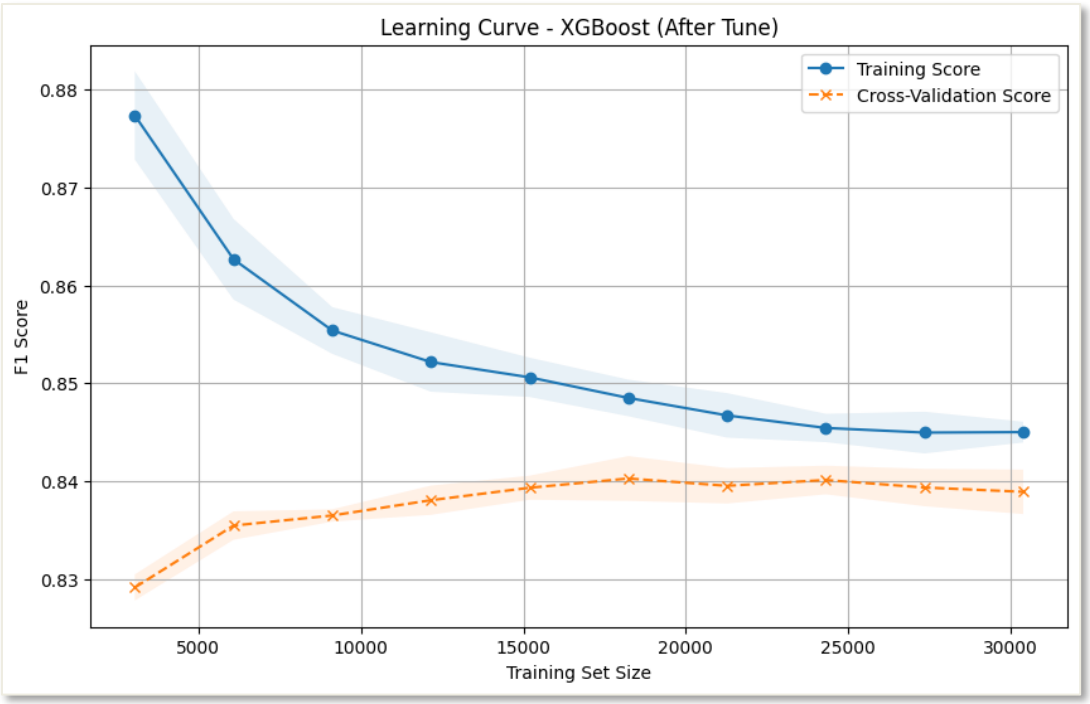


Figure 133: Visualization of Learning Curve – After Tune (XGBoost)

The figure above shows the learning curve of XGBoost after hyperparameter tuning. The XGBoost model shows good performance on both training and cross-validation data, but there is an important difference between the two scores. The training score (blue line) starts high but decreases as more data is used, eventually stabilizing at around 0.86. This indicates that the model fits well to the training data initially but is stabilizing and may start to generalize better as more data is introduced.

The cross-validation score (orange line) is consistently lower than the training score, starting at around 0.83 and slowly increasing. This suggests that while XGBoost performs well on the training data, its performance on unseen data (cross-validation) does not improve as much, which is a sign of overfitting. The gap between the training score and cross-validation score highlights that the model might be too specialized to the training data and struggles to generalize.

Although having this gap, XGBoost is still the best-performing model because it achieves the highest F1-score compared to the other models, indicating strong overall accuracy and a good balance between precision and recall. In conclusion, while there is potential for overfitting, the performance of XGBoost on both training and cross-validation data still makes it the most robust option among the models.

5.2.4.6 RF ROC-AUC (After Tune)

```
# Predict probabilities for ROC Curve (Tuned Model)
y_prob_best_xgb = best_xgb.predict_proba(X_test)[:, 1]

# Compute ROC curve for Tuned Model
fpr_best_xgb, tpr_best_xgb, thresholds_best_xgb = roc_curve(y_test, y_prob_best_xgb)
roc_auc_best_xgb = auc(fpr_best_xgb, tpr_best_xgb)

# Plot ROC-AUC Curve for Tuned Model
plt.figure(figsize=(10, 6))
plt.plot(fpr_best_xgb, tpr_best_xgb, lw=2, label='XGBoost (Tuned, AUC = %0.4f)' % roc_auc_best_xgb)
plt.plot([0, 1], [0, 1], color='gray', linestyle='--')
plt.title('ROC Curve - XGBoost (After tune)')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend(loc="lower right")
plt.grid(True)
plt.show()
```

Figure 134: Code of ROC-AUC --- After Tune (XGBoost)

This figure shows the code of ROC-AUC curves after hyperparameter tuning for XGBoost model. This is the same process with the ROC-AUC curve for the base model, the only change is the variable from base model (xgboost_model.predict_proba) to the best tuned model which is represented by (best_xgb.predict_proba) in this code. This best_xgb variable is to calculate the AUC for the tuned XGBoost model.

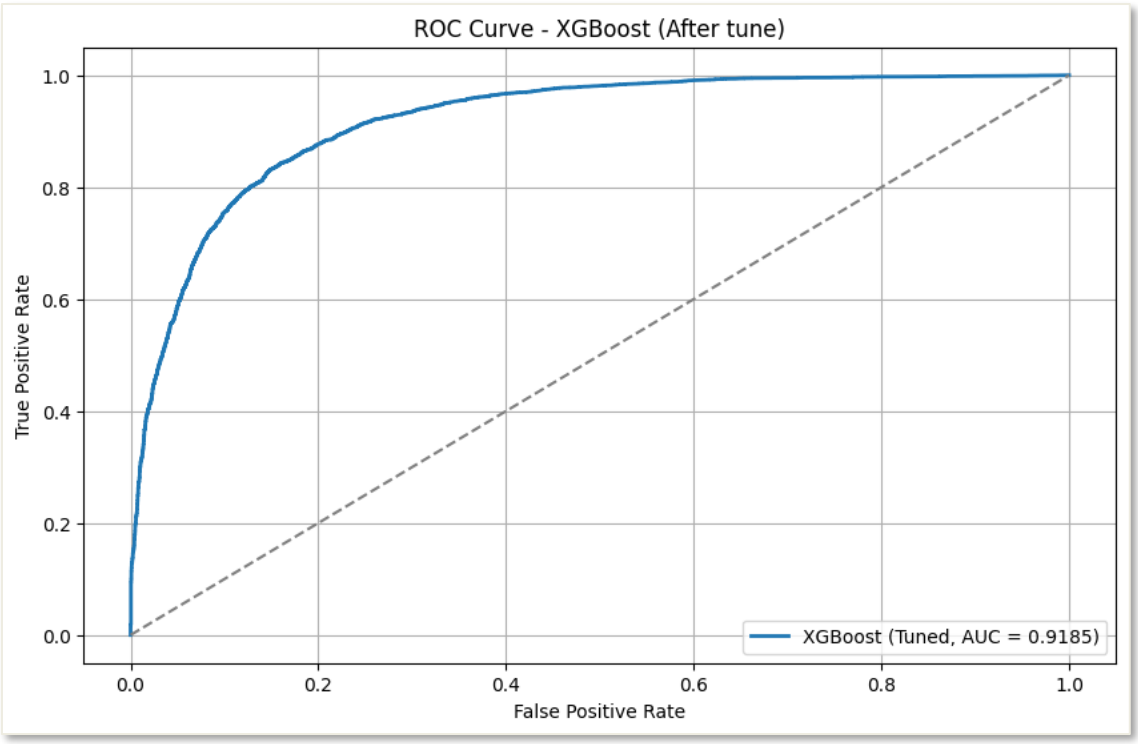


Figure 135: Visualization of ROC Curve – After Tune (XGBoost)

The figure above shows the ROC Curve and AUC score for XGBoost after hyperparameter tuning. The ROC curve for the tuned XGBoost model show a strong performance, with the curve sharply rising towards the top-left corner. This indicates that the model has a high true positive rate (accurately identifying class 1 cases) while keeping the false positive rate (incorrectly classifying class 0 as class 1) low. The AUC score of 0.9185 is excellent, as it is very close to 1, highlighting the model's ability to effectively differentiate between the two classes. The curve is clearly above the dashed diagonal line, which represents a random classifier, further confirming that the tuned XGBoost model outperforms random guessing and shows solid reliability in making predictions.

5.2.5 Comparison and Discussion

5.2.5.1 Classification Report

	Accuracy	Precision (Class 0)	Precision (Class 1)	Recall (Class 0)	Recall (Class 1)	F1-score (Class 1)
Logistic Regression						
Base	0.8006	0.77	0.82	0.76	0.83	0.83
Tuned	0.8014	0.77	0.82	0.76	0.83	0.83
Random Forest						
Base	0.8280	0.82	0.83	0.77	0.87	0.85
Tuned	0.8402	0.83	0.84	0.79	0.88	0.86
Support Vector Machine (SVM)						
Base	0.8046	0.79	0.81	0.74	0.85	0.83
Tuned	0.8372	0.84	0.84	0.77	0.89	0.86
XGBoost						
Base	0.8423	0.83	0.85	0.80	0.87	0.86
Tuned	0.8434	0.83	0.85	0.80	0.88	0.86

Table 10: Comparison on Classification Report of Base and Tuned Machine Learning Models

The table above compares the performance before and after hyperparameter tuning of four machine learning models which are Logistic Regression, Random Forest, Support Vector Machine (SVM), and XGBoost. It shows how each model's accuracy, precision, recall, and F1-score for both class 0 and class 1 change after tuning.

For **Logistic Regression,** it shows that there is only minimal improvement after tuning. The accuracy slightly increased from 0.8006 to 0.8014, while the precision, recall, and F1-scores for both classes remained the same. This suggests that Logistic Regression was already well-tuned to the data, or the hyperparameters did not significantly impact performance.

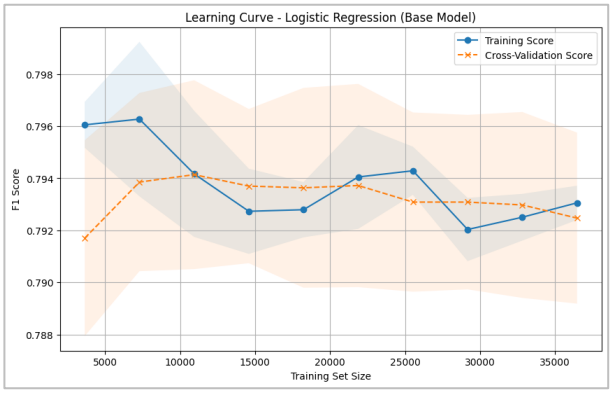
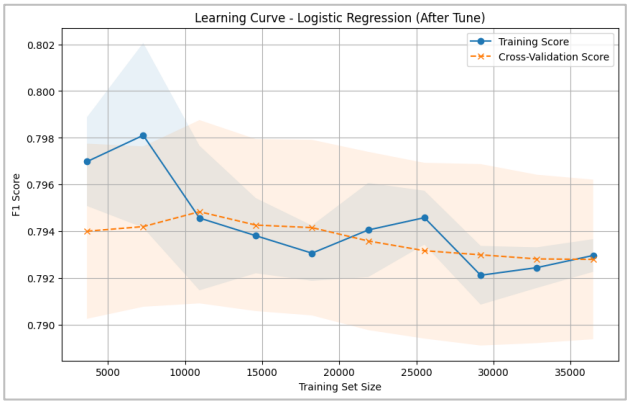
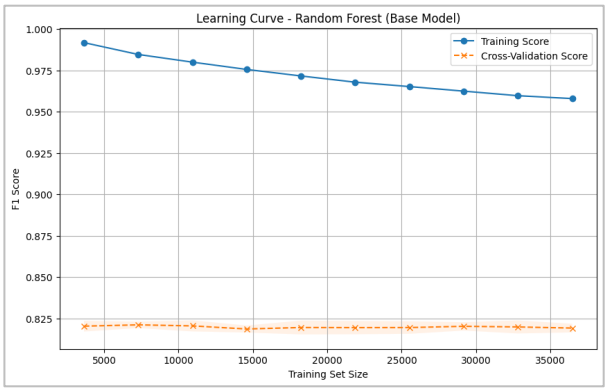
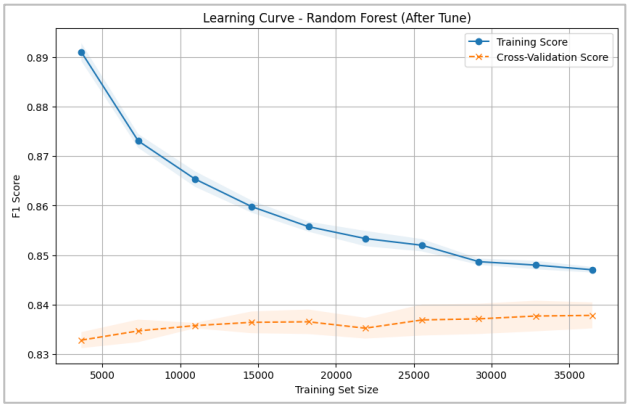
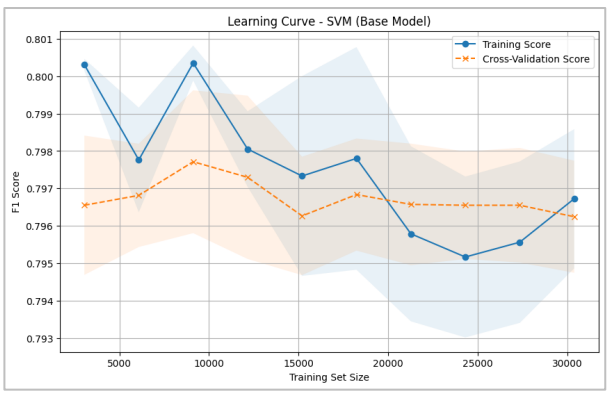
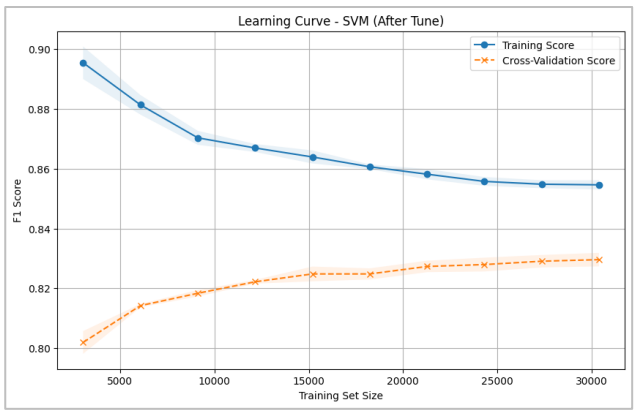
For **Random Forest**, there was a noticeable improvement after tuning. The accuracy has increased from 0.8280 to 0.8402, and both precision and recall for class 1 (students who have been bullied) improved, as did the F1-score, which increased from 0.85 to 0.86. This shows that hyperparameter tuning helped Random Forest become a more balanced and robust model, improving its ability to correctly identify both class 0 and class 1 instances.

SVM showed the largest improvement after tuning. The accuracy increased from 0.8046 to 0.8372, and both precision and recall for class 1 improved significantly. The recall for class 1 increased from 0.85 to 0.89, which indicates the model became better at identifying true positives for class 1 (student who have been bullied). The F1-score for class 1 also increased from 0.83 to 0.86, showing that tuning helped the model achieve a better balance between precision and recall.

Lastly, for **XGBoost**, the performance was already strong in the base model. It then slightly improves the accuracy from 0.8423 to 0.8434 after tuning. The precision for class 1 remained steady at 0.85, and the recall for class 1 increased slightly from 0.87 to 0.88. The F1-score for class 1 remained constant at 0.86, showing that XGBoost maintained a strong performance even after tuning.

In conclusion, while Logistic Regression showed minimal change, Random Forest and SVM both benefited significantly from hyperparameter tuning, with SVM showing the most notable improvements in recall for class 1. XGBoost remained consistent in its performance and was still the top performer overall. Therefore, based on the results, Random Forest and SVM might be considered better choices after tuning, depending on the importance of recall and precision, while **XGBoost is the best model** as it continues to be a reliable, high-performance model across all metrics.

5.2.2.1 Learning Curve

Base Model	Tuned Model
Logistic Regression	
	
Random Forest	
	
Support Vector Machine (SVM)	
	

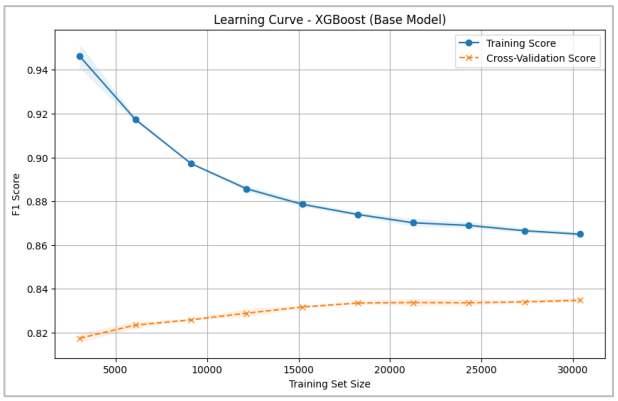
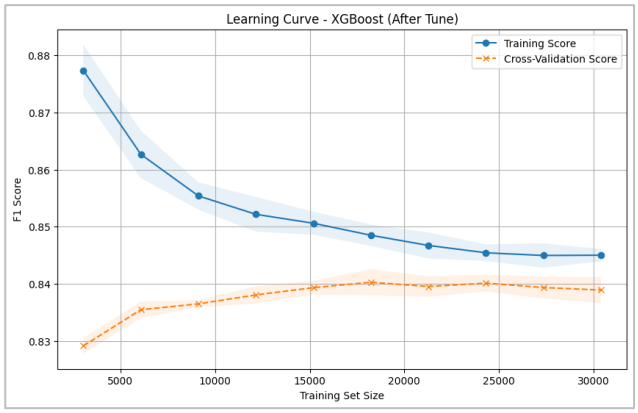
XGBoost	
	

Table 11: Comparison on Learning Curve of Base and Tuned Machine Learning Models

The figure above shows the learning curves for four machine learning models which is Logistic Regression, Random Forest, SVM, and XGBoost, comparing the performance of the base model and the tuned model. Each model has two plots, one showing the learning curve of the base model and the other showing the performance after hyperparameter tuning.

In **Logistic Regression**, the tuned model shows a slight improvement in F1-score compared to the base model. However, the overall performance of the Logistic Regression model does not significantly change after tuning, as both the training and cross-validation scores stay close to each other throughout the training set sizes.

For **Random Forest**, there is a noticeable difference between the base and tuned models. The tuned model significantly improves its performance, as indicated by the higher F1-scores and the gap between the training and cross-validation scores becoming smaller. The model performs much better after hyperparameter tuning, with the training score decreasing less rapidly, and the cross-validation score increasing, indicating better generalization.

In **Support Vector Machine (SVM)**, tuning also has a positive effect, as the F1-score increases after tuning. The learning curve of the tuned model shows better stability in the cross-validation score compared to the base model, which had a greater variation. The tuned model maintains a higher and more stable F1-score, especially as the training set size increases.

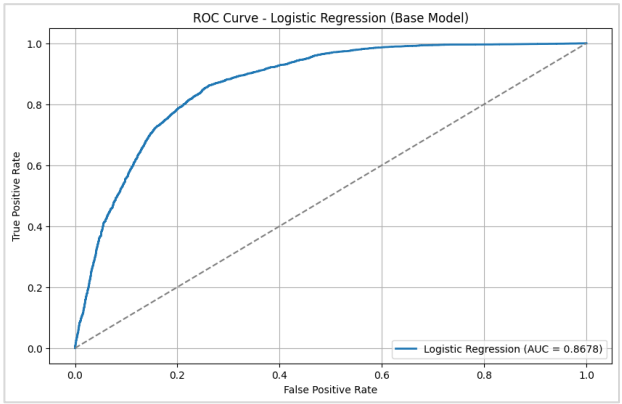
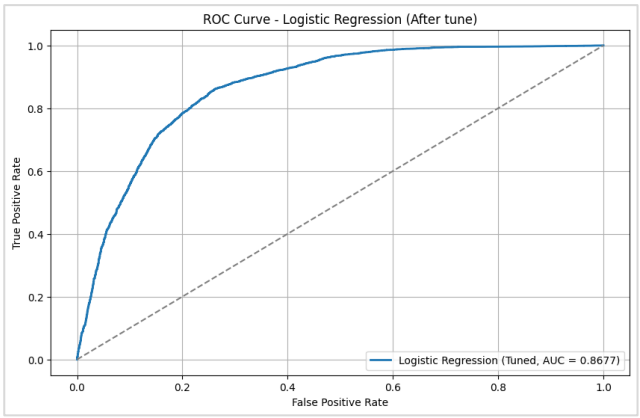
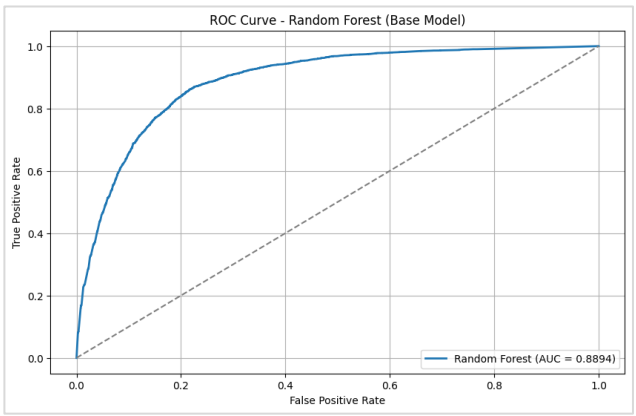
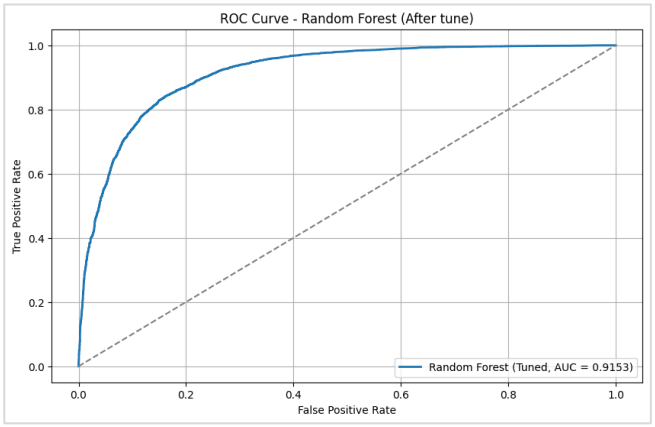
For **XGBoost**, the performance improves after hyperparameter tuning, but the change is more gradual. The gap between the training and cross-validation scores indicates that while the

model performs well, it could still benefit from further tuning, especially in terms of improving its generalization to unseen data.

As for compared to the four models, XGBoost is the best model. This is because XGBoost has the highest F1-Score and maintains the most consistent and stable performance across all training set sizes. The AUC score for XGBoost is also higher compared to the other models, meaning that its superior ability to distinguish between the two classes.

Therefore, **XGBoost (Tuned)** is the best-performing model among the ones tested.

5.2.2.2 ROC-AUC

Base Model	Tuned Model
Logistic Regression	
	
AUC = 0.8678	AUC = 0.8677
Random Forest	
	
AUC = 0.8894	AUC = 0.9153

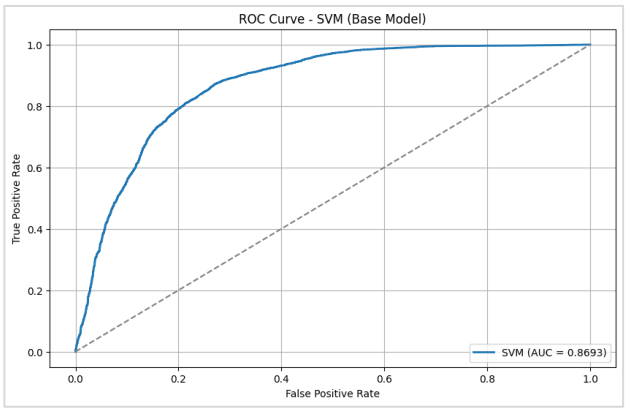
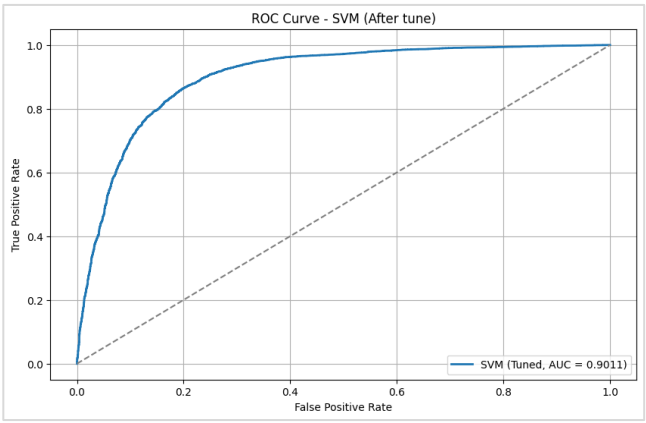
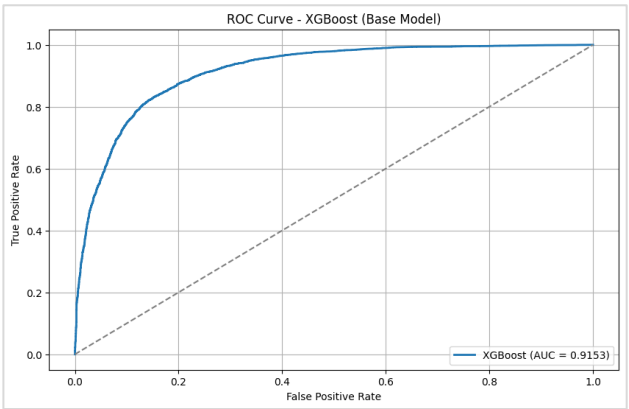
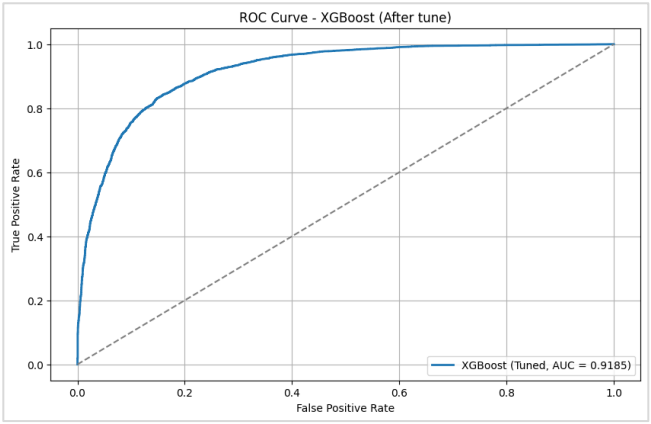
Support Vector Machine (SVM)	
	
AUC = 0.8693	AUC = 0.9011
XGBoost	
	
AUC = 0.9153	AUC = 0.9185

Table 12： Comparison on ROC-AUC curve of Base and Tuned Machine Learning Models

The figure above compares the ROC-AUC curves of the base and tuned models for Logistic Regression, Random Forest, SVM, and XGBoost. The AUC values are an indicator of how well each model distinguishes between the two classes, with higher values indicating better performance.

For **Logistic Regression**, the AUC values for the base and tuned models are very close, with the tuned model having a slightly lower AUC of 0.8677 compared to the base model's AUC of 0.8678. This indicates minimal improvement after hyperparameter tuning.

In **Random Forest**, hyperparameter tuning showed a significant improvement in AUC, with the base model having an AUC of 0.8894 and the tuned model achieving an AUC of 0.9153. This indicates a noticeable performance boost after tuning.

For **SVM**, the base model achieved an AUC of 0.8693, while the tuned model improved to 0.9011. This indicates that tuning also helped SVM to enhance its performance, but not as dramatically as Random Forest.

In **XGBoost**, both the base and tuned models had the highest AUC values among the models, with 0.9153 for the base model and 0.9185 for the tuned model. This shows that XGBoost consistently performs well, and tuning has slightly improved its AUC.

In conclusion, **XGBoost (Tuned Model) performed the best** in terms of AUC, followed by Random Forest (Tuned Model), SVM (Tuned Model), and Logistic Regression, where Logistic Regression showed the least improvement after tuning.

5.3 Feature Importance

(6.5) Best Model

```
# Compare performance metrics (accuracy and AUC) for each model
model_performance = {
    'Logistic Regression': accuracy_score(y_test, search_logreg.best_estimator_.predict(X_test)),
    'Random Forest': accuracy_score(y_test, search_rf.best_estimator_.predict(X_test)),
    'SVM': accuracy_score(y_test, search_svm.best_estimator_.predict(X_test)),
    'XGBoost': accuracy_score(y_test, search_xgb.best_estimator_.predict(X_test))
}

best_model_name = max(model_performance, key=model_performance.get)
print(f"Best model: {best_model_name}")
print(f"Accuracy: {model_performance[best_model_name]:.4f}\n")
```

```
Best model: XGBoost
Accuracy: 0.8434
```

Figure 136: Code of choosing the best model

After the comparison, the XGBoost model is the best model among all of the four models as it has the highest ACU score, highest accuracy, and strongest F1-score for Class 1, which is predicted on students who have been bullied. Therefore, below is the process of feature important for XGBoost model. This process is important to understand the significance of different features (input variables) in making predictions in a machine learning model. It tells us which features have the most influence on the model's predictions with the target variable "Bullied".

XGBoost - Feature Importance

```
# Get feature importance from the tuned XGBoost model
feature_importance_xgb = best_xgb.feature_importances_

# Create a dataframe for feature names and their corresponding importance
feature_importance_df_xgb = pd.DataFrame({
    'Feature': X_train.columns,
    'Importance': feature_importance_xgb
})

# Sort by importance (descending order)
feature_importance_df_xgb = feature_importance_df_xgb.sort_values(by='Importance', ascending=False)

# Plot the top 10 most important features
plt.figure(figsize=(10, 6))
plt.barh(feature_importance_df_xgb['Feature'][:10], feature_importance_df_xgb['Importance'][:10], color='deeppink')
plt.xlabel('Importance')
plt.title('Top 10 Feature Importance (XGBoost)')
plt.gca().invert_yaxis()
plt.grid(True)
plt.show()
```

Figure 137: Code of feature important for XGBoost

The figure above shows the code of feature important of a trained XGBoost model. First, it retrieves the importance scores of each feature using best_xgb.feature_importances_, where

best_xgb is the tuned XGBoost model. These important scores indicate how much each feature contributes to the model's decision-making process. The top 10 most important features are visualized in a bar chart as shown in figure below.

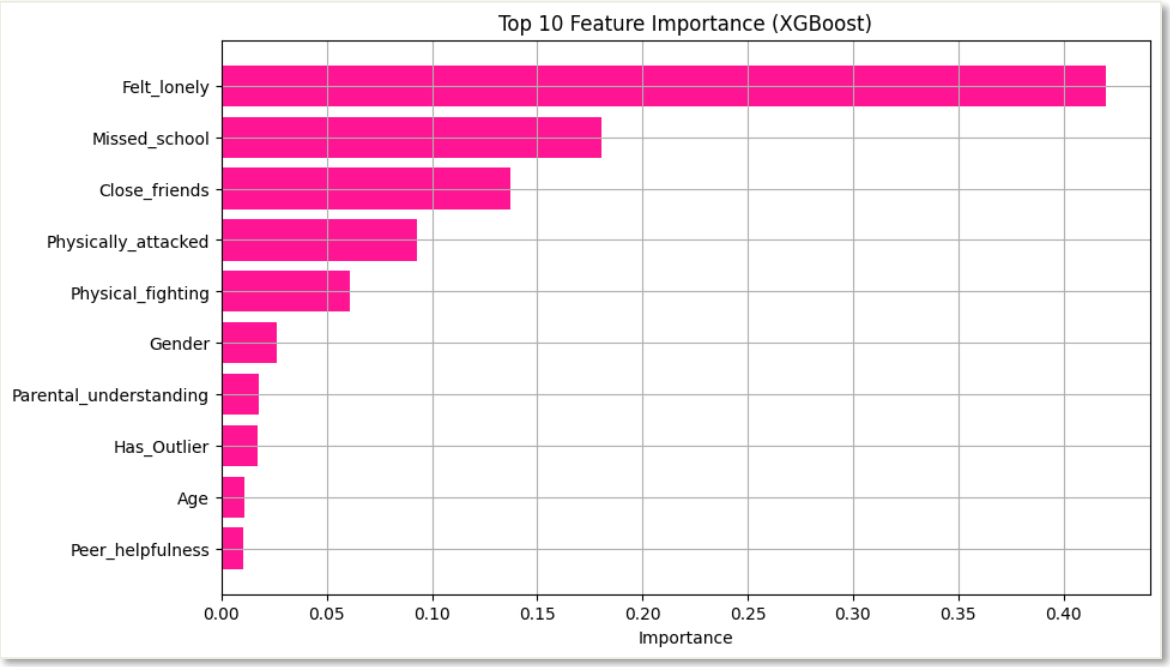


Figure 138: Top 10 features important for XGBoost

The chart above visualizes the top 10 most important features identified by the tuned XGBoost model. The feature **"Felt_lonely"** stands out as the most influential feature, with the highest importance score, followed by **"Missed_school"** and **"Close_friends".** These features are important in predicting the target variable, which could be related to social or emotional factors. The chart shows that the model assigns significant importance to social well-being factors such as feelings of loneliness and school absenteeism. Other important features include **"Physically_attacked",** "**Physical_fighting",** and **"Gender".** These features also play an important role in the model's predictions. The features **"Has_Outlier", "Age",** and **"Peer_helpfulness"** have relatively lower importance, suggesting that while they contribute to the model's decision-making process, their influence is not so strong compared to the higher-ranked features. This analysis can help in understanding which factors are most critical for the model's performance and could guide further analysis or feature engineering.

5.3.1 Analysis

As shown in the top 10 feature analysis for this dataset using XGBoost model, the variable "Felt_lonely" is the most influential feature. Loneliness is a significant predictor of bullying risk among students due to its profound impact on their social and emotional well-being. Research indicates that students who experience feelings of loneliness are more susceptible to both being bullied and engaging in bullying behavior themselves. A study published in *PubMed Central* found a strong and graded association between loneliness and bullying victimization at school and cyberbullying. The study revealed that students exposed to habitual bullying, either at school or online, had significantly higher odds of experiencing loneliness. For example, students who have been bullied at school several times a week had an adjusted odds ratio (OR) of 11.21 for loneliness, indicating a more than 11-fold increase in the likelihood of feeling lonely compared to their peers who were not bullied (Madsen et al., 2024).

Besides, the second important variable is "Missed_school". Bullying has been identified as a significant factor contributing to students' school absenteeism. Research indicates that students who are victims of bullying are more likely to miss school without permission (Sifferlin, 2014). A study in Germany involving secondary school children found that bullying victimization was significantly associated with higher school anxiety and increased school absenteeism. The structural equation modeling approach used in the study revealed that students who experienced bullying were more likely to report school displeasure and higher absenteeism rates, underscoring the impact of bullying on students' emotional well-being and school attendance (Schlesier et al., 2023). Students who being bullied are more likely to miss school, one of the reasons is they are scared to attend school due to they often being bullied in school.

Additionally, the "Close_friends" variable is the top 3 important. This is because Students who have been bullied often find it challenging to form close friendships due to a combination of emotional, cognitive, and social factors. Victims of bullying frequently experience increased levels of anxiety, depression, and low self-esteem, which can hinder their ability to initiate and maintain peer relationships.

5.4 Model Deployment

```
import joblib

joblib.dump(best_xgb, 'Bullying_model.pkl')
```

```
['Bullying_model.pkl']
```

Figure 139: Code of Save the Best Model

The figure above shows the code of saving the trained best model, which is XGBoost model. It is saved into a binary file which named “Bullying_model.pkl” by importing the joblib in python. This file is imported during model deployment.

```
import streamlit as st
import pickle
import pandas as pd

# Load the trained model
with open("Bullying_model.pkl", "rb") as file:
    model = pickle.load(file)

st.set_page_config(page_title="Bullying Prediction System", page_icon=":brain:", layout="wide")
```

Figure 140: Code of building a system

The figure above shows the code of building a system. The code starts with importing the necessary libraries which is Streamlit (for the user interface), pickle (for loading the trained machine learning model), and pandas (for data manipulation). The trained model, which is saved in a file called "Bullying_model.pkl," is loaded using the pickle library. The page of this system is titled "Bullying Prediction System".

5.4.1 Page Navigator

```
# Sidebar Navigation
st.sidebar.title("Page Navigation")
tabs = ["🏠 Home", "🎯 Prediction"]
page = st.sidebar.radio("Select a Page", tabs)

if page == "🏠 Home":
    home_page()
elif page == "🎯 Prediction":
    prediction_page()
```

Figure 141: Code for Page Navigator

This is the code for building a sidebar navigation to navigate the pages.

5.4.2 Home Page

```
# Home Page
def home_page():
    st.markdown('<div class="title"><h1>🏠 Bullying Victimization Risk Prediction System</h1>'
                '<div class="description">" Trained on a Global School-Based Student Health Survey (GSHS) Dataset in Argentina. | Dataset from Kaggle.com "</div></div>'
                , unsafe_allow_html=True)

    st.markdown('<div class="custom-image">.</div></div>'
                , unsafe_allow_html=True)
```

Figure 142: Home page interface

The figures above show the code and the interface of the home page of the system, with a big title of "Bullying Victimization Risk Prediction System". There is a page navigation on the left side of the page, which can navigate to Prediction Page.

```
with st.expander("**🧠 Understanding Bullying**"):
    st.info("""
    Bullying is an aggressive behavior that involves unwanted, negative actions.
    It usually involves a power imbalance and is repeated over time. \n\n
    Bullying can be:
    - **Physically**:  Hitting, Pushing
    - **Verbally**: Name-calling, Threats
    - **Social/Relational**: Exclusion, Spreading rumors
    - **Cyberbullying**: via social media, Texting \n\n
    The victims of bullying often experience anxiety, depression, low self-esteem, and may avoid school.
    """)


with st.expander("**🔧 This system is to:**"):
    col1, col2 = st.columns(2)
    with col1:
        st.success("#### 🎯 Prediction\nThis system is to predict whether a student are on risk to being bullid.")
    with col2:
        st.success("#### 🧠 Recommendation\nStudents, parents and educators can get recommendation on preventing bullying.")
```

Figure 143: Code for information 1 (Home Page)

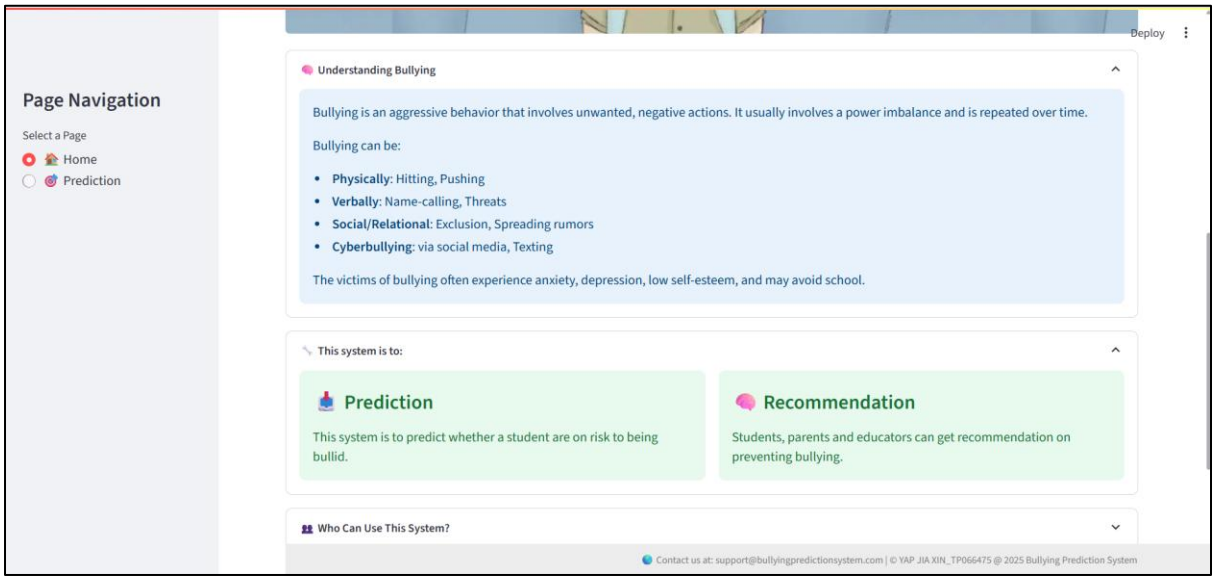


Figure 144: Interface of Information 1 (Home Page)

The figure above shows the code and interface of a bullying prediction system built using Streamlit. The code snippet begins with an expandable section titled "Understanding Bullying," which provides an explanation of bullying, its different forms, and the emotional and psychological effects it can have on victims. The explanation covers physical bullying, verbal bullying, social bullying, and cyberbullying, emphasizing the negative impacts such as anxiety, depression, and avoidance of school. Below this section, the code organizes the system's objectives into two main columns: "Prediction" and "Recommendation." The "Prediction" section highlights the system's ability to predict whether a student is at risk of being bullied, while the "Recommendation" section offers guidance for students, parents, and educators on how to prevent bullying.

```
with st.expander(" **🫂 Who Can Use This System?**"):
        st.success("""
        - 🏫 **Students**: Identify risk level by applying their informations.
        - 👪 **Parents**: Understand emotional and behavioral risk factors of their children.
        - 🏫 **Educators**: Identify at-risk students early in school environment.
        """)


with st.expander("**📈 Machine Learning Model Performance Overview**"):
    st.info("""
    **XGBoost Classifier**
    - **Accuracy**: 84%
    - **Precision**: 85%
    - **Recall**: 88%
    - **F1 Score**: 86%
    """)

st.markdown("> 🧠 *“Bullying is not a reflection of the victim's character, but of the bully's lack of it.”*")
```

Figure 145: Code for Information 2 (Home Page)

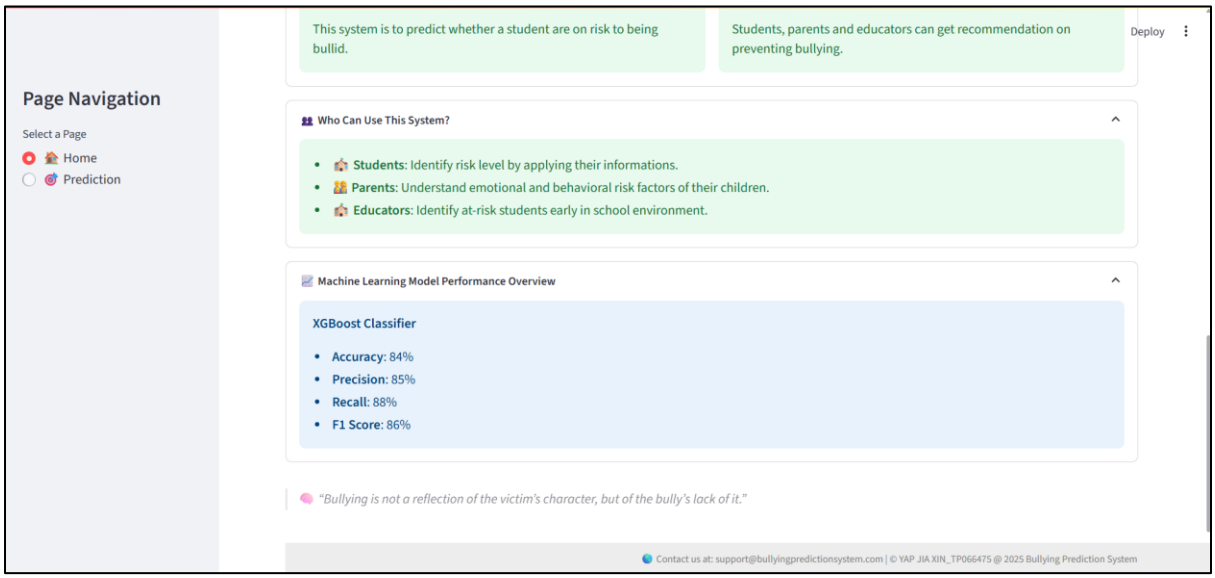


Figure 146: Interface of Information 2 (Home Page)

The figure above shows the code and interface for two additional sections of the bullying prediction system. The first section, titled "Who Can Use This System?", is designed to explain the target users of the system. The code snippet divides the users into three categories: students, parents, and educators. Students can apply their information to identify their bullying risk levels, parents can better understand the emotional and behavioral risk factors of their children, and educators can identify at-risk students early in the school environment. This helps tailor the system's functionality to meet the needs of each group.

The second section in the code is titled "Machine Learning Model Performance Overview." This section provides an overview of the XGBoost classifier's performance metrics. The accuracy of the model is 84%, with precision at 85%, recall at 88%, and an F1 score of 86%. These metrics show the model’s effectiveness in predicting bullying victimization, with recall being particularly important in minimizing false negatives. Additionally, the markdown

displays an impactful quote: "Bullying is not a reflection of the victim’s character, but of the bully’s lack of it." This serves as a reminder of the core message against bullying.

The interface mirrors the layout described in the code, showing clear and accessible information about who can use the system and the performance of the machine learning model. It offers a concise and user-friendly experience, making it easy for users to understand the purpose and effectiveness of the system.

5.4.3 Prediction Page

```python
# Map categorical data to numerical values
def map_categorical_data(input_data):
    physically_attacked_mapping = {
        "0 times": 0,
        "1 time": 1,
        "2-3 times": 2,
        "4-5 times": 4,
        "6-7 times": 6,
        "8-9 times": 8,
        "10-11 times": 10,
        "12 or more times": 12
    }

    physical_fighting_mapping = {
        "0 times": 0,
        "1 time": 1,
        "2-3 times": 2,
        "4-5 times": 4,
        "6-7 times": 6,
        "8-9 times": 8,
        "10-11 times": 10,
        "12 or more times": 12
    }

    missed_school_mapping = {
        "0 days": 0,
        "1-2 days": 1,
        "3-5 days": 3,
        "6-9 days": 6,
        "10 or more days": 10
    }

    close_friends_mapping = {
        "3 or more": 3,
    }

    # Apply the mappings
    input_data["Physically_attacked"] = input_data["Physically_attacked"].map(physically_attacked_mapping)
    input_data["Physical_fighting"] = input_data["Physical_fighting"].map(physical_fighting_mapping)
    input_data["Close_friends"] = input_data["Close_friends"].map(close_friends_mapping)
    input_data["Missed_school"] = input_data["Missed_school"].map(missed_school_mapping)


    return input_data
```

Figure 147: code for mapping categorical data to numerical data (prediction page)

The code provided is designed to map categorical data into numerical values, which is essential for machine learning algorithms that require numerical input. It begins by defining a function map_categorical_data (input_data) that takes in a dataset and applies mapping to specific columns containing categorical data. These columns are related to the frequency of physical attacks, physical fights, school absenteeism, and the number of close friends. For each of these columns, a dictionary is created to map text values to numerical values. For example, the physically_attacked_mapping dictionary converts categories like "0 times" and "1 time" to numeric values 0 and 1, respectively. Similarly, the physical_fighting_mapping and missed_school_mapping dictionaries map text descriptions like "2-3 times" and "1-2 days" into numerical values. The close_friends_mapping column is mapped to the number 3 for students who report having "3 or more" close friends. The .map() function is then applied to each relevant column in the dataset, replacing the categorical values with their corresponding

numerical representations. After performing these mappings, the function returns the updated dataset, now ready for use in machine learning models that require numerical data.

```python
# Prediction Page
def prediction_page():
    st.markdown('<div class="headerpp">🔮 Bullying Risk Predictor</div>', unsafe_allow_html=True)
    st.markdown('<div class="descriptionpp">Predict the bullying risk level of students and receive personalized recommendations from different aspects.</div>',

    # Role Selection
    role = st.selectbox("**Please select your role:**", ["Student", "Parent", "Educator"])

    # Collecting information based on the selected role
    st.subheader("**📝 Student's details:**")


    col1, col2 = st.columns(2)
    with col1:
        gender = st.selectbox("**Gender**", ["Male", "Female"])
        gender = 1 if gender == "Male" else 0
        age = st.slider("**Age**", 10, 18, 13)
        peer_helpfulness = st.slider("**Peer Helpfulness (1 = Never to 5 = Always)**", 1, 5, 3)
        physically_attacked = st.selectbox("**Physically Attacked (times)**", ["0 times", "1 time", "2-3 times", "4-5 times", "6-7 times",
                                                                               "8-9 times", "10-11 times", "12 or more times"])
        close_friends = st.selectbox("**Number of Close Friends**", [0, 1, 2, "3 or more"])

    with col2:
        missed_school = st.selectbox("**Missed School (days)**", [0, "1-2 days", "3-5 days", "6-9 days", "10 or more days"])
        felt_lonely = st.slider("**Felt Lonely (1 = Never to 5 = Always)**", 1, 5, 3)
        parental_understanding = st.slider("**Parental Understanding (1 = Never to 5 = Always)**", 1, 5, 3)
        physical_fighting = st.selectbox("**Physical Fighting (times)**",["0 times", "1 time", "2-3 times", "4-5 times", "6-7 times",
                                                                          "8-9 times", "10-11 times", "12 or more times"])
        weight_status = st.selectbox(
            "**Physical Appearence:**",
            ["Underweight", "Overweight", "Obese"]
        )

        were_underweight = 1 if weight_status == "Underweight" else 0
        were_overweight = 1 if weight_status == "Overweight" else 0
        were_obese = 1 if weight_status == "Obese" else 0
```

Figure 148: Code for prediction page

The provided code is part of a Streamlit-based application that allows users to predict bullying risks based on various student-related factors. The prediction_page function begins by displaying an introductory message on the page using st.markdown, explaining the purpose of the page. A role selection dropdown (st.selectbox) is used to collect input from the user, where they can select whether they are a "Student," "Parent," or "Educator." Based on the role selected, the application dynamically adjusts to collect relevant information.

The page layout is divided into two columns using st.columns(2) for better visual structure. In the first column, users are prompted to provide information about the student's gender, age, peer helpfulness, physical attacks experienced, and the number of close friends. For each of these attributes, the code uses widgets like st.selectbox for categorical selections and st.slider for range-based inputs. The user can choose from options like "Male" or "Female" for gender, select an age range, and rate peer helpfulness or physical attacks on various scales.

The second column collects additional data, including missed school days, feelings of loneliness, parental understanding, and physical fighting experiences. The weight status of the student is also collected here, and based on this input, the weight status is mapped to numerical values such as "Underweight," "Overweight," or "Obese." The function then stores the input data, preparing it for the next steps in the prediction process.

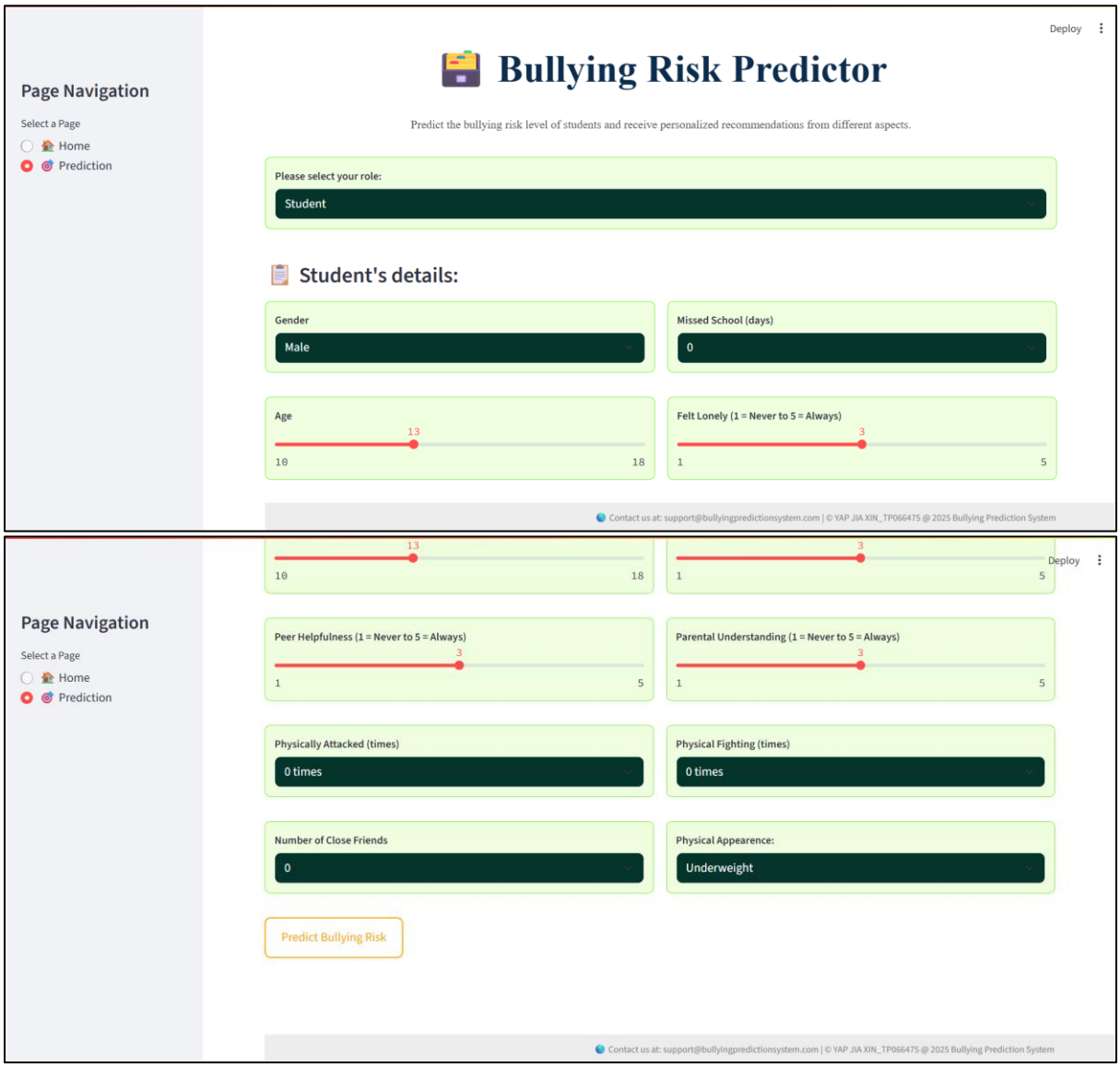


Figure 149: Interface of Prediction Page

The figures above show the interface of Prediction Page. The user should fill in the information for the students first then click on the prediction button.

```python
# Predict button
if st.button("Predict Bullying Risk"):
    input_data = pd.DataFrame([[
        age, gender, physically_attacked, physical_fighting,
        felt_lonely, close_friends, missed_school,
        peer_helpfulness, parental_understanding,
        were_underweight, were_overweight, were_obese, 0
    ]], columns=[
        "Age", "Gender", "Physically_attacked", "Physical_fighting",
        "Felt_lonely", "Close_friends", "Missed_school",
        "Peer_helpfulness", "Parental_understanding",
        "Were_underweight", "Were_overweight", "Were_obese", "Has_Outlier"
    ])

    # Map categorical data to numeric values
    input_data = map_categorical_data(input_data)

    prediction_proba = model.predict_proba(input_data)[0][1]  # prob of being bullied

    # Based on role selection, provide customized recommendations
    if role == "Student":
        if prediction_proba >= 0.65:
            st.subheader("Prediction Result:")
            st.error("**🚨 High Risk of Being Bullied**\n" \
            f"\n**📊 Probability of being bullied: `{prediction_proba * 100:.2f}%`**")

            st.markdown('<div class="custom-space"></div>', unsafe_allow_html=True)

            st.subheader("Recommendations:")
            st.write("It is crucial to intervene immediately. Seek support from a counselor, teacher, or trusted adult.")
            with st.expander("**🧠 What you can do?**"):
                st.info("""
                - Talk to a trusted adult such as a teacher, counselor, or parent.
                - Avoid isolating yourself. Stay around supportive friends.
                - Document bullying incidents (date, time, who was involved).
                - Call helplines if needed.
                """)
            with st.expander("**🛡️ How to Prevent Future Bullying?**"):
                st.info("""
                - Engage in activities that build self-esteem, like sports, hobbies, or volunteering.
                - Bullying often occurs when individuals are isolated. Stay with friends or peers when possible.
                - Learn assertive communication techniques to stand up for yourself respectfully.
                """)

        elif 0.30 <= prediction_proba < 0.65:
            st.subheader("Prediction Result:")
            st.warning("**⚠️ Medium Risk of Being Bullied**\n" \
            f"\n**📊 Probability of being bullied: `{prediction_proba * 100:.2f}%`**")

            st.markdown('<div class="custom-space"></div>', unsafe_allow_html=True)

            st.subheader("Recommendations:")
            st.write("While the situation is not urgent, regular check-ins and emotional support from peers and parents could help.")
            with st.expander("**🧠 What you can do:**"):
                st.info("""
                - Talk to someone you trust about how you feel.
                - Avoid being alone in places where bullying may occur.
                - Seek guidance from school counselors or mentors.
                """)
            with st.expander("**🛡️ Strengthen Support System**"):
                st.info("""
                - Join school clubs or activities that promote inclusivity and mutual respect.
                - Keep communicating with positive peers so that they will uplift you.
                - Participate in workshops or programs that teach communication and coping strategies.
                """)

        else:
            st.subheader("Prediction Result:")
            st.success("**✅ Low Risk of Being Bullied**\n" \
            f"\n**📊 Probability of being bullied: `{prediction_proba * 100:.2f}%`**")

            st.markdown('<div class="custom-space"></div>', unsafe_allow_html=True)

            st.subheader("Recommendations:")
            st.write("You are less likely to being bully, just maintain!!!")
            with st.expander("**🧠 What you can do:**"):
                st.info("""
                - Stay aware of your surroundings, especially in social situations.
                - Report any unusual behavior or bullying. This can help your friend who are being bullied.
                - Maintain positive peer relationships.
                """)
            with st.expander("**🛡️ Keep a Positive Environment**"):
                st.info("""
                - Be inclusive and support peers.
                - Be a role model to show kindness and inclusivity to others to help create a supportive environment.
                - Participate in anti-bullying programs that may be available in your school.
                """)
```

Figure 150: Code of prediction Button (1) – For students

The provided code snippet is for predicting the bullying risk of a student based on various input data and generating personalized recommendations for the student. When the **"Predict Bullying Risk"** button is clicked, the data is processed, and the bullying risk prediction is made.

If the prediction probability for the student being bullied is greater than or equal to 0.65 (high risk), a warning is displayed with the result showing a high risk of being bullied. The recommendations encourage immediate intervention, such as seeking support from a counselor, teacher, or trusted adult, and documenting incidents. The system also suggests preventive measures, such as engaging in activities to build self-esteem and avoiding isolation. It emphasizes staying around supportive friends and learning communication techniques to stand up for oneself.

If the prediction probability is between 0.30 and 0.65 (medium risk), a warning is displayed indicating a medium risk of being bullied. The recommendation includes regular check-ins and emotional support from peers and parents. The system suggests talking to someone trusted about feelings and ensuring safety in places where bullying may occur. It also encourages strengthening the support system through activities that promote inclusivity and mutual respect.

If the prediction probability is less than 0.30 (low risk), the system reassures that the student is less likely to be bullied but still advises maintaining a supportive environment. The recommendations encourage staying aware of surroundings, practicing positive behavior, and continuing to build positive relationships with others. The system suggests keeping a positive environment and participating in anti-bullying programs at school.

The code helps generate personalized steps for the student based on the probability of being bullied, promoting the overall well-being of the student through intervention and support.

```
elif role == "Parent":
    if prediction_proba >= 0.65:
        st.subheader("Prediction Result:")
        st.error("**🚨 High Risk of Being Bullied**\n" \
        f"\n**📊 Probability of being bullied: `{prediction_proba * 100:.2f}%`**")

        st.markdown('<div class="custom-space"></div>', unsafe_allow_html=True)

        st.subheader("Recommendations:")
        st.write("Ensure your child feels emotionally supported and work with school authorities to address the issue.")
        with st.expander("**🧠 What should a parents do?**"):
            st.info("""
            - Open a conversation with your child. Ask how they feel and encourage them to share their experiences.
            - Inform the school about the situation so they can intervene immediately.
            - Monitor your child's behavior and emotional. Look out for signs of distress such as withdrawal, anxiety, or reluctance to go to school.
            """)
        with st.expander("**🛡️ Preventive steps:**"):
            st.info("""
            - **Promote a safe environment at home**: Encourage your child to express themselves openly.
            - **Teach conflict resolution skills**: Help your child learn to handle situations calmly and assertively.
            - **Support extracurricular activities**: Encourage your child to participate in clubs, sports, or other activities to build friendships.
            """)

    elif 0.30 <= prediction_proba < 0.65:
        st.subheader("Prediction Result:")
        st.warning("**⚠️ Medium Risk of Being Bullied**\n" \
        f"\n**📊 Probability of being bullied: `{prediction_proba * 100:.2f}%`**")

        st.markdown('<div class="custom-space"></div>', unsafe_allow_html=True)

        st.subheader("Recommendations:")
        st.write("Keep the lines of communication open with your child and work together with the school to monitor behavior.")
        with st.expander("**🧠 What should a parents do?**"):
            st.info("""
            - Check in regularly with your child to understand how they are feeling and what's happening at school.
            - Collaborate with teachers and counselors to monitor the situation and ensure your child feels safe.
            - Encourage them to participate in activities where they can excel and make friends.
            """)
        with st.expander("**🏠 Prevention at home:**"):
            st.info("""
            - Talk openly about bullying and safety with your child so they understand what it is and how to react.
            - Praise and support your child's achievements oftenly.
            - Teach your child how to set boundaries with peers in a healthy way.
            """)

    else:
        st.subheader("Prediction Result:")
        st.success("**✅ Low Risk of Being Bullied**\n" \
        f"\n**📊 Probability of being bullied: `{prediction_proba * 100:.2f}%`**")

        st.markdown('<div class="custom-space"></div>', unsafe_allow_html=True)

        st.subheader("Recommendations:")
        st.write("Encourage your child to continue developing positive relationships and seek help if needed.")
        with st.expander("**🌟 How to maintain:**"):
            st.info("""
            - Keep open communication with your child. Ask your child about their day regularly, and make sure they feel comfortable talking about any issues.
            - Create an open line of communication with the school to ensure your child's well-being.
            - If your child starts showing signs of anxiety or depression, address it early.
            """)
        with st.expander("**🏠 Reinforcement at Home**"):
            st.info("""
            - Encourage your child to engage in activities that make them feel good about themselves.
            - Teach your child the importance of empathy and kindness towards others.
            - Help your child to engage with a positive group of friends who look out for each other.
            - Continue building confidence and social skills.
            """)
```

Figure 151: Code of Prediction Button (2) – For Parents

The code provided outlines a system that gives tailored recommendations based on the predicted bullying risk for a child, with specific guidance for parents. When the system identifies a **high risk** of bullying (with a probability of 0.65 or greater), it emphasizes the importance of providing emotional support to the child and working closely with school authorities to address the issue. Parents are advised to open communication with their child, asking them how they feel and encouraging them to share their experiences. It is also recommended that parents inform the school about the situation to facilitate timely intervention. Preventive steps include promoting a safe home environment, encouraging open

self-expression, teaching conflict resolution skills, and supporting the child's involvement in extracurricular activities to help them form friendships.

In cases of **medium risk** (with a probability between 0.30 and 0.65), the system suggests that parents maintain regular communication with their child and collaborate with school authorities to monitor the situation. It stresses the importance of checking in with the child frequently to understand how they feel and ensuring the child feels safe. Additionally, parents are encouraged to help their child engage in activities that can promote social connections, such as joining clubs or other group activities that allow the child to build friendships.

For **low-risk** cases (with a probability of less than 0.30), the system reassures parents that their child is less likely to be bullied but emphasizes the importance of continuing to foster positive relationships. Parents are encouraged to maintain open communication with their child and ensure they feel comfortable discussing any issues. To support the child's emotional growth, parents should promote activities that help develop their child's confidence and social skills, as well as guide them in building healthy friendships.

The system aims to offer parents actionable steps at each risk level, ensuring they can provide the necessary support and take preventive measures to safeguard their child from bullying.

```
elif role == "Educator":
    if prediction_proba >= 0.65:
        st.subheader("Prediction Result:")
        st.error("**⚠️ High Risk of Being Bullied**\n" \
        f"\n**📊 Probability of being bullied: `{prediction_proba * 100:.2f}%`**")

        st.markdown('<div class="custom-space"></div>', unsafe_allow_html=True)

        st.subheader("Recommendations:")
        st.write("Immediate intervention is necessary. Coordinate with counselors and parents to develop a safety plan for the student.")
        with st.expander("**🧠 What should an educator do?**"):
            st.info("""
            - Talk to the student in a safe environment to understand their experience.
            - Provide one-on-one support for the student.
            - Work with counselors, social workers, and parents to address the issue.
            - Monitor the student closely. Ensure that they feel safe, both physically and emotionally, at school.
            """)
        with st.expander("**🛡️ Ways to prevent and stop bullying:**"):
            st.info("""
            - Review and enforce anti-bullying policies at school.
            - Encourage students to engage in group activities and foster an inclusive school culture.
            - Ensure that all school staff are trained in recognizing bullying and knowing how to respond effectively.
            """)

    elif 0.30 <= prediction_proba < 0.65:
        st.subheader("Prediction Result:")
        st.warning("**⚠️ Medium Risk of Being Bullied**\n" \
        f"\n**📊 Probability of being bullied: `{prediction_proba * 100:.2f}%`**")

        st.markdown('<div class="custom-space"></div>', unsafe_allow_html=True)

        st.subheader("Recommendations:")
        st.write("Monitor the student's interactions closely and provide additional support through peer counseling or school resources.")
        with st.expander("**🧠 What should an educator do?**"):
            st.info("""
            - **Observe classroom dynamics**: Be vigilant in noticing any signs of bullying or social exclusion.
            - **Talk to the student**: Engage them in a conversation to understand their experience and offer support.
            - **Provide support groups**: Help the student connect with peers who can provide emotional support.
            """)
        with st.expander("**🛡️ Preventive Actions:**"):
            st.info("""
            - **Teach empathy**: Include emotional intelligence training as part of the curriculum.
            - **Provide peer mentoring**: Pair students with positive role models who can help them navigate social challenges.
            - **Regular check-ins**: Keep monitoring the student's well-being to ensure they are coping with the school environment.
            """)

    else:
        st.subheader("Prediction Result: ")
        st.success("**✅ Low Risk of Being Bullied**\n" \
        f"\n**📊 Probability of being bullied: `{prediction_proba * 100:.2f}%`**")

        st.markdown('<div class="custom-space"></div>', unsafe_allow_html=True)

        st.subheader("Recommendations:")
        st.write("Continue to monitor the student's social dynamics and offer support as needed.")
        with st.expander("**🧠 What educator can do?**"):
            st.info("""
            - **Check in with the student regularly**: Make sure they feel comfortable sharing any concerns they might have.
            - **Foster a positive school environment**: Encourage all students to engage in an inclusive and respectful manner.
            - **Maintain Safe Environment**: Ensure the student knows they have a safe space at school to talk about their concerns.
            """)
        with st.expander("**🛡️ Preventive steps:**"):
            st.info("""
            - **Promote empathy**: Continue fostering an environment of kindness and understanding.
            - **Encourage teamwork**: Have students collaborate on projects and group work to promote positive socialization.
            - **Celebrate diversity**: Emphasize the importance of embracing differences in your classroom.
            - Watch for early signs of peer conflict.
            """)
```

Figure 152: Code of Prediction Button (3) – For Educators

The provided code snippet helps implement tailored recommendations for educators in the bullying risk prediction system, adjusting advice based on the risk level of bullying. When the prediction probability is high (above 0.65), indicating a high risk of bullying, the system issues a warning with the probability of bullying and stresses the importance of immediate intervention. Educators are advised to ensure the student is in a safe environment, offer one-on-one support, and work with counselors and parents to address the bullying. Additionally,

educators are encouraged to enforce anti-bullying policies, foster an inclusive school culture through group activities, and ensure all staff are trained to handle bullying situations effectively.

For a medium risk (prediction probability between 0.30 and 0.65), the system warns that the student may still be at risk and advises monitoring the student's interactions closely. Educators are encouraged to provide additional support through peer counseling or school resources, observe classroom dynamics for signs of bullying or exclusion, and engage the student in conversations to understand their experience. Recommendations also suggest teaching emotional intelligence and pairing students with positive role models to help them cope with challenges.

In cases of low bullying risk (prediction probability below 0.30), the system gives a success message with the low probability of bullying and advises educators to maintain regular check-ins with the student to ensure their well-being. The focus here is on fostering a positive school environment, promoting empathy, and encouraging teamwork among students. Educators are also encouraged to celebrate diversity and watch for any early signs of peer conflicts, helping the student maintain healthy relationships.

Overall, the system provides customized, role-specific guidance for educators to prevent bullying and offer support based on the student's risk level, ensuring that the student's emotional and social needs are met.

```
# Display helplines
st.markdown('<div class="custom-space"></div>', unsafe_allow_html=True)
st.subheader("Need any help?")
with st.expander("**Call to Helpline**"):
    st.subheader("**Emergency Contacts and Helplines (Malaysia)**")
    st.error("""
    - **Talian Kasih (24/7 Hotline)**: 📞 15999
    - **Befrienders KL** (emotional support): 📞 03-7627 2929 / 🌐 [befrienders.org.my](https://www.befrienders.org.my)
    - **Childline Malaysia** (for youth under 18): 📞 15999

    _You can call anonymously. Everything you say is confidential._
    """)
```

Figure 153: Code for the Helpline

This code is the last functionality in the prediction page. This is the helpline which provided to help the users who are needed help.

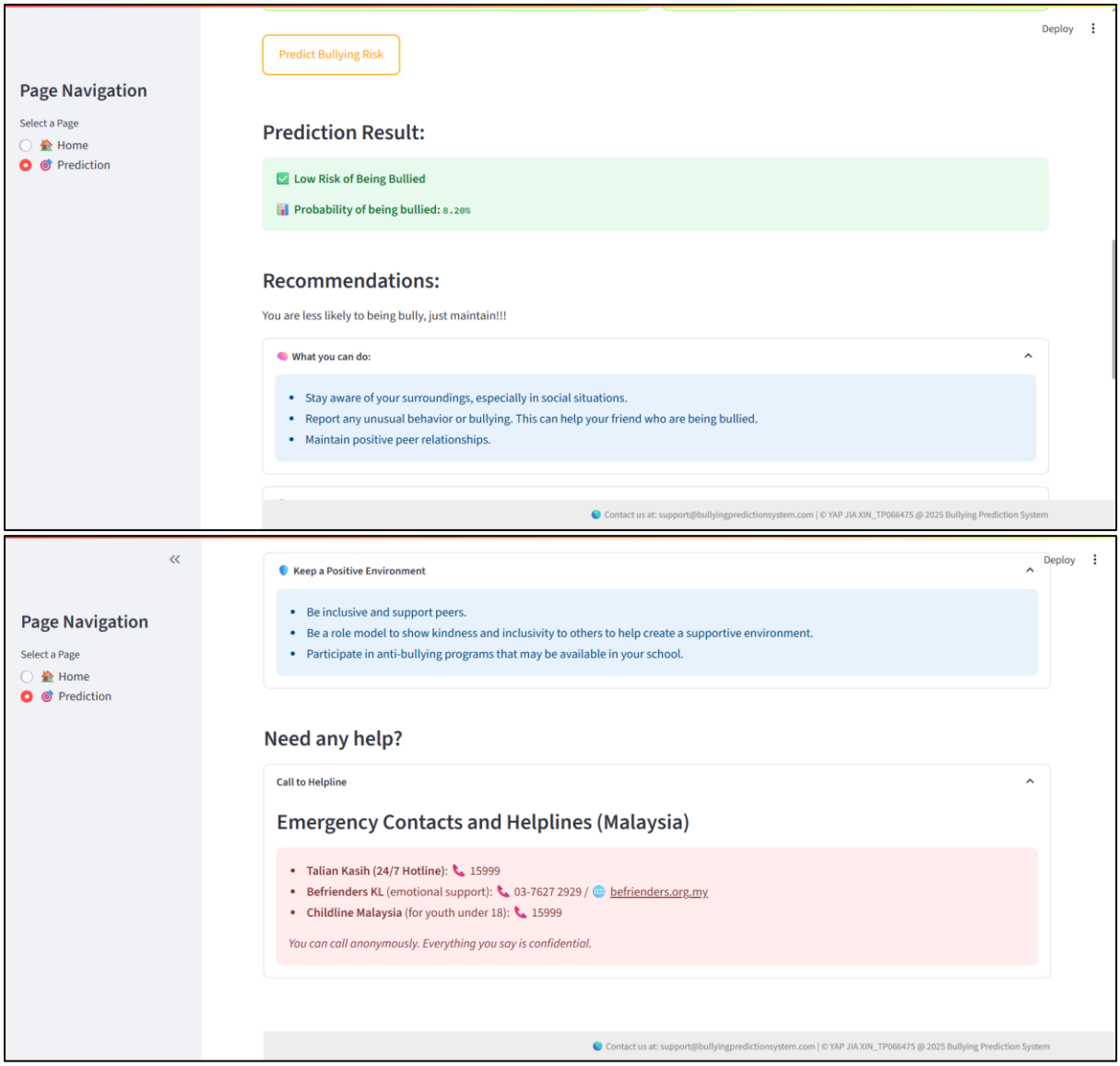


Figure 154: Sample interface after clicking prediction button - user student

5.5 Summary

This chapter have reviewed the performance of different machine learning models for predicting bullying victimization risk. The four models tested which is Logistic Regression, Random Forest, Support Vector Machine, and XGBoost were evaluated on their ability to predict whether a student had been bullied or not. The performance of each model was measured using accuracy, precision, recall, F1-score, and ROC-AUC, which showed how well each model was able to classify students who were bullied versus those who were not.

The results revealed that XGBoost was the best-performing model, with the highest accuracy, precision, and F1-score. This model consistently outperformed the others, particularly in identifying bullied students (Class 1). Random Forest also performed well, especially after hyperparameter tuning, showing improvements in accuracy, precision, and recall. On the other hand, while Logistic Regression showed minimal improvement after tuning, it still performed decently. Support Vector Machine (SVM) also benefited from tuning, with its recall for identifying bullied students improving significantly.

One of the key findings from the feature importance analysis was that emotional and social factors played a significant role in predicting bullying. Features like “Felt_lonely” and “Missed_school” were found to be among the most important predictors of bullying victimization. This suggests that students who feel isolated or miss school frequently are more likely to be bullied.

Lastly, the chapter also covered how the best model, XGBoost, was saved and deployed into a prediction system. This system is designed to help students, parents, and educators to assess bullying risks in real time, providing valuable insights to support intervention efforts. Overall, the results demonstrate that machine learning models, particularly XGBoost, can be very effective tools in predicting bullying, helping to create safer environments for students and improving early intervention strategies.

CHAPTER 6: CONCLUSION

6.1 Critical Evaluation

This project successfully created a Bullying Victimization Risk Prediction System, designed to identify students who are at risk of being bullied. The main achievement of this project was the ability to predict bullying risks early, allowing schools to intervene and reduce bullying. The system used four machine learning models which is Logistic Regression, Random Forest, Support Vector Machine (SVM), and XGBoost. After testing these models, XGBoost performed the best, giving the most accurate and reliable predictions. This success shows that the system is effective in predicting bullying and can be a valuable tool for educators, parents, and policymakers who want to protect students from the harm caused by bullying.

The project has several contributions to both the community and industry. For the community, it provides a much-needed tool to detect bullying early and offer timely support to students. Bullying can have serious long-term emotional and psychological effects, so by identifying those at risk, the system can help reduce these negative outcomes. Educators can use the system to spot students who need attention and take immediate steps to help. This can improve the overall environment at schools, making it safer for students.

For the education and mental health industries, the system offers a new way to deal with bullying. It helps by using technology and machine learning to predict and prevent bullying, something that was not possible before with traditional methods. By integrating machine learning, the system can analyze large amounts of data and predict bullying patterns in a way that humans cannot. This provides valuable insights that can help improve existing policies and approaches to bullying prevention.

One of the key strengths of this project is its ease of use. The system has been designed to be user-friendly, which means that even those with little technical knowledge, like teachers or parents, can use it effectively. The system offers customized recommendations for different groups, such as educators, parents, and students, based on their specific needs. This makes the system more practical and actionable. Another strength is that the system can be used in real-time to predict bullying risks, which makes it an important tool for immediate intervention.

Furthermore, the project highlights how technology can be used to solve social problems. By applying machine learning to bullying prevention, the system can identify risk factors and offer targeted solutions, which can lead to better support for vulnerable students.

6.2 Limitation

While the project has achieved significant results, it also has some limitations that need to be addressed in the future. First, the system was developed using data from a single source: the Global School-Based Student Health Survey. While this dataset provides useful insights, it may not cover all the factors that contribute to bullying. For example, it does not include information about family dynamics, social media use, or the effectiveness of school policies in preventing bullying. These factors could provide more context and make the predictions more accurate and relevant. Expanding the dataset to include a broader range of factors would likely improve the system's performance.

Another limitation is that the system focuses only on predicting bullying victimization and not on bullying behavior. While it can predict which students are at risk of being bullied, it does not address the students who may be bullying others. This means that the system only offers a partial solution. To be more effective, the system should also identify the bullies and suggest interventions to reduce bullying behavior.

Additionally, the machine learning models used in this project, although effective, may still have limitations in terms of data imbalance. While techniques like SMOTE were used to balance the dataset, there may still be issues that could affect the model’s accuracy. If certain categories of students are underrepresented, the model might struggle to predict bullying risk for those groups. Improving the model's ability to handle imbalanced data would help make the predictions more accurate.

Finally, while XGBoost provided the best results, it is a complex model that is harder to interpret compared to simpler models like Logistic Regression. For many users, especially teachers and parents, understanding why certain predictions are made is important. The lack of transparency in how XGBoost works could make it difficult for non-experts to trust or rely on the system. Improving the interpretability of the model would be an important step in making the system more user-friendly and trustworthy.

6.3 Recommendation

To address the limitations mentioned and improve the system, several recommendations are provided for future development. First, expanding the dataset to include additional factors would make the system more accurate and effective. By adding information such as family background, social media behavior, and details about school policies, the system would have a more complete view of the factors influencing bullying. This would allow the system to make better predictions and provide more targeted recommendations.

Second, the system should be developed to handle both bullying victimization and bullying behavior. This would create a more comprehensive solution, as it would allow the system to not only predict which students are at risk of being bullied but also identify those who are bullying others. This would enable schools to take action against bullying on both sides, providing a more holistic approach to prevention of bullying.

Additionally, a feedback loop could be added to the system, where educators, parents, and students can provide feedback on the effectiveness of the predictions and recommendations. This would allow the system to learn and improve over time based on real-world experiences. By continuously collecting data and feedback, the system could become more effective in identifying at-risk students and offering solutions.

The model's transparency should also be improved. Since XGBoost is not easily interpretable, a simpler, more transparent model or additional tools for explaining XGBoost's decisions should be integrated into the system. This would make the system more accessible and trustworthy for educators and parents who need to understand how decisions are being made.

Finally, to make the system more practical for real-time use, future versions could include features that allow for real-time interventions. This would make it possible for the system to not only predict bullying risks but also offer immediate guidance and resources to educators and students when bullying is detected.

In conclusion, while the project successfully created a predictive tool for bullying prevention, there are several areas for improvement that could increase its effectiveness and expand its impact. By incorporating these recommendations, the system could provide even greater support to schools in addressing bullying and improving the well-being of students.

REFERENCES

Fraga, S., Soares, S., Peres, F. S., & Barros, H. (2021). Household Dysfunction Is Associated With Bullying Behavior in 10-year-old Children: Do Socioeconomic Circumstances Matter? *Journal of Interpersonal Violence*, *37*(15-16), NP13877–NP13901. https://doi.org/10.1177/08862605211006352

Keylabs. (2024, October 2). *Logistic Regression: Overview and Applications | Keylabs*. Keylabs: Latest News and Updates. https://keylabs.ai/blog/logistic-regression-overview-and-applications/

Schlesier, J., Marie-Christine Vierbuchen, & Matzner, M. (2023). Bullied, anxious and skipping school? the interplay of school bullying, school anxiety and school absenteeism considering gender and grade level. *Frontiers in Education*, *8*. https://doi.org/10.3389/feduc.2023.951216

Sifferlin, A. (2014, May 5). *Many Bullying Victims Bring Weapons To School*. TIME; Time. https://time.com/86456/many-bullying-victims-bring-weapons-to-school/

Tarwidi, D., Sri Redjeki Pudjaprasetya, Didit Adytia, & Mochamad Apri. (2023). An optimized XGBoost-based machine learning method for predicting wave run-up on a sloping beach. *MethodsX*, *10*, 102119–102119. https://doi.org/10.1016/j.mex.2023.102119

Wang, H., Xiong, J., Yao, Z., Lin, M., & Ren, J. (2017). *Research Survey on Support Vector Machine*. https://doi.org/10.4108/eai.13-7-2017.2270596

Wiens, M., Verone-Boyle, A., Henscheid, N., Podichetty, J. T., & Burton, J. (2025). A Tutorial and Use Case Example of the eXtreme Gradient Boosting (XGBoost) Artificial Intelligence Algorithm for Drug Development Applications. *Clinical and Translational Science*, *18*(3). https://doi.org/10.1111/cts.70172

AdamBrian. (2020, June 15). What is CRISP – DM Methodology? InsideAIML; InsideAIML. https://insideaiml.com/blog/What-is-CRISP-%E2%80%93-DM-Methodology%3F-250

Afriani Afriani, & Denisa Denisa. (2021). Bullying Victimization Among Junior High School Students in Aceh, Indonesia: Prevalence and its Differences in Gender, Grade, and Friendship Quality. *Jurnal Ilmiah Peuradeun*, *9*(2), 251–251. https://doi.org/10.26811/peuradeun.v9i2.518

Ahmed, G. K., Metwaly, N. A., Elbeh, K., Galal, M. S., & Shaaban, I. (2022). Risk factors of school bullying and its relationship with psychiatric comorbidities: a literature review. *The Egyptian Journal of Neurology, Psychiatry and Neurosurgery*, *58*(1). https://doi.org/10.1186/s41983-022-00449-x

Ahmed, G. K., Metwaly, N. A., Khaled Elbeh, Galal, M. S., & Shaaban, I. (2022). Risk factors of school bullying and its relationship with psychiatric comorbidities: a literature review. *The Egyptian Journal of Neurology Psychiatry and Neurosurgery*, *58*(1). https://doi.org/10.1186/s41983-022-00449-x

Chen, H. (2023). Enterprise marketing strategy using big data mining technology combined with XGBoost model in the new economic era. *PLoS ONE*, *18*(6), e0285506–e0285506. https://doi.org/10.1371/journal.pone.0285506

COE - Student Bullying. (2023). Ed.gov. https://nces.ed.gov/programs/coe/indicator/a10/bullying-electronic bullying?utm_source=chatgpt.com

Coursera, S. (2024, December 9). *What Are the Advantages and Disadvantages of Random Forest?* Coursera. https://www.coursera.org/articles/advantages-and-disadvantages-of-random-forest

Ersilia Menesini, & Salmivalli, C. (2017). Bullying in schools: the state of knowledge and effective interventions. *Psychology Health & Medicine*, *22*(sup1), 240–253. https://doi.org/10.1080/13548506.2017.1279740

Galán-Arroyo, C., Gómez-Paniagua, S., Nicolás Contreras-Barraza, José Carmelo Adsuar, Olivares, P. R., & Rojo-Ramos, J. (2023). Bullying and Self-Concept, Factors Affecting the Mental Health of School Adolescents. *Healthcare*, *11*(15), 2214–2214. https://doi.org/10.3390/healthcare11152214

Galán-Arroyo, C., Gómez-Paniagua, S., Nicolás Contreras-Barraza, José Carmelo Adsuar, Olivares, P. R., & Rojo-Ramos, J. (2023). Bullying and Self-Concept, Factors Affecting the Mental Health of School Adolescents. Healthcare, 11(15), 2214–2214. https://doi.org/10.3390/healthcare11152214

GeeksforGeeks. (2021, December 29). *Machine Learning with Python Tutorial*. GeeksforGeeks. https://www.geeksforgeeks.org/machine-learning-with-python/

GeeksforGeeks. (2024, August 6). *Hardware Requirements for Machine Learning*. GeeksforGeeks. https://www.geeksforgeeks.org/hardware-requirements-for-machine-learning/

Guido, R., Ferrisi, S., Lofaro, D., & Conforti, D. (2024). An Overview on the Advancements of Support Vector Machine Models in Healthcare Applications: A Review. *Information*, *15*(4), 235–235. https://doi.org/10.3390/info15040235

Gupta, M., Rana, M., Malhi, P., Grover, S., & Kaur, M. (2020). Prevalence and correlates of bullying perpetration and victimization among school-going adolescents in Chandigarh, North India. *Indian Journal of Psychiatry*, *62*(5), 531–531. https://doi.org/10.4103/psychiatry.indianjpsychiatry_444_19

Hotz, N. (2018, September 10). *What is CRISP DM? - Data Science PM*. Data Science PM. https://www.datascience-pm.com/crisp-dm-2/

Jaana Juvonen, & Graham, S. (2013). Bullying in Schools: The Power of Bullies and the Plight of Victims. *Annual Review of Psychology*, *65*(1), 159–185. https://doi.org/10.1146/annurev-psych-010213-115030

Kanade, V. (2022, April 8). *What Is Logistic Regression? Equation, Assumptions, Types, and Best Practices*. Spiceworks. https://www.spiceworks.com/tech/artificial-intelligence/articles/what-is-logistic-regression/

Madsen, K. R., Mogens Trab Damsgaard, Petersen, K., Qualter, P., & Holstein, B. E. (2024). Bullying at School, Cyberbullying, and Loneliness: National Representative Study of Adolescents in Denmark. *International Journal of Environmental Research and Public Health*, *21*(4), 414–414. https://doi.org/10.3390/ijerph21040414

Markkanen, I., Välimaa, R., & Kannas, L. (2019). Forms of Bullying and Associations Between School Perceptions and Being Bullied Among Finnish Secondary School Students Aged 13 and 15. *International Journal of Bullying Prevention*. https://doi.org/10.1007/s42380-019-00058-y

Marsh,V.L. (Oct 2018). Bullying in School: Prevalence, Contributing Factors, and Interventions. *The Center for Urban Education Success at the University of Rochester's Warner School of Education.* https://ed.buffalo.edu/content/dam/ed/alberti/docs/bullying%20prevalence%20doc%20UofR.pdf

Microsoft. (2021, November 3). *Visual Studio Code*. Visualstudio.com; Microsoft. https://code.visualstudio.com/docs/datascience/jupyter-notebooks

Napolitano, S., & Espelage, D. (2011). *Expanding the Social-Ecological Framework of Bullying among Expanding the Social-Ecological Framework of Bullying among Youth: Lessons Learned from the Past and Directions for the Youth: Lessons Learned from the Past and Directions for the Future [Chapter 1] Future [Chapter 1]*. https://digitalcommons.unl.edu/cgi/viewcontent.cgi?article=1139&context=edpsychpapers

Patchin, J. W. (2019, May 29). *School Bullying Rates Increase by 35% from 2016 to 2019*. Cyberbullying Research Center. https://cyberbullying.org/school-bullying-rates-increase-by-35-from-2016-to-2019

Patel, M.-G., & Quan-Haase, A. (2022). The social-ecological model of cyberbullying: Digital media as a predominant ecology in the everyday lives of youth. *New Media & Society*, *26*(9), 5507–5528. https://doi.org/10.1177/14614448221136508

Qiu, T., Wang, S., Hu, D., Feng, N., & Cui, L. (2024). Predicting Risk of Bullying Victimization among Primary and Secondary School Students: Based on a Machine Learning Model. *Behavioral Sciences*, *14*(1), 73. https://doi.org/10.3390/bs14010073

Ramasamy, S., Shue, Chong, L., & Magen, J. (2024). Parental Support and Bullying: An Imperative Factor to Promote Malaysian Adolescents' Life Satisfaction. *Jurnal*

Psikologi Malaysia, *38*(2), 43–55. https://spaj.ukm.my/ppppm/jpm/article/viewFile/804/631

Rigby, K. (2003). Consequences of Bullying in Schools. *The Canadian Journal of Psychiatry*, *48*(9), 583–590. https://doi.org/10.1177/070674370304800904

Shams, H., Gholamreza Garmaroudi, Saharnaz Nedjat, & Mir Saeed Yekaninejad. (2018). Effect of education based on socio-ecological theory on bullying in students: an educational study. *Electronic Physician*, *10*(7), 7046–7053. https://doi.org/10.19082/7046

Team, C. (2025, February 10). *Python (in Machine Learning)*. Corporate Finance Institute. https://corporatefinanceinstitute.com/resources/data-science/python-in-machine-learning/

What Is Python for Machine Learning? (Definition, Uses) | Built In. (2023). Built In. https://builtin.com/machine-learning/python-machine-learning

Muneer, A., & Fati, S. M. (2020). A Comparative Analysis of Machine Learning Techniques for Cyberbullying Detection on Twitter. *Future Internet*, *12*(11), 187. https://doi.org/10.3390/fi12110187

Mehendale, N., Shah, K., Phadtare, C., & Rajpara, K. (2022). Cyber Bullying Detection for Hindi-English Language Using Machine Learning. *SSRN Electronic Journal*. https://doi.org/10.2139/ssrn.4116143

Akter, F., Umor, M., Jahangir, F., Rajarshi, R., Chowdhury, & Forhad, M. (2025). Cyberbullying Detection on Social Media Platforms Utilizing Different Machine Learning Approaches. *International Journal of Computer Applications*, *186*(61), 975–8887. https://www.ijcaonline.org/archives/volume186/number61/akter-2025-ijca-924395.pdf

Amgad Muneer, & Suliman Mohamed Fati. (2020). A Comparative Analysis of Machine Learning Techniques for Cyberbullying Detection on Twitter. *Future Internet*, *12*(11), 187–187. https://doi.org/10.3390/fi12110187

None Al-Khowarizmi, Sari, I. P., & Halim Maulana. (2023). Detecting Cyberbullying on Social Media Using Support Vector Machine: A Case Study on Twitter. *International Journal of Safety and Security Engineering*, *13*(4), 709–714. https://doi.org/10.18280/ijsse.130413

Ali Süzen, A., & Duman, B. (2023). Detection of types cyber-bullying using fuzzy c-means clustering and xgboost ensemble algorithm. *CRJ*, *1*, 27–34. https://doi.org/10.59380/crj.v1i1.2724

Mubeen, M., Muskan, A., Akram, A., Rashid, J., Alshalali, T. A. N., & Sarwar, N. (2025). Cyberbullying-Related Automated Hate Speech Detection on Social Media Platforms Using Stack Ensemble Classification Method. *International Journal of Computational Intelligence Systems*, *18*(1). https://doi.org/10.1007/s44196-025-00919-z

Muhamad Riza Kurniawanda, & Fenina Adline Twince Tobing. (2022). Analysis Sentiment Cyberbullying In Instagram Comments with XGBoost Method. *IJNMT (International Journal of New Media Technology)*, *9*(1), 28–34. https://doi.org/10.31937/ijnmt.v9i1.2670

APPENDICES

Appendix A : PPF – Title Registration Proposal

Project Title Proposal

YAP JIA XIN

Home › Title Proposal Status (Students)

Project Title Proposal

Project Title: Identifying Risk Factors of Bullying Among School Students Using Machine Learning		**Status**	APPROVED
Proposed By	YAP JIA XIN		
Description	Bullying continues to be one of the most pervasive social issues affecting individuals worldwide, particularly in environments such as schools, workplaces, homes, and increasingly online platforms, where it takes the form of cyberbullying. Bullying is typically defined as repeated aggressive behavior intended to harm someone perceived as weaker, and it can manifest in various ways—physically, verbally, and psychologically. Physical bullying includes actions such as hitting, kicking, or damaging property, while verbal bullying encompasses name-calling, threats, or inappropriate comments. Cyberbullying involves the use of digital platforms to harass, spread false information, or cause harm to individuals. The consequences of bullying are far-reaching, often leading to mental health issues such as depression, anxiety, and even suicidal thoughts, in addition to negative effects on academic performance and social well-being. In the context of schools, bullying can have a particularly devastating impact, affecting students' rights to education, mental health, and overall development. Certain groups, such as students with disabilities, minority ethnic backgrounds, or those who have migrated from other regions, are particularly vulnerable to bullying. Addressing this issue is not only crucial for protecting individuals but also essential for fostering safer and more inclusive learning environments. This project, which aims to predict bullying incidents among students using machine learning, aligns with the United Nations' Sustainable Development Goal 3 (SDG 3) that focuses on promoting well-being for all.		
	The primary aim of this project is to create an advanced machine learning model capable of predicting bullying incidents among students by analyzing the Global School-Based Student Health Survey (GSHS) dataset. The model will be designed to identify significant predictors of bullying, enabling early intervention strategies that can reduce bullying incidents in schools. By analyzing data from various factors such as age, gender, academic performance, psychological well-being, and ethnicity, the project seeks to provide actionable insights that can help educators, parents, and policymakers better address the issue of bullying. The goal is to not only predict bullying behavior but also to provide a framework for early intervention, which is essential for mitigating the harmful effects of bullying and promoting the well-being of students. This initiative is directly linked to SDG 3, which emphasizes ensuring healthy lives and well-being for all, and it aims to improve students' experiences in educational settings by creating safer environments free from bullying. The objectives of this project include four key goals. The first objective is to conduct a comprehensive exploratory data analysis (EDA) of the GSHS dataset to uncover significant patterns, trends, and correlations between various features and bullying behaviors. By examining factors such as age, gender, school performance, psychological health, and ethnicity, the EDA will help identify the key predictors of bullying that will inform the subsequent modeling phase. The second objective is to build a predictive model using machine learning techniques to classify students into categories of "at-risk" and "not-at-risk" for bullying based on the identified predictors. Various algorithms, including Decision Trees, Random Forests, and Support Vector Machines, will be applied to the cleaned and preprocessed dataset to develop the model. The third objective focuses on validating the performance of the predictive model by assessing its accuracy, precision, recall, and other relevant metrics. Cross-validation will be employed to ensure the model's robustness and its ability to generalize across different subsets of the data. Finally, the fourth objective is to generate evidence-based recommendations for students, parents, and educators to support timely interventions and prevent bullying. These recommendations will be grounded in the insights derived from the predictive model and will provide practical strategies for fostering a safe and inclusive environment in schools.		

The target users of this project include students, parents, and educators, all of whom stand to benefit from the findings and insights generated by the predictive model. For students, the project will help identify peers at risk of being bullied, enabling early interventions that can protect mental health, strengthen peer relationships, and improve overall well-being. By recognizing bullying at an early stage, students will be able to avoid the adverse effects that bullying can have on their academic performance and emotional health. Parents will also benefit from the insights provided by the project. Understanding the risk factors for bullying will enable parents to better support their children, ensure their safety, and take proactive steps to protect them from potential harm. With a clearer understanding of the factors that contribute to bullying, parents can also engage more effectively with educators to address bullying issues. Educators, including teachers, school administrators, and counselors, will benefit from the project by gaining valuable tools to identify students who may be at risk of being involved in bullying, whether as victims or perpetrators. The predictive model will help educators implement targeted anti-bullying programs, focus support on at-risk students, and create a school environment that fosters respect, safety, and inclusivity.

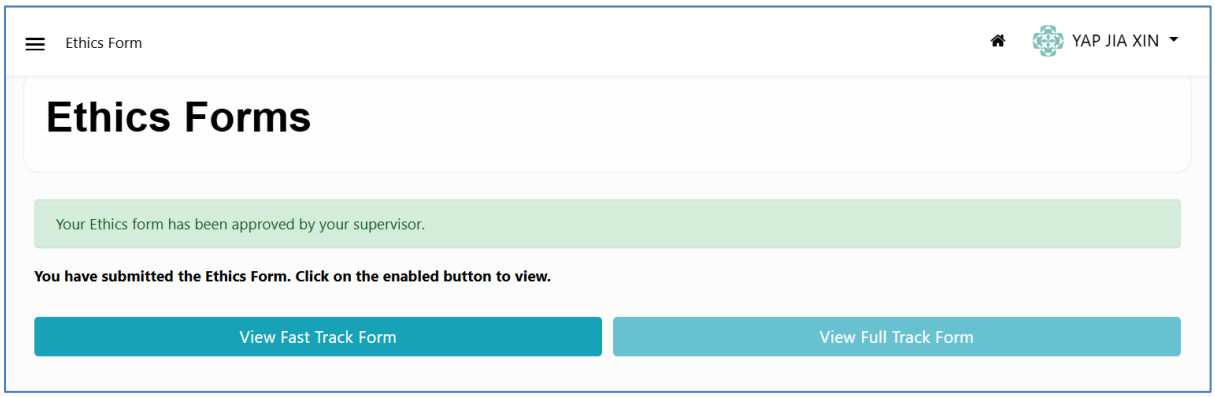
In conclusion, this project aims to leverage machine learning to predict and prevent bullying incidents within school environments, ultimately contributing to safer and healthier learning settings. By analyzing the GSHS dataset, the project seeks to identify the key predictors of bullying and develop a predictive model that can inform intervention strategies. Through these efforts, the project aligns with SDG 3, which advocates for ensuring the well-being of all individuals, regardless of their age or background. The insights gained from the predictive model will offer valuable guidance to students, parents, and educators, enabling them to take proactive steps toward preventing bullying, protecting mental health, and fostering a more inclusive and supportive educational environment. Ultimately, this project has the potential to reduce the prevalence of bullying and contribute to the overall well-being of students, helping them thrive in a safe and nurturing environment.

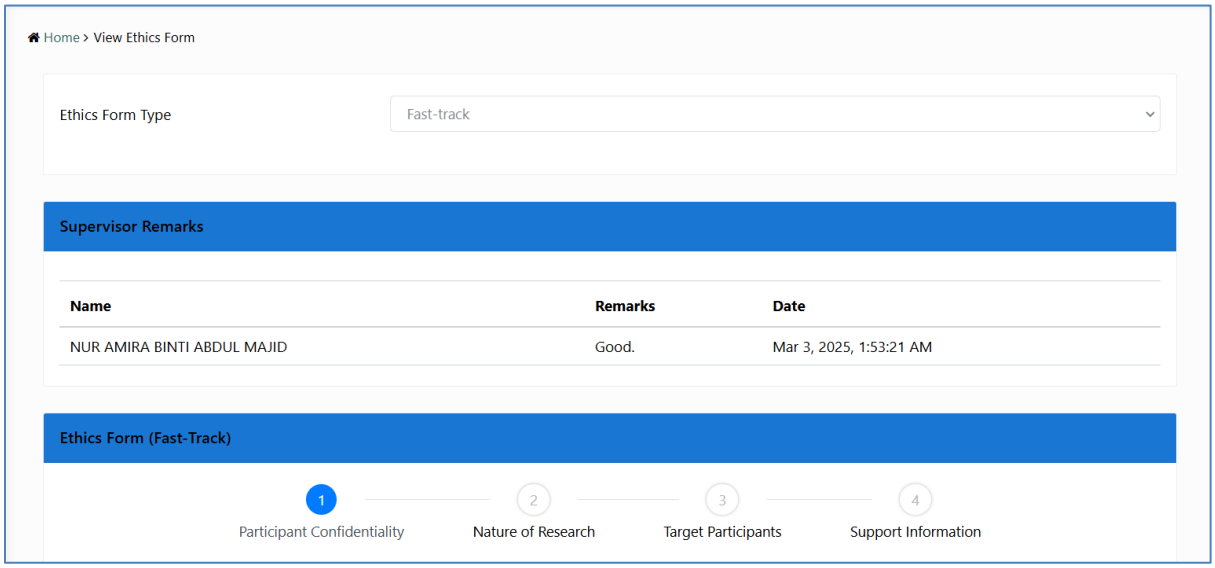
SDG	SDG3
Keywords	Education Risk Management Machine Learning Analytics and Data Good Helath and Well being
Preferred Supervisor(s)	DR. KUAN YIK JUNN, ASSOC. PROF. DR. RAJA RAJESWARI A/P PONNUSAMY, FARHANA ILLIANI BINTI HASSAN, ASSOC. PROF. DR. IMRAN MEDI, DR. VAZEERUDEEN ABDUL HAMEED
Assigned Supervisor	nuramira
Assigned Second Marker	dhason

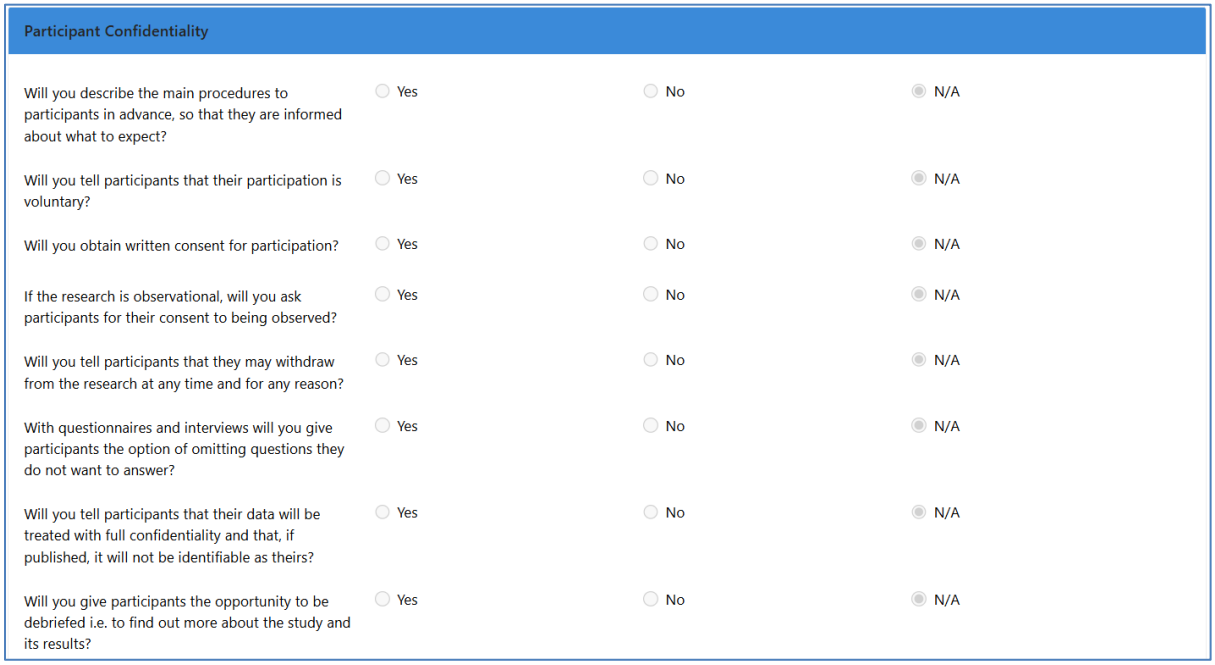
Remarks

Name	Remarks	Date
Dr. Dewi Octaviani	Poor.	Jan 31, 2025, 2:27:39 AM
Dr. Dewi Octaviani	Poor.	Jan 31, 2025, 2:27:40 AM

Appendix B : Ethics Forms (Fast Track)







Ethics Form (Fast-Track)

1 Participant Confidentiality — 2 Nature of Research — 3 Target Participants — 4 Support Information

Nature of Research

Will your project/assignment deliberately mislead participants in any way?	Yes	No	N/A
Is there any realistic risk of any participants experiencing either physical or psychological distress or discomfort? This should include details of what you will tell participants to do if they should experience any problems (e.g., who they can contact for help). You may also need to consider risk assessment issues.	Yes	No	N/A
Is the nature of the research such that contentious or sensitive issues might be involved? This includes research which could induce psychological stress, anxiety or humiliation, or cause more than minimal pain.	Yes	No	N/A
Does your research involve the use of sensitive materials? E.g., records of personal or sensitive confidential information.	Yes	No	N/A
Does your research require external agency approval?	Yes	No	N/A
Does your research use hazardous or controlled substance?	Yes	No	N/A
Does your research require you to visit participants in their home or non-public space?	Yes	No	N/A
Does your research investigate illegal activities or behaviours?	Yes	No	N/A
Does your research involve discussion or collection of information on potentially sensitive, embarrassing or distressing topics, administrative or secure data? This includes research involving respondents through internet where visual images are used, and where sensitive issues are discussed	Yes	No	N/A
Will your participants be receiving financial compensation for participating in your research?	Yes	No	N/A
Will your research data be used in the future after the conclusion of your project?	Yes	No	N/A
Will your research involve in processing sensitive data belonging to an organisation/persons?	Yes	No	N/A
Will your research be collecting photographs, videos, and audio recordings of the participants?	Yes	No	N/A
Will the participants' personal particulars be known to any third party?	Yes	No	N/A
Will the participants' data confidentiality be made known to the public?	Yes	No	N/A
Will the research be conducted where the safety of the researchers maybe in question?	Yes	No	N/A
Will be the research be conducted outside of Malaysia and/or UK?	Yes	No	N/A

Ethics Form (Fast-Track)

1 Participant Confidentiality — 2 Nature of Research — 3 Target Participants — 4 Support Information

Target Participants

Do participants fall into any of the following special groups?

- Children (under 18 years of age)
- People with communication or learning difficulties
- Patients
- People in custody
- People who could be regarded as vulnerable
- People engaged in illegal activities (e.g., drug taking)
- Groups of people whose relationship among each other allow one to have influence over the other such as: Carers and patients with chronic conditions; teachers and their students; prison authorities and prisoners; employers and employees
- Groups where permission of a gatekeeper is normally required for initial access to members.

○ Yes ○ No ◉ N/A

Note: You may also need to obtain satisfactory clearance from the relevant authorities.

Does the project/assignment involve external funding or external collaboration where the funding body or external collaborative partner requires the University to provide evidence that the project/assignment had been subject to ethical scrutiny?

○ Yes ○ No ◉ N/A

Ethics Form (Fast-Track)

1 Participant Confidentiality — 2 Nature of Research — 3 Target Participants — 4 Support Information

Support Information

I consider that this project/assignment has no significant ethical implications requiring a full ethics submission to the APU Research Ethics Committee. ☑

I am aware of APU liability policy and will make the necessary arrangement for insurance coverage of all researchers and participants of the project/assignment. ☑

Description

This project is to identify and predict the bullying factor in secondary school by using CRISP-DM methodology. There is no interview, or survey will be conducted in this project. However, an open dataset source is taken from Kaggle with more than 50,000 observations and 18 variables. Here is the link for the dataset: https://www.kaggle.com/datasets/leomartinelli/bullying-in-schools/data This dataset will proceed to data preprocessing first, such as removing outliers, handling missing value, data transformation, etc. Machine learning technique such as Decision Tree, Random Forest, SVM, and Neural Networks will be applied in this project to train and test the dataset. The Machine Learning models will then perform metrics such

Consent Form
(Maximum 3 files)

Information Sheet

Additional Files

I also confirm that:

Appendix C : Log Sheets (6 log sheets)

Proposed Meeting Schedule

10 | Status | Show

#	Meeting Name	Proposed Date	Supervisor's Proposed Date	Supervisor's Name	Meeting Schedule Status	Location	Date Modified		
1	Meeting 4	05-06-2025 09:00 am		nuramira	APPROVED	ONLINE MEETING	21-07-2025 11:23 AM	View	Meeting Log
2	Meeting 5	18-07-2025 10:45 am		nuramira	APPROVED	ONLINE MEETING	21-07-2025 11:24 AM	View	Meeting Log
3	Meeting 6	23-07-2025 12:00 pm		nuramira	APPROVED	ONLINE MEETING	23-07-2025 02:39 PM	View	Meeting Log
4	Meeting 1	14-02-2025 11:30 am		nuramira	APPROVED	ONLINE MEETING	03-03-2025 09:27 AM	View	Meeting Log
5	Meeting 2	03-03-2025 09:30 am		nuramira	APPROVED	ONLINE MEETING	12-03-2025 01:33 PM	View	Meeting Log
6	Meeting 3	12-03-2025 03:00 pm		nuramira	APPROVED	ONLINE MEETING	12-03-2025 01:33 PM	View	Meeting Log

6 items found, displaying all items.

Appendix D : Poster

Data-Driven Analysis to Predict Bullying Victimization Risk Among Adolescent Students Using Machine Learning

YAP JIA XIN - TP066475 | APU3F2411CS(DA)
B.SC (HONS) IN COMPUTER SCIENCE WITH SPECIALISM IN DATA ANALYTICS
SUPERVISED BY: MS NUR AMIRA ABDUL MAJID
SECOND MARKER: MS AZIAH ABDOLLAH

1 Introduction

Bullying is a significant global issue with profound impacts on students' mental health, academic performance, and overall well-being. It includes physical, verbal, and cyberbullying. The project aims to predict bullying victimization risks using machine learning to help educators, parents, and policymakers intervene early.

2 Problem Statement

- Effectiveness of Anti-Bullying Policies in School
- The Impact of Bullying
- Parental Awareness and Involvement in Combating Bullying

3 Aim & Objective

Aim :
Develop a machine learning model to identify students at risk of bullying.

Objective:
- Perform EDA
- Build predictive models
- Evaluate the models performance
- Provide recommendations

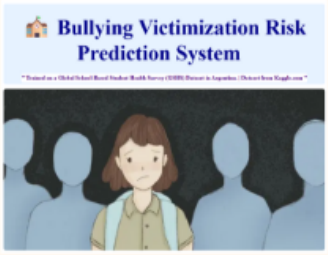
4 Methodology

CRISP-DM Methodology

Phases:
- Business Understanding.
- Data Preparation.
- Modeling.
- Evaluation.
- Deployment.

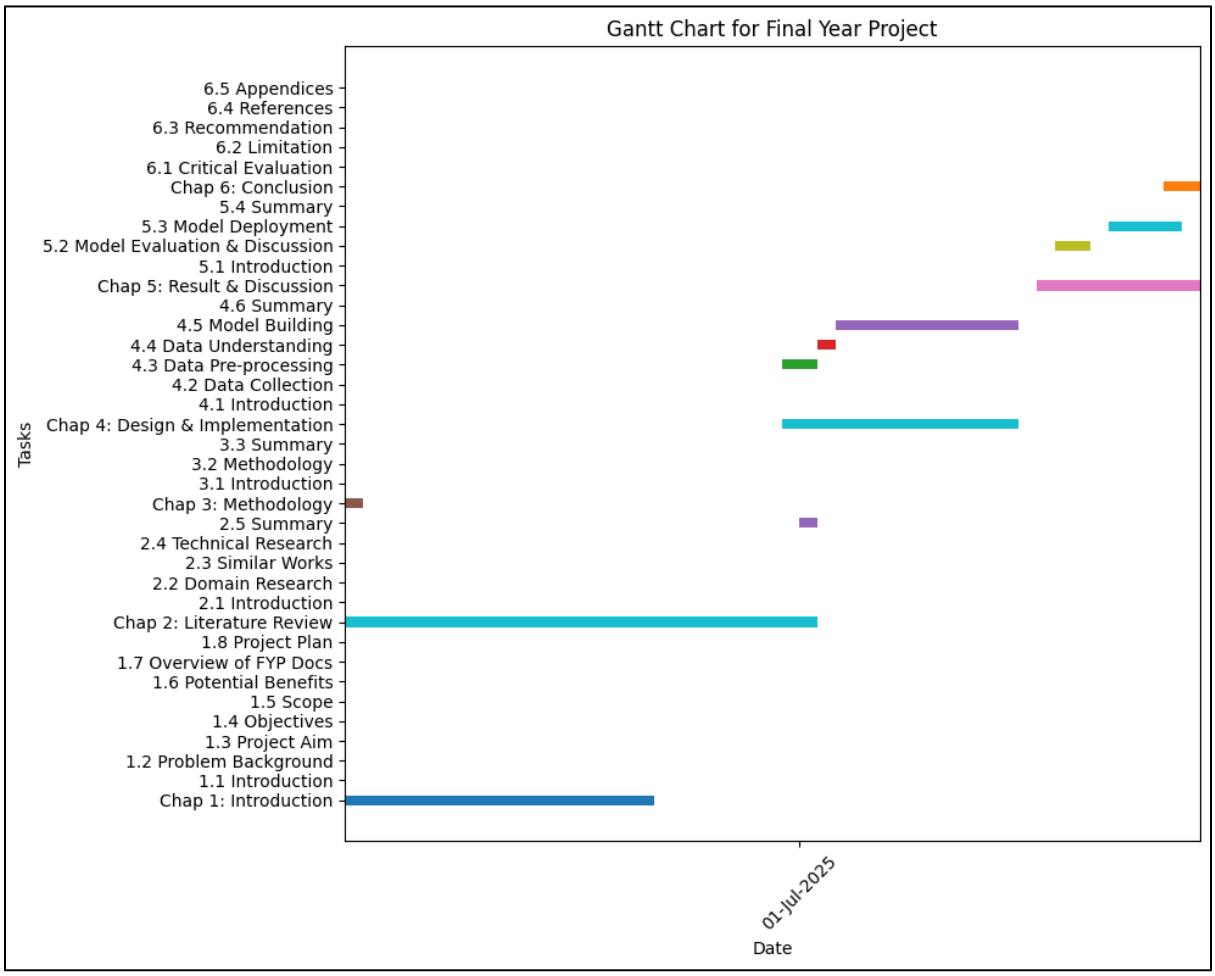
5 Models & Techniques

Models:
- Logistic Regression.
- Random Forest.
- Support Vector Machine (SVM).
- XGBoost.



6 Result

Appendix E : Gantt Chart



Appendix F : Sample Code Implementation

1. Initial Data Understanding

(1.1) Import Libraries

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import re

from tabulate import tabulate
from pprint import pprint
from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from xgboost import XGBClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import GridSearchCV, RandomizedSearchCV
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.metrics import roc_curve, auc
from sklearn.model_selection import learning_curve
```

[9] ✓ 0.0s Python

(1.2) Load Dataset and Check Basic Info

```
file_path = "C:\Bullying_2018.csv"
df = pd.read_csv(file_path)

print("(Rows, Columns)")
print(df.shape, "\n")

print(df.info(),"\n")

print(tabulate(df.head(5), headers='keys', tablefmt='pretty'))
```

[10] ✓ 0.1s Python

... (Rows, Columns)

Generate + Code + Markdown

(1.2) Load Dataset and Check Basic Info

```
file_path = "C:\Bullying_2018.csv"
df = pd.read_csv(file_path)

print("(Rows, Columns)")
print(df.shape, "\n")

print(df.info(),"\n")

print(tabulate(df.head(5), headers='keys', tablefmt='pretty'))
```

[10] ✓ 0.1s Python

```
(Rows, Columns)
(56981, 18)

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 56981 entries, 0 to 56980
Data columns (total 18 columns):
 #   Column                                        Non-Null Count  Dtype
---  ------                                        --------------  -----
 0   record                                        56981 non-null  int64
 1   Bullied_on_school_property_in_past_12_months  56981 non-null  object
 2   Bullied_not_on_school_property_in_past_12_months  56981 non-null  object
 3   Cyber_bullied_in_past_12_months               56981 non-null  object
 4   Custom_Age                                    56981 non-null  object
 5   Sex                                           56981 non-null  object
 6   Physically_attacked                           56981 non-null  object
 7   Physical_fighting                             56981 non-null  object
 8   Felt_lonely                                   56981 non-null  object
 9   Close_friends                                 56981 non-null  object
 10  Miss_school_no_permission                     56981 non-null  object
 11  Other_students_kind_and_helpful               56981 non-null  object
 12  Parents_understand_problems                   56981 non-null  object
 13  Most_of_the_time_or_always_felt_lonely        56981 non-null  object
 14  Missed_classes_or_school_without_permission   56981 non-null  object
 15  Were_underweight                              56981 non-null  object
 16  Were_overweight                               56981 non-null  object
 17  Were_obese                                    56981 non-null  object
dtypes: int64(1), object(17)
memory usage: 7.8+ MB
None

+---+--------+-----------------------------------------+-----------------------------------------+----------------------------
|   | record | Bullied on school property in past 12 months | Bullied not on school property in past 12 months | Cyber bullied in past 12 mon
```

(1.3) Initial Features Analysis

```
numeric_cols = df.select_dtypes(include=["int"]).columns
categorical_cols = df.select_dtypes(include=["object"]).columns

for col in numeric_cols:
    fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(10, 4))

    # Box Plot
    sns.boxplot(x=df[col], ax=axes[0])
    axes[0].set_title(f"Box Plot of {col}")

    # Histogram
    sns.histplot(df[col], bins=30, kde=True, ax=axes[1], color="purple")
    axes[1].set_title(f"Histogram of {col}")

    plt.tight_layout()
    plt.show()

for col in categorical_cols:
    unique_vals = df[col].dropna().unique()
    try:
        sorted_order = sorted(unique_vals, key=lambda x: float(x))
    except:
        # Fallback: sort alphabetically
        sorted_order = sorted(unique_vals)

    plt.figure(figsize=(14, 5))
    ax = sns.countplot(
        x=df[col],
        hue=df[col],
        palette="muted",
        order=sorted_order,
        hue_order=sorted_order,
    )

    for p in ax.patches:
        ax.annotate(
            f'{int(p.get_height())}',
            (p.get_x() + p.get_width() / 2., p.get_height()),
            ha='center', va='bottom', fontsize=10, color='black'
        )
    plt.title(f"Distribution of {col}")
    plt.xticks(rotation=0)
    plt.show()
```

(1.4) Check Missing Value

```
df_csv = pd.read_csv(file_path, dtype=str, na_values=["", " "])

# Calculate missing count and percentage
missing_count = df_csv.isnull().sum()
missing_percentage = (missing_count / len(df_csv)) * 100

missing_info = missing_count[missing_count > 0].astype(str) + " (" + missing_percentage[missing_percentage > 0].round(2).astype(str) + "%
print("Number of Missing Value:")
print("----------------------------------------------------------------")
print(missing_info)
```

✓ 0.1s Pytho

```
Number of Missing Value:
----------------------------------------------------------------
Bullied_on_school_property_in_past_12_months          1239 (2.17%)
Bullied_not_on_school_property_in_past_12_months       489 (0.86%)
Cyber_bullied_in_past_12_months                        571 (1.0%)
Custom_Age                                             108 (0.19%)
Sex                                                    536 (0.94%)
Physically_attacked                                    240 (0.42%)
Physical_fighting                                      268 (0.47%)
Felt_lonely                                            366 (0.64%)
Close_friends                                         1076 (1.89%)
Miss_school_no_permission                             1864 (3.27%)
Other_students_kind_and_helpful                       1559 (2.74%)
Parents_understand_problems                           2373 (4.16%)
Most_of_the_time_or_always_felt_lonely                 366 (0.64%)
Missed_classes_or_school_without_permission           1864 (3.27%)
Were_underweight                                     20929 (36.73%)
Were_overweight                                      20929 (36.73%)
Were_obese                                           20929 (36.73%)
dtype: object
```

(1.5) Check Inconsistence Value

```
columns_to_check = [
    "Physically_attacked",
    "Physical_fighting",
    "Close_friends",
    "Miss_school_no_permission"
]

unique_values = {col: df[col].unique().tolist() for col in columns_to_check}
pprint(unique_values)
```

[13] ✓ 0.0s

```
{'Close_friends': ['2', '3 or more', '0', '1', ' '],
 'Miss_school_no_permission': ['10 or more days',
                               '6 to 9 days',
                               '0 days',
                               '3 to 5 days',
                               ' ',
                               '1 or 2 days'],
 'Physical_fighting': ['0 times',
                       '2 or 3 times',
                       '1 time',
                       '4 or 5 times',
                       '6 or 7 times',
                       '8 or 9 times',
                       '10 or 11 times',
                       ' ',
                       '12 or more times'],
 'Physically_attacked': ['2 or 3 times',
                         '0 times',
                         '1 time',
                         '12 or more times',
                         '4 or 5 times',
                         '10 or 11 times',
                         '8 or 9 times',
                         '6 or 7 times',
                         ' ']}
```

(1.6) Check Outliers

```
Q1 = df[numeric_cols].quantile(0.25)
Q3 = df[numeric_cols].quantile(0.75)
IQR = Q3 - Q1

outliers = ((df[numeric_cols] < (Q1 - 1.5 * IQR)) | (df[numeric_cols] > (Q3 + 1.5 * IQR))).sum()
print("Outliers Count:")
print(outliers)
```

✓ 0.0s

```
Outliers Count:
record    0
dtype: int64
```

2. Data Pre-processing (1)

(2.1) Rename Variables

```
rename_mapping = {
    "record": "Record",
    "Bullied_on_school_property_in_past_12_months": "Bullied_in_school",
    "Bullied_not_on_school_property_in_past_12_months": "Bullied_not_in_school",
    "Cyber_bullied_in_past_12_months": "Cyberbullied",
    "Custom_Age": "Age",
    "Sex": "Gender",
    "Miss_school_no_permission": "Missed_school",
    "Other_students_kind_and_helpful": "Peer_helpfulness",
    "Parents_understand_problems": "Parental_understanding"
}
df.rename(columns=rename_mapping, inplace=True)
df.columns
print(df.info())
```

✓ 0.0s

(2.2) Hnadling Missing Value

```
# replace empty cell to null value
df.replace(r'^\s*$', pd.NA, regex=True, inplace=True)
df.fillna({"column_name": "default_value"}, inplace=True)


# 1. Yes/No data: Assume "Yes"
yes_no_columns = [
    "Bullied_in_school", "Bullied_not_in_school",
    "Cyberbullied", "Most_of_the_time_or_always_felt_lonely",
    "Missed_classes_or_school_without_permission",
    "Were_overweight", "Were_obese"
]
df[yes_no_columns] = df[yes_no_columns].fillna("Yes")
df["Were_underweight"] = df["Were_underweight"].fillna("No")


# 2. Physically_attack & Physical_fighting: Assume "0 times"
attack_columns = ["Physically_attacked", "Physical_fighting"]
df[attack_columns] = df[attack_columns].fillna("0 times")


# 3. Close_friends: Assume "0"
df["Close_friends"] = df["Close_friends"].fillna("0")


# 4. Miss_school: Assume "0 days"
df["Missed_school"] = df["Missed_school"].fillna("0 days")


# 5. Age: Extract numerical values , calculate median, reformat as string
numeric_age = df["Age"].str.extract('(\d+)').astype(float)
median_age = numeric_age.median()  # Calculate median
df["Age"] = df["Age"].fillna(f"{int(median_age)} years old")


# 6. Gender: Use mode
df["Gender"] = df["Gender"].fillna(df["Gender"].mode()[0])


# 7. Never-Always data: Assume "Never"
never_always_columns = [
    "Peer_helpfulness",
    "Parental_understanding"
]
df[never_always_columns] = df[never_always_columns].fillna("Never")
```

ter* ⊗ 0 ⚠ 9 Ln 9, Col 52 Spaces: 4 Spaces: 4 CRLF Completions quota reached Cell 102 of 104

(2.3) Feature Engineering

- Combine 3 classification of bullied

```
# Combine & Create "Bullied"
df["Bullied"] = df[["Bullied_in_school",
                    "Bullied_not_in_school",
                    "Cyberbullied"]].max(axis=1)

columns_to_remove = ["Bullied_in_school",
                     "Bullied_not_in_school",
                     "Cyberbullied"]
df.drop(columns=columns_to_remove, inplace=True)

# Reorder columns
cols = list(df.columns)
cols.remove("Bullied")

if "Record" in cols:
    first_two_cols = ["Record"]
    remaining_cols = [col for col in cols if col != "Record"]
    new_order = first_two_cols + ["Bullied"] + remaining_cols
    df = df[new_order]

print("(Rows, Columns)")
print(df.shape, "\n")

print(df.info(),"\n")
print(tabulate(df.head(10), headers='keys', tablefmt='pretty'))
```

[17] ✓ 0.0s Python

```
(Rows, Columns)
(56981, 16)
```

(2.4) Remove Redundant Data

```
columns_to_remove = ["Record",
                     "Most_of_the_time_or_always_felt_lonely",
                     "Missed_classes_or_school_without_permission"
                     ]
df.drop(columns=columns_to_remove, inplace=True)

print("(Rows, Columns)")
print(df.shape, "\n")

print(df.info(),"\n")
print(tabulate(df.head(10), headers='keys', tablefmt='pretty'))
```

[18] ✓ 0.0s Python

```
(Rows, Columns)
```

2b Features Analysis (after pre-processing 1)

```
categorical_cols = df.select_dtypes(include=["object"]).columns

for col in categorical_cols:
    unique_vals = df[col].dropna().unique()
    try:
        sorted_order = sorted(unique_vals, key=lambda x: float(x))
    except:
        # Fallback: sort alphabetically
        sorted_order = sorted(unique_vals)

    plt.figure(figsize=(14, 5))
    ax = sns.countplot(
        x=df[col],
        hue=df[col],
        palette="muted",
        order=sorted_order,
        hue_order=sorted_order,
    )

    for p in ax.patches:
        ax.annotate(
            f'{int(p.get_height())}',
            (p.get_x() + p.get_width() / 2., p.get_height()),
            ha='center', va='bottom', fontsize=10, color='black'
        )

    plt.title(f"Distribution of {col}")
    plt.xticks(rotation=0)
    plt.show()
```

19] ✓ 2.2s Python

3. Data Pre-processing (2)

(3.1) Data Transformation

- Extract Numbers only

```python
# Convert "Age" column to keep only numbers
df["Age"] = df["Age"].str.extract("(\d+)").astype(float).fillna(0).astype(int)

def extract_first_number(value):
    """Extracts the first numeric value from a string and converts it to int."""
    match = re.search(r'\d+', str(value))
    return int(match.group()) if match else None


# For ordinal columns, replace null with 0, then convert to int
ordinal_cols = ["Physically_attacked",
                "Physical_fighting",
                "Close_friends",
                "Missed_school"]
df[ordinal_cols] = df[ordinal_cols].apply(lambda col: col.map(extract_first_number)).fillna(0).astype(int)
```

✓ 0.2s

- Yes/No → 1/0
- Scale Mapping (Never - Always → 1-5)
- Gender Mapping (Female=0, Male=1)

```python
# Yes/No → 1/0
Yes_No_col = ["Bullied",
              "Were_underweight",
              "Were_overweight",
              "Were_obese"]
for col in Yes_No_col:
    df[col] = df[col].map({"Yes": 1, "No": 0})


# Scale Mapping (Never - Always -> 1-5)
scale_cols = ["Felt_lonely",
              "Peer_helpfulness",
```

4. Data Understanding (After Pre-processing)

(4.1) Check Outliers (After Pre-processing)

```
numeric_cols = df.select_dtypes(include=[int]).columns

# Calculate outlier count
outliers_count = {}
for col in df[numeric_cols]:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    outliers_count[col] = ((df[col] < lower_bound) | (df[col] > upper_bound)).sum()

# Print the count of outliers
print(f"{'Column name':<29}{'Outliers'}")
print("-" * 40)
for column, count in outliers_count.items():
    print(f"{column:<30}{count}")
```

✓ 0.0s

```
Column name                  Outliers
----------------------------------------
Bullied                       0
Age                           0
Gender                        0
Physically_attacked           2091
Physical_fighting             4019
Felt_lonely                   0
Close_friends                 0
Missed_school                 12463
Peer_helpfulness              0
Parental_understanding        0
Were_underweight              733
Were_overweight               0
Were_obese                    0
```

(4.2) Create new column for outliers

Generate + Code + Markdown

```
numeric_cols = df.select_dtypes(include='number').columns

# Precompute IQR bounds
bounds = {}
for col in numeric_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    bounds[col] = (Q1 - 1.5 * IQR, Q3 + 1.5 * IQR)

# Check outliers in a row
def detect_outliers(row):
    for col in numeric_cols:
        lower, upper = bounds[col]
        if row[col] < lower or row[col] > upper:
            return 1
    return 0

# Create new column "Has_Outlier"
df["Has_Outlier"] = df.apply(detect_outliers, axis=1)

print("(Rows, Columns)")
print(df.shape, "\n")
print(tabulate(df.head(10), headers='keys', tablefmt='pretty'))
```

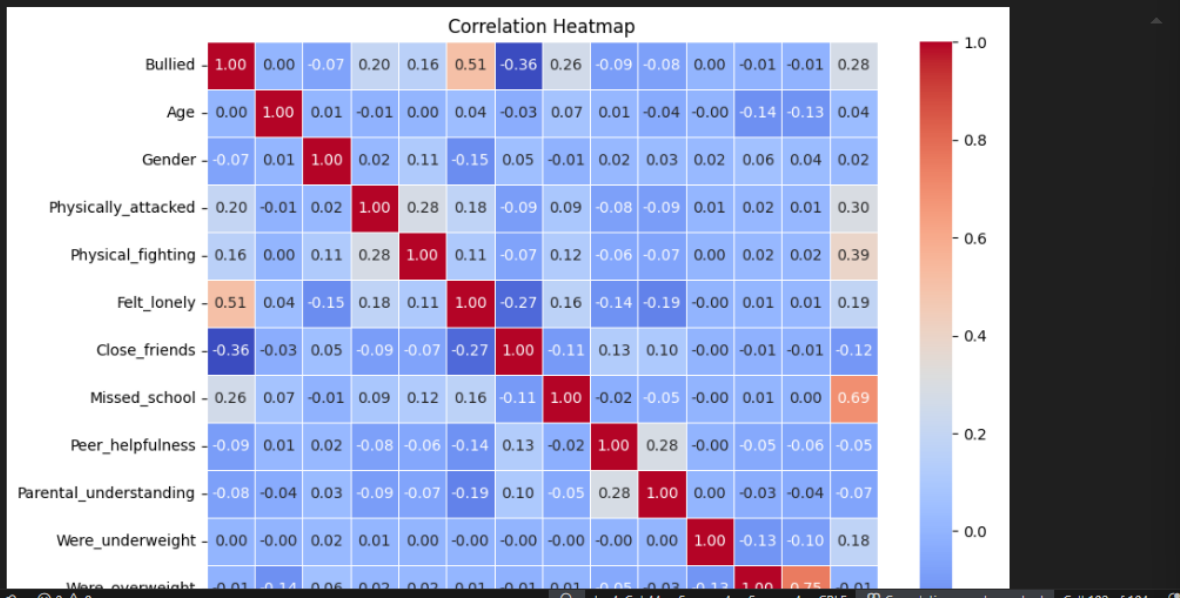
✓ 2.2s

```
(Rows, Columns)
(56981, 14)

+---+---------+-----+--------+--------------------+------------------+-------------+--------------+--------------+------------------+
|   | Bullied | Age | Gender | Physically_attacked | Physical_fighting | Felt_lonely | Close_friends | Missed_school | Peer_helpfulness |
+---+---------+-----+--------+--------------------+------------------+-------------+--------------+--------------+------------------+
| 0 |    1    | 13  |   0    |         2          |         0        |      5      |      2       |      10      |         1        |
| 1 |    1    | 13  |   0    |         0          |         0        |      4      |      3       |      6       |         3        |
| 2 |    1    | 14  |   1    |         1          |         0        |      4      |      3       |      6       |         3        |
| 3 |    0    | 16  |   1    |         0          |         2        |      1      |      3       |      0       |         3        |
| 4 |    0    | 13  |   0    |         0          |         0        |      2      |      3       |      0       |         4        |
| 5 |    1    | 13  |   1    |         0          |         1        |      4      |      0       |      3       |         4        |
| 6 |    0    | 14  |   0    |         1          |         0        |      3      |      3       |      0       |         4        |
| 7 |    0    | 12  |   0    |         0          |         0        |      2      |      3       |      0       |         4        |
| 8 |    1    | 13  |   1    |         1          |         2        |      4      |      3       |      6       |         4        |
| 9 |    1    | 14  |   0    |         1          |         0        |      5      |      0       |      6       |         3        |
```

(4.3) Correlation Heatmap

```
corr_matrix = df.corr()

# Plot heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(corr_matrix, annot=True, cmap="coolwarm", fmt=".2f", linewidths=0.5)
plt.title("Correlation Heatmap")
plt.show()
```

✓ 0.3s Python



```
df_clean = df
```

✓ 0.0s Python

5. Data Modelling

(5.1) Check Target Variable count

```
print(df_clean['Bullied'].value_counts())
```

✓ 0.0s Python

```
Bullied
1    32166
0    24815
Name: count, dtype: int64
```

(5.2) Split Data (Train & Test)

```
X = df_clean.drop('Bullied', axis=1)
y = df_clean['Bullied']

# Split into 80% training, 20% testing
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

#Training Set
print("[Train] Set Shape:")
train_count = y_train.value_counts()
print(train_count)
print("Total:", train_count.sum(), "\n")

#Testing Set
print("[Test] Set Shape:")
test_count = y_test.value_counts()
print(test_count)
print("Total:", test_count.sum(), "\n")
```

✓ 0.0s Python

```
[Train] Set Shape:
Bullied
```

6. Model building & Evaluation

(6.1) Logistic Regression (LR)

```python
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.preprocessing import StandardScaler

# Optionally scale features for Logistic Regression
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Logistic Regression Model
log_reg_model = LogisticRegression(max_iter=200)
log_reg_model.fit(X_train_scaled, y_train)

# Predict and evaluate
y_pred_log_reg = log_reg_model.predict(X_test_scaled)
accuracy = accuracy_score(y_test, y_pred_log_reg)
confusion = confusion_matrix(y_test, y_pred_log_reg)
report = classification_report(y_test, y_pred_log_reg)

# Print results
print(f"Confusion Matrix:\n {confusion}")
print(f"\n📊 Classification Report:\n{report}")
print(f"✅ Accuracy: {accuracy:.4f}")
```

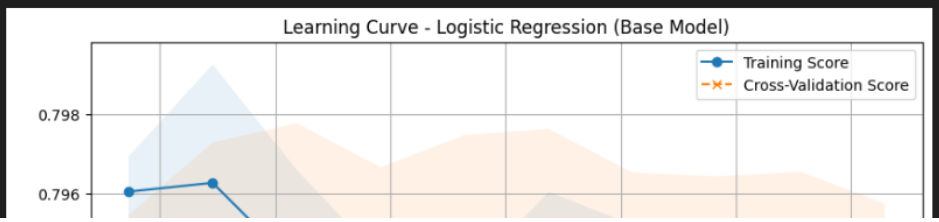
✓ 0.0s Python

```
Confusion Matrix:
 [[3749 1178]
 [1095 5375]]

📊 Classification Report:
              precision    recall  f1-score   support

           0       0.77      0.76      0.77      4927
           1       0.82      0.83      0.83      6470

    accuracy                           0.80     11397
   macro avg       0.80      0.80      0.80     11397
weighted avg       0.80      0.80      0.80     11397

✅ Accuracy: 0.8026
```

LR - Learning Curve (Base)

```python
from sklearn.model_selection import learning_curve

# Base model (Logistic Regression) learning curve
train_sizes, train_scores, test_scores = learning_curve(
    log_reg_model,
    X_train_scaled,
    y_train,
    cv=5,
    n_jobs=-1,
    train_sizes=[0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1]
)

# Calculate mean and std for train and test scores
train_mean = train_scores.mean(axis=1)
test_mean = test_scores.mean(axis=1)
train_std = train_scores.std(axis=1)
test_std = test_scores.std(axis=1)

# Plot Learning Curve
plt.figure(figsize=(10, 6))
plt.plot(train_sizes, train_mean, label=f'Training Score', linestyle='-', marker='o')
plt.plot(train_sizes, test_mean, label=f'Cross-Validation Score', linestyle='--', marker='x')
plt.fill_between(train_sizes, train_mean - train_std, train_mean + train_std, alpha=0.1)
plt.fill_between(train_sizes, test_mean - test_std, test_mean + test_std, alpha=0.1)
plt.title('Learning Curve - Logistic Regression (Base Model)')
plt.xlabel('Training Set Size')
plt.ylabel('F1 Score')
plt.legend(loc='best')
plt.grid(True)
plt.show()
```

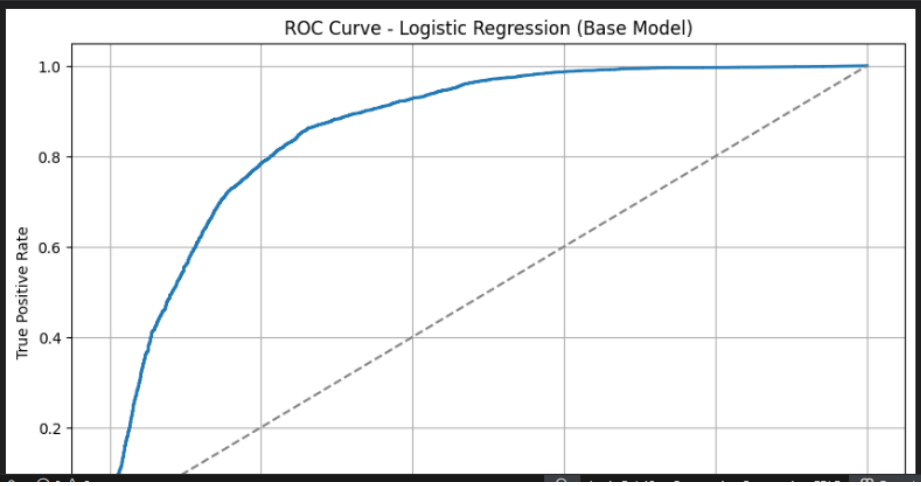
✓ 2.5s Python



LR - ROC-AUC (Base)

```python
from sklearn.metrics import roc_curve, auc

# Predict probabilities for ROC Curve
y_prob_logreg = log_reg_model.predict_proba(X_test_scaled)[:, 1]

# Compute ROC curve
fpr, tpr, thresholds = roc_curve(y_test, y_prob_logreg)
roc_auc = auc(fpr, tpr)

# Plot ROC-AUC Curve
plt.figure(figsize=(10, 6))
plt.plot(fpr, tpr, lw=2, label='Logistic Regression (AUC = %0.4f)' % roc_auc)
plt.plot([0, 1], [0, 1], color='gray', linestyle='--')
plt.title('ROC Curve - Logistic Regression (Base Model)')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend(loc="lower right")
plt.grid(True)
plt.show()
```

✓ 0.1s Python



LR - Hyperparameter Tuning

```python
from sklearn.model_selection import GridSearchCV

# Logistic Regression Model
logreg = LogisticRegression()

param_grid_logreg = {
    'C': [0.01, 0.1, 1, 10, 100],
    'solver': ['liblinear', 'saga'],
    'penalty': ['l1', 'l2'],
    'max_iter': [50, 100, 200],
}

search_logreg = GridSearchCV(logreg, param_grid_logreg, cv=5, n_jobs=-1)

# Fit the model
search_logreg.fit(X_train_scaled, y_train)

# Predict using the best model
best_logreg = search_logreg.best_estimator_
y_pred_logreg = best_logreg.predict(X_test_scaled)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred_logreg)
confusion = confusion_matrix(y_test, y_pred_logreg)
report = classification_report(y_test, y_pred_logreg)

# Display results
print(f"Confusion Matrix (After Tuning):\n {confusion}")
print(f"\n📊Classification Report:\n{report}")

# Best parameters and best score
print("✅Best Hyperparameters for LOGISTIC REGRESSION: ")
for param, value in search_logreg.best_params_.items():
    print(f"  {param.capitalize()}: {value}")

print(f"\nBest Cross-Validation Score: {search_logreg.best_score_:.4f}")
print(f"Accuracy (After Tuning): {accuracy:.4f}")
```

✓ 14.6s Python

```
Confusion Matrix (After Tuning):
 [[3754 1173]
 [1090 5380]]

📊Classification Report:
              precision    recall  f1-score   support
```

ster* ⊗0 ⚠9 Ln 1, Col 42 Spaces: 4 Spaces: 4 CRLF Completions quota reached Cell 102 of 104

LR - Learning Curve (After Tune)

```python
# After Hyperparameter Tuning Model (Best model from GridSearchCV)
train_sizes, train_scores, test_scores = learning_curve(
    best_logreg,
    X_train_scaled,
    y_train,
    cv=5,
    n_jobs=-1,
    train_sizes=[0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1]
)

# Calculate mean and std for train and test scores
train_mean = train_scores.mean(axis=1)
test_mean = test_scores.mean(axis=1)
train_std = train_scores.std(axis=1)
test_std = test_scores.std(axis=1)

# Plot Learning Curve for tuned model
plt.figure(figsize=(10, 6))
plt.plot(train_sizes, train_mean, label=f'Training Score', linestyle='-', marker='o')
plt.plot(train_sizes, test_mean, label=f'Cross-Validation Score', linestyle='--', marker='x')
plt.fill_between(train_sizes, train_mean - train_std, train_mean + train_std, alpha=0.1)
plt.fill_between(train_sizes, test_mean - test_std, test_mean + test_std, alpha=0.1)
plt.title('Learning Curve - Logistic Regression (After Tune)')
plt.xlabel('Training Set Size')
plt.ylabel('F1 Score')
plt.legend(loc='best')
plt.grid(True)
plt.show()
```

[34] ✓ 2.2s Python

LR - ROC-AUC (After Tune)

```python
# Predict probabilities for ROC Curve (tuned model)
y_prob_best_logreg = best_logreg.predict_proba(X_test_scaled)[:, 1]

# Compute ROC curve for the tuned model
fpr_best, tpr_best, thresholds_best = roc_curve(y_test, y_prob_best_logreg)
roc_auc_best = auc(fpr_best, tpr_best)

# Plot ROC-AUC Curve for tuned model
plt.figure(figsize=(10, 6))
plt.plot(fpr_best, tpr_best, lw=2, label='Logistic Regression (Tuned, AUC = %0.4f)' % roc_auc_best)
plt.plot([0, 1], [0, 1], color='gray', linestyle='--')
plt.title('ROC Curve - Logistic Regression (After tune)')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend(loc="lower right")
plt.grid(True)
plt.show()
```

✓ 0.1s

LR - Feature Importance

```python
# Get feature coefficients
coefficients = best_logreg.coef_[0]

feature_importance = pd.DataFrame({
    'Feature': X_train.columns,
    'Coefficient': coefficients
})

# Sort by absolute value of the coefficients
feature_importance['Absolute Coefficient'] = feature_importance['Coefficient'].abs()
feature_importance = feature_importance.sort_values(by='Absolute Coefficient', ascending=False)

# Plot the top 10 most important features
plt.figure(figsize=(10, 6))
plt.barh(feature_importance['Feature'][:10], feature_importance['Absolute Coefficient'][:10], color='lightcoral')
plt.xlabel('Absolute Coefficient Value')
plt.title('Top 10 Feature Importance (Logistic Regression)')
plt.gca().invert_yaxis()
plt.grid(True)
plt.show()
```

✓ 0.1s

(6.2) Random Forest (RF)

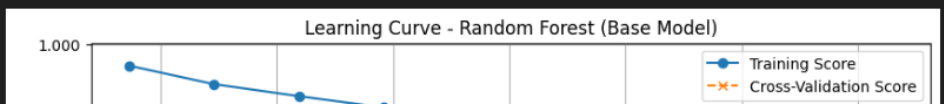
```python
from sklearn.ensemble import RandomForestClassifier

# Random Forest Model
rf_model = RandomForestClassifier(random_state=42)
rf_model.fit(X_train, y_train)

# Predict and evaluate
y_pred_rf = rf_model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred_rf)
confusion = confusion_matrix(y_test, y_pred_rf)
report = classification_report(y_test, y_pred_rf)

# Print results
print(f"Confusion Matrix:\n {confusion}")
print(f"\n📊 Classification Report:\n{report}")
print(f"✅ Accuracy: {accuracy:.4f}")
```

Python

Confusion Matrix:

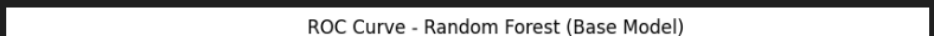
RF - Learning Curve (Base)

```python
# Random Forest Base Model Learning Curve
train_sizes, train_scores, test_scores = learning_curve(
    rf_model,
    X_train,
    y_train,
    cv=5,
    n_jobs=-1,
    train_sizes=[0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1]
)

# Calculate mean and std for train and test scores
train_mean = train_scores.mean(axis=1)
test_mean = test_scores.mean(axis=1)
train_std = train_scores.std(axis=1)
test_std = test_scores.std(axis=1)

# Plot Learning Curve
plt.figure(figsize=(10, 6))
plt.plot(train_sizes, train_mean, label=f'Training Score', linestyle='-', marker='o')
plt.plot(train_sizes, test_mean, label=f'Cross-Validation Score', linestyle='--', marker='x')
plt.fill_between(train_sizes, train_mean - train_std, train_mean + train_std, alpha=0.1)
plt.fill_between(train_sizes, test_mean - test_std, test_mean + test_std, alpha=0.1)
plt.title('Learning Curve - Random Forest (Base Model)')
plt.xlabel('Training Set Size')
plt.ylabel('F1 Score')
plt.legend(loc='best')
plt.grid(True)
plt.show()
```



RF - ROU-AUC (Base)

```python
# Predict probabilities for ROC Curve
y_prob_rf = rf_model.predict_proba(X_test)[:, 1]

# Compute ROC curve
fpr_rf, tpr_rf, thresholds_rf = roc_curve(y_test, y_prob_rf)
roc_auc_rf = auc(fpr_rf, tpr_rf)

# Plot ROC-AUC Curve for Base Model
plt.figure(figsize=(10, 6))
plt.plot(fpr_rf, tpr_rf, lw=2, label='Random Forest (AUC = %0.4f)' % roc_auc_rf)
plt.plot([0, 1], [0, 1], color='gray', linestyle='--')
plt.title('ROC Curve - Random Forest (Base Model)')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend(loc="lower right")
plt.grid(True)
plt.show()
```



RF - Hyperparameter Tuning

```python
rf = RandomForestClassifier()

param_grid_rf = {
    'n_estimators': [50, 100, 200, 300],
    'max_depth': [10, 20, 30, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'bootstrap': [True, False],
}

# Perform GridSearchCV
search_rf = GridSearchCV(rf, param_grid_rf, cv=5, n_jobs=-1)

# Fit the model
search_rf.fit(X_train, y_train)

# Predict using the best model
best_rf = search_rf.best_estimator_
y_pred_rf = best_rf.predict(X_test)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred_rf)
confusion = confusion_matrix(y_test, y_pred_rf)
report = classification_report(y_test, y_pred_rf)

# Display results
print(f"confusion_matrix (After Tuning):\n {confusion}")
print(f"\n📊Classification Report:\n{report}")

print("✅Best Hyperparameters for RANDOM FOREST: ")
for param, value in search_rf.best_params_.items():
    print(f"  {param.capitalize()}: {value}")

print(f"\nBest Cross-Validation Score: {search_rf.best_score_:.4f}")
print(f"Accuracy (After Tuning): {accuracy:.4f}")
```

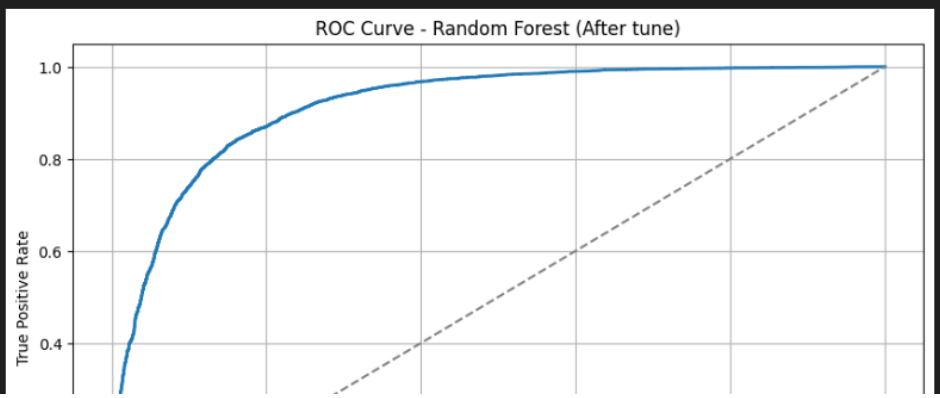
RF - Learning Curve (After Tune)

```python
# After Hyperparameter Tuning (Best Random Forest Model from GridSearchCV)
train_sizes, train_scores, test_scores = learning_curve(
    best_rf,
    X_train,
    y_train,
    cv=5,
    n_jobs=-1,
    train_sizes=[0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1]
)

# Calculate mean and std for train and test scores
train_mean = train_scores.mean(axis=1)
test_mean = test_scores.mean(axis=1)
train_std = train_scores.std(axis=1)
test_std = test_scores.std(axis=1)

# Plot Learning Curve for Tuned Model
plt.figure(figsize=(10, 6))
plt.plot(train_sizes, train_mean, label=f'Training Score', linestyle='-', marker='o')
plt.plot(train_sizes, test_mean, label=f'Cross-Validation Score', linestyle='--', marker='x')
plt.fill_between(train_sizes, train_mean - train_std, train_mean + train_std, alpha=0.1)
plt.fill_between(train_sizes, test_mean - test_std, test_mean + test_std, alpha=0.1)
plt.title('Learning Curve - Random Forest (After Tune)')
plt.xlabel('Training Set Size')
plt.ylabel('F1 Score')
plt.legend(loc='best')
plt.grid(True)
plt.show()
```

Learning Curve - Random Forest (After Tune)

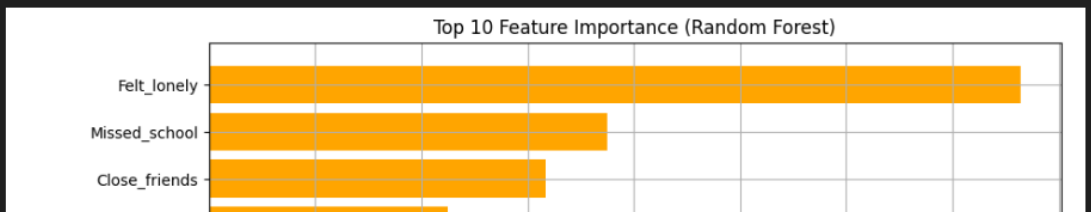
RF - ROU-AUC (After Tune)

```
# Predict probabilities for ROC Curve (Tuned Model)
y_prob_best_rf = best_rf.predict_proba(X_test)[:, 1]

# Compute ROC curve for Tuned Model
fpr_best_rf, tpr_best_rf, thresholds_best_rf = roc_curve(y_test, y_prob_best_rf)
roc_auc_best_rf = auc(fpr_best_rf, tpr_best_rf)

# Plot ROC-AUC Curve for Tuned Model
plt.figure(figsize=(10, 6))
plt.plot(fpr_best_rf, tpr_best_rf, lw=2, label='Random Forest (Tuned, AUC = %0.4f)' % roc_auc_best_rf)
plt.plot([0, 1], [0, 1], color='gray', linestyle='--')
plt.title('ROC Curve - Random Forest (After tune)')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend(loc="lower right")
plt.grid(True)
plt.show()
```



RF - Feature Importance

```
# Get feature importance from the tuned model
feature_importance_rf = best_rf.feature_importances_

# Create a dataframe for feature names and their corresponding importance
feature_importance_df = pd.DataFrame({
    'Feature': X_train.columns,
    'Importance': feature_importance_rf
})

# Sort by importance
feature_importance_df = feature_importance_df.sort_values(by='Importance', ascending=False)

# Plot the top 10 most important features
plt.figure(figsize=(10, 6))
plt.barh(feature_importance_df['Feature'][:10], feature_importance_df['Importance'][:10], color='orange')
plt.xlabel('Importance')
plt.title('Top 10 Feature Importance (Random Forest)')
plt.gca().invert_yaxis()
plt.grid(True)
plt.show()
```



(6.3) SVM

```python
from sklearn.svm import SVC

# SVM Model (use scaled data)
svm_model = SVC(kernel="linear", probability=True)
svm_model.fit(X_train_scaled, y_train)

# Predict and evaluate
y_pred_svm = svm_model.predict(X_test_scaled)
accuracy = accuracy_score(y_test, y_pred_svm)
confusion = confusion_matrix(y_test, y_pred_svm)
report = classification_report(y_test, y_pred_svm)

# Print results
print(f"Confusion Matrix:\n {confusion}")
print(f"\n📊 Classification Report:\n{report}")
print(f"✅ Accuracy: {accuracy:.4f}")
```

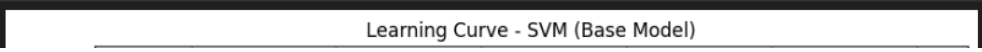
✓ 4m 37.5s

```
Confusion Matrix:
 [[3661 1266]
 [ 961 5509]]

📊 Classification Report:
              precision    recall  f1-score   support
```

SVM - Learning Curve (Base)

```python
# Learning Curve for SVM (Base Model)
train_sizes, train_scores, test_scores = learning_curve(
    svm_model,
    X_train_scaled,
    y_train,
    cv=3,
    n_jobs=-1,
    train_sizes=[0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1]
)

# Calculate mean and std for train and test scores
train_mean = train_scores.mean(axis=1)
test_mean = test_scores.mean(axis=1)
train_std = train_scores.std(axis=1)
test_std = test_scores.std(axis=1)

# Plot Learning Curve
plt.figure(figsize=(10, 6))
plt.plot(train_sizes, train_mean, label=f'Training Score', linestyle='-', marker='o')
plt.plot(train_sizes, test_mean, label=f'Cross-Validation Score', linestyle='--', marker='x')
plt.fill_between(train_sizes, train_mean - train_std, train_mean + train_std, alpha=0.1)
plt.fill_between(train_sizes, test_mean - test_std, test_mean + test_std, alpha=0.1)
plt.title('Learning Curve - SVM (Base Model)')
plt.xlabel('Training Set Size')
plt.ylabel('F1 Score')
plt.legend(loc='best')
plt.grid(True)
plt.show()
```

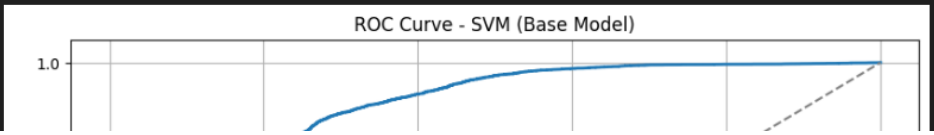
✓ 13m 9.3s



SVM - ROC-AUC (Base)

```
# Predict probabilities for ROC Curve (Base Model)
y_prob_svm = svm_model.predict_proba(X_test_scaled)[:, 1]

# Compute ROC curve
fpr_svm, tpr_svm, thresholds_svm = roc_curve(y_test, y_prob_svm)
roc_auc_svm = auc(fpr_svm, tpr_svm)

# Plot ROC-AUC Curve for Base Model
plt.figure(figsize=(10, 6))
plt.plot(fpr_svm, tpr_svm, lw=2, label='SVM (AUC = %0.4f)' % roc_auc_svm)
plt.plot([0, 1], [0, 1], color='gray', linestyle='--')
plt.title('ROC Curve - SVM (Base Model)')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend(loc="lower right")
plt.grid(True)
plt.show()
```

✓ 4.9s



SVM - Hyperparameter Tuning

```
# SVM Model
svm =  SVC(probability=True)

param_grid_svm = {
    'C': [0.1, 1, 10],
    'kernel': ['linear', 'rbf'],
    'gamma': ['scale', 'auto', 0.1, 1],
}

search_svm = RandomizedSearchCV(svm, param_grid_svm, n_iter=10,
                                cv=3, n_jobs=-1, random_state=42)

# Fit the model
search_svm.fit(X_train_scaled, y_train)

# Predict using the best model
best_svm = search_svm.best_estimator_
y_pred_svm = best_svm.predict(X_test_scaled)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred_svm)
confusion = confusion_matrix(y_test, y_pred_svm)
report = classification_report(y_test, y_pred_svm)

# Display results
print(f"Confusion Matrix (After Tuning):\n {confusion}")
print(f"\n📊Classification Report:\n{report}")

print("✅Best Hyperparameters for SVM : ")
for param, value in search_svm.best_params_.items():
    print(f"  {param.capitalize()}: {value}")

print(f"\nBest Cross-Validation Score: {search_svm.best_score_:.4f}")
print(f"Accuracy (After Tuning): {accuracy:.4f}")
```

✓ 53m 6.5s

```
Confusion Matrix (After Tuning):
 [[3792 1135]
```

SVM - Learning Curve (After Tune)

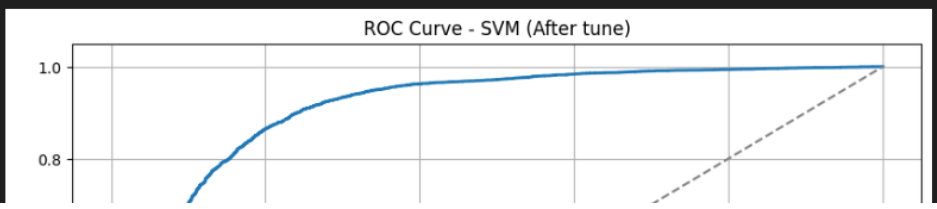
```python
# Learning Curve for Tuned SVM
train_sizes, train_scores, test_scores = learning_curve(
    best_svm,
    X_train_scaled,
    y_train,
    cv=3,
    n_jobs=-1,
    train_sizes=[0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1]
)

# Calculate mean and std for train and test scores
train_mean = train_scores.mean(axis=1)
test_mean = test_scores.mean(axis=1)
train_std = train_scores.std(axis=1)
test_std = test_scores.std(axis=1)

# Plot Learning Curve for Tuned Model
plt.figure(figsize=(10, 6))
plt.plot(train_sizes, train_mean, label=f'Training Score', linestyle='-', marker='o')
plt.plot(train_sizes, test_mean, label=f'Cross-Validation Score', linestyle='--', marker='x')
plt.fill_between(train_sizes, train_mean - train_std, train_mean + train_std, alpha=0.1)
plt.fill_between(train_sizes, test_mean - test_std, test_mean + test_std, alpha=0.1)
plt.title('Learning Curve - SVM (After Tune)')
plt.xlabel('Training Set Size')
plt.ylabel('F1 Score')
plt.legend(loc='best')
plt.grid(True)
plt.show()
```

✓ 16m 58.8s Python

SVM - ROC-AUC (After Tune)

```python
# Predict probabilities for ROC Curve (Tuned Model)
y_prob_best_svm = best_svm.predict_proba(X_test_scaled)[:, 1]

# Compute ROC curve for Tuned Model
fpr_best_svm, tpr_best_svm, thresholds_best_svm = roc_curve(y_test, y_prob_best_svm)
roc_auc_best_svm = auc(fpr_best_svm, tpr_best_svm)

# Plot ROC-AUC Curve for Tuned Model
plt.figure(figsize=(10, 6))
plt.plot(fpr_best_svm, tpr_best_svm, lw=2, label='SVM (Tuned, AUC = %0.4f)' % roc_auc_best_svm)
plt.plot([0, 1], [0, 1], color='gray', linestyle='--')
plt.title('ROC Curve - SVM (After tune)')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend(loc="lower right")
plt.grid(True)
plt.show()
```

✓ 15.4s Pyt



SVM - Feature Importance

```python
from sklearn.inspection import permutation_importance

# Get permutation importance
result = permutation_importance(best_svm, X_test_scaled, y_test, n_repeats=10, random_state=42)

# Create a DataFrame for feature importance
feature_importance_svm = pd.DataFrame({
    'Feature': X_train.columns,
    'Importance': result.importances_mean
})

# Sort the features by importance
feature_importance_svm = feature_importance_svm.sort_values(by='Importance', ascending=False)

# Plot the top 10 most important features
plt.figure(figsize=(10, 6))
plt.barh(feature_importance_svm['Feature'][:10], feature_importance_svm['Importance'][:10], color='dodgerblue')
plt.xlabel('Permutation Importance')
plt.title('Top 10 Feature Importance (SVM with RBF Kernel)')
plt.gca().invert_yaxis()
plt.grid(True)
plt.show()
```

✓ 32m 22.0s Python

(6.4) XGBoost (XGB)

```python
from xgboost import XGBClassifier

# XGBoost Model
xgboost_model = XGBClassifier(eval_metric="mlogloss", random_state=42)
xgboost_model.fit(X_train, y_train)

# Predict and evaluate
y_pred_xgboost = xgboost_model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred_xgboost)
confusion = confusion_matrix(y_test, y_pred_xgboost)
report = classification_report(y_test, y_pred_xgboost)

# Print results
print(f"Confusion Matrix:\n {confusion}")
print(f"\n📊 Classification Report:\n{report}")
print(f"✅ Accuracy: {accuracy:.4f}")
```

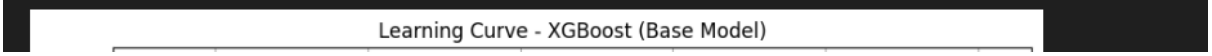
Python

```
Confusion Matrix:
 [[3953  974]
 [ 823 5647]]

📊 Classification Report:
              precision    recall  f1-score   support

           0       0.83      0.80      0.81      4927
           1       0.85      0.87      0.86      6470

    accuracy                           0.84     11397
   macro avg       0.84      0.84      0.84     11397
weighted avg       0.84      0.84      0.84     11397

✅ Accuracy: 0.8423
```

XGBoost - Learning Curve (Base)

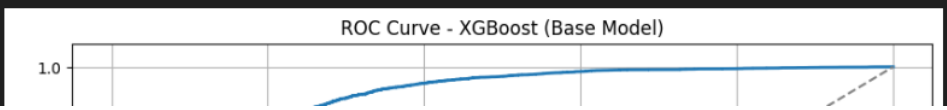
Generate + Code + Markdown

```python
# Learning Curve for XGBoost (Base Model)
train_sizes, train_scores, test_scores = learning_curve(
    xgboost_model,
    X_train,
    y_train,
    cv=3,
    n_jobs=-1,
    train_sizes=[0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1]
)

# Calculate mean and std for train and test scores
train_mean = train_scores.mean(axis=1)
test_mean = test_scores.mean(axis=1)
train_std = train_scores.std(axis=1)
test_std = test_scores.std(axis=1)

# Plot Learning Curve
plt.figure(figsize=(10, 6))
plt.plot(train_sizes, train_mean, label=f'Training Score', linestyle='-', marker='o')
plt.plot(train_sizes, test_mean, label=f'Cross-Validation Score', linestyle='--', marker='x')
plt.fill_between(train_sizes, train_mean - train_std, train_mean + train_std, alpha=0.1)
plt.fill_between(train_sizes, test_mean - test_std, test_mean + test_std, alpha=0.1)
plt.title('Learning Curve - XGBoost (Base Model)')
plt.xlabel('Training Set Size')
plt.ylabel('F1 Score')
plt.legend(loc='best')
plt.grid(True)
plt.show()
```



XGBoost - ROC-AUC (Base)

```python
# Predict probabilities for ROC Curve (Base Model)
y_prob_xgb = xgboost_model.predict_proba(X_test)[:, 1]

# Compute ROC curve
fpr_xgb, tpr_xgb, thresholds_xgb = roc_curve(y_test, y_prob_xgb)
roc_auc_xgb = auc(fpr_xgb, tpr_xgb)

# Plot ROC-AUC Curve for Base Model
plt.figure(figsize=(10, 6))
plt.plot(fpr_xgb, tpr_xgb, lw=2, label='XGBoost (AUC = %0.4f)' % roc_auc_xgb)
plt.plot([0, 1], [0, 1], color='gray', linestyle='--')
plt.title('ROC Curve - XGBoost (Base Model)')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend(loc="lower right")
plt.grid(True)
plt.show()
```



XGBoost - Hyperparameter Tuning

```python
from sklearn.model_selection import RandomizedSearchCV

# XGBoost Model
xgb = XGBClassifier()

param_grid_xgb = {
    'n_estimators': [50, 100, 200, 300],
    'max_depth': [3, 6, 10, 15],
    'learning_rate': [0.01, 0.1, 0.2, 0.5],
    'subsample': [0.6, 0.8, 1.0],
    'colsample_bytree': [0.6, 0.8, 1.0],
    'gamma': [0, 0.1, 0.2, 0.5],
}

# Perform RandomizedSearchCV
search_xgb = RandomizedSearchCV(xgb, param_grid_xgb, n_iter=10,
                                cv=5, n_jobs=-1, random_state=42)

# Fit the model
search_xgb.fit(X_train, y_train)

# Predict using the best model
best_xgb = search_xgb.best_estimator_
y_pred_xgb = best_xgb.predict(X_test)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred_xgb)
confusion = confusion_matrix(y_test, y_pred_xgb)
report = classification_report(y_test, y_pred_xgb)

# Display results
print(f"Confusion Matrix (After Tuning):\n{confusion}")
print(f"\n📊Classification Report:\n{report}")

print("✅Best Hyperparameters for XGBoost: ")
for param, value in search_xgb.best_params_.items():
    print(f"  {param.capitalize()}: {value}")

print(f"\nBest Cross-Validation Score: {search_xgb.best_score_:.4f}")
print(f"Accuracy (After Tuning): {accuracy:.4f}")
```

XGBoost - Learning Curve (After Tune)

```python
# Learning Curve for Tuned XGBoost
train_sizes, train_scores, test_scores = learning_curve(
    best_xgb, X_train, y_train, cv=3, n_jobs=-1,
    train_sizes=[0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1]
)

# Calculate mean and std for train and test scores
train_mean = train_scores.mean(axis=1)
test_mean = test_scores.mean(axis=1)
train_std = train_scores.std(axis=1)
test_std = test_scores.std(axis=1)

# Plot Learning Curve for Tuned Model
plt.figure(figsize=(10, 6))
plt.plot(train_sizes, train_mean, label=f'Training Score', linestyle='-', marker='o')
plt.plot(train_sizes, test_mean, label=f'Cross-Validation Score', linestyle='--', marker='x')
plt.fill_between(train_sizes, train_mean - train_std, train_mean + train_std, alpha=0.1)
plt.fill_between(train_sizes, test_mean - test_std, test_mean + test_std, alpha=0.1)
plt.title('Learning Curve - XGBoost (After Tune)')
plt.xlabel('Training Set Size')
plt.ylabel('F1 Score')
plt.legend(loc='best')
plt.grid(True)
plt.show()
```

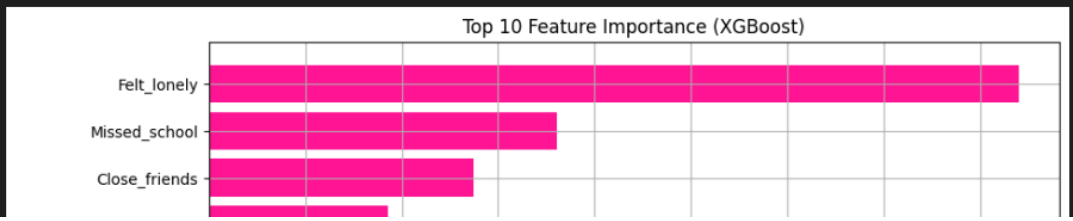
Python

XGBoost - ROC-AUC (After Tune)

```python
# Predict probabilities for ROC Curve (Tuned Model)
y_prob_best_xgb = best_xgb.predict_proba(X_test)[:, 1]

# Compute ROC curve for Tuned Model
fpr_best_xgb, tpr_best_xgb, thresholds_best_xgb = roc_curve(y_test, y_prob_best_xgb)
roc_auc_best_xgb = auc(fpr_best_xgb, tpr_best_xgb)

# Plot ROC-AUC Curve for Tuned Model
plt.figure(figsize=(10, 6))
plt.plot(fpr_best_xgb, tpr_best_xgb, lw=2, label='XGBoost (Tuned, AUC = %0.4f)' % roc_auc_best_xgb)
plt.plot([0, 1], [0, 1], color='gray', linestyle='--')
plt.title('ROC Curve - XGBoost (After tune)')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend(loc="lower right")
plt.grid(True)
plt.show()
```

XGBoost - Feature Importance

```python
# Get feature importance from the tuned XGBoost model
feature_importance_xgb = best_xgb.feature_importances_

# Create a dataframe for feature names and their corresponding importance
feature_importance_df_xgb = pd.DataFrame({
    'Feature': X_train.columns,
    'Importance': feature_importance_xgb
})

# Sort by importance (descending order)
feature_importance_df_xgb = feature_importance_df_xgb.sort_values(by='Importance', ascending=False)

# Plot the top 10 most important features
plt.figure(figsize=(10, 6))
plt.barh(feature_importance_df_xgb['Feature'][:10], feature_importance_df_xgb['Importance'][:10], color='deeppink')
plt.xlabel('Importance')
plt.title('Top 10 Feature Importance (XGBoost)')
plt.gca().invert_yaxis()
plt.grid(True)
plt.show()
```



(6.5) Best Model

```python
# Compare performance metrics (accuracy and AUC) for each model
model_performance = {
    'Logistic Regression': accuracy_score(y_test, search_logreg.best_estimator_.predict(X_test)),
    'Random Forest': accuracy_score(y_test, search_rf.best_estimator_.predict(X_test)),
    'SVM': accuracy_score(y_test, search_svm.best_estimator_.predict(X_test)),
    'XGBoost': accuracy_score(y_test, search_xgb.best_estimator_.predict(X_test))
}

best_model_name = max(model_performance, key=model_performance.get)
print(f"Best model: {best_model_name}")
print(f"Accuracy: {model_performance[best_model_name]:.4f}\n")
```

```
Best model: XGBoost
Accuracy: 0.8434

c:\Users\Yap\AppData\Local\Programs\Python\Python311\Lib\site-packages\sklearn\utils\validation.py:2732: UserWarning: X has feature names,
  warnings.warn(
```

7. Model Deployment

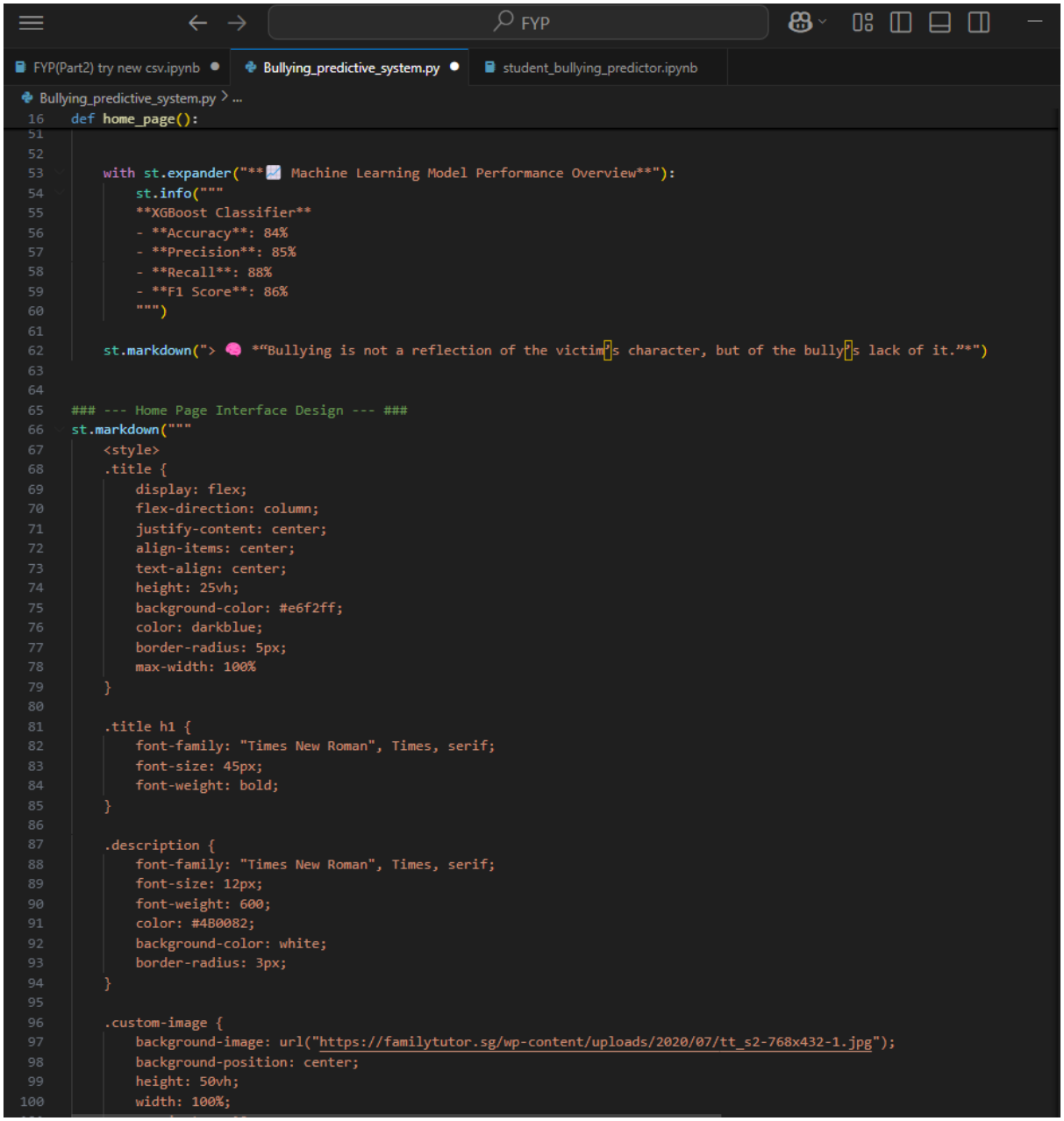
```python
import joblib

joblib.dump(best_xgb, 'Bullying_model.pkl')
```

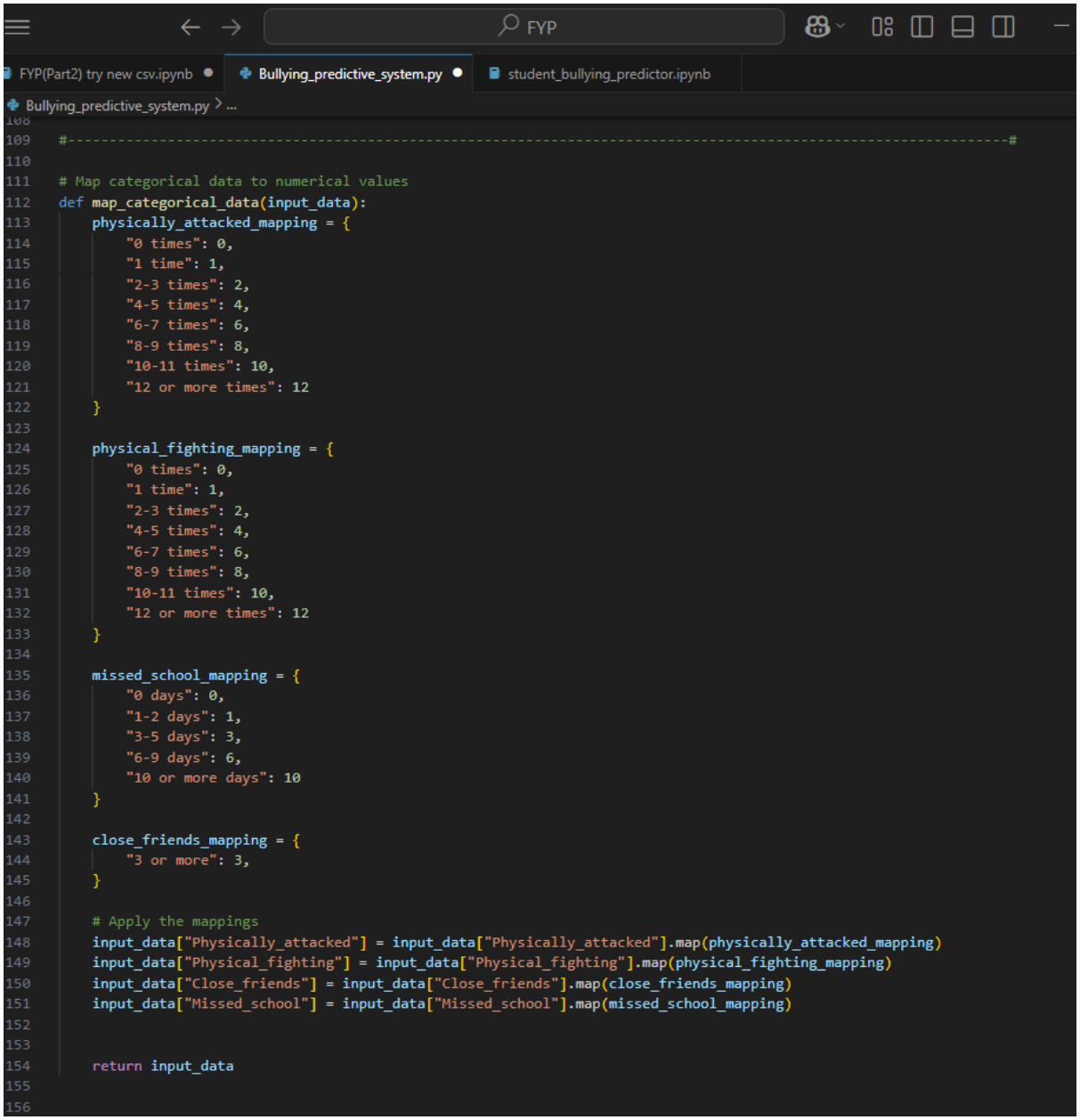
```
['Bullying_model.pkl']
```

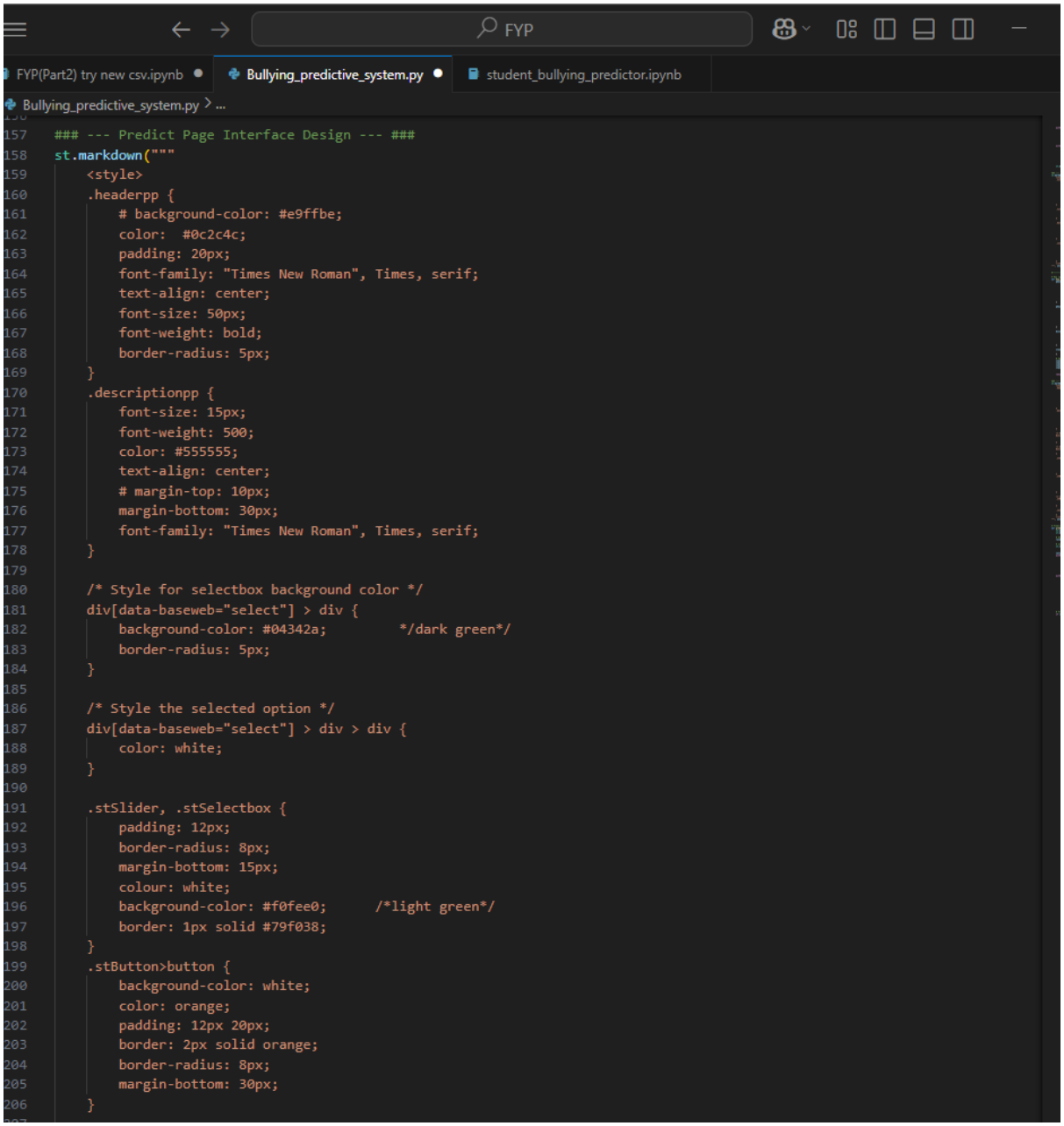
FYP(Part2) try new csv.ipynb ● | Bullying_predictive_system.py ● | student_bullying_predictor.ipynb

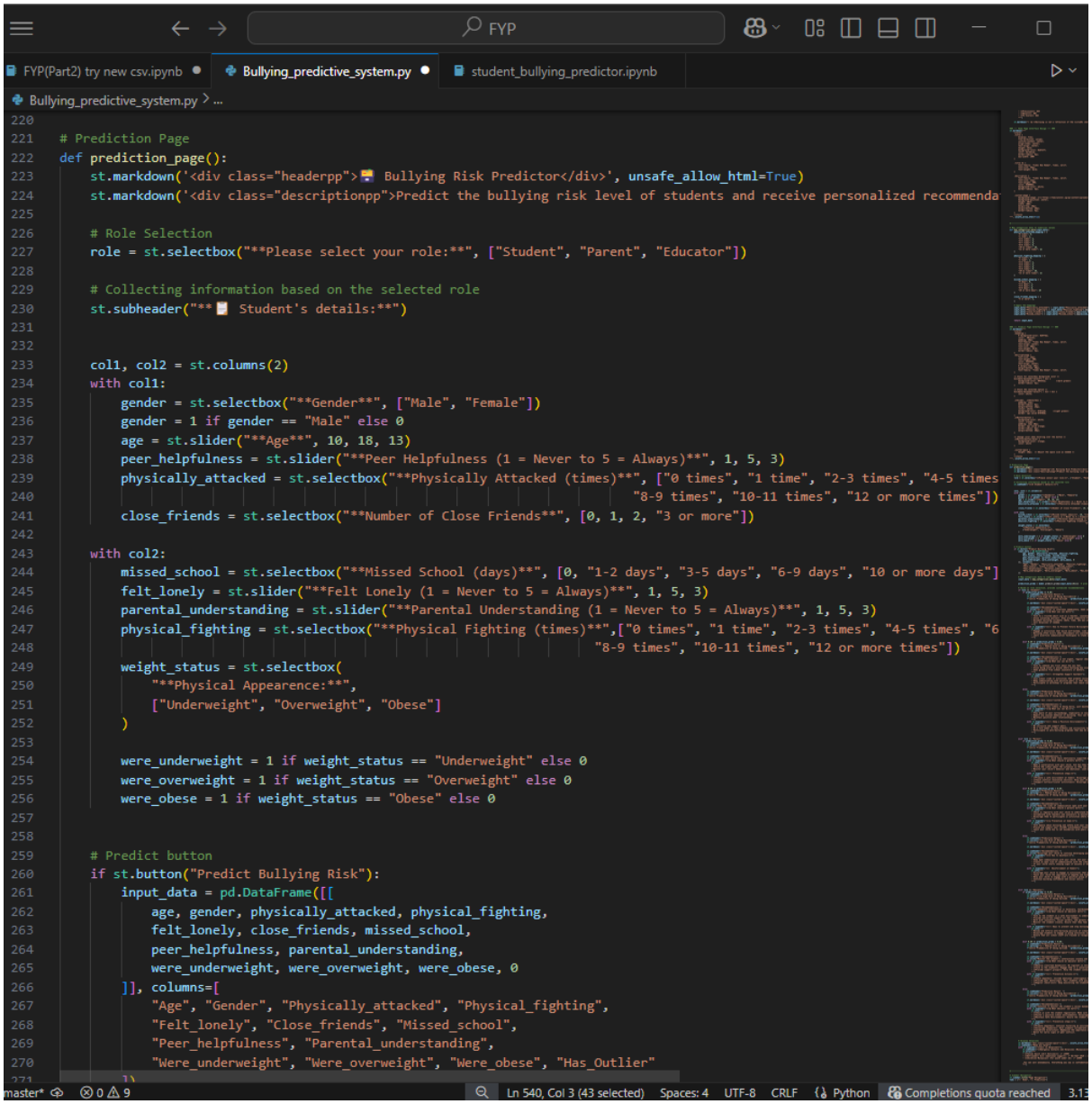
Bullying_predictive_system.py > ...

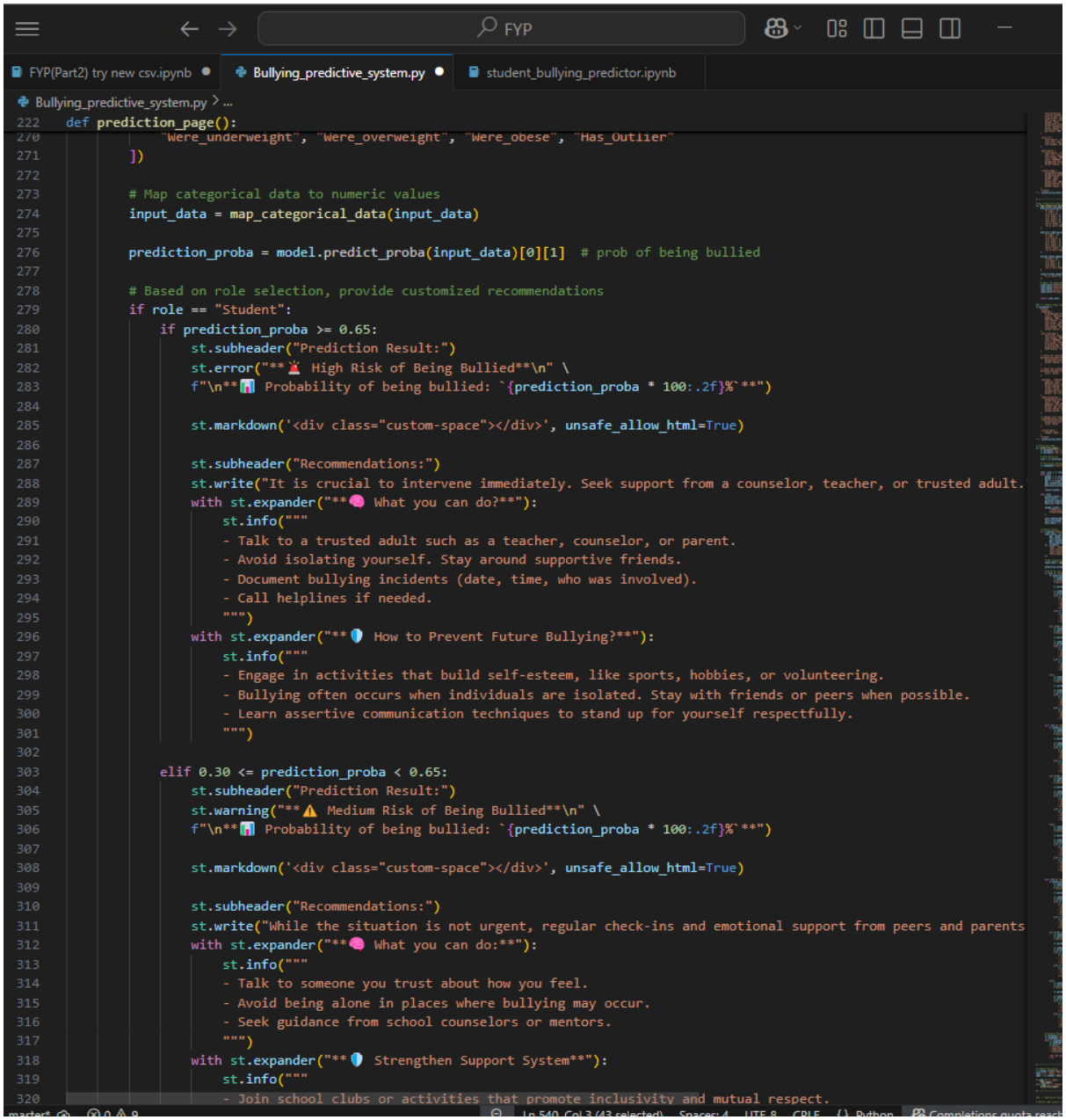
```python

import streamlit as st
import pickle
import pandas as pd

# Load the trained model
with open("Bullying_model.pkl", "rb") as file:
    model = pickle.load(file)

st.set_page_config(page_title="Bullying Prediction System", page_icon=":brain:", layout="wide")


#-------------------------------------------------------------------------------------------------------------#

# Home Page
def home_page():
    st.markdown('<div class="title"><h1>🏫 Bullying Victimization Risk Prediction System</h1>'
                '<div class="description">" Trained on a Global School-Based Student Health Survey (GSHS) Dataset in Argent
                , unsafe_allow_html=True)

    st.markdown('<div class="custom-image">.</div></div>'
                , unsafe_allow_html=True)

    with st.expander("**🧠 Understanding Bullying**"):
        st.info("""
        Bullying is an aggressive behavior that involves unwanted, negative actions.
        It usually involves a power imbalance and is repeated over time. \n\n
        Bullying can be:
        - **Physically**:  Hitting, Pushing
        - **Verbally**: Name-calling, Threats
        - **Social/Relational**: Exclusion, Spreading rumors
        - **Cyberbullying**: via social media, Texting \n\n
        The victims of bullying often experience anxiety, depression, low self-esteem, and may avoid school.
        """)


    with st.expander("**🔧 This system is to:**"):
        col1, col2 = st.columns(2)
        with col1:
            st.success("#### 🎯 Prediction\nThis system is to predict whether a student are on risk to being bullid.")
        with col2:
            st.success("#### 🧠 Recommendation\nStudents, parents and educators can get recommendation on preventing bullyi


    with st.expander(" **💜 Who Can Use This System?**"):
            st.success("""
            - 🏫 **Students**: Identify risk level by applying their informations.
            - 👪 **Parents**: Understand emotional and behavioral risk factors of their children.
            - 🏫 **Educators**: Identify at-risk students early in school environment.
            """)
```

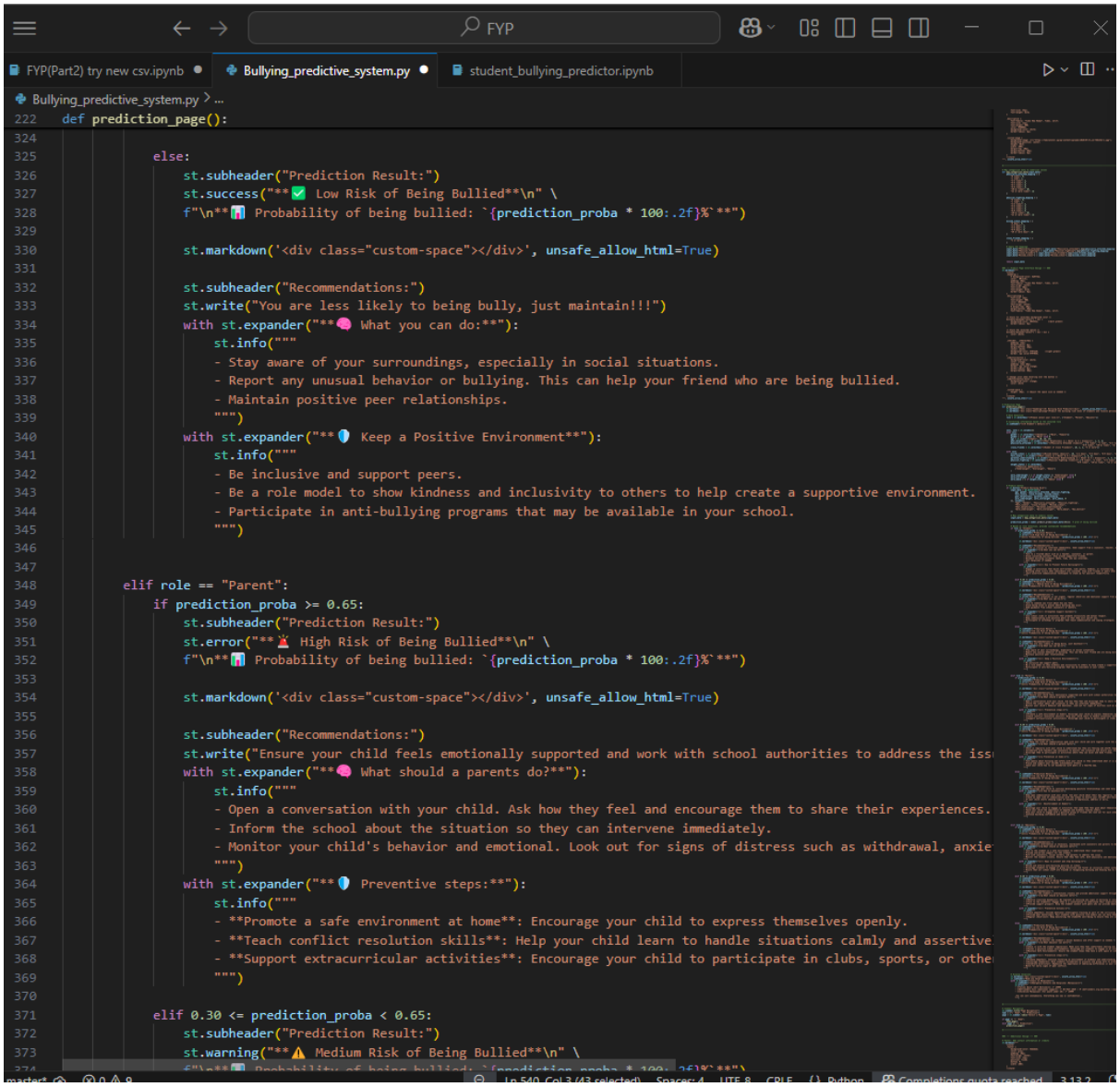

FYP
FYP(Part2) try new csv.ipynb
Bullying_predictive_system.py
student_bullying_predictor.ipynb
Bullying_predictive_system.py > ...
def home_page():
with st.expander("**📈 Machine Learning Model Performance Overview**"):
st.info("""
XGBoost Classifier
- **Accuracy**: 84%
- **Precision**: 85%
- **Recall**: 88%
- **F1 Score**: 86%
""")
st.markdown("> 🧠 *"Bullying is not a reflection of the victim's character, but of the bully's lack of it."*")
--- Home Page Interface Design ---
st.markdown("""
<style>
.title {
display: flex;
flex-direction: column;
justify-content: center;
align-items: center;
text-align: center;
height: 25vh;
background-color: #e6f2ff;
color: darkblue;
border-radius: 5px;
max-width: 100%
}
.title h1 {
font-family: "Times New Roman", Times, serif;
font-size: 45px;
font-weight: bold;
}
.description {
font-family: "Times New Roman", Times, serif;
font-size: 12px;
font-weight: 600;
color: #4B0082;
background-color: white;
border-radius: 3px;
}
.custom-image {
background-image: url("https://familytutor.sg/wp-content/uploads/2020/07/tt_s2-768x432-1.jpg");
background-position: center;
height: 50vh;
width: 100%;


FYP
FYP(Part2) try new csv.ipynb
Bullying_predictive_system.py
student_bullying_predictor.ipynb
Bullying_predictive_system.py > ...
#----------#
# Map categorical data to numerical values
def map_categorical_data(input_data):
physically_attacked_mapping = {
"0 times": 0,
"1 time": 1,
"2-3 times": 2,
"4-5 times": 4,
"6-7 times": 6,
"8-9 times": 8,
"10-11 times": 10,
"12 or more times": 12
}
physical_fighting_mapping = {
"0 times": 0,
"1 time": 1,
"2-3 times": 2,
"4-5 times": 4,
"6-7 times": 6,
"8-9 times": 8,
"10-11 times": 10,
"12 or more times": 12
}
missed_school_mapping = {
"0 days": 0,
"1-2 days": 1,
"3-5 days": 3,
"6-9 days": 6,
"10 or more days": 10
}
close_friends_mapping = {
"3 or more": 3,
}
# Apply the mappings
input_data["Physically_attacked"] = input_data["Physically_attacked"].map(physically_attacked_mapping)
input_data["Physical_fighting"] = input_data["Physical_fighting"].map(physical_fighting_mapping)
input_data["Close_friends"] = input_data["Close_friends"].map(close_friends_mapping)
input_data["Missed_school"] = input_data["Missed_school"].map(missed_school_mapping)
return input_data

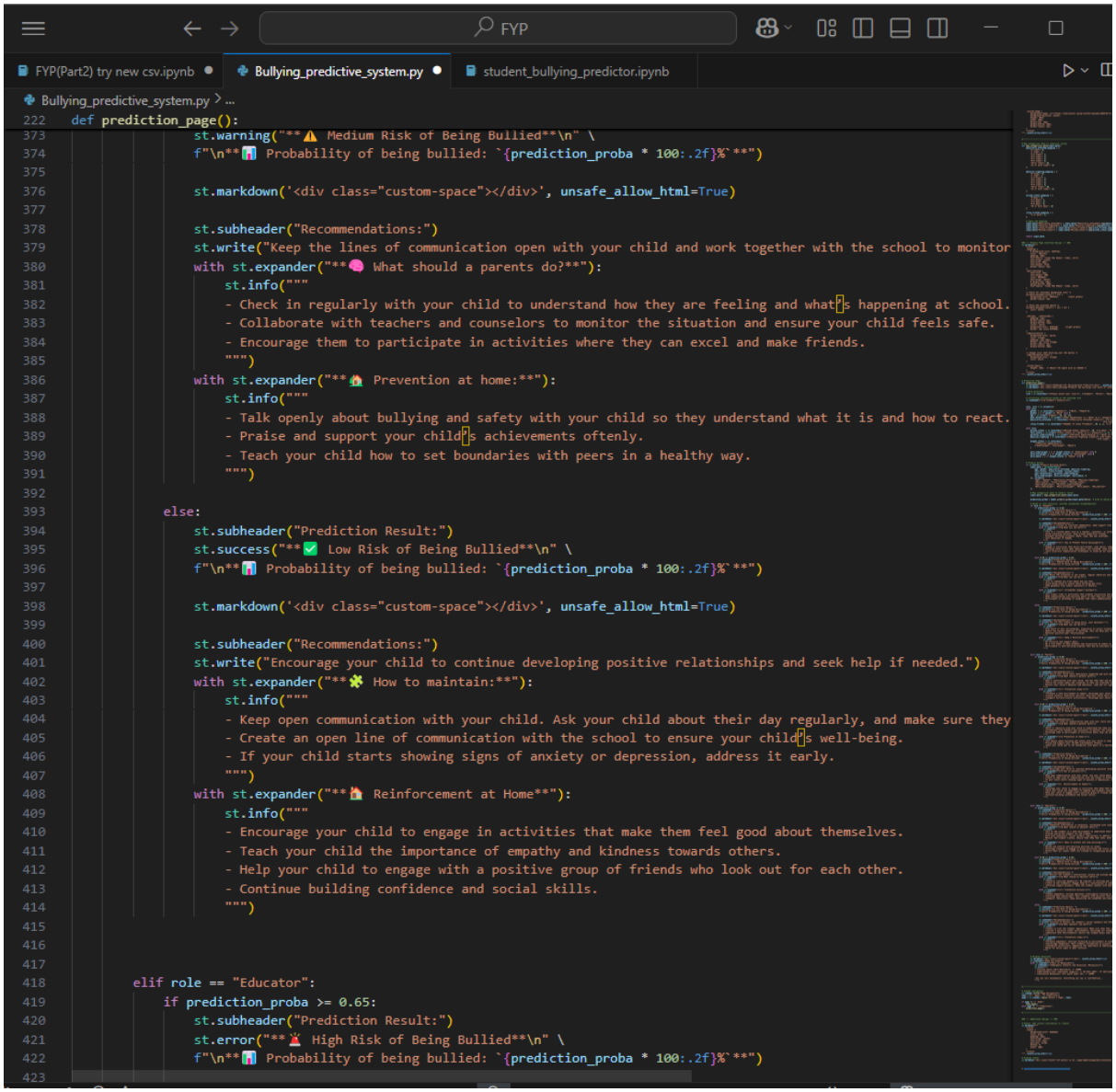

FYP
FYP(Part2) try new csv.ipynb
Bullying_predictive_system.py
student_bullying_predictor.ipynb
Bullying_predictive_system.py > ...
--- Predict Page Interface Design ---
st.markdown("""
<style>
.headerpp {
# background-color: #e9ffbe;
color: #0c2c4c;
padding: 20px;
font-family: "Times New Roman", Times, serif;
text-align: center;
font-size: 50px;
font-weight: bold;
border-radius: 5px;
}
.descriptionpp {
font-size: 15px;
font-weight: 500;
color: #555555;
text-align: center;
margin-top: 10px;
margin-bottom: 30px;
font-family: "Times New Roman", Times, serif;
}
/* Style for selectbox background color */
div[data-baseweb="select"] > div {
background-color: #04342a; */dark green*/
border-radius: 5px;
}
/* Style the selected option */
div[data-baseweb="select"] > div > div {
color: white;
}
.stSlider, .stSelectbox {
padding: 12px;
border-radius: 8px;
margin-bottom: 15px;
colour: white;
background-color: #f0fee0; /*light green*/
border: 1px solid #79f038;
}
.stButton>button {
background-color: white;
color: orange;
padding: 12px 20px;
border: 2px solid orange;
border-radius: 8px;
margin-bottom: 30px;
}

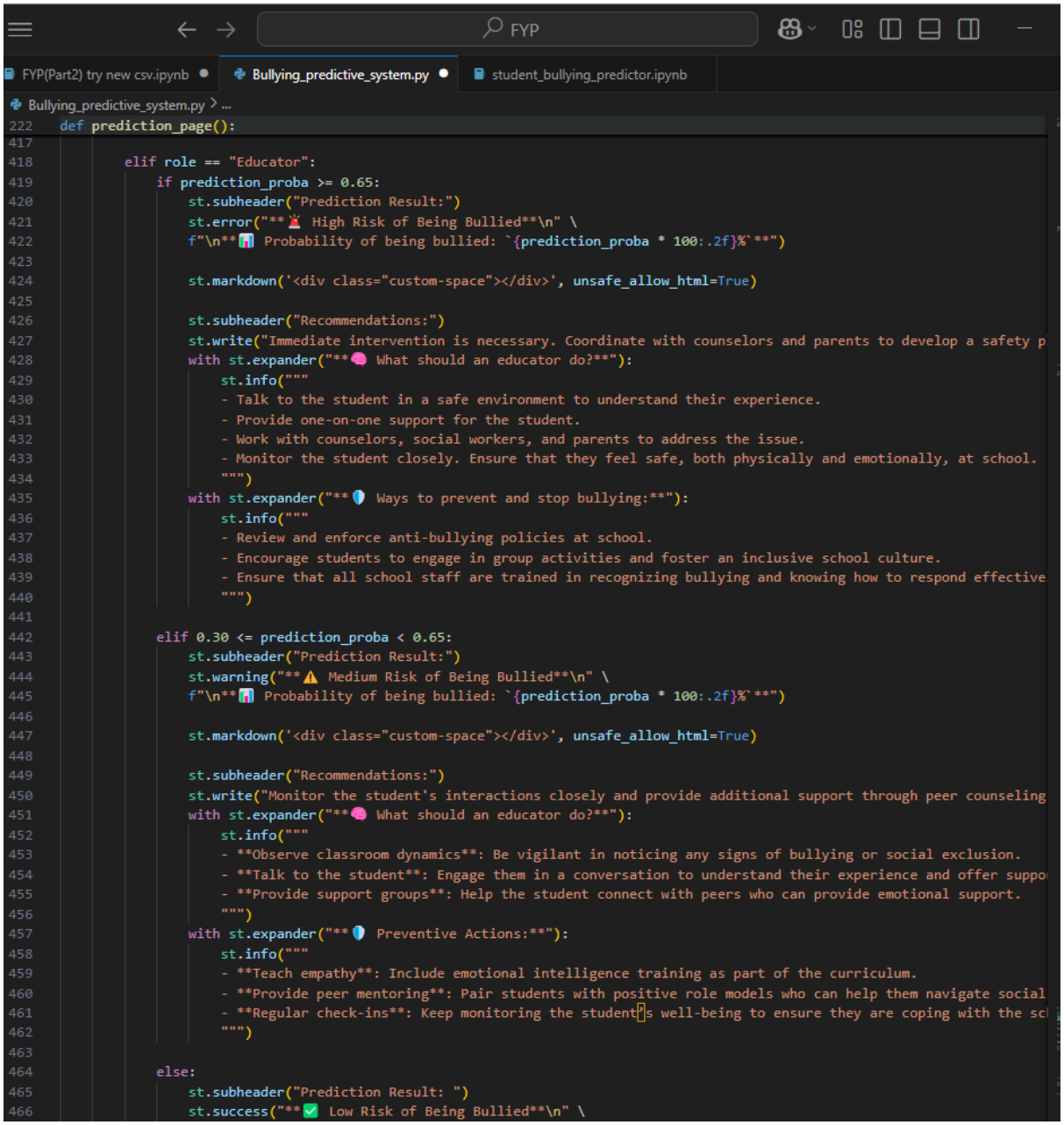

FYP
FYP(Part2) try new csv.ipynb
Bullying_predictive_system.py
student_bullying_predictor.ipynb
Bullying_predictive_system.py > ...
# Prediction Page
def prediction_page():
st.markdown(' Bullying Risk Predictor ', unsafe_allow_html=True)
st.markdown(' Predict the bullying risk level of students and receive personalized recommenda
# Role Selection
role = st.selectbox("**Please select your role:**", ["Student", "Parent", "Educator"])
# Collecting information based on the selected role
st.subheader("** Student's details:**")
col1, col2 = st.columns(2)
with col1:
gender = st.selectbox("**Gender**", ["Male", "Female"])
gender = 1 if gender == "Male" else 0
age = st.slider("**Age**", 10, 18, 13)
peer_helpfulness = st.slider("**Peer Helpfulness (1 = Never to 5 = Always)**", 1, 5, 3)
physically_attacked = st.selectbox("**Physically Attacked (times)**", ["0 times", "1 time", "2-3 times", "4-5 times
"8-9 times", "10-11 times", "12 or more times"])
close_friends = st.selectbox("**Number of Close Friends**", [0, 1, 2, "3 or more"])
with col2:
missed_school = st.selectbox("**Missed School (days)**", [0, "1-2 days", "3-5 days", "6-9 days", "10 or more days"]
felt_lonely = st.slider("**Felt Lonely (1 = Never to 5 = Always)**", 1, 5, 3)
parental_understanding = st.slider("**Parental Understanding (1 = Never to 5 = Always)**", 1, 5, 3)
physical_fighting = st.selectbox("**Physical Fighting (times)**",["0 times", "1 time", "2-3 times", "4-5 times", "6
"8-9 times", "10-11 times", "12 or more times"])
weight_status = st.selectbox(
"**Physical Appearence:**",
["Underweight", "Overweight", "Obese"]
)
were_underweight = 1 if weight_status == "Underweight" else 0
were_overweight = 1 if weight_status == "Overweight" else 0
were_obese = 1 if weight_status == "Obese" else 0
# Predict button
if st.button("Predict Bullying Risk"):
input_data = pd.DataFrame([[
age, gender, physically_attacked, physical_fighting,
felt_lonely, close_friends, missed_school,
peer_helpfulness, parental_understanding,
were_underweight, were_overweight, were_obese, 0
]], columns=[
"Age", "Gender", "Physically_attacked", "Physical_fighting",
"Felt_lonely", "Close_friends", "Missed_school",
"Peer_helpfulness", "Parental_understanding",
"Were_underweight", "Were_overweight", "Were_obese", "Has_Outlier"
])
master*
0 9
Ln 540, Col 3 (43 selected)
Spaces: 4
UTF-8
CRLF
Python
Completions quota reached

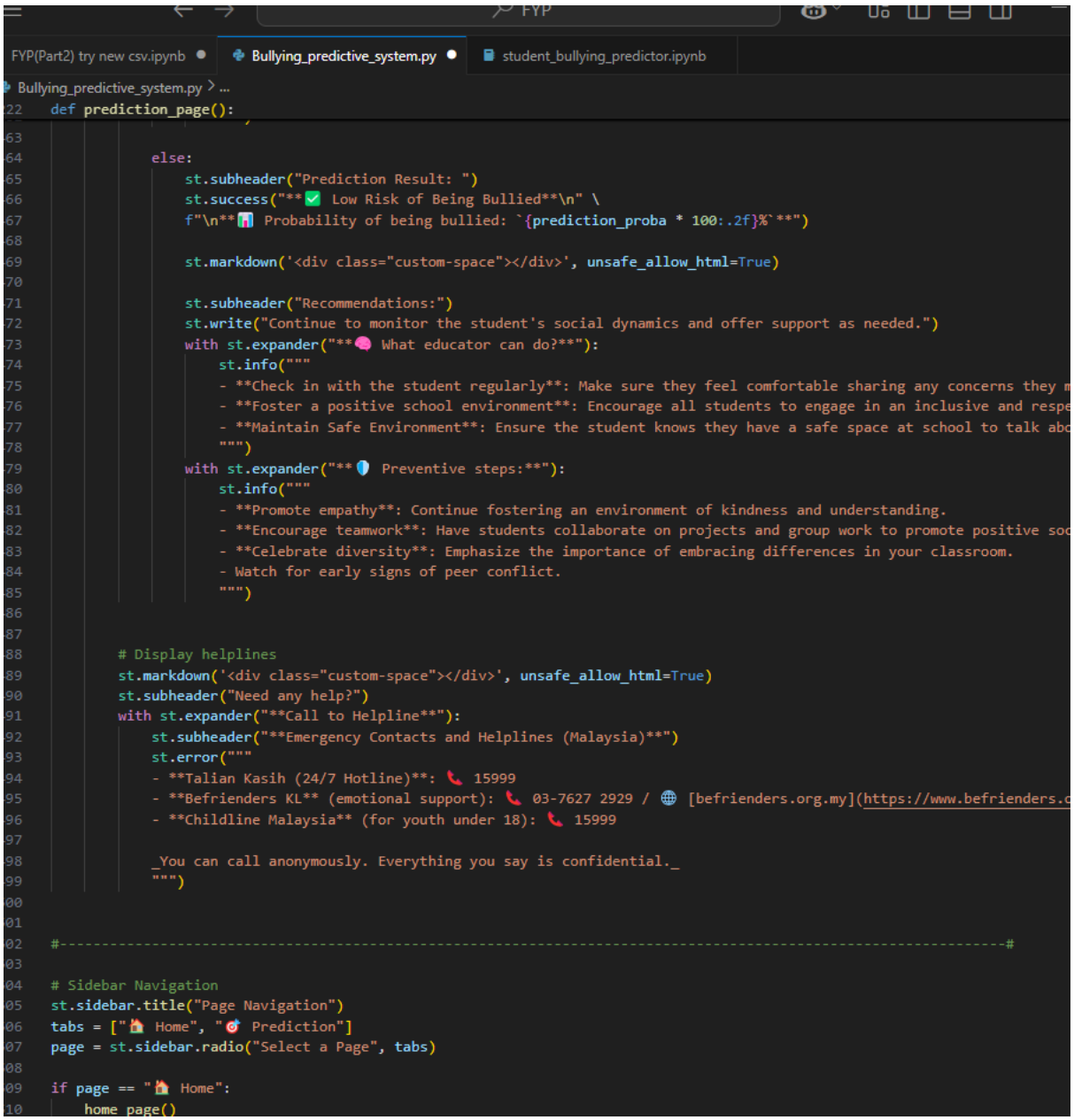

FYP
FYP(Part2) try new csv.ipynb
Bullying_predictive_system.py
student_bullying_predictor.ipynb
Bullying_predictive_system.py > ...
def prediction_page():
"Were_underweight", "Were_overweight", "Were_obese", "Has_Outlier"
])
# Map categorical data to numeric values
input_data = map_categorical_data(input_data)
prediction_proba = model.predict_proba(input_data)[0][1] # prob of being bullied
# Based on role selection, provide customized recommendations
if role == "Student":
if prediction_proba >= 0.65:
st.subheader("Prediction Result:")
st.error("**🚨 High Risk of Being Bullied**\n" \
f"\n**📊 Probability of being bullied: `{prediction_proba * 100:.2f}%`**")
st.markdown(' ', unsafe_allow_html=True)
st.subheader("Recommendations:")
st.write("It is crucial to intervene immediately. Seek support from a counselor, teacher, or trusted adult.
with st.expander("**🧠 What you can do?**"):
st.info("""
- Talk to a trusted adult such as a teacher, counselor, or parent.
- Avoid isolating yourself. Stay around supportive friends.
- Document bullying incidents (date, time, who was involved).
- Call helplines if needed.
""")
with st.expander("**🛡️ How to Prevent Future Bullying?**"):
st.info("""
- Engage in activities that build self-esteem, like sports, hobbies, or volunteering.
- Bullying often occurs when individuals are isolated. Stay with friends or peers when possible.
- Learn assertive communication techniques to stand up for yourself respectfully.
""")
elif 0.30 <= prediction_proba < 0.65:
st.subheader("Prediction Result:")
st.warning("**⚠️ Medium Risk of Being Bullied**\n" \
f"\n**📊 Probability of being bullied: `{prediction_proba * 100:.2f}%`**")
st.markdown(' ', unsafe_allow_html=True)
st.subheader("Recommendations:")
st.write("While the situation is not urgent, regular check-ins and emotional support from peers and parents
with st.expander("**🧠 What you can do:**"):
st.info("""
- Talk to someone you trust about how you feel.
- Avoid being alone in places where bullying may occur.
- Seek guidance from school counselors or mentors.
""")
with st.expander("**🛡️ Strengthen Support System**"):
st.info("""
- Join school clubs or activities that promote inclusivity and mutual respect.




```
def prediction_page():

            else:
                st.subheader("Prediction Result:")
                st.success("** Low Risk of Being Bullied**\n" \
                f"\n** Probability of being bullied: `{prediction_proba * 100:.2f}%`**")

                st.markdown('<div class="custom-space"></div>', unsafe_allow_html=True)

                st.subheader("Recommendations:")
                st.write("You are less likely to being bully, just maintain!!!")
                with st.expander("** What you can do:**"):
                    st.info("""
                    - Stay aware of your surroundings, especially in social situations.
                    - Report any unusual behavior or bullying. This can help your friend who are being bullied.
                    - Maintain positive peer relationships.
                    """)
                with st.expander("** Keep a Positive Environment**"):
                    st.info("""
                    - Be inclusive and support peers.
                    - Be a role model to show kindness and inclusivity to others to help create a supportive environment.
                    - Participate in anti-bullying programs that may be available in your school.
                    """)


        elif role == "Parent":
            if prediction_proba >= 0.65:
                st.subheader("Prediction Result:")
                st.error("** High Risk of Being Bullied**\n" \
                f"\n** Probability of being bullied: `{prediction_proba * 100:.2f}%`**")

                st.markdown('<div class="custom-space"></div>', unsafe_allow_html=True)

                st.subheader("Recommendations:")
                st.write("Ensure your child feels emotionally supported and work with school authorities to address the iss
                with st.expander("** What should a parents do?**"):
                    st.info("""
                    - Open a conversation with your child. Ask how they feel and encourage them to share their experiences.
                    - Inform the school about the situation so they can intervene immediately.
                    - Monitor your child's behavior and emotional. Look out for signs of distress such as withdrawal, anxie
                    """)
                with st.expander("** Preventive steps:**"):
                    st.info("""
                    - **Promote a safe environment at home**: Encourage your child to express themselves openly.
                    - **Teach conflict resolution skills**: Help your child learn to handle situations calmly and assertive
                    - **Support extracurricular activities**: Encourage your child to participate in clubs, sports, or othe
                    """)

            elif 0.30 <= prediction_proba < 0.65:
                st.subheader("Prediction Result:")
                st.warning("** Medium Risk of Being Bullied**\n" \
```


FYP
FYP(Part2) try new csv.ipynb
Bullying_predictive_system.py
student_bullying_predictor.ipynb
Bullying_predictive_system.py > ...
def prediction_page():
st.warning("**⚠️ Medium Risk of Being Bullied**\n" \
f"\n**📊 Probability of being bullied: `{prediction_proba * 100:.2f}%`**")
st.markdown(' ', unsafe_allow_html=True)
st.subheader("Recommendations:")
st.write("Keep the lines of communication open with your child and work together with the school to monitor
with st.expander("**🧠 What should a parents do?**"):
st.info("""
- Check in regularly with your child to understand how they are feeling and what's happening at school.
- Collaborate with teachers and counselors to monitor the situation and ensure your child feels safe.
- Encourage them to participate in activities where they can excel and make friends.
""")
with st.expander("**🏠 Prevention at home:**"):
st.info("""
- Talk openly about bullying and safety with your child so they understand what it is and how to react.
- Praise and support your child's achievements oftenly.
- Teach your child how to set boundaries with peers in a healthy way.
""")
else:
st.subheader("Prediction Result:")
st.success("**✅ Low Risk of Being Bullied**\n" \
f"\n**📊 Probability of being bullied: `{prediction_proba * 100:.2f}%`**")
st.markdown(' ', unsafe_allow_html=True)
st.subheader("Recommendations:")
st.write("Encourage your child to continue developing positive relationships and seek help if needed.")
with st.expander("**❇️ How to maintain:**"):
st.info("""
- Keep open communication with your child. Ask your child about their day regularly, and make sure they
- Create an open line of communication with the school to ensure your child's well-being.
- If your child starts showing signs of anxiety or depression, address it early.
""")
with st.expander("**🏠 Reinforcement at Home**"):
st.info("""
- Encourage your child to engage in activities that make them feel good about themselves.
- Teach your child the importance of empathy and kindness towards others.
- Help your child to engage with a positive group of friends who look out for each other.
- Continue building confidence and social skills.
""")
elif role == "Educator":
if prediction_proba >= 0.65:
st.subheader("Prediction Result:")
st.error("**🚨 High Risk of Being Bullied**\n" \
f"\n**📊 Probability of being bullied: `{prediction_proba * 100:.2f}%`**")


FYP
FYP(Part2) try new csv.ipynb
Bullying_predictive_system.py
student_bullying_predictor.ipynb
Bullying_predictive_system.py > ...
222 def prediction_page():
417
418 elif role == "Educator":
419 if prediction_proba >= 0.65:
420 st.subheader("Prediction Result:")
421 st.error("**🚨 High Risk of Being Bullied**\n" \
422 f"\n**📊 Probability of being bullied: `{prediction_proba * 100:.2f}%`**")
423
424 st.markdown(' ', unsafe_allow_html=True)
425
426 st.subheader("Recommendations:")
427 st.write("Immediate intervention is necessary. Coordinate with counselors and parents to develop a safety p
428 with st.expander("**🧠 What should an educator do?**"):
429 st.info("""
430 - Talk to the student in a safe environment to understand their experience.
431 - Provide one-on-one support for the student.
432 - Work with counselors, social workers, and parents to address the issue.
433 - Monitor the student closely. Ensure that they feel safe, both physically and emotionally, at school.
434 """)
435 with st.expander("**🛡️ Ways to prevent and stop bullying:**"):
436 st.info("""
437 - Review and enforce anti-bullying policies at school.
438 - Encourage students to engage in group activities and foster an inclusive school culture.
439 - Ensure that all school staff are trained in recognizing bullying and knowing how to respond effective
440 """)
441
442 elif 0.30 <= prediction_proba < 0.65:
443 st.subheader("Prediction Result:")
444 st.warning("**⚠️ Medium Risk of Being Bullied**\n" \
445 f"\n**📊 Probability of being bullied: `{prediction_proba * 100:.2f}%`**")
446
447 st.markdown(' ', unsafe_allow_html=True)
448
449 st.subheader("Recommendations:")
450 st.write("Monitor the student's interactions closely and provide additional support through peer counseling
451 with st.expander("**🧠 What should an educator do?**"):
452 st.info("""
453 - **Observe classroom dynamics**: Be vigilant in noticing any signs of bullying or social exclusion.
454 - **Talk to the student**: Engage them in a conversation to understand their experience and offer suppo
455 - **Provide support groups**: Help the student connect with peers who can provide emotional support.
456 """)
457 with st.expander("**🛡️ Preventive Actions:**"):
458 st.info("""
459 - **Teach empathy**: Include emotional intelligence training as part of the curriculum.
460 - **Provide peer mentoring**: Pair students with positive role models who can help them navigate social
461 - **Regular check-ins**: Keep monitoring the student's well-being to ensure they are coping with the sc
462 """)
463
464 else:
465 st.subheader("Prediction Result: ")
466 st.success("**✅ Low Risk of Being Bullied**\n" \


FYP
FYP(Part2) try new csv.ipynb
Bullying_predictive_system.py
student_bullying_predictor.ipynb
Bullying_predictive_system.py > ...
def prediction_page():
else:
st.subheader("Prediction Result: ")
st.success("**✅ Low Risk of Being Bullied**\n" \
f"\n**📊 Probability of being bullied: `{prediction_proba * 100:.2f}%`**")
st.markdown(' ', unsafe_allow_html=True)
st.subheader("Recommendations:")
st.write("Continue to monitor the student's social dynamics and offer support as needed.")
with st.expander("**🧠 What educator can do?**"):
st.info("""
- **Check in with the student regularly**: Make sure they feel comfortable sharing any concerns they
- **Foster a positive school environment**: Encourage all students to engage in an inclusive and respe
- **Maintain Safe Environment**: Ensure the student knows they have a safe space at school to talk abo
""")
with st.expander("**🛡️ Preventive steps:**"):
st.info("""
- **Promote empathy**: Continue fostering an environment of kindness and understanding.
- **Encourage teamwork**: Have students collaborate on projects and group work to promote positive soc
- **Celebrate diversity**: Emphasize the importance of embracing differences in your classroom.
- Watch for early signs of peer conflict.
""")
# Display helplines
st.markdown(' ', unsafe_allow_html=True)
st.subheader("Need any help?")
with st.expander("**Call to Helpline**"):
st.subheader("**Emergency Contacts and Helplines (Malaysia)**")
st.error("""
- **Talian Kasih (24/7 Hotline)**: 📞 15999
- **Befrienders KL** (emotional support): 📞 03-7627 2929 / 🌐 [befrienders.org.my](https://www.befrienders.c
- **Childline Malaysia** (for youth under 18): 📞 15999
You can call anonymously. Everything you say is confidential.
""")
#----------#
# Sidebar Navigation
st.sidebar.title("Page Navigation")
tabs = ["🏠 Home", "🎯 Prediction"]
page = st.sidebar.radio("Select a Page", tabs)
if page == "🏠 Home":
home_page()

```python
#-----------------------------------------------------------------------------------------------------------------#

# Sidebar Navigation
st.sidebar.title("Page Navigation")
tabs = ["🏠 Home", "🎯 Prediction"]
page = st.sidebar.radio("Select a Page", tabs)

if page == "🏠 Home":
    home_page()
elif page == "🎯 Prediction":
    prediction_page()

#-----------------------------------------------------------------------------------------------------------------#


### --- Additional Design --- ###

# Footer: Add contact information or credits
st.markdown("""
    <style>
    .footer {
        background-color: #ededed;
        color: gray;
        padding: 10px;
        text-align: center;
        font-size: 12px;
        position: fixed;
        bottom: 0;
        width: 100%;
    }
    </style>
""", unsafe_allow_html=True)

# Display footer
st.markdown('<div class="footer">🌎 Contact us at: support@bullyingpredictionsystem.com | &copy; YAP JIA XIN_TP066475 @ 20


# streamlit run Bullying_predictive_system.py
```

Appendix H : Turnitin Similarity Report

FYP - YAP JIA XIN_TP066475.docx

ORIGINALITY REPORT

25%	16%	16%	16%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	Submitted to Asia Pacific University College of Technology and Innovation (UCTI) Student Paper	5%
2	www.ncbi.nlm.nih.gov Internet Source	1%
3	www.mdpi.com Internet Source	1%
4	www.coursehero.com Internet Source	1%
5	Submitted to Georgia Institute of Technology Main Campus Student Paper	1%
6	download.bibis.ir Internet Source	<1%
7	Submitted to Queen Mary and Westfield College Student Paper	<1%
8	Submitted to Southern Arkansas University (Blackboard LTI 1.3) Student Paper	<1%

9	Abdelaziz Testas. "Distributed Machine Learning with PySpark", Springer Science and Business Media LLC, 2023 Publication	<1%
10	ecp.ep.liu.se Internet Source	<1%
11	ed.buffalo.edu Internet Source	<1%
12	H L Gururaj, Francesco Flammini, V Ravi Kumar, N S Prema. "Recent Trends in Healthcare Innovation", CRC Press, 2025 Publication	<1%
13	Submitted to Middlesex University Student Paper	<1%
14	nces.ed.gov Internet Source	<1%
15	Submitted to University of Technology, Sydney Student Paper	<1%
16	"Computer Vision and Image Processing", Springer Science and Business Media LLC, 2026 Publication	<1%
17	Submitted to University of East London Student Paper	<1%